

Big Data

- Big data means **extremely large data sets** that may be **analyzed computationally** to reveal **patterns, trends, and associations**, especially relating to human behavior and interactions.
- Big data can be **structured** (often numeric, easily formatted and stored) or **unstructured** (more free-form, less quantifiable).
- Big data is characterized by the **three Vs: volume, velocity, and variety**.
 - **Volume** refers to the **amount of data** generated and stored, which can range from terabytes to zettabytes.
 - **Velocity** refers to the **speed of data creation and collection**, which can be real-time or near-real-time.
 - **Variety** refers to the **diversity of data types and sources**, which can include text, images, audio, video, social media, sensor data, etc.
- Big data can be used for various purposes, such as **business intelligence, customer analytics, fraud detection, healthcare, education, social media, smart cities**, etc.
- Big data requires **specialized tools and techniques** to store, process, and analyze, such as **distributed systems, cloud computing, parallel processing, machine learning, data mining, visualization**, etc.
- Big data also poses **challenges and risks**, such as **data quality, data security, data privacy, data ethics, data governance**, etc.

Mnemonics and learning tricks

- To remember the three Vs of big data, you can use the following mnemonic: **Very Vast and Varied Data**.
- To remember some of the tools and techniques for big data, you can use the following acronym: **D**istributed systems, **C**loud computing, **P**arallel processing, **M**achine learning, **D**ata mining, **V**isualization (**DCPMDV**).
- To remember some of the challenges and risks of big data, you can use the following acronym: **Q**uality, **S**ecurity, **P**rivacy, **E**thics, **G**overnance (**QSPEG**).

Unit 1 - Introduction to Big Data

- Big data is a term that refers to the large, complex, and diverse datasets that are generated from various sources at high speed and volume.
- Big data challenges the traditional methods of data processing, storage, analysis, and visualization, and requires new technologies and techniques to handle it effectively and efficiently.
- Big data has four main characteristics, also known as the 4Vs: volume, variety, velocity, and veracity.
 - **Volume**: The amount of data that is generated and stored. Big data can range from terabytes to petabytes and beyond.

- Variety: The types and formats of data that are collected and analyzed. Big data can include structured, semi-structured, and unstructured data, such as text, images, audio, video, sensor data, web logs, social media data, etc.
- Velocity: The speed at which data is generated and processed. Big data can be produced and consumed in real-time or near-real-time, such as streaming data from sensors, web clicks, transactions, etc.
- Veracity: The quality and reliability of data that is collected and analyzed. Big data can be noisy, incomplete, inconsistent, inaccurate, or fraudulent, and may require data cleansing, integration, validation, and verification.
- Big data can provide valuable insights and opportunities for various domains and applications, such as business, health care, education, science, government, etc.
- Big data can also pose some challenges and risks, such as privacy, security, ethics, governance, scalability, etc.
- Big data can be processed and analyzed using different approaches, such as batch processing, stream processing, in-memory processing, etc.
- Big data can be stored and managed using different systems, such as distributed file systems, NoSQL databases, data warehouses, data lakes, etc.
- Big data can be analyzed using different techniques, such as descriptive analytics, predictive analytics, prescriptive analytics, etc.
- Big data can be visualized using different tools, such as charts, graphs, maps, dashboards, etc.

Some possible mnemonics and learning tricks for Unit 1 - Introduction to Big Data are:

- To remember the 4Vs of big data, you can use the acronym VVVV or the phrase “Very Very Very Very big data”.
- To remember the types and formats of data, you can use the acronym SSTU or the phrase “Some data is Structured, Semi-structured, or Text, and some data is Unstructured”.
- To remember the domains and applications of big data, you can use the acronym BHESG or the phrase “Big data Helps in Education, Science, and Government”.
- To remember the challenges and risks of big data, you can use the acronym PSEGS or the phrase “Privacy, Security, Ethics, Governance, and Scalability are big data challenges”.
- To remember the approaches of big data processing and analysis, you can use the acronym BSI or the phrase “Big data can be processed in Batches, Streams, or In-memory”.
- To remember the systems of big data storage and management, you can use the acronym DNDWDL or the phrase “Distributed file systems, NoSQL databases, Data warehouses, Data lakes are big data systems”.
- To remember the techniques of big data analysis, you can use the acronym

DPP or the phrase “Descriptive, Predictive, and Prescriptive analytics are big data techniques”.

- To remember the tools of big data visualization, you can use the acronym CGMD or the phrase “Charts, Graphs, Maps, and Dashboards are big data tools”.

Types of digital data in big data

- Big data is a term that refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze.
- Digital data is any data that is represented in a binary format, such as 0s and 1s, and can be manipulated by computers or other digital devices.
- There are different types of digital data in big data, depending on the nature, structure, and format of the data. Some of the common types are:
 - **Structured data:** This is the data that has a predefined schema, such as tables, records, or fields, and can be easily organized, queried, and analyzed using relational databases or other structured data management systems. Examples of structured data are customer information, sales transactions, product inventory, etc.
 - **Unstructured data:** This is the data that has no fixed schema or format, and is often text-based, such as emails, social media posts, documents, images, videos, audio, etc. Unstructured data is difficult to store, process, and analyze using traditional methods, and requires specialized tools and techniques, such as natural language processing, computer vision, or machine learning. Examples of unstructured data are tweets, blogs, reviews, photos, etc.
 - **Semi-structured data:** This is the data that has some elements of structure, such as tags, labels, or metadata, but does not conform to a rigid schema or format. Semi-structured data can be a combination of structured and unstructured data, and can be partially processed and analyzed using both relational and non-relational methods. Examples of semi-structured data are XML, JSON, HTML, etc.
 - **Multistructured data:** This is the data that is derived from the interactions or integration of different types of data, such as structured, unstructured, or semi-structured data. Multistructured data can be complex and heterogeneous, and can capture the dynamics and relationships of the data sources. Examples of multistructured data are web logs, sensor data, clickstream data, etc.
- A mnemonic to remember the types of digital data in big data is **SUSM**, which stands for **S**tructured, **U**nstructured, **S**emi-structured, and **M**ultistructured data.

History of Big Data Innovation

- The concept of big data is not new, but the term itself was coined by Roger Mougalias from O'Reilly Media in 2005, a year after they created the term Web 2.0 .
- Big data refers to a large set of data that is almost impossible to manage and process using traditional business intelligence tools. It is characterized by the 3Vs: volume, velocity, and variety .
- The origins of large data sets go back to the 1960s and 1970s, when the first data centers and the relational database model were developed .
- Some of the milestones in the history of big data innovation are:
 - 1965: The US Government plans the world's first data center to store 742 million tax returns and 175 million sets of fingerprints on magnetic tape .
 - 1970: IBM mathematician Edgar F. Codd presents his framework for a “relational database”, which allows data to be stored and accessed in a structured and efficient way .
 - 1976: The Cray-1 supercomputer is introduced, capable of processing 160 million floating-point operations per second (FLOPS).
 - 1989: Tim Berners-Lee invents the World Wide Web, which enables the sharing and linking of information across the internet.
 - 1995: The Apache web server is released, which becomes the most popular platform for hosting websites and web applications.
 - 1997: Michael Lesk publishes an article estimating that the amount of information on the internet is about 100 terabytes, and that it doubles every year.
 - 1999: Google introduces the PageRank algorithm, which uses the link structure of the web to rank the relevance of web pages.
 - 2001: Doug Laney, an analyst at Gartner, defines the 3Vs of big data: volume, velocity, and variety.
 - 2003: Google develops the MapReduce framework, which allows parallel processing of large data sets across multiple servers.
 - 2004: Google publishes a paper on the Google File System, a distributed file system designed to handle large amounts of data.
 - 2005: Yahoo creates Hadoop, an open-source software framework that implements MapReduce and the Google File System.
 - 2006: Amazon launches its cloud computing platform, Amazon Web Services, which provides on-demand access to computing resources and storage.
 - 2008: The term “big data” becomes popular after a Wired article by Chris Anderson titled “The End of Theory: The Data Deluge Makes the Scientific Method Obsolete”.
 - 2009: Google releases TensorFlow, an open-source software library for machine learning and artificial intelligence.

- 2010: Facebook launches the Open Graph protocol, which allows third-party applications to access and integrate data from the social network.
- 2011: IBM’s Watson computer defeats human champions in the quiz show Jeopardy!, demonstrating the power of natural language processing and big data analytics.
- 2012: The Obama campaign uses big data to target and mobilize voters, winning the US presidential election.
- 2013: The NSA’s PRISM program is revealed, exposing the mass surveillance of internet and phone data by the US government.
- 2014: Alibaba, the Chinese e-commerce giant, sets a record for the largest initial public offering (IPO) in history, raising \$25 billion. Alibaba’s success is attributed to its use of big data to understand and serve its customers.
- 2015: The European Union adopts the General Data Protection Regulation (GDPR), which aims to protect the privacy and rights of individuals in the digital age.
- 2016: AlphaGo, a computer program developed by Google’s DeepMind, defeats the world champion of Go, a complex board game, using deep learning and big data.
- 2017: China announces its plan to become the world leader in artificial intelligence by 2030, investing heavily in big data and machine learning research and development.
- 2018:

Introduction to Big Data Platform

- Big data platform is a type of IT solution that combines the features and capabilities of several big data applications and utilities within a single solution.
- Big data platform is designed to provide a single, unified and consistent source of data and data processing for big data analytics, data mining, data visualization, data integration, data quality, data governance and data security.
- Big data platform typically consists of the following components:
 - Data sources: The various types of data that are collected, stored and processed by the platform, such as structured, unstructured, semi-structured, streaming, batch, etc.
 - Data storage: The underlying data storage layer that provides the scalability, reliability and performance for storing and retrieving large volumes of data, such as Hadoop Distributed File System (HDFS), NoSQL databases, cloud storage, etc.
 - Data processing: The core data processing engine that performs the data transformation, manipulation, aggregation, analysis and querying operations on the data, such as MapReduce, Spark, Hive, Pig, etc.

- Data management: The data management tools and services that provide the data quality, data integration, data governance and data security functions for the platform, such as Sqoop, Flume, Kafka, Oozie, Zookeeper, etc.
- Data access: The data access layer that provides the interface for accessing and interacting with the data and the platform, such as application programming interfaces (APIs), query languages, data visualization tools, etc.
- Big data platform has the following advantages:
 - It enables the integration and analysis of diverse and complex data from multiple sources and formats, which can provide valuable insights and business intelligence for decision making.
 - It provides a scalable, reliable and high-performance solution for handling large volumes, high velocity and high variety of data, which can meet the growing and changing data needs of the organization.
 - It reduces the complexity and cost of developing and maintaining multiple big data applications and utilities, by providing a single, unified and consistent platform that can support various data use cases and scenarios.
 - It improves the data quality, data integration, data governance and data security of the organization, by providing the tools and services for ensuring the accuracy, completeness, consistency, availability and protection of the data.
- Big data platform has the following disadvantages:
 - It requires a high level of technical expertise and skills to design, implement, operate and maintain the platform, which can pose a challenge for the organization in terms of talent acquisition and retention.
 - It involves a high initial investment and ongoing operational cost for acquiring, deploying and managing the hardware, software and network resources for the platform, which can affect the return on investment (ROI) and profitability of the organization.
 - It may face compatibility and interoperability issues with the existing data systems and applications of the organization, which can result in data silos, data duplication, data inconsistency and data loss.
- Some examples of big data platforms are:
 - Cloudera: A commercial big data platform that provides an enterprise-grade distribution of Hadoop and related technologies, along with professional services and support.
 - Hortonworks: A commercial big data platform that provides an open source distribution of Hadoop and related technologies, along with professional services and support.
 - Amazon Web Services (AWS): A cloud-based big data platform that provides a suite of services and tools for storing, processing, analyzing and visualizing big data on the cloud, such as Amazon S3, Amazon EMR, Amazon Redshift, Amazon Kinesis, etc.

- Microsoft Azure: A cloud-based big data platform that provides a suite of services and tools for storing, processing, analyzing and visualizing big data on the cloud, such as Azure Data Lake, Azure HDInsight, Azure Synapse Analytics, Azure Stream Analytics, etc.
- Google Cloud Platform (GCP): A cloud-based big data platform that provides a suite of services and tools for storing, processing, analyzing and visualizing big data on the cloud, such as Google Cloud Storage, Google Dataproc, Google BigQuery, Google Dataflow, etc.
- Some applications of big data platform are:
 - Social media analytics: Analyzing the data generated by social media platforms, such as Facebook, Twitter, Instagram, etc., to understand the user behavior, preferences, sentiments, trends, etc., and to provide personalized recommendations, targeted advertising, customer service, etc.
 - E-commerce analytics: Analyzing the data generated by e-commerce platforms, such as Amazon, eBay, Alibaba, etc., to understand the customer behavior, preferences, purchase patterns, feedback, etc., and to provide personalized recommendations, dynamic pricing, inventory management, fraud detection, etc.
 - Healthcare analytics: Analyzing the data generated by healthcare platforms, such as electronic health records

Drivers for Big Data

- Drivers for big data are the factors that motivate the need for collecting, storing, processing, and analyzing large and complex datasets.
- Some of the drivers for big data are:
 - **Data explosion:** The amount of data generated by various sources such as social media, sensors, mobile devices, e-commerce, etc. is growing exponentially. According to IDC, the global data volume is expected to reach 175 zettabytes by 2025, up from 33 zettabytes in 2018. This data explosion poses challenges for traditional data management systems, which are not designed to handle such volumes and varieties of data.
 - **Data value:** The data generated by various sources has potential value for various purposes such as business intelligence, decision making, innovation, customer satisfaction, etc. However, extracting value from data requires advanced techniques and tools that can handle the complexity and diversity of data. Big data analytics can help in discovering hidden patterns, trends, insights, and opportunities from data, and provide competitive advantages for organizations and individuals.
 - **Data democratization:** The availability and accessibility of data has increased significantly in recent years, due to the advancement of technologies such as cloud computing, open source software, and

data sharing platforms. Data democratization refers to the process of making data available and understandable to a wider range of users, without requiring specialized skills or knowledge. Data democratization can empower users to leverage data for their own purposes, and foster a data-driven culture in organizations and society.

- **Data regulation:** The increasing use and misuse of data has also raised concerns about the privacy, security, ethics, and governance of data. Data regulation refers to the rules and policies that govern the collection, storage, processing, and sharing of data, and protect the rights and interests of data owners, users, and subjects. Data regulation can also influence the design and implementation of big data systems, and require compliance with various standards and regulations.

Big Data Architecture

Big data architecture is a framework that defines the components, processes, and technologies needed to capture, store, process, and analyze Big Data. Big Data refers to the large and complex data sets that are generated from various sources and require special techniques and tools to handle them. Big data architecture is the foundation of big data analytics, which aims to extract valuable insights from the data and support decision making.

A typical big data architecture consists of the following layers:

- **Data sources:** This is the layer where data is sourced from multiple inputs in a variety of formats, such as structured, semi-structured, or unstructured data. Data sources can include databases, files, web logs, social media, sensors, streaming data, etc.
- **Data storage:** This is the layer where data is ingested, stored, and converted into a common format that can be processed by the next layer. Data storage can involve different types of storage systems, such as relational databases, NoSQL databases, data warehouses, data lakes, cloud storage, etc. Data storage also involves data quality, security, and governance aspects.
- **Batch processing:** This is the layer where large volumes of data are processed in batches, usually at regular intervals, such as daily, weekly, or monthly. Batch processing can involve data transformation, aggregation, filtering, cleansing, enrichment, etc. Batch processing can use various tools and frameworks, such as Hadoop, Spark, Hive, Pig, etc.
- **Stream processing:** This is the layer where data is processed in real time or near real time, as it arrives from the data sources. Stream processing can involve data ingestion, filtering, transformation, enrichment, analysis, etc. Stream processing can use various tools and frameworks, such as Kafka, Storm, Flink, Spark Streaming, etc.
- **Data analysis:** This is the layer where data is analyzed using various techniques and methods, such as descriptive, diagnostic, predictive, or

prescriptive analytics. Data analysis can involve data mining, machine learning, statistical modeling, visualization, reporting, etc. Data analysis can use various tools and frameworks, such as R, Python, SAS, Tableau, Power BI, etc.

- **Data consumption:** This is the layer where data is consumed by the end users or applications, such as business users, analysts, data scientists, dashboards, reports, etc. Data consumption can involve data access, delivery, presentation, exploration, etc.

Some of the benefits of big data architecture are:

- It enables the handling of large and complex data sets that traditional systems cannot handle.
- It supports the integration and analysis of data from various sources and formats, which can provide a holistic view of the data and the business.
- It enables the discovery of hidden patterns, trends, and insights from the data, which can support decision making and innovation.
- It improves the performance, scalability, reliability, and availability of the data processing and analysis systems.
- It reduces the cost and complexity of data management and storage by leveraging cloud and distributed computing technologies.

Some of the challenges of big data architecture are:

- It requires a high level of knowledge and skill to design, build, test, and troubleshoot big data processes and systems.
- It involves multiple components and technologies that have to work together seamlessly and efficiently, which can pose integration and configuration challenges.
- It requires a careful balance between the trade-offs of latency, accuracy, and complexity of the data processing and analysis methods.
- It requires a robust data governance and security framework to ensure the quality, integrity, and privacy of the data.

Some of the best practices of big data architecture are:

- Define the business goals and requirements clearly and align them with the data architecture.
- Choose the appropriate data sources, storage systems, processing methods, and analysis techniques based on the data characteristics, such as volume, velocity, variety, veracity, and value.
- Design the data architecture with modularity, flexibility, and scalability in mind, and use standard and open technologies and frameworks as much as possible.
- Optimize the data architecture for performance, efficiency, and reliability, and use parallel and distributed computing techniques to leverage the resources and speed up the processes.
- Monitor and evaluate the data architecture regularly and update it as needed to meet the changing business and data needs.

A mnemonic to remember the five characteristics of big data is **Very Valuable Variety Velocity Veracity**.

Some possible additional sentences to conclude the response are:

- I hope this helps you understand the basics of big data architecture and its benefits, challenges, and best practices.
- If you have any questions or feedback, please

Big data characteristics

Big data is a term that refers to data sets that are too large, complex, or dynamic for traditional data processing methods to handle. Big data can be characterized by the following five V's:

- **Volume:** The amount of data generated and stored. Big data can range from terabytes to petabytes or even exabytes of data. For example, Facebook generates about 4 petabytes of data per day from its users' activities.
- **Velocity:** The speed at which data is created, collected, and processed. Big data can be generated and analyzed in real-time or near-real-time, requiring fast and efficient data processing methods. For example, Twitter streams about 500 million tweets per day, which need to be processed and analyzed quickly.
- **Variety:** The diversity of data types and sources. Big data can include structured, semi-structured, or unstructured data from different domains and formats, such as text, images, videos, audio, sensor data, web logs, social media, etc. For example, Netflix collects and analyzes data from various sources, such as user ratings, viewing history, preferences, etc.
- **Veracity:** The quality and reliability of data. Big data can be noisy, incomplete, inconsistent, or inaccurate, affecting the accuracy and validity of data analysis and decision making. For example, Wikipedia articles can contain errors, biases, or vandalism, which need to be detected and corrected.
- **Value:** The potential usefulness and benefits of data. Big data can provide valuable insights and opportunities for various applications and domains, such as business, health, education, science, etc. For example, Google uses big data to improve its search engine, products, and services.

A possible mnemonic to remember the five V's of big data is:

Very **Vast** **Variety** of **Valid** **Value**

5 Vs of Big Data

Big data is a term that refers to the large and complex datasets that are generated from various sources and require special techniques and tools to analyze and process. Big data can be characterized by five Vs: volume, velocity, variety, veracity, and value. These are explained below:

- **Volume:** This refers to the amount or size of data that is generated and stored. Big data can range from terabytes to petabytes or even exabytes of data. The volume of data can pose challenges for storage, processing, and analysis. For example, social media platforms like Facebook and Twitter generate huge volumes of data every day from their users' posts, likes, comments, and shares.
- **Velocity:** This refers to the speed or rate at which data is generated, collected, and analyzed. Big data can be produced and consumed in real-time or near-real-time, which requires fast and efficient processing and analysis. For example, sensor data from IoT devices, streaming data from online videos, and transaction data from e-commerce platforms are examples of high-velocity data.
- **Variety:** This refers to the diversity or heterogeneity of data sources, formats, and types. Big data can come from structured, semi-structured, or unstructured data sources, such as databases, spreadsheets, documents, images, videos, audio, text, etc. The variety of data can pose challenges for integration, cleaning, and analysis. For example, health care data can come from various sources, such as electronic health records, medical images, lab tests, prescriptions, etc.
- **Veracity:** This refers to the quality or reliability of data. Big data can be noisy, incomplete, inconsistent, inaccurate, or fraudulent, which can affect the validity and usefulness of the data. The veracity of data can pose challenges for verification, validation, and trust. For example, online reviews, ratings, and feedback can be influenced by bias, spam, or fake accounts.
- **Value:** This refers to the potential or actual benefit or impact of data. Big data can provide valuable insights, patterns, trends, and predictions that can help in decision making, problem solving, innovation, and optimization. The value of data can pose challenges for extraction, interpretation, and visualization. For example, customer data can provide value for businesses in terms of understanding customer behavior, preferences, and satisfaction.

A possible mnemonic to remember the 5 Vs of big data is:

Very **V**ast **V**ariety of **V**alid **V**alue

or

Volume **V**elocity **V**ariety **V**eracity **V**alue

Big Data technology components

Big Data technology components are the software and hardware tools that enable the processing, storage, analysis, and visualization of large and complex data sets. Some of the main components are:

- **Hadoop:** An open-source framework that allows distributed processing of large data sets across clusters of computers using simple programming models. It consists of four modules: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN.
 - Hadoop Common: The common utilities that support the other Hadoop modules.
 - Hadoop Distributed File System (HDFS): A distributed file system that provides high-throughput access to data across the Hadoop cluster.
 - Hadoop MapReduce: A programming model and software framework for writing applications that process large amounts of data in parallel on a cluster of nodes.
 - Hadoop YARN: A resource management system that allocates and schedules computing resources for applications running on the Hadoop cluster.
- **Spark:** An open-source cluster computing framework that provides fast and general data processing capabilities. It supports batch processing, streaming processing, machine learning, graph processing, and SQL queries. It can run on Hadoop, Mesos, Kubernetes, or standalone.
 - Spark Core: The foundation of the Spark framework that provides distributed task scheduling, memory management, fault recovery, and basic I/O functionalities.
 - Spark SQL: A module that supports structured and semi-structured data processing and querying using SQL or a DataFrame API.
 - Spark Streaming: A module that supports scalable and fault-tolerant stream processing of live data streams.
 - Spark MLlib: A module that provides scalable machine learning algorithms and utilities.
 - Spark GraphX: A module that supports graph processing and analysis using the GraphX API or the GraphFrames library.
- **Hive:** An open-source data warehouse system that facilitates data summarization, analysis, and querying using a SQL-like language called HiveQL. It runs on top of Hadoop and converts HiveQL queries into MapReduce, Tez, or Spark jobs.
- **Pig:** An open-source platform that provides a high-level language called Pig Latin for data analysis and manipulation. It runs on top of Hadoop and converts Pig Latin scripts into MapReduce, Tez, or Spark jobs.
- **Flume:** An open-source service that collects, aggregates, and moves large amounts of streaming data from various sources to a centralized data store such as HDFS or HBase. It uses a simple and flexible architecture based on streaming data flows.
- **Sqoop:** An open-source tool that transfers bulk data between Hadoop and structured data stores such as relational databases. It uses MapReduce to import and export data in parallel.
- **HBase:** An open-source distributed and scalable non-relational database that provides random, real-time read/write access to large data sets. It is

modeled after Google's Bigtable and runs on top of HDFS.

- **Kafka:** An open-source distributed streaming platform that provides high-throughput, low-latency, and fault-tolerant data ingestion and processing. It uses a publish-subscribe model and maintains data in topics that are partitioned and replicated across multiple nodes.
- **Storm:** An open-source distributed real-time computation system that processes unbounded streams of data. It uses a directed acyclic graph (DAG) of spouts and bolts to define the data flow and the processing logic.
- **Zookeeper:** An open-source service that provides centralized coordination and configuration management for distributed systems. It maintains a hierarchical namespace of data nodes that store configuration and status information and enables synchronization, leader election, and group membership services.

Big Data importance

- Big data refers to the large and complex datasets that are generated from various sources, such as social media, sensors, e-commerce, etc. Big data has the characteristics of volume, velocity, variety, veracity, and value.
- Big data is important because it can provide valuable insights and opportunities for businesses, governments, researchers, and individuals. Some of the benefits of big data are:
 - It can help improve decision making and problem solving by analyzing data and finding patterns, trends, correlations, and anomalies.
 - It can help enhance customer experience and satisfaction by understanding customer behavior, preferences, and feedback.
 - It can help optimize operations and efficiency by monitoring and improving processes, performance, and quality.
 - It can help innovate and create new products, services, and business models by discovering new opportunities, markets, and customer segments.
 - It can help reduce costs and risks by detecting and preventing fraud, waste, errors, and threats.
- Big data is also important because it can have a significant impact on various domains and sectors, such as health, education, agriculture, environment, energy, transportation, security, etc. Some of the examples of big data applications are:
 - It can help improve health care and outcomes by enabling personalized medicine, disease diagnosis, drug discovery, and epidemic prevention.
 - It can help improve education and learning by enabling adaptive learning, student assessment, curriculum design, and teacher training.
 - It can help improve agriculture and food security by enabling precision farming, crop yield prediction, pest control, and food traceabil-

ity.

- It can help improve environment and sustainability by enabling environmental monitoring, pollution control, climate change analysis, and natural disaster management.
- It can help improve energy and resource management by enabling smart grid, renewable energy, demand response, and energy efficiency.
- It can help improve transportation and mobility by enabling traffic management, route optimization, autonomous vehicles, and ride-sharing.
- It can help improve security and public safety by enabling crime prevention, terrorism detection, cyber security, and emergency response.

Big Data applications

Big data is the term used to describe the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, analyze and visualize. Big data applications are the software solutions that use big data to provide insights, predictions, recommendations, decisions and actions for various domains and industries. Some of the common big data applications are:

- **Business analytics:** Big data applications can help businesses to understand their customers, markets, competitors, products, services and operations better by analyzing data from various sources such as transactions, social media, web logs, sensors, etc. Big data applications can also help businesses to optimize their strategies, processes, performance and profitability by providing data-driven insights and solutions. Some examples of business analytics applications are customer segmentation, churn prediction, sentiment analysis, fraud detection, pricing optimization, etc.
- **Healthcare:** Big data applications can help healthcare providers and researchers to improve the quality, efficiency and effectiveness of healthcare services and outcomes by analyzing data from various sources such as electronic health records, medical images, genomic data, wearable devices, etc. Big data applications can also help healthcare providers and researchers to discover new treatments, drugs, diagnostics and prevention methods by providing data-driven insights and solutions. Some examples of healthcare applications are disease diagnosis, personalized medicine, drug discovery, epidemic tracking, etc.
- **Education:** Big data applications can help educators and learners to enhance the quality, accessibility and affordability of education by analyzing data from various sources such as online courses, learning management systems, student records, assessments, etc. Big data applications can also help educators and learners to personalize and optimize the learning process and outcomes by providing data-driven insights and solutions. Some examples of education applications are adaptive learning, learning analyt-

ics, student retention, curriculum design, etc.

- **Transportation:** Big data applications can help transportation providers and users to improve the safety, efficiency and convenience of transportation services and systems by analyzing data from various sources such as vehicles, roads, traffic, weather, etc. Big data applications can also help transportation providers and users to optimize the transportation planning, management and operation by providing data-driven insights and solutions. Some examples of transportation applications are route optimization, traffic prediction, accident prevention, autonomous driving, etc.
- **Social media:** Big data applications can help social media platforms and users to enhance the quality, diversity and engagement of social media content and interactions by analyzing data from various sources such as posts, comments, likes, shares, etc. Big data applications can also help social media platforms and users to understand and influence the social media trends, behaviors and outcomes by providing data-driven insights and solutions. Some examples of social media applications are content recommendation, trend analysis, sentiment analysis, influencer detection, etc.

Big Data features – security, compliance, auditing and protection

- **Security:** The process of ensuring that only authorized users can access the data and resources, and preventing unauthorized access, modification, or deletion of data.
 - Security challenges in Big Data include:
 - * **Data volume:** The large amount of data makes it difficult to apply traditional security mechanisms such as encryption, access control, or firewall.
 - * **Data variety:** The heterogeneous and unstructured nature of data makes it hard to classify and protect data according to its sensitivity or value.
 - * **Data velocity:** The high speed and frequency of data generation and processing requires real-time security solutions that can handle dynamic and streaming data.
 - * **Data veracity:** The uncertainty and inconsistency of data quality and provenance poses risks of data tampering, manipulation, or falsification.
 - Security solutions for Big Data include:
 - * **Data encryption:** The process of transforming data into an unreadable form using a secret key, so that only authorized parties can decrypt and access the data.
 - * **Data masking:** The process of hiding or replacing sensitive data elements with fictitious or anonymized data, so that the data can be used for analysis or testing without revealing the original data.

- * Data tokenization: The process of replacing sensitive data elements with non-sensitive tokens that have no meaning or value, but can be mapped back to the original data using a secure lookup table.
- * Data access control: The process of defining and enforcing policies and rules that specify who can access what data and resources, and under what conditions.
- * Data auditing: The process of monitoring and recording the activities and events related to data access, usage, and modification, and generating reports and alerts for security analysis and compliance purposes.
- * Data protection: The process of safeguarding data from loss, corruption, or theft, by using techniques such as backup, replication, recovery, or erasure.
- Compliance: The process of ensuring that the data and resources are managed and used in accordance with the applicable laws, regulations, standards, and policies.
 - Compliance challenges in Big Data include:
 - * Data governance: The lack of clear and consistent rules and responsibilities for data ownership, stewardship, quality, and life-cycle management.
 - * Data privacy: The risk of violating the rights and preferences of data subjects regarding the collection, processing, and sharing of their personal or sensitive data.
 - * Data ethics: The potential of using data for malicious, discriminatory, or harmful purposes, or without the consent or awareness of data subjects or stakeholders.
 - Compliance solutions for Big Data include:
 - * Data classification: The process of categorizing data according to its sensitivity, value, or regulatory requirements, and applying appropriate security and privacy measures accordingly.
 - * Data retention: The process of determining and implementing the optimal duration and location for storing data, based on its business value and legal obligations.
 - * Data deletion: The process of permanently removing data from the storage media, when it is no longer needed or required, or when requested by data subjects or authorities.
 - * Data anonymization: The process of removing or modifying any identifying or linkable information from data, so that the data cannot be traced back to the data subjects or sources.
 - * Data consent: The process of obtaining and managing the permission and preferences of data subjects or stakeholders for the collection, processing, and sharing of their data.
- Auditing: The process of verifying and validating that the data and resources are secure, compliant, and reliable, and that the data processing and analysis are accurate, consistent, and trustworthy.

- Auditing challenges in Big Data include:
 - * Data provenance: The difficulty of tracking and documenting the origin, history, and lineage of data, especially when data is derived from multiple and diverse sources, or when data is transformed or aggregated.
 - * Data quality: The uncertainty and inconsistency of data accuracy, completeness, timeliness, and relevance, especially when data is heterogeneous and unstructured, or when data is dynamic and streaming.
 - * Data integrity: The vulnerability of data to unauthorized or unintended modification, deletion, or corruption, especially when data is distributed and replicated across multiple nodes or locations.
 - * Data trust: The lack of confidence and credibility of data and its analysis, especially when data is incomplete, inaccurate, or biased, or when data is processed or presented in a misleading or deceptive way.
- Auditing solutions for Big Data include:
 - * Data lineage: The process of capturing and visualizing the meta-data and dependencies of data, such as its source, destination, transformation, and usage, along the data pipeline.
 - * Data quality assessment: The process of measuring and evaluating the quality of data, using metrics and indicators such as accuracy, completeness, timeliness, and relevance.
 - * Data integrity check: The process of detecting and correcting

Security of Big Data

- Big data security is the collective term for all the measures and tools used to guard both the data and analytics processes from attacks, theft, or other malicious activities that could harm or negatively affect them.
- Big data security is concerned with attacks that originate either from the online or offline spheres, and it covers the entire lifecycle of data, from its creation to its deletion.
- Big data security faces several challenges, such as:
 - The volume, variety, and velocity of big data make it difficult to secure and monitor using traditional methods.
 - The distributed nature of big data systems, such as Hadoop, introduces new vulnerabilities and risks, such as unauthorized access, data leakage, and data tampering.
 - The lack of standardization and regulation of big data technologies and practices makes it hard to enforce security policies and compliance requirements.
 - The diversity and complexity of big data sources and formats make

it challenging to identify and protect sensitive data, such as personal information, intellectual property, and trade secrets.

- Big data security requires a holistic and proactive approach that involves the following best practices :
 - Safeguard distributed programming frameworks, such as Hadoop, by implementing security features, such as encryption, authentication, authorization, auditing, and logging.
 - Secure non-relational data, such as NoSQL, by applying data encryption, access control, and data masking techniques.
 - Secure data storage and transmission, by using encryption, compression, and hashing algorithms, and by adopting secure protocols, such as HTTPS and SSL/TLS.
 - Secure data processing and analysis, by applying data anonymization, obfuscation, and tokenization methods, and by using trusted and verified algorithms and models.
 - Secure data access and usage, by enforcing role-based access control, multi-factor authentication, and data governance policies, and by educating and training users on security awareness and best practices.
 - Secure data disposal and retention, by following data deletion and archiving policies, and by using secure data erasure and shredding tools.
 - Monitor and audit data security, by using intrusion detection and prevention systems, security information and event management systems, and data loss prevention systems, and by conducting regular security assessments and audits.
 - Update and patch data security, by keeping up with the latest security trends and threats, and by applying security updates and patches to data systems and tools.
 - Leverage cloud security, by choosing a reliable and secure cloud service provider, and by using cloud security services, such as encryption, firewall, and backup.
 - Integrate data security, by aligning data security with business objectives and strategies, and by collaborating with data stakeholders and security experts.
- A possible mnemonic to remember the 10 best practices for securing big data is:
 - **S**afeguard distributed programming frameworks
 - **S**ecure non-relational data
 - **S**ecure data storage and transmission
 - **S**ecure data processing and analysis
 - **S**ecure data access and usage

- **Secure** data disposal and retention
- **Monitor** and audit data security
- **Update** and patch data security
- **Leverage** cloud security
- **Integrate** data security
- **SSSSS MULI** (pronounced as “sassy mooli”)

Compliance of Big Data

- Compliance of big data refers to the process of ensuring that the data collected, processed, and stored by an organization meets the standards and regulations of the relevant authorities and stakeholders.
- Compliance of big data is important for several reasons, such as:
 - Protecting the privacy and security of the data subjects and owners.
 - Preventing and detecting fraud, money laundering, and other criminal activities.
 - Enhancing the trust and reputation of the organization among its clients and partners.
 - Avoiding legal penalties and fines for non-compliance.
- Compliance of big data involves several challenges, such as:
 - Managing the volume, velocity, and variety of data from different sources and formats.
 - Balancing the trade-off between data quality and data quantity.
 - Adapting to the changing and evolving regulations and standards across different jurisdictions and domains.
 - Implementing effective and efficient data governance and risk management practices.
- Compliance of big data can be achieved by following some best practices, such as:
 - Adopting a data-centric approach that focuses on the data lifecycle and the data value chain.
 - Implementing data security measures, such as encryption, anonymization, and access control, to protect the data from unauthorized access and misuse.
 - Applying data transparency principles, such as informing the data subjects and owners about the purpose, scope, and duration of data collection, processing, and storage, and providing them with the right to access, correct, and delete their data.

- Applying data minimization techniques, such as collecting and retaining only the data that is necessary and relevant for the intended use, and deleting the data when it is no longer needed.
- Leveraging data analytics and predictive models to identify and mitigate data risks and compliance issues.
- A possible mnemonic to remember the key aspects of compliance of big data is:
 - **C**ollect data with consent and purpose
 - **O**bserve data quality and quantity
 - **M**onitor data flow and lifecycle
 - **P**rotect data with security and privacy
 - **L**earn from data with analytics and insights
 - **I**nforn data subjects and owners with transparency
 - **A**dapt to data regulations and standards
 - **N**arrow down data with minimization
 - **C**orrect data with access and control
 - **E**rase data with retention and deletion

Auditing of Big Data

- Big data is the term used to describe large and complex data sets that are generated from various sources and require advanced techniques and technologies to process, analyze and derive value from them .
- Auditing of big data involves applying audit principles and methods to assess the quality, reliability, security and usefulness of big data and the systems that produce and consume them .
- Auditing of big data can help auditors to:
 - Expand the scope and depth of their audit projects by using larger and more diverse data populations .
 - Enhance their audit evidence and findings by using data analytics and artificial intelligence to uncover patterns, anomalies, risks and opportunities .
 - Provide more value-added and forward-looking recommendations to the stakeholders by using big data to identify trends, benchmarks, best practices and areas for improvement .
- Auditing of big data also poses many challenges and risks for auditors, such as:
 - Knowing what data is available, relevant and reliable for the audit objectives and scope .
 - Accessing and obtaining data that might be sensitive, confidential, proprietary or regulated by laws and policies .
 - Combining and integrating data from different sources, systems and formats, such as structured data (e.g., transactions, records, tables) and unstructured data (e.g., emails, texts, images, videos) .
 - Applying appropriate data quality and governance controls to ensure

the accuracy, completeness, validity and integrity of the data and the audit results .

- Developing and maintaining the skills, tools and techniques to perform data analysis, visualization and interpretation in a timely and effective manner .
- Communicating and reporting the audit findings and recommendations in a clear, concise and actionable way to the stakeholders .
- Auditing of big data requires auditors to adopt a risk-based and data-driven approach that involves the following steps :
 - Planning: Define the audit objectives, scope, criteria and methodology; identify the data sources, requirements and availability; assess the data risks and opportunities; and design the data analysis plan and procedures.
 - Execution: Obtain and prepare the data for analysis; perform the data analysis using appropriate tools and techniques; validate and document the data quality and audit evidence; and identify and report the audit findings and issues.
 - Reporting: Analyze and interpret the audit findings and issues; formulate and prioritize the audit recommendations; communicate and discuss the audit results and implications with the stakeholders; and prepare and deliver the audit report and follow-up actions.
- Auditing of big data can benefit from some mnemonics and learning tricks, such as:
 - The 5 Vs of big data: Volume, Variety, Velocity, Veracity and Value . These are the key characteristics and challenges of big data that auditors need to consider and address in their audit projects.
 - The 4 Cs of data analysis: Collect, Clean, Compute and Communicate. These are the main steps and tasks involved in performing data analysis for audit purposes.
 - The 3 As of data quality: Accuracy, Availability and Applicability. These are the main dimensions and criteria of data quality that auditors need to assess and ensure in their audit data and results.

Protection of Big Data

Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and tools to process, analyze, and extract value from them. Big data can provide insights and opportunities for various domains, such as business, health, education, security, and social media. However, big data also poses significant challenges and risks to the privacy of individuals and organizations, as it may contain sensitive and personal information that can be exploited, misused, or breached by unauthorized parties.

To protect the privacy of big data, it is important to adopt a comprehensive and systematic approach that covers the following aspects:

- **Data collection:** This involves the process of obtaining and storing big

data from various sources, such as sensors, devices, applications, websites, and social media platforms. Data collection should follow the principles of data minimization, purpose limitation, and consent, which means that only the necessary and relevant data should be collected for a specific and legitimate purpose, and with the consent of the data subjects or owners. Data collection should also comply with the applicable laws and regulations, such as the General Data Protection Regulation (GDPR) in the European Union, which sets strict rules and standards for the protection of personal data.

- **Data retention and archiving:** This involves the process of keeping and preserving big data for a certain period of time, depending on the purpose and value of the data. Data retention and archiving should follow the principles of storage limitation, integrity, and confidentiality, which means that the data should be stored only for as long as necessary, and with appropriate measures to ensure its accuracy, security, and protection from unauthorized access, modification, or deletion. Data retention and archiving should also comply with the applicable laws and regulations, such as the GDPR, which requires the data controllers and processors to implement technical and organizational measures to safeguard the personal data.
- **Data use:** This involves the process of accessing, processing, analyzing, and sharing big data for various purposes, such as research, innovation, decision making, and service delivery. Data use should follow the principles of data quality, fairness, and transparency, which means that the data should be accurate, complete, and up-to-date, and that the data processing and analysis should be fair, lawful, and explainable to the data subjects or owners. Data use should also comply with the applicable laws and regulations, such as the GDPR, which grants the data subjects or owners certain rights and controls over their personal data, such as the right to access, rectify, erase, restrict, object, or port their data.
- **Data disclosure policies and practices:** This involves the process of informing and communicating with the data subjects or owners, as well as the relevant stakeholders and authorities, about the collection, retention, use, and protection of big data. Data disclosure policies and practices should follow the principles of accountability, responsibility, and ethics, which means that the data controllers and processors should be able to demonstrate and justify their compliance with the data protection rules and standards, and that they should respect and protect the interests, values, and rights of the data subjects or owners, as well as the public good and social welfare. Data disclosure policies and practices should also comply with the applicable laws and regulations, such as the GDPR, which requires the data controllers and processors to provide clear and concise information and notifications to the data subjects or owners, and to report any data breaches or incidents to the supervisory authorities.

Some of the techniques and tools that can help to implement the above aspects

of big data protection are:

- **Data anonymization and pseudonymization:** These are methods of transforming or modifying the data to remove or mask any identifying or sensitive information, such as names, addresses, phone numbers, or email addresses, that can link the data to a specific individual or entity. Data anonymization aims to make the data irreversibly anonymous, while data pseudonymization aims to make the data reversibly anonymous, by using a key or a code to replace the original identifiers. These methods can help to reduce the risk of re-identification or de-anonymization of the data by unauthorized parties, and to comply with the data minimization and confidentiality principles.
- **Data encryption and decryption:** These are methods of converting or transforming the data into a secret or unreadable form, using a key or a code, that can only be accessed or restored by authorized parties who have the corresponding key or code. Data encryption and decryption can help to protect the data from unauthorized access, modification, or deletion, and to comply with the data integrity and security principles.
- **

Big Data privacy

Big data privacy is the process of properly managing large and complex data sets to minimize risk and protect sensitive information. Big data privacy is important for both ethical and legal reasons, as well as for customer trust and business value. Here are some points to understand and remember about big data privacy:

- Big data privacy involves four critical data management activities: data collection, retention and archiving, data use, and disclosure policies. Each activity should be reviewed regularly and aligned with the data privacy laws and regulations of the relevant jurisdictions, as well as the expectations and preferences of the data subjects.
- Big data privacy is also a matter of customer trust. The more data you collect about users, the easier it gets to “connect the dots”: to understand their current behavior, draw inferences about their future behavior, and eventually develop deep and detailed profiles of their lives and preferences. This can create privacy risks, such as identity theft, discrimination, or unwanted marketing. Therefore, it is essential to respect the users’ consent, transparency, and control over their data, and to use it only for legitimate and beneficial purposes.
- Big data privacy is a dynamic and evolving concept, as new technologies, applications, and challenges emerge. For example, big data analytics, artificial intelligence, and machine learning can create new insights and value from big data, but also pose new privacy threats, such as re-identification, profiling, or automated decision-making. Therefore, it is important to adopt a privacy-by-design approach, which means integrating privacy prin-

ciples and safeguards into the design and development of big data systems and solutions.

- Big data privacy is not only a responsibility, but also an opportunity. By adopting a proactive and ethical approach to big data privacy, you can enhance your reputation, customer loyalty, and competitive advantage. You can also create new value from your data, such as personalized services, improved customer experience, or social good. Moreover, you can reduce the costs and risks associated with data breaches, fines, or lawsuits.

A possible mnemonic to remember the four critical data management activities for big data privacy is **CURD**:

- **Collection**: how and why you collect data
- **Use**: how and where you use data
- **Retention**: how long and where you store data
- **Disclosure**: how and to whom you share data

Big Data ethics

Big Data ethics is the study of the ethical principles and values that guide the collection, analysis, and use of large and complex datasets. Big Data ethics is concerned with the potential benefits and harms of Big Data for individuals, groups, and society, as well as the responsibilities and obligations of Big Data practitioners and stakeholders.

Some of the main topics and issues in Big Data ethics are:

- **Privacy and confidentiality**: How to protect the personal information and identity of data subjects, especially when data is collected from multiple sources, shared across platforms, and processed by algorithms that may reveal sensitive or intimate details.
- **Consent and transparency**: How to inform and obtain the permission of data subjects about the purpose, scope, and methods of data collection and analysis, as well as the potential risks and benefits of data use and sharing.
- **Ownership and control**: How to determine and respect the rights and interests of data subjects, data providers, data processors, and data users over the access, use, and disposal of data, as well as the benefits and outcomes of data analysis.
- **Quality and accuracy**: How to ensure the validity, reliability, and representativeness of data and data analysis, as well as the correction and accountability of data errors and biases.
- **Fairness and justice**: How to prevent and mitigate the discrimination, exclusion, and exploitation of data subjects, especially vulnerable and marginalized groups, as well as the promotion and protection of human dignity, autonomy, and welfare.
- **Security and safety**: How to safeguard data and data systems from unauthorized access, misuse, and harm, as well as the prevention and

response to data breaches, cyberattacks, and other threats.

- **Social and environmental impact:** How to assess and balance the positive and negative effects of Big Data for society and the environment, as well as the alignment and compliance of Big Data with social norms, values, and laws.

Some of the possible mnemonics and learning tricks for Big Data ethics are:

- **P-COQFS:** A mnemonic to remember the six main topics of Big Data ethics: Privacy, Consent, Ownership, Quality, Fairness, and Security.
- **DATA:** A mnemonic to remember the four main steps of ethical data analysis: Define the problem and purpose, Assess the data and methods, Test the results and assumptions, and Act on the findings and implications.
- **FAIR:** A mnemonic to remember the four main principles of ethical data use and sharing: Findable, Accessible, Interoperable, and Reusable.

Big Data Analytics

- Big data analytics is the process of collecting, examining, and analyzing large amounts of data to discover market trends, insights, and patterns that can help companies make better business decisions .
- Big data analytics can be applied to various types of data, such as structured, unstructured, and streaming data. Structured data is data that can be easily organized and stored in tables, such as numbers, dates, and names. Unstructured data is data that cannot be easily organized and stored in tables, such as text, images, and audio. Streaming data is data that is continuously generated and processed in real time, such as sensor data, web logs, and social media data.
- Big data analytics can use different methods, tools, and applications to collect, process, and derive insights from data. Some of the common methods include data mining, machine learning, natural language processing, and data visualization. Some of the common tools include Hadoop, Spark, Kafka, and NoSQL databases. Some of the common applications include customer segmentation, sentiment analysis, fraud detection, and recommendation systems.
- Big data analytics can provide various benefits to companies, such as:
 - Improving customer satisfaction and loyalty by understanding their needs, preferences, and feedback.
 - Enhancing operational efficiency and productivity by optimizing processes, resources, and costs.
 - Increasing innovation and competitiveness by identifying new opportunities, markets, and products.
 - Reducing risks and threats by detecting and preventing fraud, cyberattacks, and anomalies.
- Big data analytics can also pose some challenges, such as:
 - Managing the volume, velocity, and variety of data that can overwhelm the storage and processing capabilities of traditional systems.

- Ensuring the quality, accuracy, and reliability of data that can be affected by noise, errors, and inconsistencies.
- Protecting the privacy, security, and ethics of data that can be exposed to unauthorized access, misuse, and breach.
- A possible mnemonic to remember the benefits of big data analytics is **CRIER**:
 - **C**ustomer satisfaction and loyalty
 - **R**eduction of risks and threats
 - **I**nnovation and competitiveness
 - **E**nhancement of operational efficiency and productivity
 - **R**ead-time insights and actions

Challenges of conventional systems compared to Big Data

Conventional systems are the traditional data storage and processing systems that are based on relational databases and structured data. They are designed to handle small to medium-sized data sets that are consistent, homogeneous, and predictable. Big Data, on the other hand, is the term used to describe large, complex, and diverse data sets that are generated at high velocity and variety. Big Data poses several challenges to conventional systems, such as:

- **Data volume:** Big Data is characterized by its massive scale, which can range from terabytes to petabytes and beyond. Conventional systems have limited storage capacity and scalability, which makes it difficult and expensive to store and manage such large amounts of data. Big Data requires distributed and parallel systems that can store and process data across multiple nodes and clusters.
- **Data velocity:** Big Data is generated at high speed and frequency, which can be in real-time or near-real-time. Conventional systems have limited processing power and throughput, which makes it challenging and time-consuming to ingest, analyze, and act on such fast-moving data. Big Data requires streaming and batch processing systems that can handle data in motion and at rest.
- **Data variety:** Big Data is composed of different types and formats of data, which can be structured, semi-structured, or unstructured. Conventional systems have rigid data models and schemas, which makes it hard and complex to integrate and query such heterogeneous and diverse data. Big Data requires flexible and schema-less systems that can handle data in any shape and form.
- **Data veracity:** Big Data is often noisy, incomplete, inconsistent, and inaccurate, which can affect the quality and reliability of the data. Conventional systems have strict data quality and integrity rules, which makes it impossible and impractical to deal with such uncertain and dirty data. Big Data requires robust and tolerant systems that can handle data in any condition and quality.
- **Data value:** Big Data is potentially valuable and useful, but it is not al-

ways easy to extract and exploit the insights and knowledge from the data. Conventional systems have limited analytical capabilities and tools, which makes it difficult and costly to perform complex and advanced analytics on such large and complex data. Big Data requires sophisticated and intelligent systems that can perform descriptive, predictive, and prescriptive analytics on the data.

A possible mnemonic to remember these challenges is **5 Vs of Big Data**: Volume, Velocity, Variety, Veracity, and Value.

Intelligent data analysis in Big Data

- Intelligent data analysis (IDA) is the process of applying artificial intelligence techniques, such as machine learning, data mining, natural language processing, and computer vision, to extract useful information and insights from large and complex data sets.
- Big data is a term that refers to data sets that are too large, diverse, or dynamic to be analyzed by traditional methods, such as relational databases, statistical tools, or spreadsheets. Big data can be characterized by the four Vs: volume, variety, velocity, and veracity.
- IDA in big data is a challenging and important task, as it can help discover hidden patterns, trends, anomalies, and correlations in the data, and provide valuable knowledge for decision making, problem solving, innovation, and prediction in various domains, such as business, health, education, science, and security.
- Some of the steps involved in IDA in big data are:
 - Data acquisition: collecting and integrating data from various sources, such as sensors, web, social media, transactions, etc.
 - Data preprocessing: cleaning, transforming, and reducing the data to make it suitable for analysis, such as handling missing values, outliers, noise, duplicates, etc.
 - Data exploration: visualizing, summarizing, and describing the data to understand its characteristics, distribution, and relationships, such as using histograms, scatter plots, box plots, etc.
 - Data modeling: applying appropriate IDA techniques to the data to build models that can capture the underlying structure, patterns, and rules in the data, such as using clustering, classification, regression, association, etc.
 - Data evaluation: assessing the quality, validity, and usefulness of the models, and comparing them with other models or benchmarks, such as using accuracy, precision, recall, F1-score, etc.
 - Data interpretation: extracting and communicating the results and insights from the models, and applying them to the specific domain or problem, such as using reports, dashboards, charts, etc.

- Some of the challenges and opportunities of IDA in big data are:
 - Scalability: the ability to handle large volumes of data efficiently and effectively, such as using parallel, distributed, or cloud computing, or reducing the dimensionality or complexity of the data.
 - Diversity: the ability to handle different types, formats, and sources of data, such as using heterogeneous, multimodal, or multi-view data integration, or adapting to different data schemas or semantics.
 - Dynamics: the ability to handle fast-changing, streaming, or real-time data, such as using incremental, online, or adaptive learning, or detecting concept drift or change points in the data.
 - Uncertainty: the ability to handle noisy, incomplete, or inconsistent data, such as using robust, probabilistic, or fuzzy methods, or handling uncertainty propagation or quantification in the data analysis process.
 - Interpretability: the ability to provide meaningful, understandable, and explainable results and insights from the data analysis, such as using visualization, natural language generation, or explainable AI techniques, or incorporating domain knowledge or human feedback in the data analysis process.
- Some of the applications and examples of IDA in big data are:
 - Customer segmentation: grouping customers based on their behavior, preferences, or characteristics, and providing personalized recommendations, offers, or services to them, such as using k-means clustering, collaborative filtering, or association rule mining.
 - Fraud detection: identifying fraudulent or anomalous transactions or activities, and preventing or mitigating the losses or risks associated with them, such as using logistic regression, decision trees, or outlier detection.
 - Sentiment analysis: extracting and analyzing the opinions, emotions, or attitudes of people from text, speech, or images, and using them for marketing, customer service, or social media analysis, such as using naive Bayes, support vector machines, or deep neural networks.
 - Image recognition: recognizing and classifying objects, faces, or scenes from images or videos, and using them for security, surveillance, or entertainment purposes, such as using convolutional neural networks, face detection, or object detection.
 - Predictive maintenance: predicting and preventing the failure or degradation of machines, equipment, or systems, and optimizing their performance, reliability, or lifespan, such as using linear regression, random forests, or survival analysis.

Nature of data in Big Data

- Big data refers to the large, diverse, and complex datasets that are generated from various sources at high speed and volume.
- The nature of data in big data can be characterized by the following aspects:
 - **Volume:** The amount of data that is generated and stored. Big data can range from terabytes to petabytes or even exabytes of data. For example, Facebook generates about 4 petabytes of data per day from its users' activities.
 - **Variety:** The types and formats of data that are collected and analyzed. Big data can include structured, semi-structured, or unstructured data, such as text, images, videos, audio, sensor data, web logs, social media posts, etc. For example, Netflix collects and analyzes data from various sources, such as user ratings, preferences, viewing history, device information, etc.
 - **Velocity:** The speed at which data is generated and processed. Big data can be produced and consumed in real-time or near-real-time, requiring fast and efficient processing and analysis. For example, Twitter generates about 500 million tweets per day, which need to be processed and analyzed in a timely manner.
 - **Veracity:** The quality and reliability of data that are collected and analyzed. Big data can be noisy, incomplete, inconsistent, or inaccurate, requiring methods to ensure data quality and validity. For example, Amazon uses data quality checks and validation techniques to ensure the accuracy of its product reviews and recommendations.
 - **Value:** The potential and usefulness of data that are collected and analyzed. Big data can provide valuable insights and opportunities for decision making, innovation, and competitive advantage. For example, Google uses big data to improve its search engine, advertising, and other services.
- A mnemonic to remember the five aspects of the nature of data in big data is **Very Vast Varieties of Very Valuable** data.

Analytic processes and tools for Big Data

Big data analytics is the process of uncovering trends, patterns, and correlations in large amounts of raw data to help make data-informed decisions. Big data analytics can be used for various purposes, such as customer segmentation, fraud detection, market analysis, sentiment analysis, and more.

The analytic processes and tools for big data can be divided into four main steps:

1. Collection and storage: This step involves acquiring and storing the data from various sources, such as web logs, social media, sensors, etc. The

data can be structured, semi-structured, or unstructured, and can have different formats, such as text, images, audio, video, etc. The tools and technologies used for this step include:

- Hadoop: An open-source framework that stores and processes large data sets through a distributed computing environment. Hadoop consists of two components: Hadoop Distributed File System (HDFS) for storing the data, and MapReduce for processing the data in parallel. Hadoop also supports other tools, such as Hive, Pig, and Sqoop, for data manipulation and integration.
 - NoSQL databases: Non-relational data management systems that do not require a fixed schema, making them a great choice for handling unstructured or dynamic data. Some examples of NoSQL databases are MongoDB, Cassandra, and Redis.
 - Cloud storage: A service that provides scalable, reliable, and cost-effective storage for big data. Some examples of cloud storage providers are Amazon S3, Google Cloud Storage, and Microsoft Azure Blob Storage.
2. Processing: This step involves transforming and integrating the data to make it suitable for analysis. The data may need to be cleaned, filtered, aggregated, joined, or enriched, depending on the business requirements and the data quality. The tools and technologies used for this step include:
- Data integration software: A software that helps to combine data from different sources and formats, and to ensure data quality and consistency. Some examples of data integration software are Informatica, Talend, and Pentaho.
 - Data preprocessing and scrubbing tools: Tools that help to prepare the data for analysis by removing noise, outliers, missing values, duplicates, or errors. Some examples of data preprocessing and scrubbing tools are Trifacta, Dataiku, and OpenRefine.
 - MapReduce: A programming model that allows to process large data sets in parallel by dividing them into smaller chunks (map) and combining the results (reduce). MapReduce can be implemented using various languages, such as Java, Python, or Scala, and can run on Hadoop or other platforms, such as Spark or Flink.
3. Analysis: This step involves applying various techniques and algorithms to the data to discover insights, patterns, and predictions. The analysis can be descriptive, diagnostic, predictive, or prescriptive, depending on the business objectives and the data availability. The tools and technologies used for this step include:
- Data mining: A process of extracting useful information from large data sets by applying statistical, machine learning, or artificial intelligence methods. Some examples of data mining techniques are clustering, classification, association rule mining, anomaly detection, and text mining.
 - Machine learning: A branch of artificial intelligence that enables systems

to learn from data and improve their performance without explicit programming. Some examples of machine learning methods are regression, decision trees, neural networks, support vector machines, and deep learning.

- Predictive analytics: A process of using data to make predictions about future outcomes or behaviors, such as customer churn, demand forecasting, or risk assessment. Some examples of predictive analytics tools are SAS, IBM SPSS, and R.
 - Prescriptive analytics: A process of using data to recommend optimal actions or decisions, such as pricing, inventory, or marketing strategies. Some examples of prescriptive analytics tools are IBM CPLEX, Gurobi, and Solver.
4. Visualization and communication: This step involves presenting and communicating the results of the analysis to the stakeholders, such as managers, customers, or partners. The results can be visualized using charts, graphs, dashboards, or reports, and can be interactive, dynamic, or real-time, depending on the needs and preferences of the audience. The tools and technologies used for this step include:
- Data visualization tools: Tools that help to create and display graphical representations of the data, such as histograms, pie charts, scatter plots, or heat maps. Some examples of data visualization tools are Tableau, PowerBI, QlikView, and Excel.
 - Data storytelling tools: Tools that help to narrate and explain the data analysis results using text, images, audio,

Analysis vs Reporting in Big Data

- Analysis and reporting are two essential tasks in big data that help to derive insights and value from large and complex datasets.
- Analysis is the process of exploring, transforming, modeling, and interpreting data to answer specific questions or test hypotheses. Analysis can be descriptive, predictive, or prescriptive, depending on the goal and the type of data.
- Reporting is the process of presenting and communicating the results of analysis to various stakeholders, such as managers, customers, or regulators. Reporting can be done in different formats, such as dashboards, charts, tables, or narratives, depending on the audience and the purpose.
- Analysis and reporting are interrelated and complementary, but they have some key differences:
 - Analysis is more exploratory and iterative, while reporting is more structured and standardized.
 - Analysis requires more technical skills and tools, such as programming languages, statistical methods, and machine learning

- algorithms, while reporting requires more business skills and tools, such as data visualization, storytelling, and report design.
- Analysis is more focused on finding patterns, trends, and relationships in data, while reporting is more focused on summarizing, explaining, and recommending actions based on data.
- Analysis is more suitable for complex and dynamic problems, where the data and the questions are constantly changing, while reporting is more suitable for simple and stable problems, where the data and the questions are well-defined and consistent.
- A simple mnemonic to remember the difference between analysis and reporting is:
 - Analysis is about asking **why** and **how** data behaves the way it does, while reporting is about telling **what** and **so what** data means and implies.

Modern data analytic tools for Big Data

- Big Data refers to the large and complex datasets that are beyond the processing capabilities of traditional data management systems.
- Data analytics is the process of extracting meaningful insights from data using various techniques such as statistics, machine learning, data mining, visualization, etc.
- Modern data analytic tools are software applications or platforms that enable data analysts to perform data analytics tasks on Big Data efficiently and effectively.
- Some of the modern data analytic tools for Big Data are:
 - Apache Hadoop: An open-source framework that allows distributed processing of large and diverse data sets across clusters of computers using simple programming models. It consists of four main components: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common. HDFS is a distributed file system that provides high availability and fault tolerance for data. MapReduce is a programming model that enables parallel processing of data using key-value pairs. YARN is a resource management layer that allocates and schedules resources for applications. Hadoop Common is a set of libraries and utilities that support the other components.
 - Apache Spark: An open-source unified analytics engine that provides fast and general-purpose processing of large-scale data. It supports batch, streaming, interactive, and graph analytics. It also supports various languages such as Scala, Python, Java, R, and SQL. Spark consists of a core engine and several libraries that provide functionality for different domains, such as Spark SQL, Spark Streaming,

- MMLib, GraphX, and SparkR. Spark can run on Hadoop, Mesos, Kubernetes, or standalone clusters.
- Apache Kafka: An open-source distributed streaming platform that enables high-throughput, low-latency, and scalable data ingestion and processing. It is based on a publish-subscribe model, where producers publish data to topics and consumers subscribe to topics and consume data. Kafka also provides fault tolerance, replication, and durability for data. Kafka can be integrated with various data sources and sinks, such as Hadoop, Spark, Storm, Flink, Elasticsearch, etc.
 - Apache Flink: An open-source stream processing framework that provides stateful and real-time computation of large-scale data. It supports both batch and stream processing, as well as complex event processing, machine learning, and graph analytics. It also supports various languages such as Java, Scala, Python, and SQL. Flink can run on Hadoop, Mesos, Kubernetes, or standalone clusters.
 - Apache Storm: An open-source distributed real-time computation system that enables scalable and reliable processing of unbounded streams of data. It is based on a topology model, where data flows through a graph of spouts and bolts. Spouts are sources of data, and bolts are units of processing logic. Storm can be integrated with various data sources and sinks, such as Kafka, HDFS, HBase, Cassandra, etc.
 - Apache Hive: An open-source data warehouse system that provides data analysis and querying capabilities for large-scale data stored in Hadoop. It supports a SQL-like language called HiveQL, which can be extended with user-defined functions and custom data types. Hive also supports various storage formats, such as ORC, Parquet, Avro, JSON, etc. Hive can run on top of Hadoop, Spark, or Tez engines.
 - Apache Pig: An open-source platform that provides a high-level language called Pig Latin for data analysis and manipulation on Hadoop. It allows users to express complex data transformations using simple and concise scripts. Pig also supports various data types, such as tuples, bags, maps, etc. Pig can run on top of Hadoop, Spark, or Tez engines.
 - Apache Sqoop: An open-source tool that enables data transfer between Hadoop and relational databases. It supports various databases, such as MySQL, Oracle, PostgreSQL, SQL Server, etc. It also supports various data formats, such as CSV, Avro, Parquet, etc. Sqoop can run as a standalone application or as a part of a workflow system, such as Oozie or Airflow.
 - Apache Flume: An open-source service that collects, aggregates, and moves large amounts of log data from various sources to Hadoop. It supports various sources, such as syslog, HTTP, JMS, etc. and various sinks, such as HDFS, HBase, Kafka, etc. It also supports various data formats, such as JSON, Avro, etc. Flume can run as a

- standalone agent or as a part of a cluster.
- Apache Oozie: An open-source workflow scheduler that manages and coordinates data processing jobs on Hadoop. It supports various types of jobs, such as MapReduce, Pig, Hive, Sqoop, Spark, etc. It also supports various triggers, such as time

Unit 2 - Hadoop and Map Reduce

- Hadoop is an open-source framework that allows distributed processing of large-scale data sets across clusters of computers using simple programming models.
- MapReduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.
- The basic idea of MapReduce is to split the input data into independent chunks that are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps, which are then input to the reduce tasks. Typically both the input and the output of the job are stored in a distributed file system. The framework takes care of scheduling tasks, monitoring them and re-executes the failed tasks.
- The MapReduce framework consists of the following components:
 - JobTracker: the master node that coordinates all the jobs and tasks in the cluster.
 - TaskTracker: the slave node that runs the map and reduce tasks assigned by the JobTracker.
 - JobClient: the client program that submits the MapReduce job to the JobTracker and monitors its progress.
 - Hadoop Distributed File System (HDFS): the distributed file system that stores the input and output data of the MapReduce jobs.
- The MapReduce programming model has the following phases:
 - Map: the user-defined function that takes a key-value pair as input and produces a set of intermediate key-value pairs as output.
 - Shuffle: the framework phase that transfers the intermediate key-value pairs from the map tasks to the reduce tasks based on the intermediate keys.
 - Reduce: the user-defined function that takes an intermediate key and a set of values associated with that key as input and produces a set of output key-value pairs as output.
 - Output: the final output of the MapReduce job that is written to HDFS or other file systems.
- The MapReduce programming model has the following advantages:
 - It is scalable and fault-tolerant, as it can handle large data sets across multiple nodes and recover from node failures.
 - It is simple and expressive, as it abstracts the details of parallelization, distribution, load balancing and fault tolerance from the user.
 - It is flexible and extensible, as it supports various data types, formats,

sources and destinations, and allows customizing the map and reduce functions, the partitioning, sorting and grouping of the intermediate data, and the configuration of the job parameters.

- The MapReduce programming model has the following disadvantages:
 - It is not suitable for interactive or iterative applications, as it has high latency and overhead for each job execution.
 - It is not efficient for complex data processing, as it requires multiple MapReduce jobs to be chained together, which increases the I/O and network costs.
 - It is not optimal for some types of computations, such as graph algorithms, machine learning algorithms, and join operations, as it does not exploit the data locality and structure, and imposes a rigid key-value data model.
- Some examples of applications that use MapReduce are:
 - Word count: a simple application that counts the frequency of each word in a large text corpus.
 - Inverted index: an application that builds an index of words and their locations in a collection of documents, which can be used for information retrieval and text mining.
 - PageRank: an application that computes the importance of web pages based on the hyperlink structure of the web graph, which can be used for web search and ranking.
 - K-means clustering: an application that partitions a set of data points into a number of clusters based on their similarity, which can be used for data analysis and machine learning.

Hadoop

- Hadoop is an open-source framework that allows distributed processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of four main components: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN.
- Hadoop Common provides the common utilities and libraries that are used by other Hadoop modules.
- HDFS is a distributed file system that stores data across multiple nodes in a cluster, providing high availability, fault tolerance, and scalability.
- Hadoop MapReduce is a programming model that enables parallel processing of large data sets using a map function and a reduce function.
- Hadoop YARN is a resource management system that allocates and schedules resources for applications running on Hadoop clusters.
- Hadoop can handle structured, semi-structured, and unstructured data, such as text, images, videos, logs, etc.
- Hadoop can run on commodity hardware, reducing the cost and complexity of data processing.
- Hadoop can scale up from a single node to thousands of nodes, depending

on the data size and processing needs.

- Hadoop can be integrated with other frameworks and tools, such as Apache Spark, Apache Hive, Apache Pig, Apache HBase, etc.

Some mnemonics and learning tricks for Hadoop are:

- Hadoop can be remembered as **H**Ave **D**ata, **O**perate **O**n it, **P**arallelize it.
- HDFS can be remembered as **H**ighly **D**istributed **F**ile **S**ystem.
- MapReduce can be remembered as **M**A**P**ping data to key-value pairs and **R**EDUCEing them to aggregate results.
- YARN can be remembered as **Y**et **A**nother **R**esource **N**egotiator.

History of Hadoop

- Hadoop is an open-source software framework for storing and processing big data.
- Hadoop was started with Doug Cutting and Mike Cafarella in the year 2002 when they both started to work on Apache Nutch project.
- Apache Nutch project was the process of building a search engine system that can index 1 billion pages.
- Hadoop was named after an extinct specie of mammoth, a so called Yellow Hadoop, which was the toy of Cutting's son.
- Hadoop was inspired by a white paper written by Google in 2003 that described the Google File System (GFS) and the MapReduce programming model.
- GFS is a distributed file system that can store large amounts of data across multiple machines.
- MapReduce is a programming model that can process large amounts of data in parallel by dividing the work into smaller tasks.
- In 2006, Hadoop became a sub-project of Apache Lucene, a search engine library.
- In 2008, Hadoop defeated the supercomputers and became the fastest system on the planet for sorting terabytes of data.
- In 2009, Hadoop became a top-level Apache project.
- In 2010, Hadoop reached the milestone of sorting 1 petabyte of data in 16.25 hours.
- In 2011, Hadoop reached the milestone of sorting 100 terabytes of data in 72 minutes.
- In 2012, Hadoop 2.0 was released, which introduced YARN, a resource management layer that allows multiple applications to run on Hadoop.
- In 2013, Hadoop 2.2.0 was released, which supported high availability of the NameNode, the master node of HDFS.
- In 2014, Hadoop 2.6.0 was released, which supported rolling upgrades of HDFS clusters.
- In 2015, Hadoop 2.7.0 was released, which supported erasure coding, a technique that can reduce the storage overhead of HDFS.

- In 2016, Hadoop 3.0.0-alpha1 was released, which supported multiple NameNodes, improved security, and better resource utilization.
- In 2017, Hadoop 3.0.0-beta1 was released, which supported Java 8, GPU isolation, and containerized execution.
- In 2018, Hadoop 3.1.0 was released, which supported cloud storage integration, opportunistic containers, and intra-datanode balancing.
- In 2019, Hadoop 3.2.0 was released, which supported native S3A committers, ABFS file system connector, and HDFS RBF router-based federation.
- In 2020, Hadoop 3.3.0 was released, which supported ARM architecture, S3A magic committers, and HDFS disk balancer.
- In 2021, Hadoop 3.3.1 was released, which fixed some bugs and improved performance.

: <https://www.geeksforgeeks.org/hadoop-history-or-evolution/>
<https://www.geeksforgeeks.org/hadoop-an-introduction/>
<https://medium.com/@markobonaci/the-history-of-hadoop-68984a11704>
<https://data-flair.training/blogs/hadoop-history/>
https://en.wikipedia.org/wiki/Apache_Hadoop

Apache Hadoop

- Apache Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation .
- Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models .
- Apache Hadoop is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
- Apache Hadoop consists of four main components: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN .
 - Hadoop Common: The common utilities that support the other Hadoop modules.
 - Hadoop Distributed File System (HDFS): A distributed file system that provides high-throughput access to application data.
 - Hadoop MapReduce: A programming model for large-scale data processing.
 - Hadoop YARN: A framework for job scheduling and cluster resource management.
- Apache Hadoop also has several subprojects that provide additional features and functionality, such as Hadoop ZooKeeper, Hadoop Hive, Hadoop HBase, Hadoop Spark, and Hadoop Oozie .
 - Hadoop ZooKeeper: A service for coordinating processes of distributed applications.
 - Hadoop Hive: A data warehouse system for data summarization,

- query, and analysis.
 - Hadoop HBase: A distributed, scalable, big data store.
 - Hadoop Spark: A fast and general engine for large-scale data processing.
 - Hadoop Oozie: A workflow scheduler system to manage Hadoop jobs.
- Apache Hadoop is widely used for big data analytics, machine learning, data mining, web indexing, and scientific computing.
- Apache Hadoop has several advantages, such as:
 - Scalability: Hadoop can handle petabytes of data and thousands of nodes without losing performance or reliability.
 - Fault-tolerance: Hadoop can automatically recover from failures and continue processing without data loss or corruption.
 - Cost-effectiveness: Hadoop can run on commodity hardware and use open-source software, reducing the cost of ownership and maintenance.
 - Flexibility: Hadoop can process any type of data, structured or unstructured, and support various data sources and formats.
 - Compatibility: Hadoop can integrate with existing systems and tools, and support various programming languages and frameworks.
- Apache Hadoop also has some disadvantages, such as:
 - Complexity: Hadoop requires a steep learning curve and a lot of configuration and tuning to achieve optimal performance and results.
 - Security: Hadoop lacks some security features, such as encryption, authentication, and authorization, and relies on external solutions to provide them.
 - Latency: Hadoop is not suitable for real-time or interactive applications, as it has high latency and overhead for data processing and communication.
- A possible mnemonic to remember the four main components of Hadoop is: **H**ave **F**un **M**aking **Y**arn (HDFS, MapReduce, YARN).
- A possible learning trick to understand the MapReduce programming model is to think of it as a two-step process: map and reduce.
 - Map: The map function takes an input key-value pair and produces a set of intermediate key-value pairs. For example, if the input is a text document, the map function can split it into words and assign each word a value of 1.
 - Reduce: The reduce function takes an intermediate key and a set of values for that key, and merges them into a smaller set of values. For example, if the intermediate key is a word, the reduce function can sum up the values and produce the word count.
 - The map and reduce functions are executed in parallel on different nodes of the cluster, and the results are shuffled and sorted before being returned to the user.

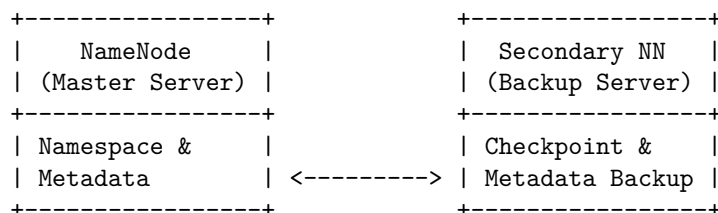
Hadoop Distributed File System

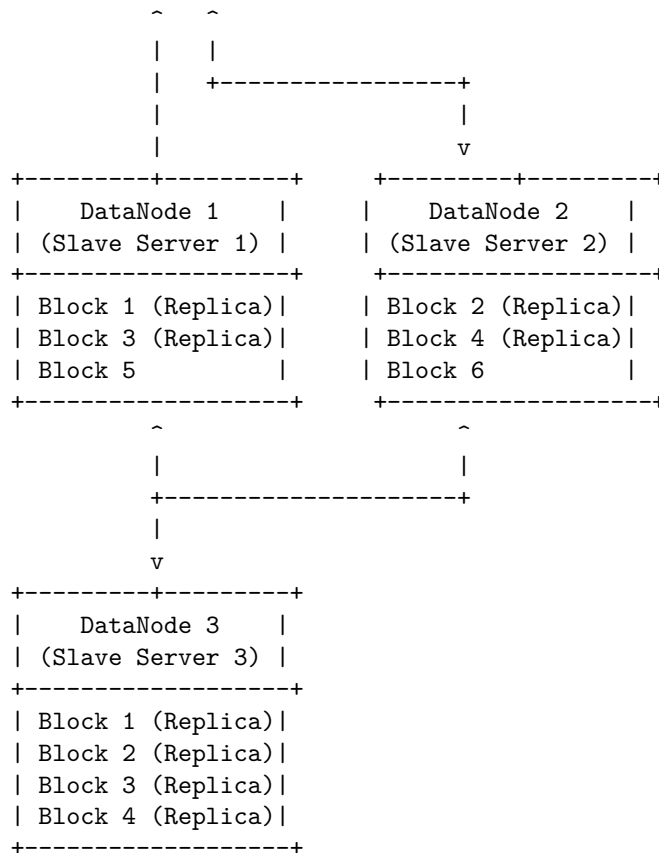
- Hadoop Distributed File System (HDFS) is a file system that provides scalable, reliable, and fault-tolerant storage for large-scale data processing applications.
- HDFS is designed to run on clusters of commodity hardware, and can store petabytes of data across thousands of nodes.
- HDFS follows a master-slave architecture, where a single NameNode manages the namespace and metadata of the file system, and multiple DataNodes store the actual data blocks.
- HDFS exposes a POSIX-like interface for clients to interact with the file system, but does not support random writes or modifications of existing files.
- HDFS splits large files into fixed-size blocks (typically 128 MB or 256 MB), and distributes them across the DataNodes in the cluster. Each block is replicated on multiple DataNodes (default replication factor is 3) for fault tolerance and availability.
- HDFS supports rack-awareness, which means that it tries to place replicas of a block on different racks to reduce the impact of rack failures and network congestion.
- HDFS provides a command-line interface (CLI) and a web-based user interface (UI) for users to perform various operations on the file system, such as creating, deleting, copying, moving, and renaming files and directories, changing permissions and ownership, and checking the status and health of the cluster.
- HDFS also provides a Java API and a REST API for programmatic access to the file system.
- HDFS is compatible with various frameworks and tools for data processing, such as MapReduce, Spark, Hive, Pig, HBase, etc.
- HDFS is suitable for storing and processing large, structured, semi-structured, or unstructured data, such as text, images, videos, logs, sensor data, etc.
- HDFS is not suitable for low-latency or interactive applications, or for storing small files that cause metadata overhead and inefficient disk utilization.

A possible mnemonic to remember the key features of HDFS is:

Highly scalable and reliable **D**istributed and replicated blocks **F**ault-tolerant and rack-aware **S**imple and POSIX-like interface

A possible ascii diagram to illustrate the HDFS architecture is:





Components of Hadoop

Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers. It consists of the following core components:

- Hadoop Distributed File System (HDFS):** This is the storage layer of Hadoop that provides a reliable and scalable file system for storing and accessing data. HDFS splits the data into blocks and distributes them across multiple nodes in the cluster, ensuring fault tolerance and high availability. HDFS also maintains metadata about the data blocks, such as their location, size, and replication factor. HDFS has two main components: NameNode and DataNode. NameNode is the master node that manages the namespace and the metadata of the file system. DataNode is the worker node that stores and serves the data blocks. A mnemonic to remember the components of HDFS is **H**ave **D**ata **F**or **S**ure.
- MapReduce:** This is the processing layer of Hadoop that provides a parallel programming model for writing applications that can process large amounts of data. MapReduce consists of two phases: Map and Reduce.

Map phase takes the input data and transforms it into key-value pairs. Reduce phase takes the output of the Map phase and aggregates the values based on the keys. MapReduce also handles the partitioning, shuffling, and sorting of the data between the phases. MapReduce has two main components: JobTracker and TaskTracker. JobTracker is the master node that coordinates the execution of the MapReduce jobs and assigns tasks to the TaskTrackers. TaskTracker is the worker node that runs the tasks and reports the progress to the JobTracker. A mnemonic to remember the components of MapReduce is **M**ake **A** **P**lan, **R**eview and **E**xecute.

- **Yet Another Resource Negotiator (YARN)**: This is the resource management layer of Hadoop that allocates and manages the resources (such as CPU, memory, disk, and network) for the applications running on the cluster. YARN also provides a platform for running various types of applications on Hadoop, such as batch, interactive, streaming, and machine learning. YARN has two main components: ResourceManager and NodeManager. ResourceManager is the master node that oversees the resource allocation and scheduling of the applications. NodeManager is the worker node that monitors and reports the resource utilization and status of the containers running on the node. A container is a unit of execution that encapsulates the resources and the application logic. A mnemonic to remember the components of YARN is **Y**ou **A**re **R**esourceful, **N**o doubt.

Besides these core components, Hadoop also has a rich ecosystem of tools and applications that help with data ingestion, transformation, analysis, and visualization. Some of these are:

- **Apache Pig**: A scripting language for writing data pipelines on Hadoop.
- **Apache Hive**: A query language for performing SQL-like operations on Hadoop.
- **Apache HBase**: A NoSQL database for storing and accessing sparse and structured data on Hadoop.
- **Apache Spark**: A fast and general-purpose engine for large-scale data processing on Hadoop.
- **Presto**: A distributed query engine for interactive analytics on Hadoop.
- **Apache Zeppelin**: A web-based notebook for data exploration and visualization on Hadoop.

Data Format Co

- Data format is a way of encoding data so that it can be read, recognized, and used by different applications and programs.
- Data format can be classified into three types: text, binary, and multimedia.
- Text data format is a format that uses plain text characters to represent data, such as CSV, XML, JSON, etc. Text data format is human-readable and easy to edit, but it may take more space and time to process.

- Binary data format is a format that uses binary digits (0 and 1) to represent data, such as EXE, ZIP, PDF, etc. Binary data format is machine-readable and efficient, but it may not be compatible with different platforms and programs.
- Multimedia data format is a format that uses a combination of text, binary, and other data types to represent data, such as JPG, MP3, MOV, etc. Multimedia data format is suitable for storing and transmitting audio, video, and image data, but it may require special software and hardware to handle.
- Data format can also be specific to certain domains or applications, such as COCO, GeoTIFF, MXF, etc. These data formats have their own syntax, structure, and metadata to store and annotate data for specific purposes, such as object detection, cartography, and audio/video production .
- Data format can be changed or converted from one type to another using various tools and methods, such as formatting, parsing, encoding, decoding, compression, decompression, etc. These processes can affect the quality, size, and compatibility of the data .
- Data format is an important aspect of data management, as it determines how data can be stored, accessed, manipulated, and communicated. Choosing an appropriate data format can improve the performance, usability, and interoperability of data.

Analyzing data with Hadoop

- Hadoop is an open source software framework and platform for storing, analyzing and processing large volumes of data in a distributed and scalable manner.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and Hadoop MapReduce.
- HDFS is a distributed file system that stores data across multiple nodes in a cluster, providing high availability, fault tolerance and scalability.
- MapReduce is a programming model and an execution engine that enables parallel processing of data on HDFS using two types of functions: map and reduce.
- Map functions take input data and transform it into intermediate key-value pairs. Reduce functions take intermediate key-value pairs and aggregate them to produce output data.
- Hadoop also provides a number of tools and frameworks that can be used to perform various types of data analysis on Hadoop, such as:
 - Hive: a data warehouse system that allows querying and analyzing data using a SQL-like language called HiveQL.
 - Pig: a data flow language and a platform that allows writing complex data transformations and analysis using a high-level scripting language called Pig Latin.
 - Spark: a fast and general-purpose cluster computing system that supports batch, streaming, SQL, machine learning and graph processing.

- HBase: a distributed and column-oriented database that provides random access and strong consistency for large-scale structured and semi-structured data.
- Sqoop: a tool that allows transferring data between Hadoop and relational databases.
- Flume: a tool that allows collecting, aggregating and moving large amounts of log data from various sources to HDFS.
- Mahout: a library that provides scalable machine learning algorithms for clustering, classification, recommendation and dimensionality reduction.
- Oozie: a workflow scheduler that allows managing and coordinating Hadoop jobs.
- To analyze data with Hadoop, one needs to follow these steps:
 - Launch a Hadoop cluster using a service provider such as Amazon EMR or Google Cloud Dataproc, or set up a local Hadoop cluster using tools such as Cloudera or Hortonworks.
 - Load the data into HDFS using tools such as Sqoop, Flume or HDFS commands.
 - Define the schema and create a table for the data using tools such as Hive or Pig, or use the existing HDFS files as input.
 - Write and execute a script or a program that performs the desired analysis using tools such as Hive, Pig, Spark, HBase or MapReduce.
 - Write the results back to HDFS or export them to other systems using tools such as Sqoop or HDFS commands.
 - Download and view the results on the local machine using tools such as Tableau or Excel.
- Some of the advantages of analyzing data with Hadoop are :
 - It can handle large, complex and unstructured data that traditional systems cannot.
 - It can scale up or down according to the data size and processing needs.
 - It can provide fast and reliable results using parallel and distributed computing.
 - It can support a variety of data sources, formats, types and analysis methods.
 - It can reduce the cost and complexity of data management and analysis.
- Some of the disadvantages of analyzing data with Hadoop are :
 - It requires a steep learning curve and specialized skills to use and maintain.
 - It may not be suitable for low-latency or real-time analysis due to the batch-oriented nature of MapReduce.
 - It may not be compatible with some existing tools and systems that rely on SQL or other standards.
 - It may pose security and privacy challenges due to the distributed and open nature of Hadoop.

- Some of the examples of applications that use Hadoop for data analysis are :
 - Facebook: uses Hadoop to store and analyze user data, such as likes, comments, messages, photos and videos, to provide personalized and relevant content and ads.
 - Netflix: uses Hadoop to process and analyze streaming data, such as user preferences, ratings, reviews and viewing history, to provide recommendations and insights.
 - Amazon: uses Hadoop to analyze customer data, such as purchases, searches, clicks and reviews, to optimize product selection, pricing and delivery.
 - LinkedIn: uses Hadoop to analyze network data, such as connections, endorsements, skills and jobs, to provide professional opportunities and insights.
 - Twitter: uses

Scaling out with Hadoop

- Scaling out is the process of adding more nodes to a distributed system to increase its capacity and performance.
- Hadoop is an open-source framework that allows for distributed processing of large datasets across clusters of computers using simple programming models.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.
- HDFS is a distributed file system that provides high-throughput access to data stored on the cluster nodes. It also handles replication and fault tolerance of the data.
- MapReduce is a programming model that enables parallel processing of large datasets by dividing them into smaller chunks (maps) and then combining the results (reduces).
- Hadoop can scale out by adding more nodes to the cluster, which increases the storage and processing capacity of the system.
- Hadoop can also scale out by using different types of nodes for different purposes, such as master nodes, worker nodes, edge nodes, etc.
- Some of the advantages of scaling out with Hadoop are:
 - It can handle very large and complex datasets that cannot fit on a single machine or a traditional database system.
 - It can leverage the existing hardware and network infrastructure without requiring expensive upgrades or specialized equipment.
 - It can provide high availability and fault tolerance by replicating the data across multiple nodes and recovering from node failures.
 - It can offer flexibility and scalability by allowing the users to add or remove nodes as needed and to run different types of applications on the same cluster.
- Some of the disadvantages of scaling out with Hadoop are:

- It can introduce more complexity and overhead in managing and maintaining the cluster, such as configuring, monitoring, securing, and troubleshooting the nodes and the data.
- It can incur more network traffic and latency due to the data transfer and communication between the nodes, which can affect the performance and efficiency of the system.
- It can require more skills and expertise to design, develop, and deploy the applications that can run on the Hadoop framework and take advantage of its features.
- Some of the examples of applications that can benefit from scaling out with Hadoop are:
 - Data analysis and mining: Hadoop can be used to perform various types of data analysis and mining tasks, such as sentiment analysis, recommendation systems, fraud detection, etc.
 - Data warehousing and business intelligence: Hadoop can be used to store and process large amounts of structured and unstructured data from various sources, such as web logs, social media, sensors, etc., and to provide insights and reports for business decision making.
 - Data processing and transformation: Hadoop can be used to process and transform raw data into more useful and meaningful formats, such as cleaning, filtering, aggregating, joining, etc.
- Some of the mnemonics and learning tricks for scaling out with Hadoop are:
 - HDFS: High Demand For Storage
 - MapReduce: Many Are Processing, Results Are Unified and Collected Efficiently
 - Scaling out: Size Can Always Lead to Increased Numbers of Nodes, Greater Output and Throughput

Hadoop streaming

- Hadoop streaming is a utility that comes with the Hadoop distribution. It allows you to create and run MapReduce jobs with any executable or script as the mapper and/or the reducer.
- Hadoop streaming works by passing the input data to the mapper as the standard input, and reading the output data from the standard output. Similarly, the reducer receives the intermediate data from the standard input, and writes the final output to the standard output.
- Hadoop streaming can be used to write MapReduce programs in any language that can read from the standard input and write to the standard output, such as Python, Ruby, Perl, etc. This makes it easier for developers who are not familiar with Java to use Hadoop.
- Hadoop streaming also supports specifying a Java class as the mapper and/or the reducer, which can be useful for some scenarios where the native Hadoop API is needed.
- Hadoop streaming can be invoked by using the `hadoop jar` command

with the `hadoop-streaming.jar` file as the argument. The command also requires specifying the input and output directories, and the mapper and reducer executables or scripts. For example:

```
hadoop jar hadoop-streaming.jar \  
-input myInputDirs \  
-output myOutputDir \  
-mapper /bin/cat \  
-reducer /usr/bin/wc
```

- This command runs a MapReduce job that uses the `cat` command as the mapper and the `wc` command as the reducer. The mapper simply passes the input data to the output, and the reducer counts the number of lines, words, and bytes in the input.
- Hadoop streaming also supports various options to customize the behavior of the MapReduce job, such as specifying the number of reducers, the input and output formats, the partitioner, the combiner, the compression codec, the environment variables, etc. For a full list of options, refer to the Hadoop streaming documentation.
- Hadoop streaming is a powerful feature that enables developers to use Hadoop with any programming language of their choice. However, it also has some limitations and drawbacks, such as:
 - The performance of Hadoop streaming may be lower than the native Java MapReduce, due to the overhead of launching external processes and serializing and deserializing data between Java and the external language.
 - The error handling and debugging of Hadoop streaming may be more difficult, as the errors and exceptions from the external processes are not captured by the Hadoop framework.
 - The external processes may not have access to some of the Hadoop features and libraries, such as the distributed cache, the counters, the configuration, etc.
 - The external processes may not be able to leverage some of the optimizations and enhancements of the Hadoop framework, such as the speculative execution, the shuffle and sort, the security, etc.
- Therefore, Hadoop streaming should be used with caution and consideration, and only when the benefits of using a different programming language outweigh the potential costs and risks.

Hadoop pipes

- Hadoop pipes is the name of the C++ interface to Hadoop MapReduce .
- Unlike Streaming, which uses standard input and output to communicate with the map and reduce code, Pipes uses sockets as the channel over which the tasktracker communicates with the process running the C++ map or reduce function . JNI is not used.
- Hadoop pipes provides a SWIG-compatible C++ API to implement

MapReduce applications (non JNI™ based).

- The MapReduce framework operates exclusively on <key, value> pairs, that is, the framework views the input to the job as a set of <key, value> pairs and produces a set of <key, value> pairs as the output of the job, conceivably of different types.
- To run Hadoop pipes, the C++ code needs to be compiled and linked with the pipes libraries . The pipes libraries are provided by the Hadoop distribution.
- A Hadoop pipes application consists of a driver class, a mapper class, and a reducer class. The driver class is responsible for configuring the job and submitting it to the cluster. The mapper class implements the map function, which takes a <key, value> pair as input and emits zero or more <key, value> pairs as output. The reducer class implements the reduce function, which takes a <key, list of values> pair as input and emits zero or more <key, value> pairs as output.
- Hadoop pipes provides a set of abstract classes and interfaces that the C++ code needs to inherit and implement. These include:
 - HadoopPipes::Mapper: The abstract base class for the mapper. It has a virtual method map, which takes a HadoopPipes::MapContext object as a parameter. The MapContext object provides methods to access the input key and value, emit output key and value pairs, and report progress and status.
 - HadoopPipes::Reducer: The abstract base class for the reducer. It has a virtual method reduce, which takes a HadoopPipes::ReduceContext object as a parameter. The ReduceContext object provides methods to access the input key and list of values, emit output key and value pairs, and report progress and status.
 - HadoopPipes::Factory: The interface for creating mapper and reducer objects. It has two pure virtual methods, createMapper and createReducer, which take a HadoopPipes::TaskContext object as a parameter and return a pointer to a HadoopPipes::Mapper or HadoopPipes::Reducer object respectively.
 - HadoopPipes::TaskContext: The abstract base class for the context objects. It provides methods to access the configuration, the counters, and the task type.
 - HadoopPipes::MapContext and HadoopPipes::ReduceContext: The subclasses of HadoopPipes::TaskContext that provide additional methods specific to the map and reduce tasks.
- A simple example of a Hadoop pipes application that performs word count is shown below:

```
// WordCount.cc
#include "hadoop/Pipes.hh"
#include "hadoop/TemplateFactory.hh"
#include "hadoop/StringUtils.hh"
```

```

class WordCountMapper : public HadoopPipes::Mapper {
public:
    // constructor: does nothing
    WordCountMapper(HadoopPipes::TaskContext& context){}
    // map function: receives a line, outputs (word,"1")
    // to reducer
    void map(HadoopPipes::MapContext& context) {
        //--- get line of text ---
        std::string line = context.getInputValue();
        //--- split it into words ---
        std::vector< std::string > words =
            HadoopUtils::splitString( line, " " );
        //--- emit each word tuple (word, "1" ) ---
        for(unsigned int i=0; i < words.size(); ++i) {
            context.emit(words[i], "1");
        }
    }
};

class WordCountReducer : public HadoopPipes::Reducer {
public:
    // constructor: does nothing
    WordCountReducer(HadoopPipes::TaskContext& context){}
    // reduce function
    void reduce(HadoopPipes::ReduceContext& context) {
        int count = 0
    }
};

```

Hadoop Ecosystem

- The Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop.
- Apache Hadoop is a software library that allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and storage .
- Hadoop enables multiple types of analytic workloads to run on the same data, at the same time, at massive scale on industry-standard hardware.
- The Hadoop Ecosystem consists of the following components:
 - **HDFS:** Hadoop Distributed File System. It is a distributed file system that stores data across multiple nodes in a cluster. It provides high availability, fault tolerance, and scalability.

- **YARN**: Yet Another Resource Negotiator. It is a resource management layer that allocates and schedules resources (such as CPU, memory, disk, network) to different applications running on Hadoop.
- **MapReduce**: Programming based Data Processing. It is a programming model that allows you to process large amounts of data in parallel by dividing the work into two phases: map and reduce. The map phase applies a function to each input record and produces intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same key and produces the final output.
- **Spark**: In-Memory data processing. It is an alternative to MapReduce that provides faster and more flexible data processing by keeping the data in memory. It supports various languages (such as Scala, Python, Java, R) and libraries (such as SQL, MLlib, GraphX, Streaming) for different types of analytics.
- **PIG, HIVE**: Query based processing of data services. They are high-level languages that allow you to write queries to analyze data stored in HDFS. PIG is a scripting language that supports a data flow model. HIVE is a SQL-like language that supports a relational model.
- **HBase**: NoSQL Database. It is a distributed, column-oriented database that provides random access and consistent updates to large amounts of structured and semi-structured data. It is based on the Google Bigtable model.
- **Sqoop, Flume, Kafka**: Data Ingestion. They are tools that help you import and export data from and to various sources and destinations. Sqoop is used to transfer data between HDFS and relational databases. Flume is used to collect and stream log data from various sources to HDFS. Kafka is used to publish and subscribe to streams of data in real-time.
- **Oozie, Zookeeper**: Coordination and Workflow. They are tools that help you manage and orchestrate the execution of various tasks and applications on Hadoop. Oozie is a workflow scheduler that allows you to define a sequence of actions and dependencies. Zookeeper is a service that provides coordination, synchronization, and configuration management for distributed systems.
- **Mahout, MLlib**: Machine Learning. They are libraries that provide various algorithms and tools for machine learning and data mining on Hadoop. Mahout is based on MapReduce, while MLlib is based on Spark.
- **Hue, Ambari, Cloudera Manager**: User Interface and Monitoring. They are web-based applications that provide graphical user interfaces and dashboards for managing and monitoring Hadoop clusters and applications. Hue is an open source project that integrates with various Hadoop components. Ambari and Cloudera Manager are products from Apache and Cloudera respectively.

- A possible mnemonic to remember the Hadoop Ecosystem components is:
 - **H**ave **Y**ou **M**et **S**ome **P**eople **H**aving **S**ome **F**un **K**icking **O**ld **Z**ebra
Making **H**uge **A**mounts of **C**ash?
 - **H**DFS, **Y**ARN, **M**apReduce, **S**park, **P**IG, **H**IVE, **S**qoop, **F**lume,
Kafka, **O**ozie, **Z**ookeeper, **M**ahout, **H**Base, **A**mbari, **C**

Map Reduce

- MapReduce is a framework for processing parallelizable problems across large datasets using a large number of computers (nodes), collectively referred to as a cluster or a grid.
- MapReduce is also the name of a programming model or pattern that enables massive scalability across hundreds or thousands of servers in a Hadoop cluster.
- The term “MapReduce” refers to two separate and distinct tasks that Hadoop programs perform:
 - The first is the map job, which takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs).
 - The second is the reduce job, which takes the output from the map job and combines those tuples into a smaller set of tuples.
- The map and reduce functions are written by the user and can be customized to perform different types of processing on the data.
- The MapReduce framework handles the distribution of data and tasks across the nodes, the coordination of the execution, the handling of failures and errors, and the aggregation of the results.
- MapReduce is suitable for processing large volumes of unstructured or semi-structured data, such as text, images, audio, video, log files, etc.
- MapReduce can be used for various applications, such as word count, web indexing, data mining, machine learning, sentiment analysis, etc.

Advantages of MapReduce

- MapReduce is scalable, as it can run on thousands of nodes and handle petabytes of data.
- MapReduce is fault-tolerant, as it can recover from node failures and data loss by replicating the data and tasks across the cluster.
- MapReduce is simple, as it abstracts the complexity of distributed computing from the user and provides a high-level interface for writing the map and reduce functions.
- MapReduce is flexible, as it can process any type of data, regardless of its format, structure, or schema.
- MapReduce is efficient, as it minimizes the network traffic by moving the computation to the data, rather than the other way around.

Disadvantages of MapReduce

- MapReduce is not suitable for interactive or real-time queries, as it has a high latency due to the batch processing nature.
- MapReduce is not optimal for complex or iterative algorithms, as it requires multiple map and reduce jobs to be chained together, which increases the overhead and reduces the performance.
- MapReduce is not compatible with traditional relational databases or SQL, as it does not support joins, aggregations, or transactions.
- MapReduce is not easy to debug or test, as it involves distributed and parallel execution, which makes it hard to trace the errors and monitor the progress.

Example of MapReduce

- Suppose we want to count the number of occurrences of each word in a large text file using MapReduce.
- The input data is the text file, which is split into smaller chunks and distributed across the nodes in the cluster.
- The map function takes each chunk of text and emits a key/value pair for each word, where the key is the word and the value is 1.
- The output of the map function is shuffled and sorted by the key, and then sent to the reduce function.
- The reduce function takes all the key/value pairs for a given word and sums up the values, resulting in a final count for that word.
- The output of the reduce function is the final word count for the entire text file.

ASCII Diagram of MapReduce

Input Data

```
+-----+
| This is a text |
| file that has  |
| some words in  |
| it.            |
+-----+
```

Map Function

```
+-----+ +-----+
| This is a text | | (This, 1) |
| file that has  | -> | (is, 1)  |
| some words in  | | (a, 1)   |
| it.            | | (text, 1) |
+-----+ | (file, 1) |
| (that, 1) |
| (has, 1)  |
| (some, 1) |
```

Map Reduce framework and basics

- Map Reduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.
- The model is inspired by the map and reduce functions commonly used in functional programming, although their purpose in the Map Reduce framework is not the same as in their original forms.
- The key idea is to split the input data set into independent chunks that are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps, which are then input to the reduce tasks. Typically both the input and the output of the job are stored in a distributed file system. The framework takes care of scheduling tasks, monitoring them and re-executes the failed tasks.
- The Map Reduce framework consists of a single master JobTracker and one slave TaskTracker per cluster-node. The master is responsible for scheduling the jobs' component tasks on the slaves, monitoring them and re-executing the failed tasks. The slaves execute the tasks as directed by the master.
- The Map Reduce framework operates exclusively on <key, value> pairs, that is, the framework views the input to the job as a set of <key, value> pairs and produces a set of <key, value> pairs as the output of the job, conceivably of different types.
- The Map Reduce framework consists of two types of tasks: map and reduce. The map task takes a set of <key, value> pairs as input and produces a set of intermediate <key, value> pairs as output. The reduce task takes the intermediate <key, value> pairs for a given key from all the map tasks and merges them to produce the final output. The user can specify a combiner function that can perform a local aggregation of the intermediate results, which can reduce the amount of data transferred between the map and reduce tasks.
- The Map Reduce framework provides a simple and powerful abstraction for parallel and distributed computing, which enables users to write scalable and fault-tolerant applications without worrying about the low-level details of the system. The framework handles the data distribution, load balancing, fault tolerance, and inter-node communication for the user.
- The Map Reduce framework can be used for a variety of applications, such as word count, inverted index, web link analysis, machine learning, data mining, and scientific computing. The framework can also be extended and customized for specific needs, such as adding new data sources, output formats, or scheduling policies.
- A possible mnemonic to remember the Map Reduce framework is: **Map Reduce Master Slave Key Value Combiner**.

How MapReduce works

MapReduce is a programming model and an associated implementation for processing and generating big data sets with a parallel, distributed algorithm on a cluster. It can perform distributed and parallel computations using large datasets across a large number of nodes.

The basic idea of MapReduce is to split the input data into smaller chunks and process them independently by applying a map function to each chunk, and then combine the results of the map functions by applying a reduce function to produce the final output.

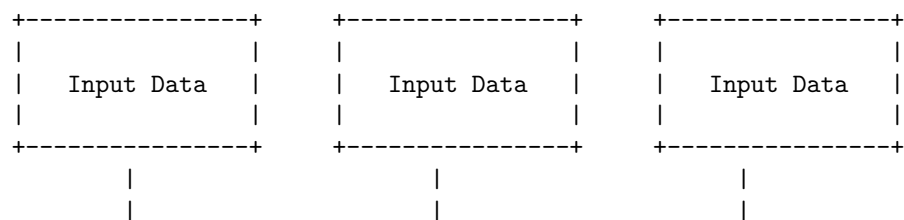
The MapReduce algorithm contains two important tasks, namely Map and Reduce.

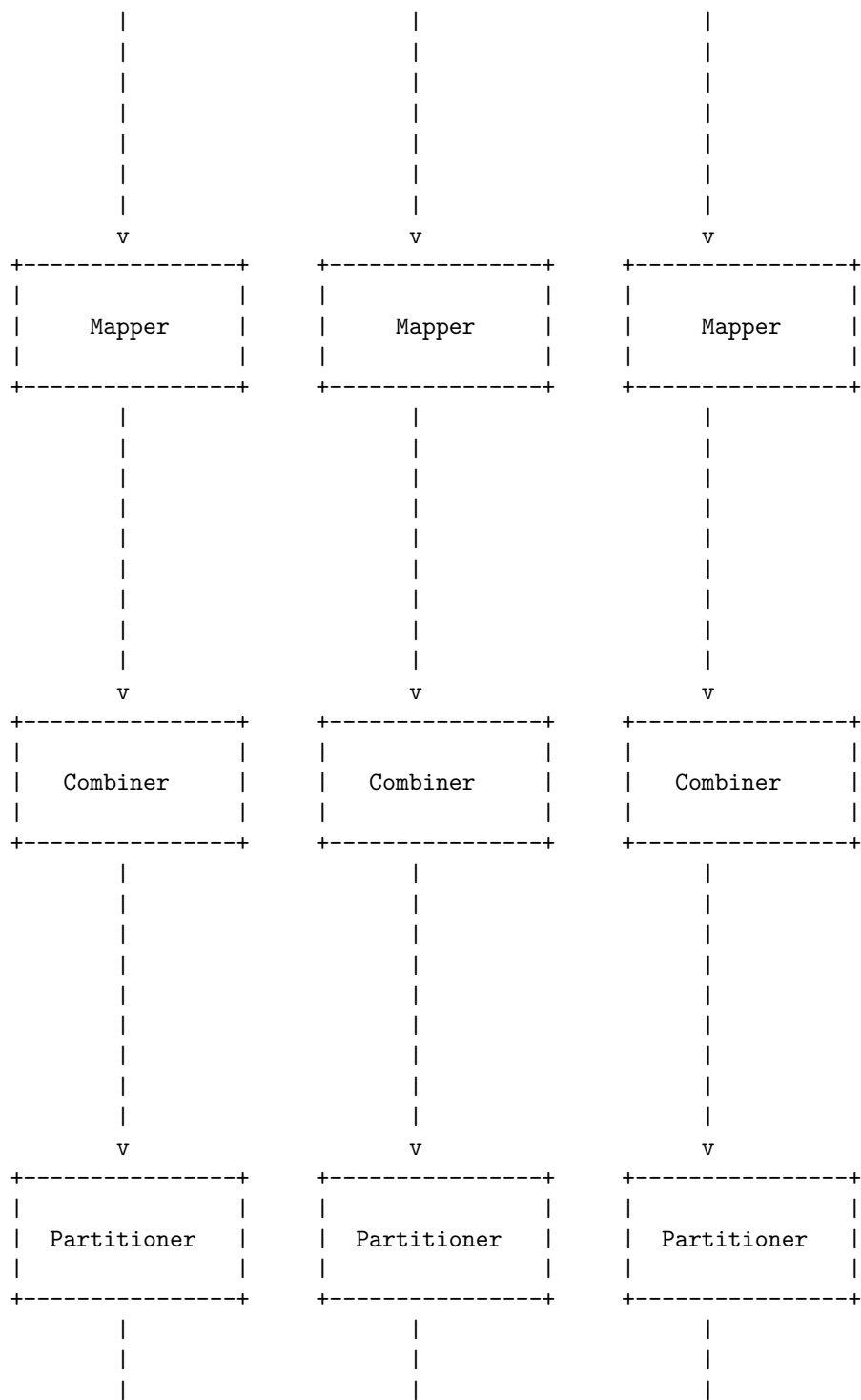
- **Map:** The map function takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key-value pairs). The map function is applied to each input split in parallel by different worker nodes in the cluster. The output of the map function is stored in a temporary storage .
- **Reduce:** The reduce function takes the output of the map function and merges the tuples with the same key. The reduce function is applied to each group of tuples with the same key in parallel by different worker nodes in the cluster. The output of the reduce function is the final result of the MapReduce job .

In addition to the map and reduce functions, there are two optional tasks that can improve the performance and scalability of the MapReduce algorithm.

- **Combine:** The combine function is a mini-reduce function that runs on each mapper node after the map function. It can reduce the amount of data that needs to be shuffled and sorted between the mapper and reducer nodes by aggregating the tuples with the same key locally. The combine function has the same signature as the reduce function, but it operates on a subset of the data.
- **Partition:** The partition function determines how the output of the map function is distributed among the reducer nodes. It takes a key and the number of reducers as input and returns the index of the reducer that should receive the key-value pair. The default partition function is a hash function, but it can be customized to implement a different logic.

The following diagram illustrates the workflow of the MapReduce algorithm:





Developing a Map Reduce application

- Map Reduce is a programming model for processing large-scale data sets in parallel and distributed environments.
- A Map Reduce application consists of two main functions: a map function and a reduce function.
- The map function takes an input key-value pair and produces a set of intermediate key-value pairs. The intermediate keys are grouped by a partitioner and sent to different reducers.
- The reduce function takes an intermediate key and a list of values associated with that key, and produces a set of output key-value pairs. The output keys are sorted by a comparator and written to the final output file.
- A Map Reduce application also requires a driver class that specifies the input and output formats, the mapper and reducer classes, and other configuration parameters.
- A typical workflow of a Map Reduce application is as follows:
 1. The input data is split into fixed-size blocks and stored in a distributed file system (such as HDFS).
 2. The driver class submits the Map Reduce job to a cluster manager (such as YARN or Mesos), which assigns tasks to available nodes in the cluster.
 3. Each map task reads a block of input data and applies the map function to each key-value pair in the block. The map function emits intermediate key-value pairs to a local buffer.
 4. The local buffer periodically spills the intermediate key-value pairs to local disk, partitioned by a hash function based on the intermediate keys.
 5. The cluster manager shuffles the intermediate key-value pairs from the local disks of the map nodes to the local disks of the reduce nodes, based on the partitioner function.
 6. Each reduce task merges and sorts the intermediate key-value pairs from the local disk, and applies the reduce function to each group of values with the same intermediate key. The reduce function emits output key-value pairs to a local buffer.
 7. The local buffer periodically writes the output key-value pairs to the distributed file system, sorted by a comparator function based on the

output keys.

- An example of a Map Reduce application is the word count program, which counts the frequency of each word in a large text file. The map function takes a line of text as the input key and emits each word in the line as the intermediate key and 1 as the intermediate value. The reduce function takes a word as the intermediate key and a list of 1s as the intermediate values, and sums up the values to get the final count of the word as the output value. The output key is the same as the intermediate key.
- A possible implementation of the word count program in Java is as follows:

```
// The driver class
public class WordCount {

    public static void main(String[] args) throws Exception {
        // Create a configuration object
        Configuration conf = new Configuration();
        // Create a job object
        Job job = Job.getInstance(conf, "word count");
        // Set the driver class
        job.setJarByClass(WordCount.class);
        // Set the mapper class
        job.setMapperClass(TokenizerMapper.class);
        // Set the reducer class
        job.setReducerClass(IntSumReducer.class);
        // Set the output key class
        job.setOutputKeyClass(Text.class);
        // Set the output value class
        job.setOutputValueClass(IntWritable.class);
        // Set the input and output paths
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        // Wait for the job to complete
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

// The mapper class
public class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>{

    // A constant value of 1
    private final static IntWritable one = new IntWritable(1);
    // A reusable text object
    private Text word = new Text();

    public void map(Object key, Text value, Context context) throws IOException, InterruptedException{
```



```

        // Tokenize the input line
        StringTokenizer itr = new StringTokenizer(value.toString());
        // For each word in the line
        while (itr.hasMoreTokens()) {
            // Set the word as the intermediate key
            word.set(itr.nextToken());
            // Emit the word and 1 as the intermediate key-value pair
            context.write(word, one);
        }
    }
}

// The reducer class
public class IntSumReducer extends Reducer<Text,IntWritable,Text,IntWritable> {
    // A reusable int object
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException {
        // Initialize the sum to 0
        int sum =

```

Unit tests with MR unit

- MR unit is a Java library that helps developers unit test Hadoop MapReduce jobs .
- MR unit allows you to do test-driven development (TDD) and write lightweight unit tests that accommodate Hadoop's specific architecture and constructs.
- MR unit supports testing mappers and reducers separately as well as testing MapReduce computations as a whole .
- With MR unit, you can craft test input, push it through your mapper and/or reducer, and verify its output all in a JUnit test.
- As do other JUnit tests, this allows you to debug your code using the JUnit test as a driver.
- A map/reduce pair can be tested using MR unit's MapReduceDriver.
- MR unit also supports testing combiners, counters, partitioners, and custom writable classes .
- MR unit can be used with other testing frameworks such as Mockito, PowerMock, and EasyMock.
- MR unit can be integrated with Maven or Gradle for dependency management and build automation .

Example

- Suppose we are processing road surface data used to create maps.
- The input contains both linear surfaces and intersections.

- The mapper extracts the surface type and the length of each segment.
- The reducer sums up the lengths of each surface type and outputs the total length per type.
- The following code snippet shows how to write a unit test for this map/reduce pair using MR unit.

```
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.MapReduceDriver;
import org.junit.Before;
import org.junit.Test;

public class RoadSurfaceTest {

    MapReduceDriver<Object, Text, Text, IntWritable, Text, IntWritable> mapReduceDriver;

    @Before
    public void setUp() {
        RoadSurfaceMapper mapper = new RoadSurfaceMapper();
        RoadSurfaceReducer reducer = new RoadSurfaceReducer();
        mapReduceDriver = MapReduceDriver.newMapReduceDriver(mapper, reducer);
    }

    @Test
    public void testMapReduce() {
        mapReduceDriver.withInput(new Text("road1"), new Text("asphalt,100"));
        mapReduceDriver.withInput(new Text("road2"), new Text("concrete,200"));
        mapReduceDriver.withInput(new Text("road3"), new Text("asphalt,150"));
        mapReduceDriver.withInput(new Text("road4"), new Text("gravel,50"));
        mapReduceDriver.withOutput(new Text("asphalt"), new IntWritable(250));
        mapReduceDriver.withOutput(new Text("concrete"), new IntWritable(200));
        mapReduceDriver.withOutput(new Text("gravel"), new IntWritable(50));
        mapReduceDriver.runTest();
    }
}
```

Advantages

- MR unit makes it easy to develop and maintain Hadoop MapReduce code bases.
- MR unit allows you to test your code without setting up a Hadoop cluster or writing files to HDFS .
- MR unit provides a simple and intuitive API for writing test cases .
- MR unit can help you catch bugs and errors early in the development cycle .
- MR unit can improve the quality and reliability of your MapReduce pro-

grams .

Disadvantages

- MR unit does not test the performance or scalability of your MapReduce programs .
- MR unit does not test the integration of your MapReduce programs with other components such as input/output formats, compression codecs, or distributed cache .
- MR unit does not test the behavior of your MapReduce programs under different cluster configurations or failure scenarios .
- MR unit may not cover all the features or edge cases of your MapReduce programs .

Mnemonics and learning tricks

- MR unit stands for MapReduce unit testing

Test data and local tests in map reduce

- Test data is a set of input data that is used to verify the correctness and performance of a map reduce program.
- Local tests are tests that are run on a single machine or a small cluster, without using the full-scale distributed environment of map reduce.
- Test data and local tests are important for map reduce development, because they can help to:
 - Debug the logic and syntax of the map and reduce functions.
 - Check the output format and content of the map reduce program.
 - Identify and fix any errors or exceptions that may occur during the execution of the map reduce program.
 - Optimize the performance and efficiency of the map reduce program by tuning the parameters and settings.
 - Validate the scalability and reliability of the map reduce program by simulating different scenarios and workloads.
- Some of the best practices for test data and local tests in map reduce are:
 - Use a small but representative sample of the real data as test data, to ensure that the test results are consistent and meaningful.
 - Use a variety of test data, including normal, abnormal, and edge cases, to cover all the possible situations and outcomes.
 - Use a local file system or a pseudo-distributed mode for local tests, to avoid the overhead and complexity of the distributed file system and the map reduce framework.
 - Use a testing framework or a tool, such as JUnit, MRUnit, or Hadoop Unit, to automate and simplify the testing process and the verification of the test results.

- Use logging, debugging, and profiling tools, such as Log4j, Eclipse, or Hadoop Vaidya, to monitor and analyze the behavior and performance of the map reduce program during the local tests.
- Use a mnemonic or a learning trick to remember the steps and the benefits of test data and local tests in map reduce, such as:
 - * **Test data:** small, representative, and varied
 - * **Local tests:** local, pseudo-distributed, and automated
 - * **Benefits:** debug, check, fix, optimize, and validate

Anatomy of a MapReduce job run

- A MapReduce job is a unit of work that consists of a map phase and a reduce phase, which operate on a large dataset in parallel.
- A MapReduce job can be executed by calling the `submit()` or `waitForCompletion()` methods on a `Job` object, which represents the configuration and status of the job.
- The `Job` object communicates with the `JobTracker`, which is a master service that coordinates the execution of the job across the cluster.
- The `JobTracker` assigns the job a unique ID and splits the input data into fixed-size pieces called **input splits**, each of which is processed by a single map task.
- The `JobTracker` also determines the number of reduce tasks to create, based on the `mapred.reduce.tasks` property in the `JobConf`, which can be set by the `setNumReduceTasks()` method.
- The `JobTracker` then schedules the map and reduce tasks to run on the available `TaskTrackers`, which are slave services that run on each node in the cluster and execute the tasks assigned by the `JobTracker`.
- The `TaskTrackers` launch separate JVM processes for each map and reduce task, and report their progress and status to the `JobTracker`.
- The map tasks read the input data from the **Hadoop Distributed File System (HDFS)** or other sources, and apply the user-defined map function to each key-value pair in the input split.
- The map function emits intermediate key-value pairs, which are partitioned, sorted, and written to the local disk of the `TaskTracker`.
- The reduce tasks fetch the intermediate data from the map tasks, either directly or through an intermediate service called `ShuffleHandler`, and merge and sort them by key.
- The reduce tasks then apply the user-defined reduce function to each group of values that share the same key, and emit the final output key-value pairs, which are written to the **HDFS** or other destinations.
- The `JobTracker` monitors the progress and status of the tasks, and handles failures and retries if necessary.
- The `JobTracker` also provides a web interface for the user to view the details and statistics of the job, such as the number of tasks, the input and output sizes, the execution time, and the counters.
- The `Job` object returns the final status of the job to the user, and cleans

up any temporary files and resources used by the job.

Failures in MapReduce

MapReduce is a programming model and framework for processing large-scale data sets in parallel using clusters of commodity machines. However, MapReduce applications may encounter various types of failures during their execution, which can affect their performance and correctness. Some of the common failures in MapReduce are:

- **Task failure:** This occurs when a map or reduce task fails due to an error in the user code, such as a runtime exception, a bad record, or a bug. The task may also fail due to a hardware failure, such as a disk or memory error, on the machine where it is running. When a task fails, the MapReduce framework reports the error to the application master, which is responsible for managing the job execution. The application master then schedules a new attempt of the failed task on a different machine. The failed task attempt does not affect the progress of other tasks in the same job. However, if a task fails too many times, the job may be aborted. A common technique to handle task failures is to use a custom error handler that can skip or ignore bad records or exceptions in the user code.
- **Node failure:** This occurs when a node in the cluster becomes unavailable due to a network partition, a power outage, or a hardware failure. A node failure can affect both the map and reduce tasks that are running on that node, as well as the intermediate data that is stored on the local disk of that node. When a node failure is detected, the MapReduce framework marks the node as dead and reschedules the tasks that were running on that node on other nodes. The intermediate data that was stored on the failed node is also lost and needs to be regenerated by rerunning the map tasks that produced it. This can cause a significant delay in the job completion, especially if the failed node was storing a large amount of intermediate data. A common technique to handle node failures is to use replication or erasure coding to store the intermediate data on multiple nodes, so that the data can be recovered from other nodes in case of a node failure.
- **Job failure:** This occurs when the entire MapReduce job fails to complete due to a fatal error in the application master, the MapReduce framework, or the cluster. A job failure can be caused by a bug in the MapReduce code, a configuration error, a resource exhaustion, or a cluster-wide failure. When a job failure occurs, the MapReduce framework aborts the job and reports the error to the user. The user may need to debug the error and resubmit the job with a different configuration or a different version of the MapReduce code. A common technique to handle job failures is to use checkpoints or snapshots to save the state of the job periodically, so that the job can be resumed from the last checkpoint or snapshot in case of a

job failure.

Some possible mnemonics and learning tricks for the failures in MapReduce are:

- Task failure: **T**ry again or **T**olerate errors
- Node failure: **N**eed replication or **N**eed recovery
- Job failure: **J**ust abort or **J**ump to checkpoint

Job scheduling in MapReduce

- Job scheduling is the process of assigning tasks to workers in a distributed system, such as MapReduce, to optimize the performance and resource utilization of the system.
- MapReduce is a programming model and framework for processing large-scale data sets in parallel using multiple machines, or nodes, in a cluster.
- A MapReduce job consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase applies another user-defined function to all the intermediate values associated with the same intermediate key and produces a set of output key-value pairs.
- Job scheduling in MapReduce involves deciding how to partition the input data, how to assign map and reduce tasks to nodes, how to balance the load among nodes, and how to handle failures and stragglers.
- There are different job scheduling algorithms and policies for MapReduce, such as FIFO, Fair, Capacity, and Delay. Each one has its own advantages and disadvantages, depending on the characteristics of the jobs and the cluster.
- FIFO (First-In-First-Out) is the simplest job scheduling policy, where jobs are executed in the order they are submitted to the system. FIFO is easy to implement and understand, but it can cause starvation and poor resource utilization if there are long-running jobs or jobs with different resource requirements.
- Fair scheduling is a job scheduling policy that aims to allocate resources fairly among jobs, such that each job gets an equal share of the cluster over time. Fair scheduling can improve the response time and throughput of the system, but it can also introduce overhead and complexity in managing the resources and the queues.
- Capacity scheduling is a job scheduling policy that divides the cluster into multiple queues, each with a predefined capacity and priority. Jobs are submitted to different queues based on their attributes, such as user, group, or deadline. Capacity scheduling can improve the resource isolation and utilization of the system, but it can also cause fragmentation and underutilization if the queues are not configured properly.
- Delay scheduling is a job scheduling technique that improves the data locality of the system, which is the degree to which tasks access data that is stored on or near the nodes where they are executed. Data locality can reduce the network traffic and improve the performance of the system.

Delay scheduling allows a job to wait for a short period of time before assigning a task to a node, in order to increase the chance of finding a node that has the data for the task. Delay scheduling can improve the data locality and the throughput of the system, but it can also increase the waiting time and the complexity of the scheduler.

Shuffle and sort in map reduce

- Shuffle and sort is the process of transferring the intermediate key-value pairs from the mappers to the reducers in a map reduce framework.
- Shuffle and sort ensures that all the values associated with the same key are sent to the same reducer, and that the keys are sorted in ascending order.
- Shuffle and sort consists of the following steps:
 1. Partitioning: The mapper divides the output key-value pairs into partitions, one for each reducer. The partitioning function is usually a hash function of the key, but it can be customized by the user.
 2. Sorting: The mapper sorts the key-value pairs within each partition by the key. This is done to facilitate merging at the reducer side. The sorting can be done in memory or on disk, depending on the size of the data and the available resources.
 3. Merging: The mapper merges the sorted partitions into a single sorted file and sends it to the reducer. The merging can be done in parallel with the sorting, or after the sorting is finished.
 4. Copying: The reducer copies the sorted files from the mappers over the network. The copying can be done in parallel with the merging, or after the merging is finished.
 5. Grouping: The reducer groups the key-value pairs by the key, and passes each group to the reduce function. The grouping can be done in memory or on disk, depending on the size of the data and the available resources.
- Shuffle and sort is a crucial and expensive phase in map reduce, as it involves a lot of network and disk I/O, and can affect the performance and scalability of the system.
- Some of the challenges and optimizations in shuffle and sort are:
 - Balancing the load among the reducers, by choosing a suitable partitioning function and number of reducers.
 - Minimizing the network traffic, by compressing the data, using combiners to aggregate the values at the mapper side, and using speculative execution to handle slow or failed nodes.
 - Reducing the disk I/O, by using in-memory sorting and merging, and spilling the data to disk only when necessary.

- Improving the sorting efficiency, by using external sorting algorithms, such as merge sort or quick sort, and exploiting the locality and similarity of the keys.
- Enhancing the grouping performance, by using secondary sorting to sort the values within each key group, and using iterators to avoid loading the entire group into memory.

Task execution in map reduce

- Map reduce is a programming model for processing large-scale data sets in parallel and distributed environments.
- Map reduce consists of two phases: map and reduce, each of which can be executed by multiple tasks on different nodes of a cluster.
- A map task takes an input key-value pair and produces a set of intermediate key-value pairs as output. The map function defines how the input is transformed into the intermediate output.
- A reduce task takes a set of intermediate key-value pairs that share the same key and produces a final output value for that key. The reduce function defines how the intermediate values are combined into the final output.
- The map reduce framework handles the partitioning, shuffling, sorting, and grouping of the intermediate key-value pairs, as well as the fault tolerance, load balancing, and scalability of the tasks.
- The map reduce framework can be implemented on various platforms, such as Hadoop, Spark, Google Cloud Dataflow, etc.

Some points to remember about task execution in map reduce are:

- The number of map tasks is determined by the number of input splits, which are chunks of the input data that can be processed independently. The input splits are usually based on the size of the input files or blocks.
- The number of reduce tasks is determined by the user or the framework, depending on the desired level of parallelism and the available resources. The reduce tasks are assigned a range of keys to process, which are determined by a partition function that maps each intermediate key to a reduce task ID.
- The map tasks and the reduce tasks are executed by workers, which are processes or threads that run on the nodes of the cluster. The workers communicate with a master, which is a process or thread that coordinates the execution of the tasks and monitors their progress and status.
- The map tasks write their intermediate output to local disks, which are then transferred to the reduce tasks via the network. The reduce tasks read the intermediate input from the local disks or the network, sort and merge them by key, and apply the reduce function to produce the final output.
- The map reduce framework provides fault tolerance by detecting and re-executing failed or slow tasks, as well as replicating the input and output

data across multiple nodes. The framework also provides load balancing by dynamically assigning tasks to workers based on their availability and performance. The framework also provides scalability by allowing the addition or removal of nodes without affecting the execution of the tasks.

Some examples of applications that can be implemented using map reduce are:

- Word count: count the frequency of each word in a large collection of documents.
- Inverted index: build an index of words and the documents that contain them for a search engine.
- Page rank: compute the importance of each web page based on the links between them.
- K-means clustering: group a large set of points into k clusters based on their distance.
- Matrix multiplication: multiply two large matrices in parallel.

Map Reduce types in map reduce

Map Reduce is a programming model for processing large-scale data sets in parallel and distributed environments. It consists of two main phases: map and reduce. In the map phase, a user-defined function is applied to each input record (key-value pair) and produces a set of intermediate key-value pairs. In the reduce phase, another user-defined function is applied to all the intermediate values associated with the same key and produces a set of output key-value pairs.

There are different types of map reduce operations, depending on the nature of the input, output, and intermediate data. Some of the common types are:

- Identity map reduce: This is the simplest type of map reduce, where the map function does not change the input key-value pairs and the reduce function simply concatenates the values for each key. This type of map reduce can be used to copy or partition data across different machines or files.
- Filtering map reduce: This type of map reduce filters out some of the input records based on a condition. The map function checks the condition and either emits the input key-value pair or discards it. The reduce function is the same as the identity reduce function. This type of map reduce can be used to remove unwanted or invalid data from a data set.
- Aggregation map reduce: This type of map reduce performs some aggregation operation on the values for each key, such as sum, count, average, min, max, etc. The map function emits the input key-value pair or transforms it into a different key-value pair. The reduce function applies the aggregation function to the values for each key and emits a single key-value pair. This type of map reduce can be used to compute statistics or summaries from a data set.

- **Join map reduce:** This type of map reduce performs a join operation on two or more data sets based on a common key. The map function emits key-value pairs where the key is the join key and the value is the record from the input data set. The reduce function merges the records from different data sets that have the same key and emits a single key-value pair. This type of map reduce can be used to combine related data from different sources.
- **Group by map reduce:** This type of map reduce performs a group by operation on a data set based on one or more attributes. The map function emits key-value pairs where the key is the group by attribute(s) and the value is the record from the input data set. The reduce function collects the records from the same group and emits a single key-value pair. This type of map reduce can be used to organize data into different categories or clusters.
- **Sorting map reduce:** This type of map reduce sorts a data set based on one or more attributes. The map function emits key-value pairs where the key is the sort attribute(s) and the value is the record from the input data set. The reduce function is the same as the identity reduce function. The sorting is done by the framework based on the key order. This type of map reduce can be used to order data by some criteria.
- **Inverted index map reduce:** This type of map reduce creates an inverted index from a collection of documents. The map function emits key-value pairs where the key is a word from the document and the value is the document identifier. The reduce function collects the document identifiers for each word and emits a single key-value pair. This type of map reduce can be used to support text search or analysis.
- **Word count map reduce:** This is a classic example of map reduce, where the goal is to count the frequency of each word in a collection of documents. The map function emits key-value pairs where the key is a word from the document and the value is 1. The reduce function sums up the values for each word and emits a single key-value pair. This type of map reduce can be used to measure the popularity or importance of words in a corpus.

A mnemonic to remember these types of map reduce is:

- I F A J G S I W
- Identity, Filtering, Aggregation, Join, Group by, Sorting, Inverted index, Word count

Input formats in map reduce

- Input formats are classes that define how the input data is split into input splits and how to read the records from the input splits.
- Input splits are logical chunks of the input data that are assigned to different mappers for parallel processing.

- Input formats also provide a `RecordReader` implementation that is responsible for extracting key-value pairs from the input split.
- There are two types of input formats: `FileInputFormat` and `InputFormat`.
- `FileInputFormat` is an abstract class that extends `InputFormat` and provides common functionality for reading files as input.
- `FileInputFormat` has several subclasses that can handle different types of files, such as `TextInputFormat`, `KeyValueTextInputFormat`, `SequenceFileInputFormat`, `NLineInputFormat`, etc.
- `InputFormat` is an interface that defines the methods for creating input splits and record readers. It can be implemented by custom classes that need to read data from sources other than files, such as databases, web services, etc.
- The default input format is `TextInputFormat`, which reads each line of a text file as a record and splits the line by a tab character into a key-value pair.
- The key is the byte offset of the line from the beginning of the file and the value is the line itself.
- The input format can be specified by setting the `mapreduce.inputformat.class` property in the job configuration or by using the `setInputFormatClass()` method of the `Job` class.
- The input format can also be specified by using the `@InputFormat` annotation on the mapper class.
- A custom input format can be created by implementing the `InputFormat` interface and overriding the `getSplits()` and `createRecordReader()` methods.
- The `getSplits()` method should return a list of `InputSplit` objects that represent the logical partitions of the input data.
- The `createRecordReader()` method should return a `RecordReader` object that can read the key-value pairs from the input split.
- A custom record reader can be created by implementing the `RecordReader` interface and overriding the `initialize()`, `nextKeyValue()`, `getCurrentKey()`, `getCurrentValue()`, and `close()` methods.
- The `initialize()` method should initialize the record reader with the input split and the task context.
- The `nextKeyValue()` method should advance to the next key-value pair and return true if one exists or false if the end of the input split is reached.
- The `getCurrentKey()` and `getCurrentValue()` methods should return the current key and value respectively.
- The `close()` method should close the record reader and release any resources.

Some mnemonics and learning tricks for the input formats in map reduce are:

- Remember that input formats are responsible for splitting and reading the input data.
- Remember that input splits are logical chunks of the input data and record

readers are the ones that extract key-value pairs from them.

- Remember that `FileInputFormat` is a common base class for reading files and `InputFormat` is an interface for reading other sources of data.
- Remember that the default input format is `TextInputFormat` and it splits the lines by tabs into key-value pairs.
- Remember that the input format can be specified by setting a property, using a method, or using an annotation.
- Remember that a custom input format can be created by implementing the `InputFormat` interface and a custom record reader can be created by implementing the `RecordReader` interface.

Output formats in map reduce

- Output formats are the classes that define how the output of a map reduce job is stored.
- Output formats are responsible for creating the output files, writing the records, and optionally compressing the data.
- Output formats can be specified by setting the `mapreduce.output.fileoutputformat.class` property in the job configuration.
- The default output format is `TextOutputFormat`, which writes plain text files with one record per line and a tab character as the separator between the key and the value.
- Some other common output formats are:
 - **SequenceFileOutputFormat**: Writes binary files that store key-value pairs in a serialized format. Sequence files are efficient and compact, and can be used as input for subsequent map reduce jobs.
 - **KeyValueTextOutputFormat**: Writes plain text files with one record per line and a configurable separator between the key and the value. The default separator is a tab character.
 - **MultipleOutputs**: A utility class that allows writing to multiple output files from a single map reduce job. Each output file can have a different output format and a different name or path. Multiple outputs can be useful for partitioning the output by some criteria, such as the key or the value.
 - **LazyOutputFormat**: A wrapper class that prevents the creation of empty output files. By default, map reduce creates one output file per reducer, even if some reducers do not produce any output. Lazy output format only creates output files when they are needed, which can save disk space and reduce the number of output files.
- A custom output format can be implemented by extending the `FileOutputFormat` abstract class and overriding the `getRecordWriter` method. The record writer is the class that writes the key-value pairs to the output file.

- A custom output format can also implement the `OutputCommitter` interface to perform any cleanup or finalization tasks after the job is completed. The output committer is responsible for committing or aborting the output, depending on the success or failure of the job.

Map Reduce features

- Map Reduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.
- Map Reduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase aggregates all the intermediate values associated with the same intermediate key and produces the final output.
- Map Reduce is designed to handle large-scale data processing in a fault-tolerant and scalable way. Some of the features of Map Reduce are:
 - It abstracts the details of parallelization, data distribution, load balancing, and fault tolerance from the programmer, allowing them to focus on the logic of the application.
 - It supports a variety of data types and formats, such as text, binary, structured, and unstructured data. It can also handle data that does not fit in memory, by using external storage and streaming techniques.
 - It provides a simple and expressive interface for defining the map and reduce functions, which can be written in any programming language that can be executed on the cluster nodes.
 - It allows the programmer to specify custom partitioning, grouping, sorting, and combiner functions for optimizing the performance and resource utilization of the cluster.
 - It enables the reuse of existing code and libraries, by allowing the programmer to invoke external programs or scripts as part of the map and reduce functions.
 - It supports incremental and iterative processing, by allowing the programmer to chain multiple Map Reduce jobs together and to reuse the intermediate results.
 - It supports distributed caching, by allowing the programmer to specify files or data that should be copied to the local disk of each cluster node before executing the map and reduce functions.
 - It supports speculative execution, by launching backup tasks for the slowest or failed tasks, to improve the overall throughput and reliability of the cluster.
 - It supports counters and status updates, by allowing the programmer to monitor the progress and statistics of the Map Reduce job, such

as the number of input, output, and intermediate records, the CPU and disk usage, and the error messages.

Real-world Map Reduce

MapReduce is a programming model that allows processing large-scale data sets in parallel on a cluster of machines. It consists of two phases: map and reduce. The map phase applies a user-defined function to each input record and produces a set of intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same key and produces the final output.

Some examples of real-world applications of MapReduce are:

- **Twitter:** Twitter uses MapReduce to analyze the tweets of its users and generate trending topics, recommendations, and advertisements. The map phase processes each tweet and extracts hashtags, mentions, and keywords. The reduce phase counts the occurrences of each term and ranks them by popularity.
- **Google:** Google uses MapReduce to index the web pages and provide search results. The map phase parses each web page and extracts words and links. The reduce phase builds an inverted index that maps each word to a list of web pages that contain it.
- **Netflix:** Netflix uses MapReduce to analyze the viewing habits of its subscribers and provide personalized recommendations. The map phase processes each user's rating and viewing history and generates a profile of preferences. The reduce phase compares the profiles of different users and finds similar ones.
- **The True Size Of:** The True Size Of is a web application that allows users to compare the sizes of different countries on a map. The map phase projects each country's shape onto a Mercator projection and calculates its area. The reduce phase scales each country's shape according to its true area and displays it on the map.

Some advantages of MapReduce are:

- **Scalability:** MapReduce can handle very large data sets by distributing the work across multiple machines in a cluster.
- **Fault-tolerance:** MapReduce can handle failures of machines or tasks by automatically reassigning the work to other machines or tasks.
- **Simplicity:** MapReduce abstracts away the details of parallelization, distribution, and coordination from the user. The user only needs to provide the map and reduce functions.

Some disadvantages of MapReduce are:

- **Overhead:** MapReduce introduces some overhead in terms of network communication, data serialization, and disk I/O. This can affect the performance of some applications that require low latency or high throughput.

- **Rigidity:** MapReduce imposes a fixed structure of map and reduce phases on the user. This can limit the expressiveness and flexibility of some applications that require more complex or iterative algorithms.
- **Inefficiency:** MapReduce can generate a lot of intermediate data that needs to be shuffled and sorted between the map and reduce phases. This can consume a lot of resources and time for some applications that require only a small amount of output.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

- HDFS is a distributed file system that is designed to store and process large amounts of data across a cluster of machines.
- HDFS is one of the core components of the Hadoop ecosystem, which also includes MapReduce, YARN, Hive, Pig, Spark, etc.
- HDFS follows a master-slave architecture, where one node acts as the NameNode (master) and the rest of the nodes act as DataNodes (slaves).
- The NameNode is responsible for managing the metadata of the file system, such as the file names, directories, permissions, locations of blocks, etc.
- The DataNodes are responsible for storing the actual data blocks of the files, and for serving read and write requests from the clients.
- HDFS splits the files into fixed-size blocks (typically 128 MB or 256 MB) and distributes them across the DataNodes, with replication for fault tolerance.
- HDFS provides a high-level abstraction of the file system to the clients, who can access the files using a standard interface (such as Java API, command-line, web browser, etc.).
- HDFS is optimized for batch processing of large and sequential data, rather than random and interactive access.
- HDFS supports the write-once-read-many (WORM) model, where the files are appended to but not modified after creation.
- HDFS is designed to run on commodity hardware, which means that it can handle hardware failures and network partitions gracefully.

Some of the advantages of HDFS are:

- **Scalability:** HDFS can scale to thousands of nodes and petabytes of data, by adding more machines to the cluster.
- **Reliability:** HDFS can tolerate the loss of some DataNodes or blocks, by replicating the data across multiple nodes and by performing checksums to detect corruption.
- **Availability:** HDFS can provide high availability of the NameNode, by using a secondary or standby NameNode that can take over in case of failure.
- **Cost-effectiveness:** HDFS can run on low-cost hardware, which reduces the total cost of ownership and maintenance.

- Compatibility: HDFS can integrate with various tools and frameworks in the Hadoop ecosystem, such as MapReduce, Hive, Pig, Spark, etc.

Some of the disadvantages of HDFS are:

- Latency: HDFS has a high latency for read and write operations, due to the network overhead and the replication factor.
- Overhead: HDFS consumes a lot of disk space and memory for storing the metadata and the blocks, which reduces the effective storage capacity and performance of the cluster.
- Limitations: HDFS has some limitations in terms of the file size, the number of files, the number of concurrent clients, the append operations, etc.

Some of the applications of HDFS are:

- Data warehousing: HDFS can store and process large volumes of structured and unstructured data, such as logs, transactions, social media, etc.
- Data analytics: HDFS can support various types of analytics, such as batch, real-time, streaming, machine learning, etc., by using different tools and frameworks in the Hadoop ecosystem.
- Data backup: HDFS can provide a reliable and cost-effective backup solution for the data, by replicating and archiving the data across multiple nodes and locations.
- Data archive: HDFS can store and preserve the data for a long time, by using compression and encryption techniques, and by applying retention and deletion policies.

Some of the mnemonics and learning tricks for HDFS and Hadoop Environment are:

- HDFS: Hadoop Distributed File System
 - H: Huge files
 - D: Distributed blocks
 - F: Fault tolerant replication
 - S: Scalable cluster
- NameNode: Master node that manages the metadata
 - N: Name of the files and directories
 - A: Access permissions and ownership
 - M: Map of the blocks and their locations
 - E: Edit log and fsimage of the file system
- DataNode: Slave node that stores the data blocks
 - D: Data blocks of the files
 - A: Acknowledgement of the write requests
 - T: Transfer of the blocks to other nodes
 - A: Availability report to the NameNode
- Block: Fixed-size unit of data storage
 - B: Big size (128 MB or 256 MB)
 - L: Location on the DataNodes

- O: Offset within the file
- C: Checksum for error detection
- K: Key for identification
- Replication: Process of copying the blocks across multiple DataNodes
 - R: Redundancy for fault tolerance
 -

HDFS

- HDFS stands for Hadoop Distributed File System. It is a file system that stores large amounts of data across multiple nodes in a cluster.
- HDFS is designed to handle failures gracefully, by replicating data blocks on different nodes and providing fault tolerance and high availability.
- HDFS follows a master-slave architecture, where one node acts as the NameNode (master) and the rest of the nodes act as DataNodes (slaves).
- The NameNode is responsible for managing the namespace, the metadata, and the access control of the files and directories. It also coordinates the placement and replication of data blocks on the DataNodes.
- The DataNodes are responsible for storing the actual data blocks and serving read and write requests from the clients. They also periodically report to the NameNode about the status of the data blocks they hold.
- HDFS splits a large file into fixed-size blocks (typically 128 MB) and distributes them across the DataNodes. Each block is replicated a number of times (default is 3) for fault tolerance.
- HDFS provides a Java-based API for clients to interact with the file system. It also supports a web-based interface and a command-line interface.
- HDFS supports a write-once-read-many model, where a file can be written only once and then read multiple times. It does not support random writes or updates to a file.
- HDFS is optimized for streaming access of large files, rather than random access of small files. It is suitable for applications that perform batch processing or analytics on large datasets.
- HDFS is not a POSIX-compliant file system, meaning it does not support some of the features and semantics of a standard Unix file system, such as hard links, soft links, file permissions, etc.

Design of HDFS

HDFS is a distributed file system that is designed to store very large files across clusters of commodity hardware. It is part of the Apache Hadoop project, which provides a framework for processing and analyzing large-scale data sets using parallel and distributed computing.

Some of the main features and design principles of HDFS are:

- **Fault tolerance:** HDFS can tolerate failures of nodes, disks, and network by replicating data blocks across multiple machines. The default

replication factor is three, which means that each block is stored on three different nodes. If a node or a disk fails, the system can automatically recover the data from the remaining replicas. HDFS also performs periodic checksums to detect and correct corrupted blocks.

- **Scalability:** HDFS can scale to thousands of nodes and petabytes of data by distributing the workload and the data across the cluster. HDFS uses a master-slave architecture, where a single NameNode manages the namespace and the metadata of the file system, and multiple DataNodes store and serve the data blocks. The NameNode and the DataNodes communicate through heartbeats and block reports to maintain the consistency and availability of the file system.
- **Streaming data access:** HDFS is optimized for batch processing rather than interactive use. The emphasis is on high throughput of data access rather than low latency. HDFS supports a write-once-read-many model, where files are appended to rather than modified. HDFS also provides a high-level abstraction of data as input and output streams, rather than as random access files.
- **Large files:** HDFS is designed to store very large files, typically in the range of gigabytes to terabytes. HDFS splits files into fixed-size blocks, which are stored on different DataNodes. The default block size is 128 MB, which can be configured according to the needs of the application. HDFS can handle files that are larger than the disk capacity of a single node by distributing the blocks across the cluster.
- **Simple and robust coherency model:** HDFS follows a simple and robust coherency model, where a file is visible to all the readers once it is closed by the writer. HDFS does not support caching, locking, or concurrent writes. HDFS also does not support POSIX semantics, which impose many hard requirements that are not needed for applications that are targeted for HDFS. HDFS provides a simple and consistent view of the data to the users and the applications.

A possible mnemonic to remember the design features of HDFS is **FSSL**:

- **F**ault tolerance
- **S**calability
- **S**teaming data access
- **L**arge files

HDFS concepts

HDFS stands for Hadoop Distributed File System. It is a distributed file system that runs on a cluster of commodity hardware (low-cost machines) and provides high-throughput access to large data sets. HDFS is one of the core components of Apache Hadoop, along with MapReduce and YARN.

Some of the key concepts of HDFS are:

- **Blocks:** HDFS stores files as a sequence of fixed-size blocks, usually 128

MB or 256 MB. Each block is replicated across multiple data nodes (usually three) for fault tolerance. The block size and replication factor are configurable per file or per cluster.

- **NameNode:** The NameNode is the master node of HDFS. It maintains the metadata of the file system, such as the file names, directories, permissions, and locations of blocks. The NameNode also coordinates the read and write operations from clients and manages the replication and recovery of blocks. There is only one active NameNode in a cluster, and optionally one or more standby NameNodes for high availability.
- **DataNode:** The DataNode is the worker node of HDFS. It stores and serves the blocks of files to clients and other DataNodes. It also performs periodic block reports and heartbeats to the NameNode to indicate its health and availability. There can be hundreds or thousands of DataNodes in a cluster, depending on the scale and configuration.
- **Client:** The client is the application or user that interacts with HDFS. The client can perform operations such as creating, reading, writing, appending, deleting, or renaming files and directories. The client communicates with the NameNode to get the metadata of the file system and with the DataNodes to read or write the blocks of files.
- **Rack Awareness:** HDFS is rack-aware, meaning that it considers the physical location of the DataNodes in a cluster when placing or replicating blocks. HDFS tries to minimize the network bandwidth consumption by placing blocks on the same or nearby racks as much as possible. This also improves the reliability and performance of the file system.

A possible mnemonic to remember these concepts is:

Big **N**erds **D**o **C**ool **R**esearch

where B stands for Blocks, N for NameNode, D for DataNode, C for Client, and R for Rack Awareness.

Benefits of HDFS

HDFS stands for Hadoop Distributed File System, which is a scalable and reliable file system for storing and processing large volumes of data. HDFS has the following benefits:

- **Distributed storage:** HDFS splits large files into smaller blocks and distributes them across multiple nodes in a cluster. This enables parallel processing and reduces network congestion.
- **Fault tolerance:** HDFS replicates each block of data to multiple nodes, ensuring data availability and reliability in case of node failure or network outage. HDFS also detects and recovers from faults automatically, without requiring human intervention.
- **Cost effectiveness:** HDFS runs on commodity hardware, which lowers the storage cost and allows for horizontal scaling. HDFS is also an open source software, which means there is no licensing or support fee.

- **Speed:** HDFS can deliver more than 2 GB of data per second, thanks to its cluster architecture and data locality. Data locality means that HDFS tries to place the data close to where it is processed, minimizing data movement and network latency.
- **Scalability:** HDFS can scale up to thousands of nodes and petabytes of data, by adding more nodes to the cluster. HDFS also supports heterogeneous hardware, which means that nodes can have different capacities and configurations.

Challenges of HDFS

HDFS is a distributed file system that is designed to store and process large amounts of data across multiple nodes in a cluster. HDFS has many advantages, such as scalability, fault tolerance, high throughput, and low cost. However, HDFS also faces some challenges and limitations, such as:

- **Issues with small files:** HDFS is not suitable for storing and accessing small files, because each file is stored as a separate block (typically 128 MB or 256 MB) and occupies a slot in the namenode's memory. This can lead to inefficient use of disk space and namenode memory, as well as increased network traffic and latency. Moreover, HDFS does not support random access to files, which means that the entire file has to be read sequentially, even if only a small portion is needed .
- **Slow processing speed:** HDFS relies on MapReduce, a batch processing framework, to perform data analysis. MapReduce processes data in two phases: map and reduce, which involve shuffling and sorting large amounts of data across the network. This can result in slow processing speed, especially for complex and iterative algorithms. Furthermore, MapReduce does not support real-time or streaming data processing, which limits the applicability of HDFS for some use cases .
- **Support for batch processing only:** HDFS only supports batch processing, which means that data has to be loaded into HDFS before it can be processed. This can introduce delays and overheads, especially for dynamic and interactive data analysis. HDFS does not support transactions, updates, or deletes, which makes it difficult to handle mutable data. HDFS also does not support indexing, caching, or compression, which can improve the performance and efficiency of data processing .
- **No real-time processing:** HDFS does not support real-time processing, which means that data has to be processed in batches after it is stored in HDFS. This can result in high latency and stale data, which can affect the quality and accuracy of data analysis. HDFS also does not support stream processing, which means that data has to be processed as it arrives, without storing it in HDFS. This can limit the scalability and fault tolerance of data processing, as well as the ability to perform historical and comparative analysis .
- **Iterative processing:** HDFS does not support iterative processing,

which means that data has to be processed multiple times with different parameters or algorithms. This can result in high overheads and low performance, as data has to be read and written repeatedly from and to HDFS. HDFS also does not support in-memory processing, which means that data has to be stored on disk, rather than in memory, which can reduce the speed and efficiency of data processing .

- **Latency:** HDFS has high latency, which means that data takes a long time to be stored and retrieved from HDFS. This is because HDFS has to perform several operations, such as splitting, replicating, and check-summing data blocks, as well as communicating with the namenode and the datanodes, before data can be stored or accessed. Moreover, HDFS has to perform periodic operations, such as heartbeat, block report, and checkpoint, which can affect the availability and performance of HDFS .
- **No ease of use:** HDFS is not easy to use, as it requires a lot of configuration, tuning, and maintenance. HDFS has to be installed and configured on each node in the cluster, which can be tedious and error-prone. HDFS also has to be monitored and managed, which involves checking the status, health, and performance of the cluster, as well as performing backup, recovery, and upgrade operations. HDFS also has a complex and unfamiliar programming model, which requires users to write MapReduce programs or use other frameworks, such as Pig or Hive, to access and process data in HDFS .
- **Security issue:** HDFS has a weak security model, which means that data is not protected from unauthorized access or modification. HDFS does not support encryption, which means that data is stored in plain text, and can be read or tampered with by anyone who has access to the cluster. HDFS also does not support authentication, authorization, or auditing, which means that users cannot be verified, restricted, or tracked when they access or modify data in HDFS [2]

File sizes in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed file system designed to store and process large amounts of data across multiple nodes in a cluster.
- HDFS breaks down files into fixed-size blocks, which are stored as independent units on different nodes. The default block size in HDFS is 128 MB, but it can be configured manually .
- The block size in HDFS is much larger than the typical block size in a traditional file system, such as 4 KB. This is because HDFS is optimized for large files, which can range from gigabytes to terabytes in size.
- The advantages of having a large block size in HDFS are:
 - It reduces the number of disk seeks, which improves the data transfer rate and the overall performance of the system.
 - It reduces the amount of metadata that needs to be stored and managed by the NameNode, which is the master node that keeps track

- of the location and status of all the blocks in the cluster.
 - It reduces the network overhead, as fewer requests and responses are needed to read or write a file.
- The disadvantages of having a large block size in HDFS are:
 - It increases the disk space wastage, as the last block of a file may not be fully utilized. For example, if a file is 129 MB in size, it will occupy two blocks of 128 MB each, but the second block will only use 1 MB of space, leaving 127 MB unused.
 - It increases the replication factor, which is the number of copies of each block that are stored on different nodes for fault tolerance. By default, the replication factor is 3, which means that each block is replicated three times. This means that a file of 129 MB will consume 768 MB of disk space in total (2 blocks x 3 replicas x 128 MB).
- To overcome the disadvantages of having a large block size in HDFS, some techniques are used, such as:
 - Compression, which reduces the size of the files and the blocks, and improves the disk space utilization and the data transfer rate.
 - Variable block size, which allows different files to have different block sizes, depending on their characteristics and requirements.
 - Small files handling, which involves combining multiple small files into a single large file, or storing them in a different file system, such as HBase or Hive.
- To check the size of a file or a directory in HDFS, some commands are used, such as:
 - **hadoop fs -ls <path>**, which lists the files and directories in the given path, along with their details, such as permissions, owner, group, size, and modification time. The size of the file or directory is shown in the fifth column of the output, in bytes.
 - **hadoop fs -du <path>**, which displays the disk usage of the files and directories in the given path, in bytes. The output shows the base size of the file or directory before replication, which means the actual amount of space that is used by the data.
 - **hadoop fs -du -s <path>**, which summarizes the disk usage of the files and directories in the given path, in bytes. The output shows the total size of the file or directory before replication, which means the sum of the sizes of all the files and directories that match the given path.

Block sizes in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed file system that stores large amounts of data across multiple nodes in a cluster.
- HDFS splits a file into one or more blocks, and stores each block on a different node. The default block size in HDFS is 128 MB, which can be configured by changing the parameter `dfs.blocksize` in the `hdfs-site.xml`

file.

- The block size in HDFS is much larger than the block size in a traditional file system, such as 4 KB or 8 KB. This is because HDFS is designed for storing and processing large files, and having a large block size reduces the number of disk seeks and network transfers required to access a file.
- Some of the advantages of having a large block size in HDFS are:
 - It improves the throughput of data transfer, as each block can be read or written in a single disk operation.
 - It reduces the metadata overhead, as each file requires fewer blocks and hence fewer entries in the namenode's memory.
 - It increases the fault tolerance, as each block can be replicated across multiple nodes and recovered in case of node failure.
- Some of the disadvantages of having a large block size in HDFS are:
 - It wastes disk space, as each block is padded to the full block size regardless of the actual file size. For example, if a file is 130 MB, it will occupy two blocks of 128 MB each, wasting 126 MB of disk space.
 - It increases the latency of data access, as each block may be located on a different node and require a network transfer to read or write.
 - It reduces the parallelism of data processing, as each block can be processed by only one mapper or reducer at a time.
- Some of the factors that affect the choice of block size in HDFS are:
 - The size and type of the files stored in HDFS. For example, if the files are small and text-based, a smaller block size may be preferred to avoid wasting disk space and increase parallelism. If the files are large and binary, a larger block size may be preferred to improve throughput and reduce metadata overhead.
 - The network bandwidth and latency of the cluster. For example, if the network is fast and reliable, a larger block size may be preferred to reduce the number of network transfers and disk seeks. If the network is slow and unreliable, a smaller block size may be preferred to increase the fault tolerance and recovery speed.
 - The processing framework and application logic used on the data. For example, if the processing framework is MapReduce, a larger block size may be preferred to match the input split size and reduce the number of mappers. If the processing framework is Spark, a smaller block size may be preferred to increase the number of partitions and parallelism.
- A possible mnemonic to remember the default block size in HDFS is: **128 MB = 1 HDFS block**. This can be associated with the fact that HDFS is a part of the Hadoop ecosystem, which has the letter H as its logo. The letter H has one horizontal line and two vertical lines, which can be seen as the digits 1, 2, and 8.

Block abstraction in HDFS

- HDFS is a distributed file system that stores large files across multiple machines.
- HDFS breaks down each file into smaller units called blocks, which are stored on different nodes in the cluster.
- Each block has a fixed size, typically 64 MB or 128 MB, which is configurable by the administrator.
- Blocks are the smallest unit of data that can be read or written by HDFS.
- Blocks are also replicated across multiple nodes for fault tolerance and high availability.
- The default replication factor is 3, which means each block has 3 copies on different nodes.
- HDFS maintains a metadata file called the namespace, which records the file names, directories, permissions, and the locations of the blocks for each file.
- The namespace is stored on a special node called the NameNode, which is the master node of the cluster.
- The NameNode also manages the block allocation, replication, and recovery.
- The other nodes in the cluster are called DataNodes, which store the actual blocks and serve read and write requests from clients.
- The DataNodes periodically send heartbeat and block report messages to the NameNode, to report their status and the blocks they have.
- The NameNode uses these messages to keep track of the cluster health and the block locations.
- If a DataNode fails or a block becomes corrupted, the NameNode can initiate the replication of the missing or corrupted block from another DataNode that has a copy of the block.
- The NameNode also balances the load of the cluster by moving blocks from one DataNode to another, if needed.
- A simple mnemonic to remember the block abstraction in HDFS is:
 - **B**reak files into **B**locks
 - **R**eplicate **B**locks across **R**acks
 - **S**ore metadata in the **S**ingle NameNode
 - **M**anage blocks with heart**B**eat and block report
 - **R**ecover and balance blocks with **R**eplication and **R**ebalancing

Data replication in HDFS

- Data replication is the process of copying data from one HDFS service to another, or to and from other storage systems such as Amazon S3 or Microsoft ADLS.
- Data replication is used for fault tolerance, backup, disaster recovery, and data availability.
- HDFS stores each file as a sequence of blocks, and each block is replicated across multiple nodes in the cluster .
- The block size and replication factor are configurable per file, and can be changed by the user or the administrator .
- The default block size is 128 MB and the default replication factor is 3.
- HDFS uses a master-slave architecture, where the master node (NameNode) manages the metadata of the file system, and the slave nodes (DataNodes) store the actual data blocks .
- The NameNode is responsible for maintaining the namespace tree, the file-to-block mapping, and the block-to-DataNode mapping .
- The DataNodes are responsible for serving read and write requests from the clients, and performing block creation, deletion, and replication as instructed by the NameNode .
- HDFS follows a write-once-read-many model, where a file once created, written, and closed, cannot be changed .
- HDFS supports appending data to existing files, but not random writes or updates .
- HDFS replicates each block to a different rack in the cluster, to ensure high availability and reliability in case of rack failure .
- HDFS uses a pipeline mechanism to write data blocks to multiple DataNodes in parallel, to improve the write performance .
- HDFS uses a heartbeat mechanism to monitor the health and status of the DataNodes, and a block report mechanism to keep track of the blocks stored on each DataNode .
- HDFS can detect and handle node failures, network failures, and corrupted blocks, by re-replicating the missing or damaged blocks to other DataNodes .
- HDFS can balance the load of the cluster, by moving blocks from over-utilized DataNodes to under-utilized DataNodes .

A possible mnemonic to remember the key features of HDFS replication is:

Block size and replication factor are configurable **R**eplication across racks for fault tolerance **W**rite-once-read-many model for files **P**ipeline mechanism for parallel writes **H**eartbeat and block report for monitoring **R**e-replication and load balancing for recovery

How does HDFS store

- HDFS stands for Hadoop Distributed File System, which is the primary

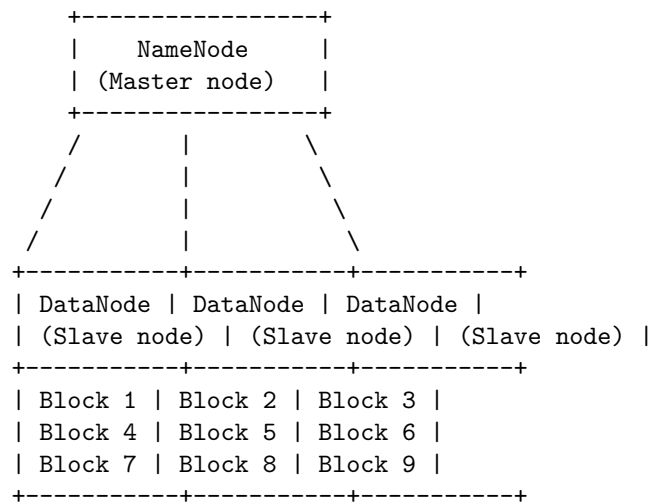
storage system used by Hadoop applications.

- HDFS stores data in a distributed manner, meaning that it divides the data into small pieces and stores them on different DataNodes in the cluster .
- Each piece of data is called a block, and the default size of each block is 128 MB, which can be configured in the hdfs-site.xml file.
- HDFS replicates each block across multiple DataNodes for fault tolerance and high availability. The default replication factor is 3, which can also be configured in the hdfs-site.xml file.
- HDFS has a master-slave architecture, where the master node is called the NameNode and the slave nodes are called the DataNodes.
- The NameNode manages the file system namespace and the metadata of all the files and directories. It also coordinates the placement and replication of blocks across the DataNodes.
- The DataNodes store the actual blocks of data and serve read and write requests from the clients. They also send periodic reports to the NameNode about the blocks they are storing and their health status.
- HDFS provides a way for MapReduce to process a subset of large data sets broken into blocks, parallelly on several nodes. This reduces the network overhead and increases the performance of data processing.

A possible mnemonic to remember the key components of HDFS is:

Hadoop **D**istributed **F**ile **S**ystem = **H**uge **D**ata **F**ragmented and **S**tored

A possible ascii diagram to illustrate the HDFS architecture is:



Read operations in HDFS

- To read a file from HDFS, a client needs to interact with the NameNode, which stores the metadata about the file location, size, block IDs, and

DataNodes that store the blocks.

- The NameNode returns a list of DataNodes that have a copy of the file blocks, along with a token that authorizes the client to access the data.
- The client then contacts one of the DataNodes (preferably the closest one) and requests to read the file blocks.
- The DataNode sends the data to the client in a streaming fashion, using a TCP connection.
- The client can read the data from multiple DataNodes in parallel, to improve the performance and reliability of the read operation.
- If the client encounters any error or failure while reading from a DataNode, it can switch to another DataNode that has the same block and resume the read operation.
- The client periodically sends a heartbeat to the NameNode, to indicate that it is still reading the file and to renew the token.
- The client also verifies the checksum of the data received from the DataNode, to ensure the data integrity and quality.
- The read operation is completed when the client has read all the blocks of the file from the DataNodes.

A sample code to read a file from HDFS using Java API is as follows:

```
// Create a configuration object and get a handle to the filesystem
Configuration conf = new Configuration();
FileSystem fileSystem = FileSystem.get(conf);

// Create a path object for the file to be read
Path path = new Path("/path/to/file.ext");

// Check if the file exists
if (!fileSystem.exists(path)) {
    System.out.println("File does not exist");
    return;
}

// Create an input stream to read the file
FSDataInputStream in = fileSystem.open(path);

// Read the file content and display it on the console
String line = in.readLine();
while (line != null) {
    System.out.println(line);
    line = in.readLine();
}

// Close the input stream and the filesystem
in.close();
fileSystem.close();
```

Some mnemonics and learning tricks for the read operations in HDFS are:

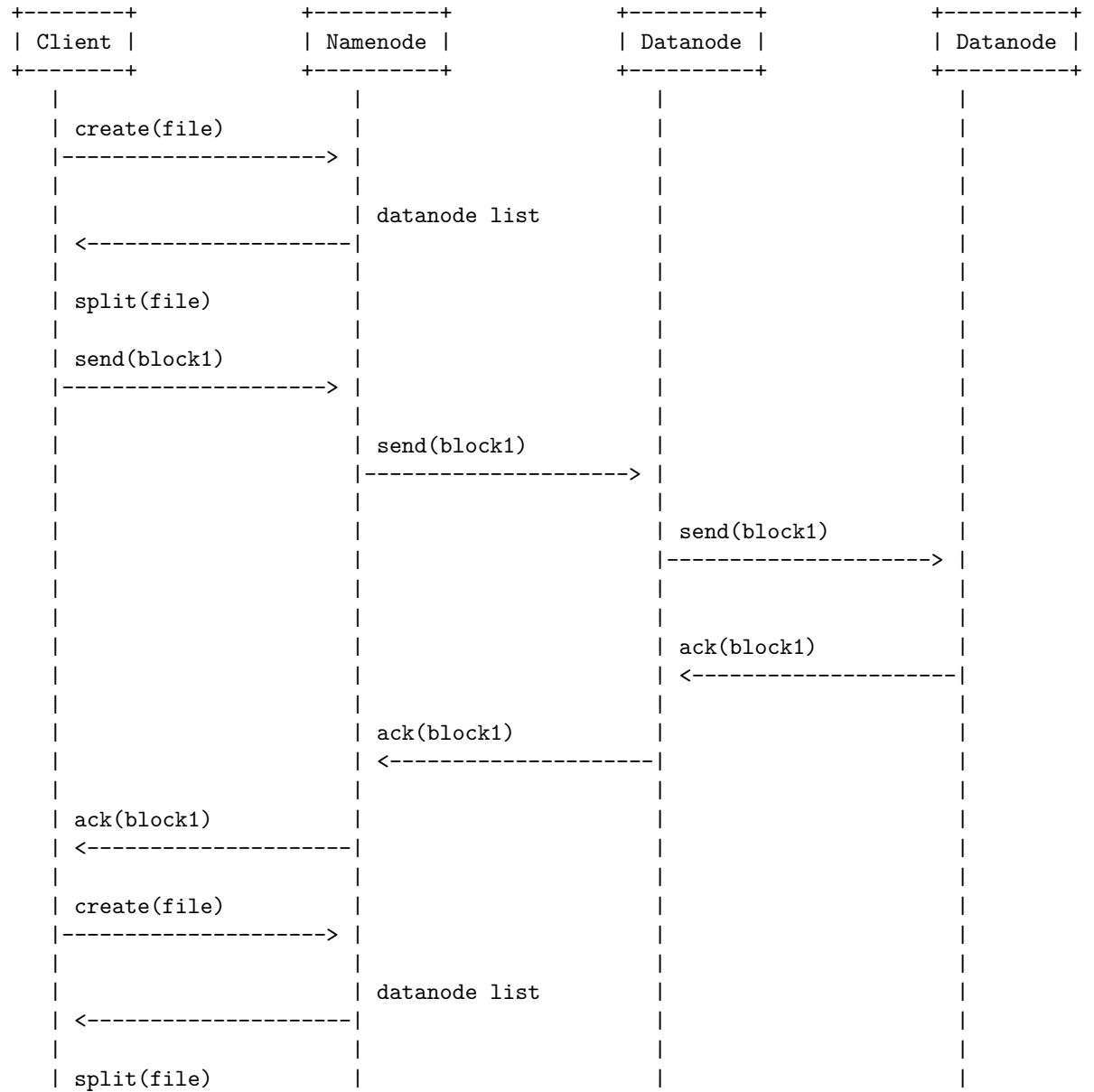
- Remember the acronym **NDRD**: NameNode, DataNode, Read, Data. This is the sequence of steps involved in reading a file from HDFS.
- Remember the formula $R = N * B / S$: Read time = Number of blocks * Block size / Streaming rate. This is an approximation of how long it takes to read a file from HDFS, assuming no failures or errors.
- Remember the difference between **replication** and **replica**: Replication is the process of creating multiple copies of a file block on different DataNodes, while replica is one of those copies. Replication improves the availability and reliability of the data, while replica provides the actual data to the client.

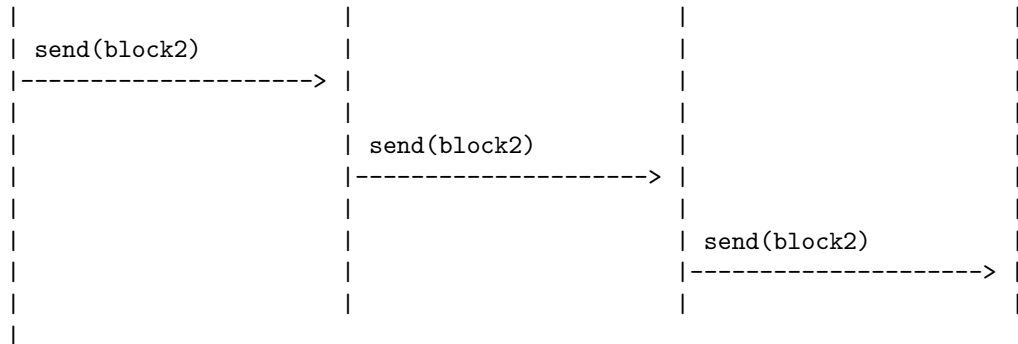
Write operations in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS supports write-once-read-many model, which means a file can be written once and then read multiple times, but not modified or appended.
- HDFS write operations involve three main components: the client, the namenode, and the datanodes.
- The client is the application that initiates the write request to HDFS.
- The namenode is the master node that manages the metadata of the file system, such as the file name, size, location, replication factor, etc.
- The datanodes are the worker nodes that store the actual data blocks of the file.
- The following steps describe the write operations in HDFS:
 1. The client contacts the namenode and requests to create a new file with a given name, replication factor, and block size.
 2. The namenode checks if the file already exists or if the namespace quota is exceeded. If not, it grants the write permission to the client and returns a list of datanodes that can store the first block of the file.
 3. The client splits the file into fixed-size blocks (default 128 MB) and sends the first block to the first datanode in the list. The datanode stores the block locally and forwards it to the next datanode in the list. This process continues until the block is replicated to the desired number of datanodes (default 3).
 4. The client receives an acknowledgment from the last datanode in the pipeline and contacts the namenode again for the next block location. The namenode returns another list of datanodes for the second block.
 5. The client repeats steps 3 and 4 until all the blocks of the file are written and replicated.

6. The client sends a close request to the namenode, indicating that the file write is complete. The namenode updates the metadata and commits the file creation.

- A possible mnemonic to remember the write operations in HDFS is: **Create, Split, Send, Acknowledge, Close.** (CSSAC)
- A possible ascii diagram to illustrate the write operations in HDFS is:





Java interfaces to HDFS

- HDFS is the Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for large-scale data processing.
- HDFS can be accessed and manipulated through various interfaces, such as command-line, web, REST, and Java.
- The Java interface of HDFS is based on the abstract `FileSystem` class, which defines the common operations for all file systems, such as create, delete, rename, copy, etc.
- HDFS provides a concrete implementation of the `FileSystem` class, called `DistributedFileSystem`, which extends the `FileSystem` class and implements the specific operations for HDFS, such as replication, block management, etc.
- To use the Java interface of HDFS, one needs to import the `org.apache.hadoop.fs` package, which contains the `FileSystem` and `DistributedFileSystem` classes, as well as other classes and interfaces related to HDFS.
- To create a `FileSystem` object, one can use the static `get()` method of the `FileSystem` class, which takes a `Configuration` object as a parameter. The `Configuration` object contains the HDFS configuration settings, such as the name node address, the default block size, the replication factor, etc.
- The `FileSystem` object can then be used to perform various operations on HDFS, such as creating, reading, writing, deleting, or listing files and directories. For example, to create a file in HDFS, one can use the `create()` method of the `FileSystem` object, which takes a `Path` object as a parameter. The `Path` object represents the HDFS file or directory name.
- To read or write data from or to a file in HDFS, one can use the `FSDataInputStream` or `FSDataOutputStream` classes, which are subclasses of the `java.io.DataInputStream` and `java.io.DataOutputStream` classes, respectively. These classes provide methods for reading and writing primitive data types, such as int, long, byte, etc.
- To close a `FileSystem` object, one can use the `close()` method of the `FileSystem` class, which releases the resources associated with the object.

Here is an example of a Java program that creates a file in HDFS and writes some data to it:

```
// Import the HDFS classes
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.fs.FSDataOutputStream;

// Create a configuration object
Configuration conf = new Configuration();

// Set the HDFS name node address
conf.set("fs.defaultFS", "hdfs://namenode:8020");

// Create a file system object
FileSystem fs = FileSystem.get(conf);

// Create a path object for the file to be created
Path path = new Path("/user/hadoop/test.txt");

// Create a file in HDFS
FSDataOutputStream out = fs.create(path);

// Write some data to the file
out.writeInt(42);
out.writeUTF("Hello, HDFS!");

// Close the file and the file system
out.close();
fs.close();
```

Command line interface to HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed and scalable file system that stores large amounts of data across multiple nodes in a cluster.
- HDFS provides a command line interface (CLI) that allows users to interact with the file system using commands similar to Unix/Linux shell commands.
- The CLI can be accessed by typing `hdfs dfs` followed by the command name and its arguments. For example, `hdfs dfs -ls /` will list the files and directories in the root directory of HDFS.
- Some of the common commands and their functions are:

Command	Function
-cat	Concatenate files and print on the standard output
-chgrp	Change group association of files
-chmod	Change permissions of files
-chown	Change owner of files
-copyFromLocal	Copy files from the local file system to HDFS
-copyToLocal	Copy files from HDFS to the local file system
-cp	Copy files from source to destination within HDFS
-du	Display disk usage of files and directories
-get	Copy files from HDFS to the local file system
-getmerge	Get all the files in the directories that match the source file pattern and merge and sort them to only one file on the local file system
-ls	List the contents of a directory
-mkdir	Create a directory in HDFS
-moveFromLocal	Move files from the local file system to HDFS
-moveToLocal	Move files from HDFS to the local file system
-mv	Move files from source to destination within HDFS
-put	Copy files from the local file system to HDFS
-rm	Delete files from HDFS
-rmdir	Remove a directory from HDFS
-tail	Display the last kilobyte of a file
-test	Test a file for existence, accessibility, zero length, etc.

- To get more information about a command, use the **-help** option. For example, **hdfs dfs -help ls** will display the usage and options of the **ls** command.
- To get a list of all the commands, use the **-help** option without any arguments. For example, **hdfs dfs -help** will display the usage and options of all the commands.
- A mnemonic to remember some of the common commands is: **C**ats **C**hange **C**hange **C**hange **C**opy **C**opy **C**opy **C**opy **D**isk **G**et **G**et **L**ist **M**ake **M**ove **M**ove **M**ove **P**ut **R**emove **R**emove **T**ail **T**est. The first letter of

each word corresponds to the first letter of the command. For example, **C**ats corresponds to **-cat**, **C**hange corresponds to **-chgrp**, **-chmod**, and **-chown**, etc.

Hadoop file system interfaces

- Hadoop file system (HDFS) is a distributed file system that runs on a cluster of nodes and provides high-throughput access to large data sets.
- HDFS exposes two main interfaces for interacting with the file system: the Java API and the command-line interface (CLI).
- The Java API provides a set of classes and methods that allow applications to perform various operations on HDFS, such as creating, reading, writing, deleting, and appending files, as well as listing, renaming, and moving directories.
- The Java API also supports some advanced features, such as file checksums, snapshots, encryption, erasure coding, and quotas.
- The Java API is organized into three main packages: `org.apache.hadoop.fs`, `org.apache.hadoop.hdfs`, and `org.apache.hadoop.hdfs.client`.
- The `org.apache.hadoop.fs` package contains the core interfaces and classes that define the abstract file system model, such as `FileSystem`, `Path`, `FileStatus`, `FSDatInputStream`, and `FSDatOutputStream`.
- The `org.apache.hadoop.hdfs` package contains the implementation of HDFS-specific classes and methods, such as `DistributedFileSystem`, `DFSClient`, `DFSInputStream`, and `DFSOutputStream`.
- The `org.apache.hadoop.hdfs.client` package contains some additional classes and methods that provide more functionality and convenience for HDFS users, such as `HdfsConfiguration`, `HdfsUtils`, and `HdfsAdmin`.
- The command-line interface (CLI) provides a set of commands that allow users to perform various operations on HDFS from the shell, such as `hdfs dfs -ls`, `hdfs dfs -cat`, `hdfs dfs -put`, and `hdfs dfs -get`.
- The CLI also supports some advanced commands, such as `hdfs dfsadmin`, `hdfs fsck`, `hdfs balancer`, and `hdfs crypto`.
- The CLI is implemented by the `org.apache.hadoop.fs.shell` package, which parses the user input and invokes the corresponding Java API methods.
- The CLI can be accessed by using the `hdfs` script, which is located in the `bin` directory of the Hadoop installation.
- The CLI can also be accessed by using the `hadoop fs` command, which is a wrapper for the `hdfs` script.
- The CLI supports some common options, such as `-conf`, `-D`, `-help`, and `-usage`, as well as some HDFS-specific options, such as `-skipTrash`, `-safely`, and `-storagepolicy`.
- The CLI follows the Unix convention of using `-` for standard input and output, and `*` for wildcard matching.

Data flow in HDFS

HDFS stands for Hadoop Distributed File System, which is the storage layer of Hadoop that stores data in a distributed and reliable manner. HDFS splits a file into one or more blocks and stores them across a cluster of DataNodes. The NameNode manages the file system namespace and the metadata of the blocks. The client interacts with the NameNode and the DataNodes to perform read and write operations on HDFS files.

The following are the main steps involved in the data flow of HDFS read and write operations:

- **HDFS write operation:**

1. The client creates the file by calling `create()` on the Distributed-FileSystem (DFS) object, which is an instance of the `FileSystem` class that represents HDFS.
2. DFS makes a remote procedure call (RPC) to the NameNode to create a new file in the file system namespace, with no blocks associated with it. The NameNode checks if the file already exists or if the parent directory is valid, and returns an exception if any of these conditions are not met.
3. The client obtains a `DFSOutputStream` object to write data to the file. The `DFSOutputStream` splits the data into packets, which are stored in an internal queue called the data queue. The data queue is consumed by the `DataStreamer`, which is responsible for asking the NameNode to allocate new blocks and to locate DataNodes to store those blocks.
4. The `DataStreamer` picks a list of suitable DataNodes to store a replica of each block. The list of DataNodes forms a pipeline, and the default replication factor is three. The `DataStreamer` streams the packets to the first DataNode in the pipeline, which stores the packet and forwards it to the second DataNode, and so on.
5. The `DFSOutputStream` also maintains an internal queue of packets that are waiting to be acknowledged by the DataNodes, called the ack queue. A packet is removed from the ack queue only when it has been acknowledged by all the DataNodes in the pipeline.
6. When the client has finished writing data, it calls `close()` on the `DFSOutputStream`. This flushes all the remaining packets to the DataNodes and waits for the acknowledgments. Then, the `DFSOutputStream` tells the NameNode that the file is complete. The NameNode commits the file creation operation to its persistent store.

- **HDFS read operation:**

1. The client opens the file by calling `open()` on the DFS object, which returns a `DFSInputStream` object to read data from the file.
2. The `DFSInputStream` communicates with the NameNode to obtain the list of blocks and their locations for the file. The locations of each block are ordered by their proximity to the client.

3. The client reads data from the closest DataNode for each block. If the read fails or the DataNode is not available, the client tries the next DataNode in the list.
4. The client can also perform seek operations to move to a different position in the file. This may involve contacting the NameNode again to get the new locations of the blocks.

A mnemonic to remember the data flow in HDFS is:

- Write: Create, Queue, Stream, Ack, Close
- Read: Open, Locate, Read, Seek

Data ingest with Flume and Scoop in HDFS

- Data ingestion is the process of transferring data from various sources to a data storage system, such as HDFS (Hadoop Distributed File System).
- Flume and Scoop are two tools that can be used for data ingestion in HDFS.
- Flume is a distributed, reliable, and available service for efficiently collecting, aggregating, and moving large amounts of log data from various sources to HDFS.
- Scoop is a tool for transferring data between relational databases and HDFS using parallel processes.
- Flume and Scoop have different use cases and advantages depending on the type and source of data.

Flume

- Flume can ingest data from sources such as web servers, application servers, social media, sensors, etc. that generate streaming or event-based data.
- Flume has a flexible and scalable architecture that consists of three main components: sources, channels, and sinks.
- Sources are the components that consume data from the data sources and pass them to channels.
- Channels are the components that store the data temporarily and provide reliability and fault tolerance.
- Sinks are the components that deliver the data from channels to the destination, such as HDFS, Kafka, etc.
- Flume supports various types of sources, channels, and sinks that can be configured and customized according to the data ingestion requirements.
- Flume also supports interceptors and selectors that can be used to modify, filter, or route the data during the ingestion process.
- Flume can handle high volumes of data with high throughput and low latency.
- Flume can also handle complex and multi-hop data flows using agents and collectors.

- An agent is a JVM process that hosts the sources, channels, and sinks and runs on the nodes where the data is generated or collected.
- A collector is a special type of agent that receives data from other agents and delivers it to the final destination.
- Flume can be integrated with other tools such as Spark Streaming, Kafka, Hive, etc. for further processing and analysis of the ingested data.

Scoop

- Scoop can ingest data from sources such as MySQL, Oracle, PostgreSQL, etc. that store structured or semi-structured data in relational databases.
- Scoop can also export data from HDFS to relational databases.
- Scoop uses JDBC (Java Database Connectivity) to connect to the databases and perform the data transfer operations.
- Scoop can perform full or incremental data ingestion using various options and parameters.
- Scoop can also perform data partitioning, compression, and transformation during the ingestion process.
- Scoop can leverage the parallelism and scalability of Hadoop by launching multiple map tasks to transfer data in parallel.
- Scoop can also use the Hadoop security features such as Kerberos and encryption to ensure the data security and privacy.
- Scoop can be integrated with other tools such as Hive, Pig, Spark, etc. for further processing and analysis of the ingested data.

Mnemonics and learning tricks

- A possible mnemonic to remember the difference between Flume and Scoop is: Flume for streaming, Scoop for SQL.
- A possible learning trick to remember the Flume architecture is: Sources consume data, Channels store data, Sinks deliver data.
- A possible learning trick to remember the Scoop options and parameters is: Use `-help` to see the available options and parameters for each command.

Hadoop archives in HDFS

- Hadoop archives or HAR files are a file archiving facility that packs files into HDFS blocks more efficiently, thereby reducing Name Node memory usage while still allowing transparent access to files .
- Hadoop archives can be used as input to MapReduce jobs by specifying a different input filesystem than the default file system, such as `har:///user/zoo/foo.har` .
- Hadoop archives are created from a collection of files and the archiving tool (a simple command) will run a MapReduce job to process the input files in parallel and create an archive file.
- Hadoop archives have a hierarchical structure that consists of an index file, a master index file, and data files . The index file contains the metadata

of the files in the archive, such as name, size, and offset. The master index file contains the metadata of the index files, such as name, size, and offset. The data files contain the actual data of the files in the archive.

- Hadoop archives can be accessed using the `har://` scheme or the `hadoop archive` command . The `har://` scheme allows users to browse, read, and write files in the archive as if they were normal files in HDFS. The `hadoop archive` command allows users to create, list, and get files from the archive.
- Hadoop archives can be used to tackle the small files problem in Hadoop, which occurs when there are too many small files in HDFS that consume a lot of Name Node memory and degrade the performance of MapReduce jobs . By packing small files into larger blocks, Hadoop archives reduce the number of files and blocks in HDFS and improve the efficiency of MapReduce jobs.
- Hadoop archives have some limitations, such as not supporting append or overwrite operations, not preserving the permissions and ownership of the original files, and not being compatible with some HDFS features, such as snapshots, encryption zones, and erasure coding .

A possible mnemonic to remember the main features of Hadoop archives is:

Hierarchical structure **A**ccessible by `har://` scheme or `hadoop archive` command **R**educes Name Node memory usage and small files problem **F**ormed by a MapReduce job **I**nput to MapReduce jobs **L**imitations on append, overwrite, permissions, and compatibility **E**fficient packing of files into blocks

Hadoop I/O

- Hadoop I/O is a set of primitives for data input and output in Hadoop. It includes techniques such as data integrity, compression, serialization, and file formats that are suitable for large-scale data processing.
- Data integrity is the ability to detect and correct errors in data. Hadoop uses checksums to ensure data integrity at different levels, such as in HDFS blocks, MapReduce shuffle transfers, and SequenceFiles. Checksums are stored separately from data, and are verified by Hadoop before reading or writing data.
- Compression is the technique of reducing the size of data by removing redundancy. Compression can save disk space, network bandwidth, and I/O time. Hadoop supports various compression codecs, such as Gzip, Bzip2, Snappy, and LZO. Compression can be applied at different levels, such as in HDFS files, MapReduce intermediate outputs, and SequenceFiles. Compression can be either splittable or non-splittable, depending on whether the compressed data can be split into independent chunks for parallel processing.
- Serialization is the process of converting data structures into a binary for-

mat that can be stored or transmitted. Serialization is used by Hadoop to transfer data between nodes, and to store data in files. Hadoop has its own serialization framework, called Writable, that provides a compact and fast way of serializing data for MapReduce. Writable classes implement the Writable interface, which defines two methods: `readFields()` and `write()`. Hadoop provides a number of built-in Writable classes, such as `IntWritable`, `Text`, and `NullWritable`, and also allows users to define their own custom Writable classes.

- File formats are the ways of organizing data in files. Hadoop supports various file formats, such as plain text, binary, and structured. Some of the common file formats used by Hadoop are:
 - `SequenceFile`: A binary file format that stores key-value pairs in a sequence. `SequenceFiles` can be compressed, split, and indexed. They are suitable for storing intermediate outputs of MapReduce, or small files that are otherwise inefficient to store in HDFS.
 - `MapFile`: A directory of files that stores key-value pairs in a sorted order. `MapFiles` are built on top of `SequenceFiles`, and provide random access to the values by using an index file. They are suitable for storing lookup tables or inverted indexes that are used by MapReduce or other applications.
 - `Avro`: A binary file format that stores data in a schema-based way. `Avro` files have a self-describing schema that is stored with the data, and can be read by any program that supports `Avro`. `Avro` files can be compressed, split, and processed in parallel. They are suitable for storing complex or evolving data structures that are used by MapReduce or other applications.
- Some of the advantages of using Hadoop I/O are:
 - It provides a consistent and reliable way of handling data in Hadoop.
 - It optimizes the performance and efficiency of data processing in Hadoop.
 - It supports various types and formats of data that are common in big data scenarios.
 - It allows users to customize and extend the data I/O functionality according to their needs.
- Some of the disadvantages of using Hadoop I/O are:
 - It requires users to learn and use the specific APIs and conventions of Hadoop I/O.
 - It may not be compatible or interoperable with other data I/O frameworks or tools that are outside of Hadoop.
 - It may not support some of the advanced or specialized features or requirements of data I/O that are specific to certain domains or applications.

Compression in Hadoop io

- Compression is a technique to reduce the size of data by encoding it in a different format that uses fewer bits.
- Compression is useful in Hadoop for saving storage space, reducing network bandwidth, and improving performance of mapreduce jobs.
- Hadoop supports various compression codecs, such as DEFLATE, gzip, bzip2, LZO, LZ4, and Snappy. Each codec has different characteristics in terms of compression ratio, speed, and splittability.
- Splittability means the ability to split a compressed file into smaller chunks that can be processed independently by different map tasks. This is important for parallel processing and load balancing in Hadoop.
- Only bzip2 is splittable among the standard codecs, but it is also the slowest and most CPU-intensive. LZO, LZ4, and Snappy are fast and lightweight, but not splittable. DEFLATE and gzip are somewhere in between, but gzip is more widely supported and compatible.
- Hadoop provides a CodecFactory class that can detect the compression format of an input file based on its extension, and return the appropriate CompressionCodec object. For example:

```
CompressionCodecFactory factory = new CompressionCodecFactory(new Configuration());
CompressionCodec codec = factory.getCodec(inputPath); //inputPath is a Path object
```

- Hadoop also provides a CompressionInputStream and a CompressionOutputStream class that can be used to read and write compressed data from and to HDFS. For example:

```
//To read a compressed file
CompressionCodec codec = ... //get the codec as above
CompressionInputStream in = codec.createInputStream(fs.open(inputPath)); //fs is a FileSystem
//To write a compressed file
CompressionCodec codec = ... //get the codec as above
CompressionOutputStream out = codec.createOutputStream(fs.create(outputPath)); //fs is a FileSystem
```

- Hadoop allows users to configure the compression codec and format for map output and reduce output. This can be done by setting the following properties in the Configuration object:

```
//To enable map output compression
conf.setBoolean("mapreduce.map.output.compress", true);
//To specify the map output compression codec
conf.setClass("mapreduce.map.output.compress.codec", codecClass, CompressionCodec.class);
//To enable reduce output compression
conf.setBoolean("mapreduce.output.fileoutputformat.compress", true);
//To specify the reduce output compression codec
conf.setClass("mapreduce.output.fileoutputformat.compress.codec", codecClass, CompressionCodec.class);
//To specify the reduce output compression type
conf.set("mapreduce.output.fileoutputformat.compress.type", type); //type is either NONE, RLE, or DELTA
```

- A possible mnemonic to remember the compression codecs in Hadoop is: **Big Dogs Go Lick Lazy Snakes** (bzip2, DEFLATE, gzip, LZO, LZ4, Snappy).
- A possible learning trick to understand the trade-offs between compression codecs is: **The more you squeeze, the harder it gets** (higher compression ratio means slower speed and lower splittability).

Serialization in Hadoop io

- Serialization is the process of converting structured data (such as objects) into a byte stream for transmission over the network or permanent storage on disk .
- Deserialization is the reverse process of converting a byte stream back to the original structured data .
- Serialization is used in Hadoop for interprocess communication between nodes in the system using remote procedure calls (RPCs) and for storing data in HDFS or other file systems.
- Hadoop provides a mechanism for using different serialization frameworks by defining a list of **Serialization** classes that know how to create **Serializer** and **Deserializer** instances for different data types .
- The property **io.serializations** in the configuration file specifies the list of **Serialization** classes to be used by Hadoop .
- Hadoop also provides some built-in serialization frameworks, such as **Writable**, **Avro**, and **Protocol Buffers**.
- **Writable** is the default serialization framework in Hadoop that implements the **org.apache.hadoop.io.Writable** interface . It provides methods for writing and reading data to and from a **DataOutput** or **DataInput** stream .
- **Avro** is a serialization framework that supports schema evolution, meaning that the data can be read by older or newer versions of the schema. It also supports dynamic typing, meaning that the data types are not fixed at compile time.
- **Protocol Buffers** is a serialization framework that uses a predefined schema to generate code for different languages. It supports efficient binary encoding and decoding of data.
- Some advantages of using serialization in Hadoop are:
 - It reduces the network bandwidth and disk space required for data transmission and storage.
 - It enables interoperability between different languages and platforms.
 - It facilitates data processing and analysis by providing a common format for data representation.
- Some disadvantages of using serialization in Hadoop are:
 - It adds some overhead for serializing and deserializing data.
 - It may introduce compatibility issues if the serialization frameworks or schemas are changed.
 - It may require additional libraries or dependencies for using different

serialization frameworks.

- An example of using serialization in Hadoop is:

```
// Define a custom data type that implements Writable
public class Employee implements Writable {
    private int id;
    private String name;
    private double salary;

    // Default constructor
    public Employee() {}

    // Parameterized constructor
    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    // Getter and setter methods
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    // Implement the write method to serialize the object
    @Override
```

```

    public void write(DataOutput out) throws IOException {
        out.writeInt(id);
        out.writeUTF(name);
        out.writeDouble(salary);
    }

    // Implement the readFields method to deserialize the object
    @Override
    public void readFields(DataInput in) throws IOException {
        id = in.readInt();
        name = in.readUTF();
        salary = in.readDouble();
    }
}

// Create an instance of Employee and serialize it to a file
Employee emp = new Employee(101, "Alice", 5000.0);
Configuration conf = new Configuration();
FileSystem fs = FileSystem.get(conf);
Path path = new Path("/user/hadoop/emp.ser");
DataOutputStream dos = fs.create(path);
emp.write(dos);
dos.close();

// Read the serialized file and deserialize it to an Employee object
DataInputStream dis = fs.open(path);
Employee emp2 = new Employee();
emp2.readFields(dis);
dis.close();
System.out.println(emp2.getId() + " " + emp2.getName() + " " + emp2.getSalary());
[assistant](

```

Avro and file based data structures in Hadoop io

- Avro is a data serialization framework that is widely supported throughout Hadoop and its ecosystem .
- Avro creates a data file where it keeps data along with schema in its metadata section, which makes the file self-describing .
- Avro data files support compression and are splittable, which is crucial for a MapReduce data input format.
- Avro data files are line-oriented, meaning the value of each row in the file are stored consecutively.
- Avro provides a rich data structure which makes it more popular than other similar solutions.
- Avro is the best fit for Big Data processing and is quite popular in Hadoop

and Kafka world for its faster processing.

- Avro also supports schema evolution, which means the schema can be changed without breaking the compatibility with existing data.

Some advantages of using Avro are:

- It reduces the size of the data by using binary format and compression.
- It preserves the schema information in the file, which makes it easy to read and write data without any external schema files.
- It allows schema evolution, which means the schema can be modified without affecting the existing data or applications.
- It provides a rich set of data types, such as arrays, maps, unions, enums, fixed, etc.
- It integrates well with Hadoop and its ecosystem, such as Sqoop, Flume, Pig, Hive, Spark, etc.

Some disadvantages of using Avro are:

- It requires a schema to read and write data, which may not be suitable for some use cases where the schema is not known in advance or is dynamic.
- It does not support nested data structures, such as nested arrays or maps, which may limit its expressiveness.
- It does not provide any built-in indexing or querying capabilities, which may require additional tools or frameworks to perform data analysis.

Some examples of using Avro are:

- Importing and exporting data from RDBMS to HDFS using Sqoop in Avro format.
- Producing and consuming data from Kafka topics using Avro serializers and deserializers.
- Writing and reading data from HDFS using Avro data files and Avro input and output formats.
- Performing data transformations and analysis using Pig, Hive, or Spark with Avro data files .

Some mnemonics and learning tricks for Avro and file based data structures in Hadoop are:

- Avro is a **A**pproach to **V**alidate and **R**epresent **O**bjects in Hadoop.
- Avro data files are **S**elf-describing, **S**plittable, and **S**chema-evolving, which makes them **S**uperior to other formats.
- Avro data types are **A**rrays, **M**aps, **U**nions, **E**nums, **F**ixed, and **R**ecords, which can be remembered as **AMUEFR**.

Hadoop Environment

- Hadoop is an open source software framework that allows for the distributed storage and processing of large datasets across clusters of computers using simple programming models .

- Hadoop requires the Java Runtime Environment (JRE) 1.6 or higher and Secure Shell (SSH) set up between nodes in the cluster.
- Hadoop consists of four main components: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN.
- Hadoop Common is a set of utilities and libraries that support other Hadoop modules.
- HDFS is a distributed file system that provides high-throughput access to application data and can store large amounts of data across multiple machines.
- Hadoop MapReduce is a programming model and software framework for writing applications that process large amounts of data in parallel on clusters of nodes.
- Hadoop YARN is a resource management platform that manages computing resources in clusters and schedules applications to run on them.
- Hadoop also has a rich ecosystem of related projects that extend its functionality, such as Apache Hive, Apache Pig, Apache Spark, Apache HBase, Apache ZooKeeper, Apache Sqoop, Apache Flume, Apache Oozie, and Apache Mahout.
- Hadoop can be used for various applications, such as data warehousing, data mining, data analysis, machine learning, web indexing, and log processing.
- Hadoop can handle structured, semi-structured, and unstructured data, such as text, images, audio, video, and XML.
- Hadoop can scale up from single servers to thousands of machines, each offering local computation and storage.
- Hadoop can provide fault tolerance, reliability, and availability by replicating data blocks across different nodes and automatically recovering from failures.
- Hadoop can reduce the cost of data processing by using commodity hardware and open source software.

Some mnemonics and learning tricks for Hadoop Environment are:

- Hadoop = HDFS + MapReduce + YARN + Common
- HDFS = High-throughput Distributed File System
- MapReduce = Map + Reduce + Shuffle + Sort
- YARN = Yet Another Resource Negotiator
- Hive = SQL-like queries on Hadoop
- Pig = Scripting language for Hadoop
- Spark = Fast and general engine for large-scale data processing
- HBase = NoSQL database on Hadoop
- ZooKeeper = Coordination service for distributed applications
- Sqoop = SQL to Hadoop and Hadoop to SQL
- Flume = Data ingestion service for Hadoop
- Oozie = Workflow scheduler for Hadoop
- Mahout = Machine learning library for Hadoop

Setting up a Hadoop cluster in Hadoop Environment

A Hadoop cluster is a collection of machines that run the Hadoop software and store the data in a distributed manner. A Hadoop cluster can be configured in different modes, such as standalone, pseudo-distributed, or fully distributed. The following steps describe how to set up a Hadoop cluster in a fully distributed mode, which is the most common and scalable way to run Hadoop applications.

1. Install the required software on all the nodes in the cluster, such as Java, SSH, and Hadoop. You can download the Hadoop software from the official website: <https://hadoop.apache.org/releases.html>. Make sure all the nodes have the same version of the software and the same configuration files.
2. Configure the environment variables for the Hadoop daemons, such as `HADOOP_HOME`, `HADOOP_CONF_DIR`, `JAVA_HOME`, etc. You can edit the `~/.bashrc` file or the `/etc/profile` file to set these variables. You also need to add the Hadoop bin directory to the `PATH` variable.
3. Configure the Hadoop parameters for the cluster, such as the cluster name, the namenode address, the datanode directories, the replication factor, the memory and CPU allocation, etc. You can edit the following files in the `HADOOP_CONF_DIR` directory: `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, and `yarn-site.xml`. You can refer to the official documentation for the details of each parameter: <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/ClusterSetup.html>.
4. Configure the SSH access for the Hadoop daemons, such as the namenode, the secondary namenode, the datanodes, the resourcemanager, the nodemanagers, and the webappproxy. You need to generate a SSH key pair on the master node and copy the public key to the `authorized_keys` file on all the slave nodes. You also need to disable the password authentication and enable the public key authentication in the `/etc/ssh/sshd_config` file on all the nodes. You can use the following commands to test the SSH access:

```
# On the master node
ssh-keygen -t rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 0600 ~/.ssh/authorized_keys
# Copy the public key to all the slave nodes
for host in $(cat $HADOOP_CONF_DIR/slaves); do
    ssh-copy-id -i ~/.ssh/id_rsa.pub $host
done
# Test the SSH access
for host in $(cat $HADOOP_CONF_DIR/slaves); do
    ssh $host hostname
done
```

5. Format the HDFS namenode on the master node. This step is required only for the first time you set up the cluster. You can use the following command to format the namenode as the hdfs user:

```
# On the master node
su - hdfs
hdfs namenode -format
```

6. Start the Hadoop daemons on the cluster. You can use the following commands to start the HDFS and YARN daemons as the hdfs and yarn users respectively:

```
# On the master node
su - hdfs
start-dfs.sh
su - yarn
start-yarn.sh
```

7. Verify the status of the Hadoop cluster. You can use the following commands to check the running processes and the logs of the Hadoop daemons:

```
# On any node
jps
# On the master node
hdfs dfsadmin -report
yarn node -list
# On any node
tail -f $HADOOP_LOG_DIR/*.log
```

You can also use the web interfaces to monitor the cluster, such as the namenode web UI (<http://:9870>), the resourcemanager web UI (<http://:8088>), and the job history server web UI (<http://:19888>).

8. Run some Hadoop applications on the cluster. You can use the following commands to run some example applications, such as the wordcount and the pi programs:

```
“‘bash # On the master node su - hdfs hdfs dfs -mkdir /user/hdfs hdfs dfs
-put
```

Cluster specification in Hadoop Environment

- A Hadoop cluster is a special type of computational cluster designed specifically for storing and processing large amounts of data.
- A Hadoop cluster is often referred to as a shared-nothing system because the only thing that is shared is the network.
- A Hadoop cluster consists of a network of master and slave nodes that are connected to each other.
- The main components of a Hadoop cluster are:

- Hadoop Distributed File System (HDFS): A distributed file system that provides high-throughput access to application data.
- MapReduce: A programming model and framework for parallel data processing on large-scale clusters.
- YARN: A resource management layer that allocates and manages the cluster resources for other daemons to run.
- The types of Hadoop clusters are:
 - Single-node cluster: A cluster with only one node that acts as both master and slave. This is the simplest and most basic type of Hadoop cluster.
 - Multi-node cluster: A cluster with more than one node that are configured as master and slave nodes.
 - Pseudo-distributed cluster: A cluster with only one node that acts as both master and slave. This is a simulated distributed environment.
- A possible mnemonic to remember the components of a Hadoop cluster is: ****H**ave **M**any **S**laves**

Cluster setup and installation in Hadoop Environment

- A Hadoop cluster is a collection of machines that run the Hadoop software and store and process data.
- A Hadoop cluster consists of one or more master nodes and multiple worker nodes.
- The master nodes are responsible for managing the cluster resources, coordinating the data, and scheduling tasks.
- The worker nodes are responsible for storing the data blocks and executing the tasks assigned to them.
- The Hadoop software includes the following components:
 - Hadoop Common: The common utilities and libraries that support the other Hadoop modules.
 - Hadoop Distributed File System (HDFS): The distributed file system that stores the data.
 - Hadoop YARN: The resource management framework that allocates the cluster resources and schedules tasks.
 - Hadoop MapReduce: The programming model and execution engine that processes the data in parallel.
- To set up a Hadoop cluster, you need to perform the following steps:
 - Install the required software on all the nodes in the cluster, such as Java, SSH, and Hadoop.
 - Configure the Hadoop environment variables and properties on all the nodes, such as HADOOP_HOME, HADOOP_CONF_DIR, and HADOOP_OPTS.
 - Set up the SSH access between the nodes, so that the master nodes can communicate with the worker nodes.
 - Format the HDFS on the node designated as the NameNode, which will initialize the file system.
 - Start the HDFS daemons on the master and worker nodes, such as NameNode, DataNode, and SecondaryNameNode.
 - Start the YARN daemons on the master and worker nodes, such as ResourceManager and NodeManager.
 - Start the MapReduce daemons on the master node, such as JobTracker and TaskTracker.
 - Verify the cluster status and functionality using the web interfaces, command-line tools, and logs.
- A possible mnemonic to remember the steps of cluster setup and installation is:
 - Install, Configure, SSH, Format, Start, Verify (ICSSFSV)

Hadoop configuration in Hadoop Environment

- Hadoop configuration is the process of setting up the parameters and properties of the Hadoop system.
- Hadoop configuration files are XML files that contain the key-value pairs of the configuration parameters.
- The main Hadoop configuration files are:
 - core-site.xml: This file contains the core settings of the Hadoop system, such as the default filesystem, the Hadoop home directory, and the logging configuration.
 - hdfs-site.xml: This file contains the settings of the Hadoop Distributed File System (HDFS), such as the replication factor, the block size, and the namenode address.

- mapred-site.xml: This file contains the settings of the MapReduce framework, such as the
- yarn-site.xml: This file contains the settings of the Yet Another Resource Negotiator (YARN)
- To configure Hadoop in a Hadoop environment, the following steps are required:
 - Edit the configuration files according to the desired settings and the cluster specific
 - Copy the configuration files to all the nodes in the cluster. This can be done using the
 - Restart the Hadoop services on all the nodes. This can be done using the start-all.sh script
- Some mnemonics and learning tricks for Hadoop configuration are:
 - Remember the four main configuration files by the acronym CHMY: Core, HDFS, MapReduce, YARN
 - Remember the default configuration directory by the acronym CHEC: /etc/hadoop/conf.
 - Remember the default file system URI by the acronym HDFS: hdfs://namenode:port.
 - Remember the default replication factor by the number 3: dfs.replication=3.

Security in Hadoop in Hadoop Environment

- Security in Hadoop is the process of protecting the data and resources of a Hadoop cluster
- Security in Hadoop can be divided into four pillars: authentication, authorization, encryption, and audit
- Authentication is the process of verifying the identity of a user or a service before allowing access
- Authorization is the process of granting or denying access to specific data or resources based on the user's role
- Encryption is the process of transforming data into an unreadable form to prevent unauthorized access
- Audit is the process of recording and monitoring the activities and events that occur in the cluster
- A mnemonic to remember the four pillars of security in Hadoop is **AAEA** (Authentication, Authorization, Encryption, Audit)
- A learning trick to understand the difference between authentication and authorization is to remember that authentication is about who you are, and authorization is about what you can do
- Some advantages of security in Hadoop are:
 - It protects the data and resources of the Hadoop cluster from unauthorized access, modification, and deletion
 - It ensures the compliance with the regulatory and legal requirements for data privacy and protection
 - It enhances the trust and confidence of the users and customers who use or consume the data and services
- Some disadvantages of security in Hadoop are:
 - It adds complexity and overhead to the Hadoop cluster, which can affect the performance and scalability
 - It requires additional configuration and management of the security components and policies
 - It may not be compatible or interoperable with some legacy or third-party applications and tools

Administering Hadoop in Hadoop Environment

- Hadoop is a framework for distributed processing of large-scale data sets across clusters of commodity hardware
- Hadoop administration is the process of managing and maintaining the Hadoop clusters and their components
- Hadoop administration involves the following tasks:
 - Installing and configuring Hadoop components and services on the cluster nodes.
 - Setting up and securing Hadoop users and groups.

- Monitoring and troubleshooting the cluster performance and health.
- Managing the Hadoop file system (HDFS) and the data replication and availability.
- Scheduling and executing Hadoop jobs and applications using MapReduce, YARN, or other frameworks.
- Tuning the cluster parameters and the Hadoop daemons for optimal performance and resource utilization.
- Applying patches and updates to the Hadoop software and the operating system.
- Handling the backup and recovery of the cluster data and metadata.
- Scaling the cluster up or down as per the business needs and the data growth.
- Hadoop administration requires the following skills and knowledge:
 - A good understanding of the Hadoop architecture and the components such as HDFS, MapReduce, YARN, and Hadoop Common.
 - A strong background in Linux/Unix administration and shell scripting.
 - A familiarity with the Java programming language and the Hadoop APIs.
 - A proficiency in using the Hadoop command-line tools and the web-based interfaces such as the Hadoop Distributed Shell (HDFS Shell) and the Hadoop Job Tracker Web UI.
 - A working knowledge of the network and security concepts and protocols such as SSH, Kerberos, and SSL.
 - A basic knowledge of the data analysis and processing techniques and tools such as SQL, Pig, and Hive.
- Hadoop administration can be performed using the following steps:
 - Setting up the Hadoop environment variables in the ~/.bashrc file or the etc/hadoop/hadoop-env.sh file.
 - Configuring the Hadoop cluster parameters in the etc/hadoop/core-site.xml, etc/hadoop/hdfs-site.xml, and etc/hadoop/yarn-site.xml files.
 - Starting and stopping the Hadoop daemons using the bin/hadoop-daemon.sh, bin/hdfs, bin/yarn, and bin/mapred commands.
 - Managing the HDFS using the bin/hdfs dfs or bin/hadoop fs commands. These commands can be used to create, delete, and manage files and directories in the HDFS.
 - Running the Hadoop jobs and applications using the bin/hadoop, bin/yarn, or bin/mapred commands.
 - Monitoring the Hadoop cluster using the web-based interfaces or the command-line tools.
 - Tuning the Hadoop cluster using the configuration files, the command-line options, or the Hadoop Tuning Guide.

HDFS monitoring & maintenance in Hadoop Environment

- HDFS (Hadoop Distributed File System) is a distributed file system that stores large data
- NameNode is the master node that manages the file system metadata, such as the directory t
- HDFS monitoring is the process of checking the health and performance of the HDFS cluster
- HDFS maintenance is the process of repairing and optimizing the HDFS cluster, such as the
- HDFS monitoring and maintenance are essential for ensuring the reliability, scalability, a
- Some of the common tools and commands for HDFS monitoring and maintenance are :
 - ****hdfs fsck****: This command checks the consistency and validity of the HDFS files and b
 - ****hdfs dfsadmin****: This command performs various administrative tasks on the HDFS cluste
 - ****hdfs balancer****: This command balances the disk space usage across the DataNodes by mo
 - ****hdfs snapshot****: This command creates a point-in-time copy of a directory or a file in
 - ****hdfs distcp****: This command copies large amounts of data from one HDFS cluster to anot
 - ****hdfs fetchdt****: This command gets a delegation token from the NameNode, which can be u
 - ****Hadoop web UI****: This is a graphical interface that provides a comprehensive view of t
 - ****Hadoop JMX****: This is a Java-based interface that exposes various management and monit
 - ****Hadoop Metrics2****: This is a framework that collects and publishes various performance
 - ****Hadoop Audit Logs****: These are logs that record the file system activities of the HDFS

Hadoop benchmarks in Hadoop Environment

- Hadoop benchmarks are tests that measure the performance of various aspects of the Hadoop ecosystem.
- Hadoop benchmarks can help users to evaluate the hardware and software configurations of their Hadoop clusters.
- Hadoop benchmarks can be classified into two types: built-in benchmarks and external benchmarks.
- Built-in benchmarks are those that are included in the Hadoop distribution and can be run directly from the Hadoop command-line interface.
 - TestDFSIO: This benchmark tests the read and write performance of HDFS by generating and reading large amounts of data.
 - TeraSort: This benchmark tests the sorting performance of MapReduce by generating, sorting, and reducing a large amount of data.
 - PiEstimator: This benchmark tests the computation performance of MapReduce by estimating the value of pi.
 - NNbench: This benchmark tests the performance of the NameNode by creating, renaming, and deleting a large number of files.
- External benchmarks are those that are developed by third-party organizations or researchers.
 - HiBench: This benchmark suite covers a wide range of Hadoop workloads, such as micro-benchmarks, streaming, and analytics.
 - BigDataBench: This benchmark suite focuses on big data analytics workloads, such as classification, regression, and clustering.
 - GridMix: This benchmark suite simulates a mix of realistic Hadoop workloads by replaying real-world Hadoop logs.
- Hadoop benchmarks can be run in two modes: standalone mode or remote mode. Standalone mode runs the benchmarks on the local Hadoop cluster, while remote mode runs the benchmarks on a remote Hadoop cluster.
- Hadoop benchmarks can be customized by using the Hadoop command-line Generic Options, such as `-D`, `-x`, and `-y`.
- Hadoop benchmarks can be analyzed by using various tools or metrics, such as the Hadoop Performance Analyzer (HADOOP-PERF) or the Hadoop Performance Metrics (HADOOP-PERF-METRICS).

Hadoop in the cloud in Hadoop Environment

- Hadoop is an open source framework that allows for the distributed storage and processing of large amounts of data.
- Hadoop in the cloud refers to running Hadoop clusters on cloud platforms such as Google Cloud Platform, Amazon Web Services, and Microsoft Azure.
- Hadoop in the cloud has several advantages over traditional on-premises Hadoop environment.
 - Lower upfront and operational costs, as cloud providers offer pay-as-you-go pricing models.
 - Higher scalability and elasticity, as cloud providers allow users to provision and deprovision resources on-demand.
 - Faster data ingestion and processing, as cloud providers offer various data sources and connectors.
 - Improved data security and compliance, as cloud providers offer various features and tools to protect data.
- Hadoop in the cloud also has some challenges and trade-offs, such as:
 - Data transfer and latency, as moving large amounts of data between on-premises and cloud can be expensive and slow.
 - Data governance and quality, as managing and ensuring the consistency and accuracy of data in the cloud can be challenging.
 - Data sovereignty and privacy, as storing and processing data in cloud regions that are subject to different laws and regulations.
 - Vendor lock-in and compatibility, as relying on specific cloud providers and services can limit flexibility.
- Hadoop in the cloud requires a different approach and mindset from running Hadoop on-premises.
 - Shifting from monolithic, persistent clusters to specialized, ephemeral clusters, that are optimized for specific workloads.
 - Leveraging cloud-native services and tools to complement or replace Hadoop components, such as object storage, managed services, and serverless computing.
 - Adopting best practices and patterns for cloud-based data architectures, such as microservices, data lakes, and data warehouses.
- Hadoop in the cloud is a viable and beneficial option for many use cases and scenarios involving big data.
 - Exploratory data analysis and experimentation, as cloud platforms offer flexibility and scalability for running ad-hoc queries and analyses.
 - Batch and stream processing, as cloud platforms offer high performance and scalability for running large-scale data processing jobs.
 - Machine learning and artificial intelligence, as cloud platforms offer various frameworks and tools for training and deploying models.
 - Data integration and transformation, as cloud platforms offer various tools and pipelines for moving and processing data between different systems.

Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala

- Hadoop is a framework for distributed storage and processing of large-scale data sets using
- Hadoop Ecosystem is a collection of tools and applications that interact with the core components
- YARN (Yet Another Resource Negotiator) is a component of Hadoop that provides resource management
- NoSQL databases are databases that do not follow the relational model and do not use SQL as a query language
- MongoDB is a popular NoSQL database that stores data as documents in JSON-like format. MongoDB is a cross-platform, open-source, document-oriented database.
- Spark is a fast and general-purpose framework for large-scale data processing. Spark can run on a variety of hardware architectures and can be used for a wide range of applications.
- Scala is a programming language that combines the features of object-oriented and functional programming.

Hadoop Eco System and YARN

- Hadoop Eco System is a collection of open source projects and tools that work together to process and analyze data.
- Hadoop Eco System consists of four core components: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common.
- HDFS is a distributed file system that stores large volumes of data across multiple nodes in a cluster.
- MapReduce is a programming model and framework for parallel processing of data using key-value pairs.
- YARN is a resource management and scheduling layer that allocates and manages resources for applications running on Hadoop.
- Hadoop Common is a set of libraries and utilities that support the other components of Hadoop.
- Some of the most well-known tools of the Hadoop Eco System include Hive, Pig, Spark, HBase, Mahout, and Tez.
- Hive is a data warehouse system that provides a SQL-like interface for querying and analyzing data stored in HDFS.
- Pig is a scripting language and platform for data transformation and analysis using MapReduce.
- Spark is a fast and general engine for large-scale data processing that supports batch, streaming, and interactive queries.
- HBase is a distributed and scalable NoSQL database that provides random access and strong consistency.
- Oozie is a workflow scheduler and coordinator that manages and executes Hadoop jobs.
- Sqoop is a tool for transferring data between HDFS and relational databases.
- Zookeeper is a service for maintaining configuration information, naming, synchronization, and distributed coordination.
- YARN stands for Yet Another Resource Negotiator and it is the second generation of Hadoop.
- YARN was introduced in Hadoop 2.0 to overcome the limitations of Hadoop 1.0 MapReduce, such as its inability to support multiple types of applications.
- YARN splits the functionalities of resource management and job scheduling/monitoring into two separate components: ResourceManager and ApplicationMaster.
- ResourceManager is a global daemon that runs on a master node and is responsible for managing the cluster's resources.
- ApplicationMaster is a per-application daemon that runs on a worker node and is responsible for managing the application's execution.
- A container is a unit of resource allocation that consists of a certain amount of memory, CPU, and other resources.
- An application is either a single job or a DAG (directed acyclic graph) of jobs that run on containers.
- YARN supports multiple types of applications, such as MapReduce, Spark, Hive, Pig, etc., by using different ApplicationMasters.
- YARN also supports dynamic resource allocation, node labels, security, high availability, and other features.
- A possible mnemonic to remember the four core components of Hadoop Eco System is: **H**adoop, **M**apReduce, **Y**ARN, and **H**adoop Common.
- A possible mnemonic to remember the main functions of ResourceManager and ApplicationMaster is: **R**esourceManager and **A**pplicationMaster.

Hadoop ecosystem components

Hadoop ecosystem is a collection of software components and tools that are used to process,

- **HDFS**: Hadoop Distributed File System is the backbone of Hadoop that runs on Java language.
- **MapReduce**: MapReduce is a programming model and a framework for processing large-scale data.
- **YARN**: Yet Another Resource Negotiator is a resource management layer that allocates the resources.
- **Hive**: Hive is a data warehouse system that provides a SQL-like interface for querying the data.
- **Pig**: Pig is a data flow language and a platform for analyzing large data sets stored in HDFS.
- **Spark**: Spark is a fast and general-purpose engine for large-scale data processing. It runs on YARN.
- **HBase**: HBase is a distributed and scalable NoSQL database that stores data in HDFS. It runs on YARN.
- **Sqoop**: Sqoop is a tool that transfers data between Hadoop and relational databases. It runs on the client.
- **Flume**: Flume is a tool that collects, aggregates, and moves large amounts of streaming data into HDFS.
- **Oozie**: Oozie is a workflow scheduler that coordinates and executes the Hadoop jobs. It runs on the client.

Schedulers in Hadoop ecosystem

- Schedulers are algorithms that manage the execution of tasks on a Hadoop cluster, based on the resources available.
- Schedulers help in ensuring optimal utilization of the resources and the access to the underlying hardware.
- There are mainly three types of schedulers in Hadoop: FIFO, Capacity, and Fair.

FIFO Scheduler

- FIFO (First In First Out) is the default scheduling policy used in Hadoop.
- FIFO Scheduler gives more preference to the application coming first than those coming later.
- It places the applications in a queue and executes them in the order of their submission.
- FIFO Scheduler is simple and easy to implement, but it does not consider the priority or the size of the applications.
- FIFO Scheduler can cause starvation for the applications that are submitted later, especially if the cluster is overloaded.

Capacity Scheduler

- Capacity Scheduler is designed to run multiple applications on a Hadoop cluster, with different priorities.
- Capacity Scheduler divides the cluster into multiple queues, each with a configurable capacity.
- Each queue can have sub-queues to further divide the resources among different users or groups.
- Capacity Scheduler allocates resources to the applications in a queue based on their priority.
- Capacity Scheduler supports elasticity, security, and multi-tenancy.

Fair Scheduler

- Fair Scheduler is another scheduler that supports running multiple applications on a Hadoop cluster.

- Fair Scheduler assigns resources to applications such that each application gets an equal
- Fair Scheduler also allows the creation of multiple queues, each with a minimum and maximum
- Fair Scheduler dynamically adjusts the resource allocation based on the demand and the av

Mnemonics and learning tricks

- A possible mnemonic to remember the three types of schedulers in Hadoop is ****F**air **C**apacity**
- A possible learning trick to compare the three types of schedulers in Hadoop is to use a r

Fair and Capacity in Hadoop Ecosystem

- Fair and Capacity are two types of schedulers in Hadoop YARN that are responsible for assi
- A scheduler is a kind of algorithm that we use to schedule tasks in a Hadoop cluster when
- The main difference between Fair and Capacity schedulers is how they balance the resources

Fair Scheduler

- Fair Scheduler allows YARN applications to justly share resources in large Hadoop clusters
- With this scheduler, there is no need for reserving a set amount of capacity because it w
- Fair Scheduler supports hierarchical queues, which can have different scheduling policies.
- Fair Scheduler also supports preemption, which means that it can reclaim resources from s
- Fair Scheduler can be configured by using a file called fair-scheduler.xml, which defines

Capacity Scheduler

- Capacity Scheduler is designed to run Hadoop applications as a shared, multi-tenant cluste
- With this scheduler, the available resources in the Hadoop cluster are partitioned among m
- Capacity Scheduler supports multiple queues, each of which is guaranteed a minimum share o
- Capacity Scheduler also supports elasticity, which means that an organization can access a
- Capacity Scheduler can be configured by using a file called capacity-scheduler.xml, which

Mnemonics and Learning Tricks

- A possible mnemonic to remember the difference between Fair and Capacity schedulers is: ****F**air **C**apacity**
- A possible learning trick to understand the concept of preemption in Fair Scheduler is to
- A possible learning trick to understand the concept of elasticity in Capacity Scheduler is

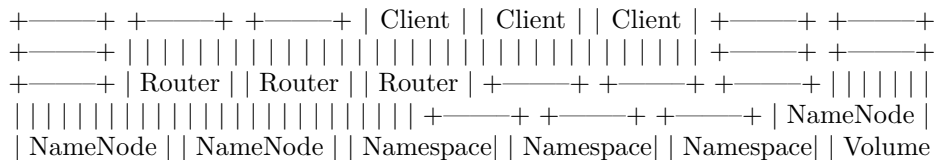
Hadoop 2.0 New Features - NameNode high availability

- NameNode is the master node in HDFS that maintains the filesystem tree and the metadata of
- In Hadoop 1.x, NameNode was a single point of failure (SPOF) in an HDFS cluster. If the Na
- Hadoop 2.0 overcomes this SPOF problem by providing support for multiple NameNodes. It int
- The Active NameNode is the one that serves the client requests and performs all the namesp
- The Standby NameNode synchronizes its state with the Active NameNode by reading the share
- The DataNodes send block reports and heartbeat messages to both the Active and the Standby
- The failover process can be triggered manually or automatically by using a ZooKeeper-based

- The advantages of Hadoop 2.0 High Availability feature are:
 - It eliminates the NameNode as a potential single point of failure in an HDFS cluster.
 - It provides a fast and automatic failover mechanism that minimizes the downtime and data loss.
 - It simplifies the administration and maintenance of HDFS clusters by avoiding the need for a secondary NameNode.
- The disadvantages of Hadoop 2.0 High Availability feature are:
 - It introduces additional complexity and overhead in the HDFS architecture and configuration.
 - It requires a reliable and secure shared storage system for the edit logs and a reliable network.
 - It may not guarantee a consistent view of the filesystem across the Active and the Standby NameNodes.
- A possible mnemonic to remember the main components of Hadoop 2.0 High Availability feature are:
 - **A**ctive NameNode
 - **S**tandby NameNode
 - **S**hared edit logs
 - **Z**ooKeeper
 - **Z**KFC
- **ASSZZ** (sounds like "as is")

HDFS federation in Hadoop ecosystem

- HDFS stands for Hadoop Distributed File System, which is a storage system for Hadoop that stores data in multiple machines.
- HDFS federation is a feature introduced in Hadoop 2.x that enhances the existing HDFS architecture by allowing multiple NameNodes to manage different namespaces within the same cluster.
- A NameNode is a master node that manages the metadata of the files and directories in HDFS.
- In the original HDFS architecture, there was only one NameNode/namespace for the entire cluster.
 - Single point of failure: If the NameNode fails, the entire cluster becomes inaccessible.
 - Scalability bottleneck: The NameNode has to store all the metadata in memory, which limits the number of files and directories that can be stored.
 - Performance bottleneck: The NameNode has to process all the requests from the clients and DataNodes.
- HDFS federation overcomes these limitations by allowing the use of more than one NameNode/namespace.
 - Isolation: Each NameNode/namespace operates independently and does not affect the availability of other namespaces.
 - Scalability: The cluster can store more files and blocks by distributing the metadata across multiple NameNodes.
 - Performance: The cluster can handle more concurrent requests by load balancing the traffic across multiple NameNodes.
- The HDFS federation architecture consists of the following components:
 - Namespace volume: A self-contained management unit that consists of a NameNode and a block of storage.
 - DataNode: A slave node that stores the blocks of data in the local disks. A DataNode can belong to multiple namespaces.
 - Client: A user or an application that accesses the files and directories in HDFS. A client can access files in any namespace.
 - Router: An optional component that acts as a proxy between the clients and the NameNodes.
- The following diagram illustrates the HDFS federation architecture:



and job scheduling .

- YARN consists of the following main components :
 - **Resource Manager:** It is the master daemon of YARN that runs on a dedicated machine. It is responsible for allocating resources to different applications and managing the cluster-level policies. It has two sub-components:
 - * **Scheduler:** It performs scheduling based on the resource requests from the applications and the available resources in the cluster. It is a pure scheduler that does not perform any monitoring or tracking of the applications. It supports different scheduling policies, such as FIFO, fair, and capacity.
 - * **Application Manager:** It is responsible for accepting the application submissions, negotiating the first container for the application, and launching the Application Master for each application. It also maintains the state of the applications and handles the failures of the Application Masters.
 - **Node Manager:** It is the slave daemon of YARN that runs on each node in the cluster. It is responsible for managing the containers, monitoring the resource usage, and reporting to the Resource Manager. It also executes the commands from the Application Master and the Resource Manager.
 - **Application Master:** It is the per-application daemon of YARN that runs in a container allocated by the Resource Manager. It is responsible for negotiating the resources from the Resource Manager, managing the life cycle of the tasks, and monitoring the progress and status of the application. It also communicates with the Node Manager to launch and stop the containers.
 - **Container:** It is the basic unit of resource allocation in YARN. It represents a fixed amount of memory, CPU, disk, and network bandwidth that can be used by an application. A container can run one or more tasks of an application, depending on the application logic and configuration.
- YARN supports different types of applications, not only MapReduce. For example, YARN can run Spark, Hive, Pig, and Tez applications. Each application has its own Application Master that implements the application-specific logic and interacts with the YARN framework.
- YARN provides the following benefits:
 - **Scalability:** YARN can scale up to thousands of nodes and handle millions of tasks, by distributing the resource management and job scheduling functions among multiple daemons.
 - **Efficiency:** YARN can utilize the cluster resources more efficiently, by allowing multiple applications to share the same resources and by supporting dynamic resource allocation based on the application needs and the cluster availability.
 - **Flexibility:** YARN can support different types of applications and frameworks, by providing a common platform for resource manage-

ment and a pluggable interface for application-specific logic.

- **Compatibility:** YARN can run the existing MapReduce applications without any changes, by providing a backward-compatible MapReduce Application Master that mimics the behavior of the JobTracker.
- A possible mnemonic to remember the components of YARN is **RANC** (Resource Manager, Application Master, Node Manager, Container). A possible learning trick to understand the role of each component is to compare them with the roles in a school:
 - The Resource Manager is like the principal, who allocates the classrooms and teachers to different courses and enforces the school policies.
 - The Scheduler is like the timetable, which decides when and where each course will be held based on the course requests and the classroom availability.
 - The Application Manager is like the admission officer, who accepts the course enrollments, assigns the first classroom for each course, and launches the course coordinator for each course.
 - The Node Manager is like the janitor, who maintains the classrooms, monitors the resource usage, and reports to the principal. He also executes the commands from the course coordinator and the principal.
 - The Application Master is like the course coordinator, who negotiates the classrooms and teachers from the principal, manages the life cycle of the lessons, and monitors the progress and status of the course. He also communicates with the janitor to start and stop the classrooms.
 - The Container is like the classroom, which represents a fixed amount of space, furniture, equipment, and utilities that can be used by a course. A classroom can hold one or more lessons of a course, depending on the course logic and configuration.

Running MRv1 in YARN

- MRv1 stands for MapReduce version 1, which is the original framework for processing large-scale data sets in parallel using the map and reduce functions.
- YARN stands for Yet Another Resource Negotiator, which is the resource management layer of Hadoop that allows multiple applications to run on the same cluster and share resources dynamically.
- Running MRv1 in YARN means using YARN as the execution engine for MRv1 applications, instead of the default JobTracker and TaskTracker daemons that MRv1 uses.
- To run MRv1 in YARN, the following steps are required:
 - Enable the MRv1 compatibility mode in YARN by setting the `mapreduce.framework.name` property to `yarn` in the `mapred-site.xml` file.
 - Use the `yarn` command in the Hadoop-YARN bin folder to submit

- MRv1 applications, instead of the `hadoop` command. For example, `yarn jar wordcount.jar WordCount input output`.
- Monitor the MRv1 applications using the ResourceManager web interface, which shows the basic cluster metrics, list of applications, and nodes associated with the cluster. The ResourceManager web interface can be accessed at `http://<ResourceManager-Host>:8088`.
- Running MRv1 in YARN has the following advantages:
 - It allows MRv1 applications to coexist with other applications that use YARN, such as Spark, Hive, and Pig, and share the cluster resources efficiently.
 - It improves the scalability and reliability of MRv1 applications, as YARN can handle more concurrent jobs and recover from failures faster than the JobTracker and TaskTracker daemons.
 - It simplifies the migration from MRv1 to MRv2, which is the newer version of MapReduce that is designed to run on YARN natively. MRv2 has more features and performance improvements than MRv1, such as support for high-availability, speculative execution, and combiners.

NoSQL Databases

- NoSQL databases are non-relational databases that can store and process large amounts of unstructured or semi-structured data.
- NoSQL databases do not use tables, rows, and columns to store data, but rather use different data models, such as document, key-value, wide-column, and graph.
- NoSQL databases provide flexible schemas, meaning that the data structure can vary within the same database or collection, and can be changed without affecting the existing data.
- NoSQL databases are designed to scale horizontally, meaning that they can distribute the data across multiple servers or nodes, and handle high concurrency and availability demands.
- NoSQL databases are often used for applications that deal with dynamic, complex, or unpredictable data, such as social media, e-commerce, real-time analytics, IoT, etc.

Some examples of NoSQL databases are:

- MongoDB: A document-based database that stores data in JSON-like format and supports dynamic queries and aggregation.
- Redis: A key-value database that stores data in memory and supports various data structures, such as strings, lists, sets, hashes, etc.
- Cassandra: A wide-column database that stores data in tables with rows and columns, but allows each row to have a different set of columns and supports high availability and scalability.
- Neo4j: A graph database that stores data as nodes and relationships, and supports graph traversal and analysis.

Some advantages of NoSQL databases are:

- They can handle large volumes and variety of data with high performance and scalability.
- They can support flexible and agile development, as the data model can be changed without affecting the existing data or requiring complex migrations.
- They can support complex and rich data types and structures, such as nested documents, arrays, geospatial data, etc.
- They can support various query languages and paradigms, such as SQL-like, map-reduce, graph, etc.

Some disadvantages of NoSQL databases are:

- They may not provide full ACID (atomicity, consistency, isolation, durability) guarantees, meaning that the data may not be consistent or durable in some scenarios, such as network failures, concurrent updates, etc.
- They may not support complex joins or transactions across multiple collections or tables, meaning that the data may need to be denormalized or duplicated to avoid multiple queries or updates.
- They may not have a standard or unified query language or API, meaning that the developers may need to learn different syntaxes or tools for different NoSQL databases.
- They may not have a mature or comprehensive ecosystem, meaning that the NoSQL databases may lack some features or support that are available for relational databases, such as security, backup, monitoring, etc.

Some mnemonics and learning tricks for NoSQL databases are:

- NoSQL stands for Not Only SQL, meaning that NoSQL databases can support SQL-like queries, but also other types of queries and data models.
- NoSQL databases can be classified into four main types based on their data model: Document, Key-Value, Wide-Column, and Graph. A simple way to remember them is by using the acronym D-K-W-G.
- Document databases store data as documents, which are similar to JSON objects. A simple way to remember them is by using the word DOCS, which stands for Documents, Objects, Collections, and Schemaless.
- Key-Value databases store data as key-value pairs, which are similar to hash tables. A simple way to remember them is by using the word KEYS, which stands for Keys, Elements, Yields, and Speed.
- Wide-Column databases store data as tables with rows and columns, but allow each row to have a different set of columns. A simple way to remember them is by using the word WIDE, which stands for Wide, Indexed, Distributed, and Extensible.
- Graph databases store data as nodes and relationships, which are similar to graphs. A simple way to remember them is by using the word GRAP, which stands for Graph, Relationships, Attributes, and Paths.

Introduction to NoSQL databases

- NoSQL stands for **Not only SQL**, which means that it is a type of database that does not use the relational model and SQL language of traditional relational database management systems (RDBMS).
- NoSQL databases are designed to handle **large-scale, distributed, and unstructured** data that cannot be easily stored and queried using relational tables and schemas.
- NoSQL databases are also known as **non-relational, schema-less, or schema-free** databases, because they do not enforce a fixed and predefined structure for the data, but allow the data to be stored and accessed in various ways, such as key-value pairs, documents, graphs, or columns.
- NoSQL databases offer several advantages over relational databases, such as:
 - **Scalability:** NoSQL databases can scale horizontally by adding more nodes or servers to the cluster, without affecting the performance or availability of the data. This is also known as **sharding** or **partitioning**. Relational databases, on the other hand, can only scale vertically by upgrading the hardware of a single server, which is more costly and complex.
 - **Flexibility:** NoSQL databases can handle different types of data, such as structured, semi-structured, or unstructured data, without requiring a predefined schema or data model. This allows the data to evolve and change over time, without affecting the existing data or queries. Relational databases, on the other hand, require a fixed and rigid schema for the data, which can be difficult to modify or extend.
 - **Performance:** NoSQL databases can provide faster and more efficient data access and processing, especially for large and complex data sets, by using simpler and more expressive query languages, such as JSON, XML, or graph queries. Relational databases, on the other hand, use SQL, which can be more verbose and complicated for some types of data and queries.
 - **Availability:** NoSQL databases can ensure high availability and fault tolerance of the data, by replicating and distributing the data across multiple nodes or servers, which can operate independently and asynchronously. This is also known as **replication** or **sharding**. Relational databases, on the other hand, rely on a single master server or a cluster of synchronized servers, which can create a single point of failure or a bottleneck for the data.
- NoSQL databases are not a one-size-fits-all solution, but rather a family of different database systems that have different characteristics, features, and trade-offs. Some of the common types of NoSQL databases are:
 - **Key-value databases:** These databases store the data as a collection of key-value pairs, where each key is a unique identifier for a value, which can be any type of data, such as a string, a number, a bi-

nary object, or a JSON document. Examples of key-value databases are Redis, Riak, and DynamoDB.

- **Document databases:** These databases store the data as a collection of documents, where each document is a self-contained and semi-structured data unit, usually in a format such as JSON, XML, or BSON. Each document can have different fields and values, and can be nested or embedded within other documents. Examples of document databases are MongoDB, CouchDB, and Couchbase.
- **Column-family databases:** These databases store the data as a collection of columns, where each column is a group of related values that share a common key, and each column can have a different number and type of values. Each column can also be further divided into sub-columns or families, which can have different attributes and properties. Examples of column-family databases are Cassandra, HBase, and Bigtable.
- **Graph databases:** These databases store the data as a collection of nodes and edges, where each node is an entity that has a unique identifier and a set of properties, and each edge is a relationship that connects two nodes and has a type and a direction. Graph databases are useful for modeling and querying complex and interconnected data, such as social networks, recommendation systems, or knowledge graphs. Examples of graph databases are Neo4j, OrientDB, and Titan.

MongoDB

- MongoDB is an open source, nonrelational database management system (DBMS) that uses flexible documents instead of tables and rows to process and store various forms of data .
- MongoDB is a document database, which means it stores data in JSON-like documents with optional schemas . This allows for more flexibility and scalability than traditional relational databases that use fixed schemas and tables.
- MongoDB is a distributed database at its core, so high availability, horizontal scaling, and geographic distribution are built in and easy to use . MongoDB can run on multiple servers and replicate data across them for fault tolerance and load balancing.
- MongoDB supports ad-hoc queries, secondary indexing, and real-time aggregations to provide powerful ways to access and analyze your data . MongoDB also supports various programming languages and frameworks through its drivers and APIs.
- MongoDB is free to use under the Server Side Public License (SSPL), which is a copyleft license that requires users to make their source code available if they offer MongoDB as a service . MongoDB also offers a cloud-based version called MongoDB Atlas, which is a fully managed and secure database service.

Some possible mnemonics and learning tricks for MongoDB are:

- MongoDB stands for “humongous” database, which reflects its ability to handle large amounts of data.
- MongoDB documents are like JSON objects, but with some extra features such as binary data types and date types.
- MongoDB collections are like tables in relational databases, but without a predefined schema.
- MongoDB uses the BSON (Binary JSON) format to store and transfer data, which is more efficient and compact than plain JSON.
- MongoDB supports CRUD (Create, Read, Update, Delete) operations, which are the basic functions of any database.

Introduction to MongoDB

- MongoDB is a cross-platform, document-oriented database that works on the concept of collections and documents.
- MongoDB is a NoSQL database, which means it does not use the traditional relational model of tables, rows, and columns. Instead, it stores data as JSON-like documents with dynamic schemas, allowing for more flexibility and scalability.
- MongoDB is one of the most popular databases for modern web applications, as it can handle large volumes of data, support various data types, and provide high performance and availability.
- Some of the features of MongoDB are:
 - BSON: MongoDB uses a binary representation of JSON called BSON (Binary JSON) to store and manipulate data. BSON supports additional data types, such as date, binary, and decimal128.
 - Collections: MongoDB organizes data into collections, which are analogous to tables in relational databases. A collection can contain any number of documents, and each document can have a different structure or schema.
 - Documents: MongoDB stores data as documents, which are analogous to rows or records in relational databases. A document is a set of key-value pairs, where the value can be any BSON data type, including arrays, subdocuments, or references to other documents.
 - Indexes: MongoDB supports indexing on any field or combination of fields in a document, to improve the efficiency of queries. MongoDB also supports text, geospatial, and hashed indexes, as well as unique and sparse indexes.
 - Aggregation: MongoDB provides a powerful aggregation framework that allows for complex data analysis and transformation. The aggregation framework consists of a pipeline of stages, each of which performs an operation on the input documents and produces an out-

put for the next stage. Some of the common aggregation stages are match, project, group, sort, and unwind.

- Replication: MongoDB supports replication, which is the process of synchronizing data across multiple servers. Replication provides redundancy, fault tolerance, and high availability. MongoDB uses a replica set, which is a group of servers that maintain the same data set and elect a primary server to handle write operations. The other servers, called secondaries, replicate the data from the primary and can handle read operations.
 - Sharding: MongoDB supports sharding, which is the process of distributing data across multiple servers. Sharding allows for horizontal scaling, which means adding more servers to handle larger data sets and higher throughput. MongoDB uses a sharded cluster, which consists of three components: shards, mongos, and config servers. Shards are the servers that store the data, mongos are the routers that direct the queries to the appropriate shards, and config servers are the servers that store the metadata and configuration of the cluster.
- Some of the advantages of MongoDB are:
 - Schemaless: MongoDB does not enforce a fixed schema for the documents in a collection, which allows for more flexibility and adaptability to changing data requirements.
 - Scalable: MongoDB can scale horizontally by adding more servers to the cluster, which can handle larger data sets and higher throughput. MongoDB also supports auto-sharding, which automatically distributes the data across the shards based on a shard key.
 - Performant: MongoDB can provide high performance and low latency for read and write operations, as it uses memory-mapped files and in-memory computing. MongoDB also supports various query operators and indexes to optimize the queries.
 - Expressive: MongoDB can store and manipulate various data types, such as arrays, subdocuments, references, and binary data. MongoDB also supports rich and complex queries, such as text search, geospatial queries, and aggregation queries.
 - Some of the disadvantages of MongoDB are:
 - No joins: MongoDB does not support joins, which are a common feature of relational databases that allow for combining data from multiple tables. MongoDB relies on embedding or referencing data within documents, which can result in data duplication or inconsistency.
 - No transactions: MongoDB does not support transactions, which are a common feature of relational databases that allow for atomic and consistent operations on multiple documents. MongoDB relies on document-level atomicity, which means that an operation on a single document is either fully completed or not executed at all. However,

MongoDB does not guarantee atomicity across multiple documents or collections.

- No constraints: MongoDB does not support constraints, which are a common feature of relational databases that enforce rules on the data, such as uniqueness, foreign keys, and check constraints. MongoDB relies on the application logic to validate and enforce the data integrity.

Data types in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format called BSON (Binary JSON).
- BSON supports various data types, such as strings, numbers, booleans, arrays, objects, dates, binary data, object ids, regular expressions, code, symbols, timestamps, decimals, min key and max key.
- Some of the data types in MongoDB are:
 - **String:** This is the most common data type. It is used to store text values. It must be valid UTF-8. The maximum size of a string is 16 MB.
 - **Number:** This is used to store numeric values. There are two subtypes: NumberInt and NumberLong for 32-bit and 64-bit integers, and NumberDecimal for 128-bit decimal floating point values. NumberDecimal supports exact arithmetic and is useful for financial calculations.
 - **Boolean:** This is used to store true or false values.
 - **Array:** This is used to store an ordered list of values. An array can contain values of different types, including nested arrays. The maximum size of an array is 16 MB.
 - **Object:** This is used to store an embedded document, which is a set of key-value pairs. An object can contain values of different types, including nested objects. The maximum size of an object is 16 MB.
 - **Date:** This is used to store the date and time as a 64-bit integer that represents the number of milliseconds since the Unix epoch (1 January 1970). The date range is from 1 January 1970 to 19 January 2038.
 - **Binary Data:** This is used to store binary data, such as images, audio, video, etc. It can have a subtype that indicates the type of data. The maximum size of binary data is 16 MB.
 - **Object Id:** This is used to store a unique identifier for a document. It is a 12-byte value that consists of a 4-byte timestamp, a 5-byte random value, and a 3-byte incrementing counter.
 - **Regular Expression:** This is used to store a regular expression pattern that can be used for pattern matching. It is stored as a string with two options: i for case-insensitive matching and m for

multiline matching.

- **Code**: This is used to store JavaScript code that can be executed by the database. It can have an optional scope that is an object containing variables that are accessible by the code. The maximum size of code is 16 MB.
 - **Symbol**: This is used to store a string that is intended to be used as a symbol. It is similar to a string, but it is not indexed and can only be used in certain contexts. It is deprecated and should not be used.
 - **Timestamp**: This is used to store a special internal value that is used by MongoDB for replication and sharding. It is a 64-bit value that consists of a 32-bit seconds value and a 32-bit incrementing ordinal value. It is not the same as a date and should not be used by applications.
 - **Decimal**: This is used to store a 128-bit decimal floating point value that supports exact arithmetic. It is similar to NumberDecimal, but it is not a BSON type and can only be used in aggregation expressions.
 - **Min Key**: This is used to store a value that compares less than all other values. It is used for internal purposes and should not be used by applications.
 - **Max Key**: This is used to store a value that compares greater than all other values. It is used for internal purposes and should not be used by applications.
- A possible mnemonic to remember the data types in MongoDB is:
 - Strings are **S**imple and **S**tandard
 - Numbers are **N**ecessary and **N**umerous
 - Booleans are **B**asic and **B**inary
 - Arrays are **A**wesome and **A**bundant
 - Objects are **O**rganized and **O**riginal
 - Dates are **D**ynamic and **D**efined
 - Binary data are **B**ig and **B**old
 - Object ids are **O**utstanding and **O**ptimal
 - Regular expressions are **R**elevant and **R**efined
 - Code is **C**reative and **C**omplex
 - Symbols are **S**pecial

Creating documents in MongoDB

- MongoDB is a document-oriented database that stores data in collections of JSON-like documents.
- To create a document in MongoDB, you can use the `insertOne()` or `insertMany()` methods of the collection object.
- The `insertOne()` method takes a single document as an argument and inserts it into the collection. It returns a result object that contains the

_id field of the inserted document.

- The `insertMany()` method takes an array of documents as an argument and inserts them into the collection. It returns a result object that contains an array of `_id` fields of the inserted documents.
- If you do not specify an `_id` field for a document, MongoDB will automatically generate a unique `ObjectId` value for it.
- You can also use the `db.collection.insert()` method, which is a wrapper for `insertOne()` and `insertMany()`, depending on the argument type.
- To insert a document into a collection that does not exist, MongoDB will create the collection for you.
- Example: Inserting a document into the `users` collection using `insertOne()`:

```
db.users.insertOne({
  name: "Alice",
  age: 25,
  email: "alice@example.com"
})
```

- Result:

```
{
  "acknowledged" : true,
  "insertedId" : ObjectId("60f9f0b4a4f9c1f4f4f4f4")
}
```

- Example: Inserting multiple documents into the `users` collection using `insertMany()`:

```
db.users.insertMany([
  {
    name: "Bob",
    age: 28,
    email: "bob@example.com"
  },
  {
    name: "Charlie",
    age: 30,
    email: "charlie@example.com"
  }
])
```

- Result:

```
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("60f9f0b5a4f9c1f4f4f4f5"),
    ObjectId("60f9f0b5a4f9c1f4f4f4f6")
  ]
}
```

```
]
}
```

- Mnemonic: To remember the methods for creating documents in MongoDB, you can use the acronym **I'M IN**:
 - **I**nsertOne
 - **M**any
 - **I**nsert
 - **N**o collection, no problem

Updating documents in MongoDB

- MongoDB provides various methods to update documents in a collection, such as `updateOne()`, `updateMany()`, `replaceOne()`, and `bulkWrite()`.
- The `updateOne()` method updates a single document that matches the filter condition. It takes three parameters: the filter, the update, and an optional options object. The filter specifies the criteria for selecting the document to update. The update specifies the modifications to apply to the document. The options object can include parameters such as `upsert`, which creates a new document if none matches the filter, and `writeConcern`, which specifies the level of acknowledgment for the write operation.
- The `updateMany()` method updates all documents that match the filter condition. It takes the same parameters as `updateOne()`, but applies the update to multiple documents. The update can be either a replacement document or an update document that uses update operators such as `$set`, `$inc`, `$push`, etc.
- The `replaceOne()` method replaces a single document that matches the filter condition with a new document. It takes the same parameters as `updateOne()`, but the update parameter must be a replacement document, not an update document. The replacement document cannot contain update operators.
- The `bulkWrite()` method performs multiple write operations in bulk. It takes an array of write models, which specify the type of operation (`insertOne`, `updateOne`, `updateMany`, `replaceOne`, `deleteOne`, or `deleteMany`) and the document or documents involved. It also takes an optional options object, which can include parameters such as `ordered`, which determines whether the operations are executed in sequence or in parallel, and `writeConcern`, which specifies the level of acknowledgment for the write operation.
- Here is an example of using the `updateOne()` method to update the name field of a document in the `users` collection:

```

db.users.updateOne(
  { _id: ObjectId("60f9a3c9c9d77c23d8a9ce31") }, // filter
  { $set: { name: "Alice" } } // update
)

```

- Here is an example of using the `updateMany()` method to increment the age field of all documents in the users collection by 1:

```

db.users.updateMany(
  {}, // filter
  { $inc: { age: 1 } } // update
)

```

- Here is an example of using the `replaceOne()` method to replace a document in the users collection with a new document:

```

db.users.replaceOne(
  { _id: ObjectId("60f9a3c9c9d77c23d8a9ce31") }, // filter
  { name: "Alice", age: 25, email: "alice@example.com" } // replacement
)

```

- Here is an example of using the `bulkWrite()` method to perform multiple write operations in the users collection:

```

db.users.bulkWrite([
  { insertOne: { document: { name: "Bob", age: 30, email: "bob@example.com" } } }, // insert
  { updateOne: { filter: { name: "Alice" }, update: { $set: { age: 26 } } } }, // update the
  { deleteOne: { filter: { name: "Charlie" } } } // delete the document of Charlie
])

```

- Some mnemonics and learning tricks for updating documents in MongoDB are:
 - Remember the CRUD acronym: Create, Read, Update, Delete. These are the four basic operations that MongoDB supports. The update methods are part of the Update operation.
 - Remember the ONE acronym: One, Many, Replace. These are the three types of update methods that MongoDB provides. The `updateOne()` and `replaceOne()` methods affect one document, while the `updateMany()` method affects many documents.
 - Remember the SET acronym: Set, Increment, Push. These are some of the common update operators that MongoDB supports. The `$set` operator sets the value of a field, the `$inc` operator increments the value of a field, and the `$push` operator appends a value to an array field.
 - Remember the BOW acronym: Bulk, Ordered, Write. These are some of the key concepts of the `bulkWrite()` method. The `bulkWrite()` method performs bulk write operations, which can be ordered or unordered, and which return a write result object.

Deleting documents in MongoDB

- MongoDB provides two methods to delete documents from a collection: `db.collection.deleteOne()` and `db.collection.deleteMany()`.
- The `deleteOne()` method deletes the first document that matches the specified filter. The `deleteMany()` method deletes all documents that match the specified filter.
- Both methods return a `DeleteResult` object that contains the number of deleted documents and a boolean flag indicating whether the operation acknowledged by the server.
- To delete all documents from a collection, use `deleteMany()` method with an empty filter `{}`.
- To delete a single document by its `_id` field, use `deleteOne()` method with the `_id` value as the filter.
- To delete multiple documents that match a condition, use `deleteMany()` method with the condition as the filter.
- To delete documents in a transaction, use `session.withTransaction()` method and pass the delete operations as callbacks.

Examples

- Delete the first document where the field `status` is equal to "D":

```
db.orders.deleteOne({ status: "D" })
```

- Delete all documents where the field `status` is equal to "D":

```
db.orders.deleteMany({ status: "D" })
```

- Delete all documents from the `orders` collection:

```
db.orders.deleteMany({})
```

- Delete the document with the `_id` value of `ObjectId("5f6a4b8c9d1e4a3b3c9b0f88")`:

```
db.orders.deleteOne({ _id: ObjectId("5f6a4b8c9d1e4a3b3c9b0f88") })
```

- Delete all documents where the field `qty` is less than 20:

```
db.orders.deleteMany({ qty: { $lt: 20 } })
```

- Delete documents in a transaction:

```
// Start a session
const session = db.getMongo().startSession();

// Start a transaction
session.startTransaction();

// Delete one document from the inventory collection
session.withTransaction(() => {
  db.inventory.deleteOne({ sku: "abc123" }, { session });
});
```

```
});

// Delete many documents from the orders collection
session.withTransaction(() => {
  db.orders.deleteMany({ sku: "abc123" }, { session });
});

// Commit the transaction
session.commitTransaction();

// End the session
session.endSession();
```

Mnemonics and learning tricks

- Remember the CRUD acronym for the four basic operations on MongoDB collections: Create, Read, Update, and Delete.
- Remember the difference between `deleteOne()` and `deleteMany()` by their names: one deletes one document, many deletes many documents.
- Remember that both methods require a filter argument, which can be an empty object `{}` to match all documents.
- Remember that both methods return a `DeleteResult` object, which has two properties: `deletedCount` and `acknowledged`.
- Remember that to delete a document by its `_id`, you need to use the `ObjectId()` constructor to match the `_id` value.
- Remember that to delete documents in a transaction, you need to use a session object and the `withTransaction()` method.

Querying documents in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format.
- To query documents in MongoDB, you can use the `find()` method on a collection, which returns a cursor to the matching documents.
- The `find()` method takes two optional parameters: a query filter and a projection.
- The query filter specifies the conditions that the documents must match to be returned. It is a JSON object that can contain various operators, such as `$eq`, `$gt`, `$in`, `$or`, etc.
- The projection specifies which fields of the documents to include or exclude in the result. It is also a JSON object that can contain either 1 or 0 values for each field, indicating inclusion or exclusion, respectively. By default, the `_id` field is always included, unless explicitly excluded.
- For example, to find all documents in the `users` collection where the `age` field is greater than 25 and the `name` field starts with “A”, you can use the following query:

```
db.users.find(
  { age: { $gt: 25 }, name: { $regex: "^A" } }, // query filter
  { name: 1, age: 1, _id: 0 } // projection
)
```

- The result will be a cursor that can be iterated over to access the matching documents, such as:

```
{ "name" : "Alice", "age" : 30 }
{ "name" : "Adam", "age" : 28 }
{ "name" : "Anna", "age" : 26 }
```

- To limit the number of documents returned by the `find()` method, you can use the `limit()` method on the cursor, which takes a positive integer as an argument.
- To skip some documents before returning the result, you can use the `skip()` method on the cursor, which also takes a positive integer as an argument.
- To sort the documents by one or more fields, you can use the `sort()` method on the cursor, which takes a JSON object as an argument, where the keys are the field names and the values are either 1 or -1, indicating ascending or descending order, respectively.
- For example, to find the first 10 documents in the `users` collection where the `age` field is greater than 25, sorted by the `name` field in ascending order, you can use the following query:

```
db.users.find(
  { age: { $gt: 25 } }, // query filter
  { name: 1, age: 1, _id: 0 } // projection
).sort(
  { name: 1 } // sort by name ascending
).limit(10) // limit to 10 documents
```

- The result will be a cursor that can be iterated over to access the matching documents, such as:

```
{ "name" : "Adam", "age" : 28 }
{ "name" : "Alice", "age" : 30 }
{ "name" : "Anna", "age" : 26 }
{ "name" : "Bob", "age" : 27 }
{ "name" : "Carol", "age" : 29 }
{ "name" : "David", "age" : 31 }
{ "name" : "Eve", "age" : 32 }
{ "name" : "Frank", "age" : 33 }
{ "name" : "Grace", "age" : 34 }
{ "name" : "Helen", "age" : 35 }
```

- To count the number of documents that match a query filter, you can use the `count()` method on the cursor, which returns an integer.

- For example, to count the number of documents in the `users` collection where the `age` field is greater than 25, you can use the following query:

```
db.users.find(
  { age: { $gt: 25 } } // query filter
).count()
```

- The result will be an integer, such as:

15

- To update one or more documents that match a query filter, you can use the `update()` method on a collection, which takes three parameters: a query filter, an update document, and an optional options object.
- The update document specifies the changes to be applied to the matching documents. It can contain various operators, such as `$set`, `$inc`, `$push`, `$pull`, etc.
- The options object can contain various flags, such as `multi`, `'up`

Indexing in MongoDB

- Indexing is a process that improves the performance of queries in MongoDB by creating data structures that store a small portion of the collection's data.
- Indexes support the efficient execution of queries by reducing the number of documents that the database needs to scan.
- MongoDB provides various types of indexes, such as:
 - Single field indexes: indexes that are created on a single field of a document.
 - Compound indexes: indexes that are created on multiple fields of a document, in a specified order.
 - Multikey indexes: indexes that are created on array fields, or fields that contain subdocuments, and index each element of the array or subdocument.
 - Geospatial indexes: indexes that are created on geospatial data, such as points, lines, and polygons, and support queries that calculate geometries and distances.
 - Text indexes: indexes that are created on string fields and support text search queries on the collection.
 - Hashed indexes: indexes that are created on a single field and use a hash function to compute the index key for each document.
 - Wildcard indexes: indexes that are created on all fields that match a specified pattern, and support queries on unknown or arbitrary fields.
- To create an index, use the `db.collection.createIndex()` method, which takes the following parameters:

- **keys**: a document that specifies the field(s) to index and the index type (ascending, descending, or special)
- **options**: an optional document that specifies additional properties of the index, such as name, uniqueness, expiration, etc.
- For example, to create a single field index on the **name** field of the **users** collection, use the following command:

```
db.users.createIndex({name: 1})
```

This creates an ascending index on the **name** field, which means that the index entries are sorted in ascending order of the **name** values.

- To create a compound index on the **name** and **age** fields of the **users** collection, use the following command:

```
db.users.createIndex({name: 1, age: -1})
```

This creates a compound index on the **name** and **age** fields, which means that the index entries are sorted in ascending order of the **name** values, and within each **name** value, in descending order of the **age** values.

- To create a multikey index on the **hobbies** field of the **users** collection, use the following command:

```
db.users.createIndex({hobbies: 1})
```

This creates a multikey index on the **hobbies** field, which means that the index entries are created for each element of the **hobbies** array. For example, if a document has the following **hobbies** field:

```
hobbies: ["reading", "writing", "coding"]
```

The index will have three entries for this document, one for each hobby.

- To create a geospatial index on the **location** field of the **places** collection, use the following command:

```
db.places.createIndex({location: "2dsphere"})
```

This creates a geospatial index on the **location** field, which means that the index supports queries that perform geospatial calculations on the **location** values, which are assumed to be GeoJSON objects. For example, if a document has the following **location** field:

```
location: {
  type: "Point",
  coordinates: [-73.97, 40.77]
}
```

The index will store the geospatial data of this document, and support queries that find places near, within, or intersecting with a given geometry.

- To create a text index on the `title` and `content` fields of the `posts` collection, use the following command:

```
db.posts.createIndex({title: "text", content: "text"})
```

This creates a text index on the `title` and `content` fields, which means that the index supports text search queries on the `posts` collection, using the `title` and `content` fields as the search fields. For example, to find posts that contain the term “MongoDB”, use the following query:

```
db.posts.find({$text: {$search: "MongoDB"}})
```

The index will use a text search engine to perform linguistic analysis and ranking on the `title` and `content` fields, and return the matching documents

aggregation in MongoDB

- Aggregation is a process of transforming and analyzing data from a collection or a view in MongoDB.
- Aggregation can perform various operations such as filtering, grouping, sorting, calculating, and projecting data.
- Aggregation can be done using three methods: aggregation pipeline, map-reduce function, and single purpose aggregation methods.
- Aggregation pipeline is a sequence of stages that process documents from a collection or a view. Each stage applies an aggregation operator to the input documents and produces output documents for the next stage. The output of the final stage is the result of the aggregation.
- Map-reduce function is a way of processing large data sets by applying a map function to each document and then reducing the results by a reduce function. The map function emits key-value pairs, and the reduce function combines the values with the same key. The output of the map-reduce function is a collection of documents.
- Single purpose aggregation methods are simple and fast ways of performing common aggregation tasks, such as counting documents, finding the minimum or maximum value, or calculating the average. Some examples are `db.collection.count()`, `db.collection.distinct()`, and `db.collection.group()`.
- Aggregation can be useful for various purposes, such as data analysis, reporting, data mining, business intelligence, and performance optimization.
- Aggregation can also be combined with other MongoDB features, such as indexes, sharding, and transactions, to improve the efficiency and scalability of the queries.

Some mnemonics and learning tricks for aggregation in MongoDB are:

- Remember the acronym PAGS for the four types of aggregation methods: Pipeline, map-reduce, single purpose Aggregation, and Stages.

- Remember the acronym GASP for the four common aggregation operators: Group, project, Sort, and match.
- Remember the word PIPE for the structure of an aggregation pipeline: [{ stage1 }, { stage2 }, ... , { stageN }].
- Remember the word MAPR for the structure of a map-reduce function: `db.collection.mapReduce( mapFunction, reduceFunction, options )`.
- Remember the word SPAM for the structure of a single purpose aggregation method: `db.collection.method( query, options )`.
- Remember the word FAME for the four benefits of aggregation: Flexibility, Analysis, performance, and scalability.

Capped collections in MongoDB

- Capped collections are fixed-size collections that support high-throughput operations that insert and retrieve documents based on insertion order .
- Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection .
- You must create capped collections explicitly using the `db.createCollection()` method, which is a mongosh helper for the create command .
- When creating a capped collection you must specify the maximum size of the collection in bytes, which MongoDB will pre-allocate for the collection .
- You can also specify the maximum number of documents that the capped collection can store, but this is optional.
- Capped collections have the following characteristics and limitations :
 - Capped collections guarantee preservation of the insertion order. As a result, queries do not need an index to return documents in insertion order. Without this indexing overhead, capped collections can support higher insertion throughput.
 - Capped collections only allow insertion at the end of the collection and in contiguously allocated space. Updates to a document must not increase the document's size, to prevent the document from exceeding the allocated space. If a document size increases, MongoDB will remove the document from the capped collection and insert it as a new document at the end of the collection, if space is available.
 - Capped collections cannot be sharded. However, you can create a sharded system that writes to a non-sharded capped collection on a single shard. See Sharded Cluster with Non-Sharded Capped Collection for an example.
 - Capped collections automatically remove the oldest documents in the collection without requiring scripts or explicit remove operations.
 - Capped collections do not support the `db.collection.remove()` method. To remove all documents from a capped collection, use the `db.collection.drop()` method to drop the collection and recreate the capped collection.

- Capped collections support `createIndex()` on any field, but only support `dropIndex()` on the `_id` field.
- Capped collections support the `db.collection.renameCollection()` method with the `dropTarget` option set to `true`.
- Capped collections are useful for storing log information, high volume of data, and cache information.
- Capped collections can be converted to regular (uncapped) collections using the `db.collection.convertToCapped()` method, which takes the maximum size of the collection in bytes as a parameter.
- A possible mnemonic to remember the features of capped collections is: **C**ircular, **A**llocated, **P**reserved, **P**re-ordered, **E**nd-insertion, **D**eletion.

Spark

- Spark is an open-source, distributed computing framework that provides a fast and general way to process large-scale data.
- Spark supports multiple programming languages, such as Scala, Python, Java, and R, and provides various libraries for data analysis, machine learning, graph processing, streaming, and SQL.
- Spark runs on a cluster of machines, where each machine is called a node. The nodes are organized into a master-slave architecture, where one node acts as the driver or master, and the rest are workers or slaves.
- Spark uses a data abstraction called resilient distributed dataset (RDD), which is a collection of elements that can be partitioned across the nodes and operated on in parallel.
- Spark also supports a higher-level data abstraction called `DataFrame`, which is a distributed collection of rows organized into named columns, similar to a relational table or a spreadsheet.
- Spark can read and write data from various sources, such as HDFS, S3, Cassandra, Hive, Kafka, etc.
- Spark can run on various cluster managers, such as YARN, Mesos, Kubernetes, or its own standalone mode.
- Spark has a web-based user interface (UI) that shows the status and metrics of the applications, jobs, stages, and tasks.
- Spark has a shell or REPL (read-eval-print loop) that allows users to interactively run commands and queries using Scala, Python, or R.
- Spark has a core API that provides basic operations, such as `map`, `filter`, `reduce`, `join`, etc., on RDDs and `DataFrames`.
- Spark has a SQL API that allows users to run SQL queries on `DataFrames` and view them as tables or views.
- Spark has a streaming API that allows users to process real-time data from various sources, such as Kafka, Flume, Twitter, etc., and apply transformations and actions on them.
- Spark has a machine learning library (MLlib) that provides various algorithms and tools for classification, regression, clustering, recommendation, etc., on RDDs and `DataFrames`.

- Spark has a graph processing library (GraphX) that provides various algorithms and tools for manipulating and analyzing graphs on RDDs.
- Spark has a natural language processing library (Spark NLP) that provides various algorithms and tools for text analysis, sentiment analysis, named entity recognition, etc., on DataFrames.

Installing spark

Spark is an open-source distributed computing framework that can process large-scale data using in-memory caching and parallel operations. Spark can run on various platforms, such as Hadoop, Mesos, Kubernetes, standalone, or in the cloud. To install Spark, you need to follow these steps:

- Download the latest version of Spark from <https://spark.apache.org/downloads.html>. Choose the package type that matches your cluster manager (for example, Hadoop 3.2 and later).
- Extract the downloaded file to a location of your choice. This will create a directory called `spark-x.y.z-bin-hadoopx.y`, where `x.y.z` is the Spark version and `hadoopx.y` is the Hadoop version.
- Set the environment variable `SPARK_HOME` to point to the extracted directory. For example, on Linux, you can use the command `export SPARK_HOME=/path/to/spark-x.y.z-bin-hadoopx.y`.
- Optionally, you can also add the `bin` directory of Spark to your `PATH` variable, so that you can run Spark commands without specifying the full path. For example, on Linux, you can use the command `export PATH=$PATH:$SPARK_HOME/bin`.
- To verify the installation, you can run the command `spark-shell` to launch an interactive Scala shell, or `pyspark` to launch an interactive Python shell. You should see a welcome message and a prompt to enter commands. You can also run some basic Spark operations, such as creating a `SparkSession` object, reading a file, or performing some transformations and actions on a `DataFrame` or an `RDD`.
- To run Spark applications, you need to either use `spark-submit` command or use an IDE that supports Spark integration, such as IntelliJ IDEA, Eclipse, or PyCharm. You can find more details on how to write and run Spark applications in the official documentation: <https://spark.apache.org/docs/latest/quick-start.html>.

Spark Applications

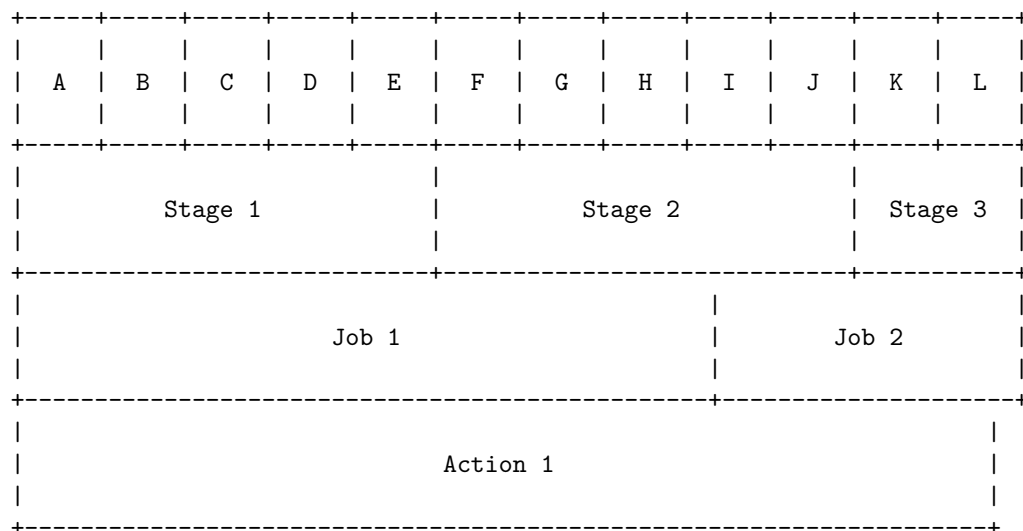
- A Spark application is a program that runs on a Spark cluster and performs parallel data processing using the Spark API.
- A Spark application consists of a driver program and one or more executors that run on the cluster nodes.
- The driver program is the main program that defines the logic and the transformations of the data. It also creates a `SparkSession` object that

represents the connection to the cluster.

- The executors are the processes that run the tasks assigned by the driver program and store the data in memory or disk.
- A Spark application can be written in Scala, Java, Python, or R, and can use various libraries and frameworks that are compatible with Spark, such as Spark SQL, Spark Streaming, MLlib, GraphX, etc.
- A Spark application can be submitted to the cluster using the `spark-submit` command, which takes various options and parameters to configure the application, such as the master URL, the application name, the number of cores, the memory, the jars, the files, the arguments, etc.
- A Spark application can also be run interactively using the Spark shell or the PySpark shell, which provide a REPL (read-eval-print loop) environment for testing and debugging the code.
- A Spark application can also be run in a notebook environment, such as Jupyter or Zeppelin, which provide a web-based interface for writing and executing the code, as well as visualizing the results and the performance metrics.
- Some of the common use cases and applications of Spark are:
 - Big data analytics: Spark can process large volumes of structured, semi-structured, or unstructured data from various sources, such as HDFS, S3, Kafka, etc., and perform SQL queries, aggregations, joins, filters, etc., using Spark SQL or the DataFrame API.
 - Streaming analytics: Spark can process real-time data streams from various sources, such as Kafka, Flume, Twitter, etc., and perform transformations, windowing, stateful operations, etc., using Spark Streaming or the Structured Streaming API.
 - Machine learning: Spark can perform various machine learning tasks, such as classification, regression, clustering, recommendation, etc., using MLlib or the ML API, which provide scalable and distributed implementations of various algorithms and models.
 - Graph analytics: Spark can perform various graph processing tasks, such as finding shortest paths, connected components, page rank, etc., using GraphX or the GraphFrames API, which provide scalable and distributed implementations of various graph algorithms and data structures.
 - Natural language processing: Spark can perform various natural language processing tasks, such as sentiment analysis, topic modeling, named entity recognition, etc., using various libraries and frameworks, such as Spark NLP, Stanford CoreNLP, etc.
 - Image processing: Spark can perform various image processing tasks, such as face detection, object recognition, segmentation, etc., using various libraries and frameworks, such as OpenCV, TensorFlow, etc.

Jobs in Spark

- A job in Spark is a parallel computation that consists of multiple tasks that get spawned in response to a Spark action (e.g., save, collect); you can also use the web UI to check the jobs that have been scheduled or run.
- A Spark job can consist of more than one stage, which are the smaller units of a job and are scheduled by Spark's scheduler. A stage contains tasks based on the same shuffle dependency. Spark tries to pipeline as many RDD operations as possible within a stage, but it has to break the pipeline if one of the RDD operations relies on a shuffle (e.g., reduceByKey, join, groupByKey).
- The boundaries of a stage are often referred to as shuffle boundaries, or just shuffles. A shuffle is a data exchange that copies data across the executors and the network. Shuffles are expensive operations because they involve disk I/O, data serialization, and network I/O. To organize data for the shuffle, Spark creates buckets of data called shuffle partitions (or just partitions), which are configurable through `spark.sql.shuffle.partitions` or `spark.default.parallelism` depending on whether you are working in SQL or RDDs.
- A task is a unit of execution that runs on a single executor (a process running on a worker node). One task is executed for each partition of the shuffled data; tasks are multithreaded and share the same executor JVM. The number of tasks to run is the same as the number of partitions of the data. Each task performs a range of transformations on a single partition of the data, and optionally writes out a result.
- A simple way to remember the relationship between jobs, stages, and tasks is: one action triggers one job, which consists of one or more stages, each of which has one or more tasks. The following diagram illustrates this relationship:



- In this example, action 1 triggers job 1, which has two stages: stage 1 and stage 2. Stage 1 has four tasks: A, B, C, and D; stage 2 has four tasks: E, F, G, and H. There is a shuffle boundary between stage 1 and stage 2. Job 2 is triggered by a different action and has one stage: stage 3, which has four tasks: I, J, K, and L. There is no shuffle boundary between job 1 and job 2, as they are independent computations.
- To optimize the performance of your Spark applications, you should minimize the number of shuffles and the amount of data shuffled, and tune the level of parallelism to match the resources available. You can use the Spark UI to monitor the progress and performance of your Spark jobs, stages, and tasks, and identify the bottlenecks or skewness in your data or computation.

Stages and Tasks in Spark

- Spark is a distributed computing framework that can process large-scale data in parallel using clusters of machines.
- Spark divides the data into partitions, which are logical chunks of data that can be processed by different executors (workers) in parallel.
- Spark also divides the computation into stages, which are groups of tasks that perform the same operation on different partitions of data.
- A task is a unit of work that is sent to an executor by the driver (master) to process a partition of data and produce an output.
- A stage consists of one or more tasks that have the same shuffle dependency, which means they depend on the same set of partitions from the previous stage or the original data source.
- A shuffle dependency occurs when the data needs to be redistributed across the cluster, such as when performing a join, groupBy, reduceByKey, or sortBy operation.
- A shuffle dependency can be narrow or wide, depending on how many partitions of the parent RDD are used by each partition of the child RDD.
- A narrow dependency means that each partition of the child RDD depends on at most one partition of the parent RDD, such as when performing a map, filter, or union operation.
- A wide dependency means that each partition of the child RDD depends on multiple partitions of the parent RDD, such as when performing a join, groupBy, reduceByKey, or sortBy operation.
- Spark tries to minimize the number of shuffles and the amount of data shuffled, as they are expensive operations that involve network communication and disk I/O.
- Spark creates a DAG (directed acyclic graph) of stages based on the RDD transformations and actions in the application code, and optimizes the DAG by applying various rules, such as pipelining, coalescing, and caching.
- Pipelining means that Spark combines multiple narrow transformations into a single stage, and executes them in memory without writing intermediate data to disk.

- Coalescing means that Spark reduces the number of partitions of an RDD by combining adjacent partitions, which can reduce the shuffle cost and the memory usage.
- Caching means that Spark can persist an RDD in memory or on disk, and reuse it in subsequent stages without recomputing it.
- Spark executes the DAG of stages in a lazy manner, which means that it only computes the RDDs when an action is triggered, such as count, collect, save, or show.
- Spark also supports dynamic allocation of executors, which means that it can scale up or down the number of executors based on the workload and the available resources.
- Spark provides a web UI that can show the details of the stages, tasks, executors, and RDDs in the application, as well as the metrics, logs, and events.

Here is an example of a Spark application that reads a text file, counts the number of words in each line, and saves the result as a CSV file.

```
# Import SparkSession
from pyspark.sql import SparkSession

# Create a SparkSession
spark = SparkSession.builder.appName("WordCount").getOrCreate()

# Read a text file as an RDD
rdd = spark.sparkContext.textFile("input.txt")

# Split each line into words and count the number of words
rdd = rdd.map(lambda line: line.split()) \
    .map(lambda words: (len(words), 1)) \
    .reduceByKey(lambda x, y: x + y)

# Convert the RDD to a DataFrame
df = rdd.toDF(["word_count", "line_count"])

# Save the DataFrame as a CSV file
df.write.csv("output.csv")

# Stop the SparkSession
spark.stop()
```

The DAG of stages for this application is shown below:

```
input.txt
|
|  map
|  map
|  reduceByKey
```

```

      v
(word_count, line_count)
      |
      | toDF
      v
DataFrame
      |
      | write.csv
      v
output.csv

```

There are two stages in this DAG:

- Stage 1: This stage has a narrow dependency, as each partition of the input RDD depends on one partition of the text file. It performs the map, map, and reduceByKey transformations on the input RDD, and produces an output RDD with the word count and line count for each partition.
- Stage 2: This stage has a wide dependency, as each partition of the output RDD depends on multiple partitions of the input RDD. It performs the toDF and write.csv actions on the output RDD, and saves the result as a CSV file.

Each stage consists of one or more tasks, depending on the number of partitions of the RDDs involved. For example, if the input RDD has

Resilient Distributed Databases in spark

- Resilient Distributed Databases (RDDs) are the primary data structure in Spark .
- RDDs are immutable distributed collections of objects that can be operated on in parallel .
- RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes.
- Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster .
- RDDs are reliable and memory-efficient when it comes to parallel processing.
- RDDs support two types of operations: transformations and actions.
 - Transformations create a new RDD from an existing one, such as map, filter, join, etc.
 - Actions return a value to the driver program after running a computation on the RDD, such as count, collect, save, etc.
- RDDs are lazy evaluated, meaning that they are only computed when an action is performed on them .
- RDDs are fault-tolerant, meaning that they can recover from failures by using lineage information .
 - Lineage is the sequence of transformations that were applied to create

- an RDD.
 - Spark can recompute the lost partitions of an RDD by using the lineage information.
- RDDs can be created from various sources, such as external files, parallelized collections, or other RDDs.
- RDDs can be cached or persisted in memory or disk for faster access.
- RDDs can be controlled by two parameters: persistence level and partitioning scheme.
 - Persistence level determines how and where the RDD is stored, such as `MEMORY_ONLY`, `DISK_ONLY`, `MEMORY_AND_DISK`, etc.
 - Partitioning scheme determines how the RDD is distributed across the cluster, such as `HashPartitioner`, `RangePartitioner`, etc.
- RDDs are the primary user-facing API in Spark since its inception, but they have some limitations, such as:
 - RDDs are low-level and verbose, requiring users to specify how to do things instead of what to do.
 - RDDs do not support schema inference, meaning that users have to manually define the schema of the data.
 - RDDs do not support complex operations, such as SQL queries, machine learning algorithms, graph processing, etc.
- To overcome these limitations, Spark introduced higher-level APIs, such as `DataFrames`, `Datasets`, and `Structured Streaming`, which are built on top of RDDs.
- However, RDDs are still useful for some scenarios, such as:
 - When the data is unstructured or semi-structured, such as text files, JSON files, etc.
 - When the data processing requires fine-grained control over the partitioning and caching.
 - When the data processing requires custom or complex transformations that are not supported by the higher-level APIs.

Anatomy of a Spark job run

- A Spark job is a user-defined computation that transforms one or more RDDs (Resilient Distributed Datasets) into a final result.
- A Spark job consists of one or more stages, which are parallel tasks that operate on a subset of the data.
- A stage is a collection of tasks that perform the same computation on different partitions of the RDD.
- A task is a unit of execution that runs on a single executor (a process that runs on a worker node).
- A Spark job is executed by the driver program, which coordinates the execution of tasks on the executors.
- The driver program splits the job into stages based on the dependencies between the RDDs.
- The driver program submits each stage to the `DAGScheduler`, which cre-

ates a DAG (Directed Acyclic Graph) of tasks and assigns them to executors.

- The DAGScheduler tracks the progress of each stage and handles failures and retries.
- The executors run the tasks assigned by the DAGScheduler and report the status and results back to the driver program.
- The driver program collects the results from the executors and returns the final result to the user.

Some mnemonics and learning tricks for the anatomy of a Spark job run are:

- Remember the acronym RDD: Resilient Distributed Dataset.
- Remember the acronym DAG: Directed Acyclic Graph.
- Remember the hierarchy of Spark components: Job > Stage > Task > Executor > Worker.
- Remember the roles of the driver program and the DAGScheduler: Driver splits the job into stages and submits them to the DAGScheduler. DAGScheduler creates a DAG of tasks and assigns them to executors.
- Remember the difference between a stage and a task: A stage is a collection of tasks that perform the same computation on different partitions of the RDD. A task is a unit of execution that runs on a single executor.

Spark on YARN

- Spark on YARN is a mode of running Spark applications on a cluster of machines managed by YARN (Yet Another Resource Negotiator), which is a resource management framework for Hadoop.
- Spark on YARN requires a binary distribution of Spark which is built with YARN support. Binary distributions can be downloaded from the downloads page of the project website. To build Spark yourself, refer to Building Spark .
- Spark on YARN supports two deployment modes: **client mode** and **cluster mode**.
 - In client mode, the Spark driver runs on the machine that launches the Spark application, and the Spark executors run on the YARN nodes. The driver communicates with the YARN ResourceManager to request resources for the executors, and communicates with the executors to coordinate the execution of tasks. The driver also exposes a web UI for monitoring the application.
 - In cluster mode, the Spark driver runs on a YARN node, and the Spark executors run on other YARN nodes. The driver is launched by the YARN ApplicationMaster, which is a process that runs on a YARN node and manages the lifecycle of the Spark application. The driver communicates with the YARN ResourceManager to request resources for the executors, and communicates with the executors to coordinate the execution of tasks. The driver also exposes a web UI for monitoring the application, which can be accessed through the

YARN ResourceManager web UI.

- To run a Spark application on YARN, you need to set some configuration properties, such as `spark.master`, `spark.submit.deployMode`, `spark.yarn.archive` or `spark.yarn.jars`, etc. For details, please refer to Spark Properties .
- To submit a Spark application to YARN, you can use the `spark-submit` script, which is located in the `bin` directory of the Spark distribution. For example, to submit a Spark application in cluster mode, you can use the following command:

```
./bin/spark-submit \  
--class org.apache.spark.examples.SparkPi \  
--master yarn \  
--deploy-mode cluster \  
--driver-memory 4g \  
--executor-memory 2g \  
--executor-cores 1 \  
--queue thequeue \  
/path/to/examples.jar \  
10
```

- The command above will launch a Spark application that runs the `SparkPi` example with 10 partitions. The application will run on the YARN cluster with the specified driver and executor memory, cores, and queue. The application jar file (`examples.jar`) needs to be accessible from the YARN nodes, either by using a shared file system (such as HDFS) or by uploading it to the distributed cache (by using `spark.yarn.archive` or `spark.yarn.jars`).
- To monitor the status of the Spark application, you can use the YARN ResourceManager web UI, which shows the list of running and completed applications, and the details of each application, such as the ApplicationMaster URL, the driver log, the allocated resources, etc. You can also use the Spark driver web UI, which shows the list of running and completed stages, tasks, executors, etc. The Spark driver web UI can be accessed through the ApplicationMaster URL, which is shown in the YARN ResourceManager web UI.
- To stop a Spark application on YARN, you can use the `yarn application -kill <applicationId>` command, where `<applicationId>` is the YARN application ID of the Spark application, which can be obtained from the YARN ResourceManager web UI or the `spark-submit` output.
- Some advantages of running Spark on YARN are:
 - You can leverage the existing YARN cluster and its features, such as security, resource management, high availability, etc.
 - You can run multiple Spark applications on the same YARN cluster, and share the resources among them.

- You can use different versions of Spark on the same YARN cluster, by specifying different Spark binaries for each application.
- You can dynamically adjust the number of executors for a Spark application, by using `spark.dynamicAllocation.enabled` and `spark.shuffle.service.enabled` properties.
- Some disadvantages of running Spark on YARN are:
 - You need to install and configure both Spark and YARN on the cluster machines, and ensure their compatibility and interoperability.
 - You need to tune the Spark and YARN configuration properties to optimize the performance and resource utilization of

SCALA

- Scala is a **strong statically typed** general-purpose programming language that supports both **object-oriented programming** and **functional programming**.
- Scala is designed to be **concise**, meaning that it allows programmers to write less code to achieve the same functionality as in other languages.
- Scala runs on the **Java Virtual Machine (JVM)** and can interoperate with Java code and libraries. It can also compile to **JavaScript** and run on web browsers or Node.js.
- Scala has many features that make it a powerful and expressive language, such as:
 - **Case classes** and **pattern matching**, which enable concise and elegant data manipulation and processing.
 - **Traits**, which are like interfaces with default implementations that can be mixed into classes.
 - **Implicits**, which allow implicit conversions and parameters that can enhance existing types and methods.
 - **Higher-order functions**, which are functions that can take other functions as arguments or return them as results.
 - **Lazy evaluation**, which means that expressions are only evaluated when they are needed, avoiding unnecessary computations.
 - **Type inference**, which means that the compiler can infer the types of variables and parameters from the context, reducing the need for explicit type annotations.
- Scala is used for a variety of applications, such as web development, data analysis, concurrent and distributed systems, and more. Some of the popular frameworks and libraries that use Scala are:
 - **Play**, which is a web framework that supports RESTful APIs, web sockets, and reactive programming.
 - **Spark**, which is a big data processing engine that supports batch and stream processing, machine learning, and graph analytics.
 - **Akka**, which is a toolkit for building concurrent and distributed systems based on the actor model.

- **Scala.js**, which is a compiler that translates Scala code to JavaScript, allowing Scala to run on web browsers or Node.js.
- Scala has a rich and active community of developers and users, who contribute to the language development, documentation, and support. Scala also has several online resources for learning and practicing, such as:
 - **Scala Documentation**, which provides tutorials, guides, and reference materials for Scala.
 - **Scala Exercises**, which is an interactive platform that offers exercises and quizzes for learning Scala.
 - **Scala Fiddle**, which is an online editor and REPL that allows users to write and run Scala code in the browser.
- Scala is a modern and versatile programming language that combines the best of object-oriented and functional paradigms. It offers a concise and expressive syntax, a powerful type system, and a seamless integration with Java and JavaScript. Scala is suitable for a wide range of applications, from web development to data analysis, and from concurrent and distributed systems to browser-based applications. Scala is a language that can help programmers to write better code, faster and easier.

Introduction to Scala

Scala is a modern multi-paradigm programming language that combines features of object-oriented and functional programming. It was designed to express common programming patterns in a concise, elegant, and type-safe way. It seamlessly integrates with the Java platform and interoperates with existing Java code and libraries. Scala also supports concurrency and distributed computing, and can run on multiple platforms, including JVM, Android, and Scala.js.

Some of the main characteristics and advantages of Scala are:

- **Pure object-oriented:** Every value in Scala is an object, and every operation is a method call. There are no primitive types or operators. Classes and traits are used to define types and behaviors of objects. Scala also supports multiple inheritance through a flexible mixin-based composition mechanism.
- **Functional:** Scala supports immutable data, higher-order functions, pattern matching, and lazy evaluation. It also has a rich collection library that provides many persistent and immutable data structures. Functional programming in Scala enables concise and expressive code, and avoids side effects and null references.
- **Statically typed:** Scala has a sophisticated type system that enforces type safety and allows the compiler to infer types and optimize code. Scala also supports advanced type features, such as generic classes, variance annotations, higher-kinded types, and implicit parameters and conversions. Scala's type system enables abstraction and modularity, and can prevent many common errors at compile time.

- **Extensible:** Scala allows programmers to easily create new constructs and DSLs through its powerful syntax and metaprogramming features. Scala supports operator overloading, infix notation, optional parentheses, and multiple parameter lists. Scala also has macros, implicits, and reflection, which enable code generation and manipulation at compile time and runtime.
- **Scalable:** Scala is designed to scale from small scripts to large and complex systems. It has a concise and readable syntax that reduces boilerplate and enhances productivity. It also has features that support concurrency and parallelism, such as futures, promises, actors, and Akka framework. Scala can handle big data and distributed computing with frameworks like Spark, Flink, and Kafka.

Classes and objects in Scala

- A class in Scala is a blueprint for creating objects. It defines the state and behavior of the objects of that class.
- A class can have fields, methods, constructors, and nested classes or objects.
- A class can also inherit from another class or trait, or implement an interface.
- A class can be defined using the `class` keyword, followed by the name of the class and an optional parameter list.
- For example, `class Person(name: String, age: Int)` defines a class named `Person` with two parameters: `name` and `age`.
- A class can also have a body, enclosed by curly braces, where the fields and methods of the class are defined.
- For example, `class Person(name: String, age: Int) { val greeting = s"Hello, I am $name and I am $age years old." def sayHello() = println(greeting) }` defines a class named `Person` with two parameters, a field named `greeting`, and a method named `sayHello`.
- An object in Scala is an instance of a class. It can be created by using the `new` keyword, followed by the name of the class and an optional argument list.
- For example, `val alice = new Person("Alice", 25)` creates an object named `alice` of the class `Person`, with the arguments `"Alice"` and `25`.
- An object can access the fields and methods of its class by using the dot notation.
- For example, `alice.sayHello()` invokes the `sayHello` method of the `alice` object, which prints `"Hello, I am Alice and I am 25 years old."`
- An object can also be defined without a corresponding class, using the `object` keyword. Such an object is called a singleton object, as there can be only one instance of it in the program.
- A singleton object can have fields, methods, and nested classes or objects,

but it cannot have parameters or inherit from another class or trait.

- A singleton object can be used to define constants, utility functions, or entry points for the program.
- For example, `object Math { val PI = 3.14 def square(x: Int) = x * x }` defines a singleton object named `Math`, with a constant named `PI` and a function named `square`.
- A singleton object can be accessed by using its name directly, without the `new` keyword.
- For example, `Math.PI` returns the value of the constant `PI`, and `Math.square(5)` returns the value of 25.
- A singleton object can also have the same name as a class, in which case it is called a companion object. A companion object can access the private members of its companion class, and vice versa.
- A companion object can be used to define factory methods, apply methods, or implicit conversions for its companion class.
- For example, `class Person(name: String, age: Int) { override def toString = s"Person($name, $age)" } object Person { def apply(name: String, age: Int) = new Person(name, age) }` defines a class named `Person` and a companion object named `Person`, with an `apply` method that creates a new `Person` object.
- A companion object can be used to create objects of its companion class without the `new` keyword, by using the `apply` method implicitly.
- For example, `val bob = Person("Bob", 30)` creates an object named `bob` of the class `Person`, by invoking the `apply` method of the `Person` object.

Basic types and operators in Scala

- Scala has a rich set of built-in types, such as `Int`, `Double`, `Boolean`, `String`, etc.
- Scala also supports user-defined types, such as classes, traits, and objects.
- Scala has a unified type system, which means that every value is an object and every operation is a method call.
- Scala has two main categories of types: value types and reference types.
- Value types are stored directly on the stack or embedded in objects, and they have no identity. Examples of value types are `Int`, `Double`, `Char`, `Boolean`, etc.
- Reference types are stored on the heap and they have an identity. Examples of reference types are `String`, `Array`, `List`, etc.
- Scala has a special type called `Unit`, which represents the absence of a meaningful value. It is similar to `void` in Java, but it is an object. The only value of type `Unit` is `()`.
- Scala has a special type called `Nothing`, which represents the bottom of the type hierarchy. It is a subtype of every other type, but it has no values. It is used to indicate abnormal termination, such as throwing an exception.

- Scala has a special type called `Any`, which represents the top of the type hierarchy. It is a supertype of every other type, and it has two direct subtypes: `AnyVal` and `AnyRef`.
- `AnyVal` is the parent type of all value types, and `AnyRef` is the parent type of all reference types. `AnyRef` is equivalent to `java.lang.Object` in Java.
- Scala has a rich set of operators, which are actually methods defined on types. For example, `+` is a method defined on `Int`, `Double`, `String`, etc.
- Scala allows user-defined operators, which are methods with symbolic names, such as `++`, `::`, `+=`, etc.
- Scala supports operator precedence and associativity, which determine the order of evaluation of operators. For example, `*` has higher precedence than `+`, and `+` is left-associative, which means that `a + b + c` is equivalent to `(a + b) + c`.
- Scala supports infix, prefix, and postfix notation for operators, which are syntactic sugar for method calls. For example, `a + b` is equivalent to `a.+(b)`, `!a` is equivalent to `a.unary_!`, and `a++` is equivalent to `a.++()`.
- Scala supports compound assignment operators, such as `+=`, `-=`, `*=`, etc., which are shorthand for assigning the result of an operation to a variable. For example, `a += b` is equivalent to `a = a + b`.
- Scala supports relational and logical operators, such as `==`, `!=`, `<`, `>`, `&&`, `||`, etc., which are used to compare values and produce boolean results.
- Scala supports bitwise operators, such as `&`, `|`, `^`, `~`, `<<`, `>>`, etc., which are used to manipulate bits of integers.
- Scala supports special operators, such as `??`, `?:`, `->`, `<-`, etc., which are used for various purposes, such as null handling, conditional expressions, tuples, pattern matching, etc.

Built-in control structures in Scala

Scala has several built-in control structures that can be used to write concise and expressive code. Some of the most common ones are:

- **if-else:** This is a conditional expression that evaluates a boolean condition and executes one of two branches depending on whether the condition is true or false. For example:

```
val x = 10
val y = if (x > 0) "positive" else "negative" // y is "positive"
```

- **while:** This is a loop that executes a block of code repeatedly as long as a boolean condition is true. For example:

```
var i = 0
while (i < 10) {
  println(i)
  i += 1
}
```

- **for**: This is a loop that iterates over a collection or a range of values and executes a block of code for each element. For example:

```
for (i <- 1 to 10) {
  println(i)
}
```

- **match**: This is a pattern matching expression that compares a value with a series of cases and executes the first case that matches. For example:

```
val x = 10
val y = x match {
  case 0 => "zero"
  case 1 => "one"
  case _ => "other" // default case
} // y is "other"
```

- **try-catch-finally**: This is an exception handling expression that tries to execute a block of code and catches any exceptions that may occur. It can also execute a block of code regardless of whether an exception occurs or not. For example:

```
try {
  val x = 10 / 0 // throws an ArithmeticException
} catch {
  case e: ArithmeticException => println("Cannot divide by zero")
} finally {
  println("This is always executed")
}
```

Some of the advantages of using these built-in control structures are:

- They are concise and expressive, avoiding unnecessary boilerplate code.
- They are consistent and uniform, following the same syntax and semantics as other expressions in Scala.
- They are functional and immutable, returning values instead of modifying variables or performing side effects.

Some of the disadvantages of using these built-in control structures are:

- They may not be familiar to programmers coming from other languages that use different syntax or semantics for control structures.
- They may not cover all the possible scenarios or use cases that require more complex or custom logic.

Some of the mnemonics and learning tricks for these built-in control structures are:

- **if-else**: Remember that it is an expression that returns a value, not a statement that performs an action. Think of it as a ternary operator in other languages.

- **while**: Remember that it is a loop that checks the condition before executing the block of code. Think of it as a “pre-test” loop.
- **for**: Remember that it is a loop that iterates over a collection or a range of values. Think of it as a “foreach” loop or a “for-in” loop in other languages.
- **match**: Remember that it is a pattern matching expression that compares a value with a series of cases. Think of it as a “switch” statement or a “case” statement in other languages, but more powerful and flexible.
- **try-catch-finally**: Remember that it is an exception handling expression that tries to execute a block of code and catches any exceptions that may occur. Think of it as a “try-except-finally” statement or a “try-catch-finally” statement in other languages.

Functions and closures in Scala

- A function in Scala is a piece of code that takes some input, performs some computation, and returns some output.
- A function can be defined using the `def` keyword, followed by the name of the function, a list of parameters, an optional return type, and a body enclosed in curly braces.
- For example, the following function takes two integers and returns their sum:

```
def add(x: Int, y: Int): Int = {
  x + y
}
```

- A function can also be defined as an expression, without using the `def` keyword or the curly braces. This is called a function literal or an anonymous function.
- For example, the following function literal takes two integers and returns their sum:

```
(x: Int, y: Int) => x + y
```

- A function literal can be assigned to a variable or passed as an argument to another function. This is called a function value or a first-class function.
- For example, the following code assigns a function literal to a variable called `sum` and passes it as an argument to the `map` function:

```
val sum = (x: Int, y: Int) => x + y
val numbers = List(1, 2, 3, 4, 5)
val result = numbers.map(sum(_, 10)) // result is List(11, 12, 13, 14, 15)
```

- A closure in Scala is a function that can access variables from its enclosing scope, even if they are not passed as parameters.
- A closure can capture both mutable and immutable variables, but it is recommended to use only immutable variables to avoid side effects and concurrency issues.

- For example, the following code defines a closure that can access the variable `factor` from its enclosing scope:

```
val factor = 2
val multiplier = (x: Int) => x * factor
val result = multiplier(10) // result is 20
```

- A closure can also modify the variables from its enclosing scope, if they are mutable. However, this is not a good practice and should be avoided.
- For example, the following code defines a closure that can modify the variable `counter` from its enclosing scope:

```
var counter = 0
val incrementer = () => {
  counter += 1
  counter
}
val result = incrementer() // result is 1
val result2 = incrementer() // result2 is 2
```

- A mnemonic to remember the difference between a function and a closure is that a function is a code block, while a closure is a code block with a backpack. The backpack contains the variables from the enclosing scope that the closure can access or modify.
- A learning trick to understand how closures work is to think of them as objects that have a hidden field for each variable they capture from the enclosing scope. The hidden field stores the value or the reference of the variable, depending on whether it is immutable or mutable. The closure can then use the hidden field to access or modify the variable.

Inheritance in Scala

- Inheritance is an important pillar of OOP (Object Oriented Programming).
- It is the mechanism in Scala by which one class is allowed to inherit the features (fields and methods) of another class.
- Scala supports various types of inheritance including single, multilevel, multiple, and hybrid.
- You can use single, multilevel and hierarchal inheritance in your class.
- Multiple and hybrid inheritance can only be achieved by using traits.

Single Inheritance

- Single inheritance is the most simple form of inheritance.
- As shown in the example below, one class (subclass or child class) inherits from another class (superclass or parent class).

```
// Superclass
class Animal {
```

```

    def eat(): Unit = {
        println("Animal is eating")
    }
}

// Subclass
class Dog extends Animal {
    def bark(): Unit = {
        println("Dog is barking")
    }
}

// Main object
object Main {
    def main(args: Array[String]): Unit = {
        // Creating an instance of Dog class
        val dog = new Dog()
        // Calling methods of both classes
        dog.eat()
        dog.bark()
    }
}

```

- The output of the above program is:

```

Animal is eating
Dog is barking

```

- The subclass can access the public and protected members of the superclass, but not the private ones.
- The subclass can also override the methods of the superclass by using the `override` keyword.

Multilevel Inheritance

- Multilevel inheritance is the extension of single inheritance, where one class inherits from another class, which in turn inherits from another class.
- As shown in the example below, the class `Puppy` inherits from the class `Dog`, which inherits from the class `Animal`.

```

// Superclass
class Animal {
    def eat(): Unit = {
        println("Animal is eating")
    }
}

// Subclass 1

```

```

class Dog extends Animal {
  def bark(): Unit = {
    println("Dog is barking")
  }
}

// Subclass 2
class Puppy extends Dog {
  def play(): Unit = {
    println("Puppy is playing")
  }
}

// Main object
object Main {
  def main(args: Array[String]): Unit = {
    // Creating an instance of Puppy class
    val puppy = new Puppy()
    // Calling methods of all classes
    puppy.eat()
    puppy.bark()
    puppy.play()
  }
}

```

- The output of the above program is:

```

Animal is eating
Dog is barking
Puppy is playing

```

- The subclass can access the public and protected members of its direct and indirect superclasses, but not the private ones.
- The subclass can also override the methods of its direct and indirect superclasses by using the **override** keyword.

Multiple Inheritance

- Multiple inheritance is the scenario where one class inherits from more than one class.
- Scala does not support multiple inheritance for classes, but it does for traits.
- Traits are similar to interfaces in Java, but they can also have concrete methods and fields.
- As shown in the example below, the class **Bird** inherits from two traits **Flyable** and **Singable**.

```

// Trait 1

```

```

trait Flyable {
  def fly(): Unit = {
    println("Flying in the sky")
  }
}

// Trait 2
trait Singable {
  def sing(): Unit = {
    println("Singing a song")
  }
}

// Class
class Bird extends Flyable with Singable {
  def chirp(): Unit = {
    println("Chirping happily")
  }
}

// Main object
object Main {
  def main(args: Array[String]): Unit = {
    // Creating an instance of Bird class
    val bird = new Bird()
    // Calling methods of all traits and class
    bird.fly()
    bird.sing()
    bird.chirp()
  }
}

```

- The output of the above program is:

```

Flying in the sky
Singing a song
Chirping happily

```

- The class can access the public and protected members of all the traits it inherits from, but not the private ones.
- The class can also override the methods of the traits it inherits from by

Hadoop Eco System Frameworks , Pig , Hive and HBase

- Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of two main components: Hadoop Distributed File Sys-

tem (HDFS) and MapReduce.

- HDFS is a distributed file system that provides high-throughput access to data stored in Hadoop clusters.
- MapReduce is a programming model and an execution engine for parallel processing of large data sets using key-value pairs.
- Hadoop also includes several additional modules that provide additional functionality, such as Hive, Pig, and HBase.

Pig

- Pig is a high-level platform for creating MapReduce programs using a scripting language called Pig Latin.
- Pig Latin is a data-flow language that allows users to express complex transformations and analysis of data in a simple and declarative way.
- Pig can handle both structured and unstructured data, and supports various data formats, such as text, binary, or JSON.
- Pig can also interact with other Hadoop components, such as HDFS, HBase, or Hive, to read and write data.
- Pig helps to achieve ease of programming and optimization, and hence is a major segment of the Hadoop ecosystem.

Hive

- Hive is a data warehouse infrastructure that provides data summarization and ad-hoc querying using a SQL-like query language called HiveQL.
- HiveQL is a declarative language that allows users to perform various operations on data stored in Hadoop, such as filtering, grouping, joining, aggregating, and sorting.
- Hive can also support custom functions and scripts written in other languages, such as Python, Java, or Ruby, to extend its functionality.
- Hive can handle both structured and semi-structured data, and supports various data formats, such as text, ORC, Parquet, or Avro.
- Hive can also interact with other Hadoop components, such as HDFS, HBase, or Pig, to read and write data.
- Hive performs reading and writing of large data sets using SQL methodology and interface, and hence is a major segment of the Hadoop ecosystem.

HBase

- HBase is a distributed column-oriented database that supports structured data storage for large tables.
- HBase is an open-source project and horizontally scalable, meaning that it can handle increasing amounts of data and requests by adding more nodes to the cluster.
- HBase is a data model that is similar to Google's BigTable, and designed to provide quick random access to huge amounts of structured data.

- HBase can also support atomic operations, such as increment, decrement, or append, on individual cells or rows.
- HBase can also interact with other Hadoop components, such as HDFS, MapReduce, or Hive, to read and write data.
- HBase is a major segment of the Hadoop ecosystem for applications that require low-latency and high-throughput access to data.

Mnemonics and learning tricks

- A possible mnemonic to remember the names and functions of the Hadoop ecosystem frameworks is:
 - **H**ave **P**izza **H**ere **B**efore **M**idnight
 - **H**adoop: distributed processing of large data sets
 - **P**ig: high-level platform for creating MapReduce programs
 - **H**ive: data warehouse infrastructure for data summarization and ad-hoc querying
 - **B**ase: distributed column-oriented database for structured data storage
 - **M**apReduce: programming model and execution engine for parallel processing of large data sets
- A possible learning trick to understand the differences and similarities between Pig and Hive is:
 - Pig and Hive are both high-level platforms for creating MapReduce programs, but they use different languages and data models.
 - Pig uses a scripting language called Pig Latin, which is a data-flow language that allows users to express complex transformations and analysis of data in a simple and declarative way.
 - Hive uses a query language called HiveQL, which is a SQL-like language that allows users to perform various operations on data stored in Hadoop, such as filtering, grouping, joining, aggregating, and sorting.
 - Pig can handle both structured and unstructured data, and supports various data formats, such as text, binary, or JSON.
 - Hive can handle both structured and semi-structured data, and supports various data formats, such as text, ORC, Parquet, or Avro.
 - Pig and Hive can both interact with other Hadoop components, such as HDFS, HBase, or MapReduce, to

Hadoop Eco System Frameworks

- Hadoop is a framework that enables processing of large data sets which reside in the form of clusters. Being a framework, Hadoop is made up of several modules that are supported by a large ecosystem of technologies.
- The Hadoop ecosystem is a platform or a suite that provides various services to solve the big data problems. It includes Apache projects and

various commercial tools and solutions .

- There are four major elements of Hadoop i.e. HDFS, MapReduce, YARN, and Hadoop Common.
 - HDFS (Hadoop Distributed File System) is a distributed file system that has the capability to store a large stack of data sets. It provides high availability, fault tolerance, scalability, and reliability .
 - MapReduce is a programming model that allows for the parallel processing of large data sets across multiple nodes in a cluster. It consists of two phases: map and reduce. The map phase transforms the input data into key-value pairs, and the reduce phase aggregates the values based on the keys .
 - YARN (Yet Another Resource Negotiator) is a resource management layer that allocates the resources (CPU, memory, disk, network) to the applications running on the cluster. It also schedules the tasks and monitors their execution .
 - Hadoop Common is a set of common utilities and libraries that support the other Hadoop modules. It provides the basic functionality such as configuration, logging, security, and I/O operations .
- Apart from the core components, the Hadoop ecosystem also consists of various tools and frameworks that help in data ingestion, data processing, data analysis, data visualization, and data management. Some of the popular ones are :
 - Apache Pig: A high-level scripting language that allows for the analysis of large data sets using a data flow approach. It can run on top of MapReduce or Tez engines .
 - Apache Hive: A data warehouse system that provides a SQL-like interface to query and analyze structured and semi-structured data stored in HDFS. It can also run on top of MapReduce or Tez engines .
 - Apache HBase: A distributed, column-oriented database that provides random access and consistent read/write operations on large data sets. It is based on the Google Bigtable model .
 - Apache Spark: A fast and general-purpose cluster computing system that supports in-memory processing and a variety of data processing tasks such as batch, streaming, machine learning, and graph analysis. It can run on top of Hadoop or standalone .
 - Apache Kafka: A distributed messaging system that provides high-throughput, low-latency, and fault-tolerant data transfer between applications. It can be used for data ingestion, data integration, and data streaming .
 - Apache Flume: A distributed service that collects, aggregates, and moves large amounts of log data from various sources to HDFS or other destinations. It can be used for data ingestion and data integration .
 - Apache Sqoop: A tool that transfers data between Hadoop and relational databases. It can be used for data ingestion and data export

- Apache Oozie: A workflow scheduler that coordinates and executes Hadoop jobs. It can be used for data processing and data management .
- Apache Zeppelin: A web-based notebook that allows for interactive data analysis and visualization using various languages and frameworks such as Spark, Hive, Pig, etc. It can be used for data exploration and data visualization .
- Apache Mahout: A library that provides scalable machine learning algorithms for clustering, classification, recommendation, and dimensionality reduction. It can run on top of Hadoop or Spark .
- A possible mnemonic to remember the Hadoop ecosystem components is: **H**ave **M**ore **F**un **S**olving **P**roblems **H**appily **S**parking **K**nowledge **F**rom **O**ozie **Z**eppelin ****M**

Applications on Big Data using Pig

- Pig is a high-level scripting language that allows users to write complex data transformations and analysis using a simple syntax.
- Pig can run on top of Hadoop, a distributed framework for storing and processing large-scale data sets across multiple nodes.
- Pig can also run on other platforms such as Apache Spark, Apache Tez, and Apache Flink, which offer different execution models and performance benefits.
- Some of the applications of Pig on Big Data are:
 - Data cleansing: Pig can be used to filter, join, group, and aggregate data from various sources and formats, such as structured, semi-structured, or unstructured data. Pig can also handle missing or invalid data, and perform data validation and quality checks.
 - Data integration: Pig can be used to combine data from multiple sources, such as relational databases, NoSQL databases, web logs, social media, sensor data, etc. Pig can also perform complex joins and merges across different data sets, and handle schema evolution and data type conversions.
 - Data analysis: Pig can be used to perform various types of data analysis, such as descriptive, exploratory, predictive, or prescriptive analysis. Pig can also support advanced analytics, such as machine learning, natural language processing, sentiment analysis, etc. Pig can leverage existing libraries and frameworks, such as Apache Mahout, Apache UIMA, Apache OpenNLP, etc. for these tasks.
 - Data visualization: Pig can be used to generate data visualizations, such as charts, graphs, maps, dashboards, etc. Pig can also export data to various formats, such as CSV, JSON, XML, etc. for further processing or presentation. Pig can also interact with other tools and

platforms, such as Apache Hive, Apache HBase, Apache Phoenix, etc. for data visualization.

- Some of the advantages of using Pig on Big Data are:
 - Ease of use: Pig provides a simple and expressive syntax that is similar to SQL, but more flexible and powerful. Pig also supports user-defined functions (UDFs) and macros, which allow users to extend the functionality and reuse the code. Pig also provides a shell and a graphical user interface (GUI) for interactive development and debugging.
 - Scalability: Pig can handle large-scale data sets that are distributed across multiple nodes. Pig can also parallelize the execution of data transformations and analysis, and optimize the data flow and resource utilization. Pig can also handle data skew and data locality issues, and provide fault tolerance and recovery mechanisms.
 - Extensibility: Pig can work with various data sources and formats, and support different data models and schemas. Pig can also integrate with other tools and platforms, such as Apache Hadoop, Apache Spark, Apache Tez, Apache Flink, Apache Hive, Apache HBase, Apache Phoenix, etc. Pig can also leverage existing libraries and frameworks, such as Apache Mahout, Apache UIMA, Apache OpenNLP, etc. for advanced analytics.
- Some of the disadvantages of using Pig on Big Data are:
 - Performance: Pig may not be as fast or efficient as other tools and platforms, such as Apache Spark, Apache Tez, Apache Flink, etc. Pig may also incur some overhead and latency due to the compilation and execution of scripts. Pig may also have some limitations and trade-offs in terms of memory usage, data partitioning, data shuffling, etc.
 - Debugging: Pig may not provide enough feedback or error messages when something goes wrong. Pig may also have some challenges in debugging complex scripts and UDFs, and tracing the data lineage and dependencies. Pig may also have some compatibility and versioning issues with other tools and platforms, such as Apache Hadoop, Apache Spark, Apache Tez, Apache Flink, etc.
 - Documentation: Pig may not have enough documentation or tutorials for beginners or advanced users. Pig may also have some gaps or inconsistencies in the documentation or the syntax. Pig may also have some changes or updates in the features or the functionality that are not well documented or communicated.

Applications on Big Data using Hive

- Hive is a data warehouse system that provides a SQL-like interface to query and analyze large-scale structured and semi-structured data stored

in Hadoop Distributed File System (HDFS) or other compatible storage systems.

- Hive is mainly used for batch processing, data mining, business intelligence, and data analytics applications that require fast and scalable data processing over big data.
- Some of the common applications of Hive are:
 - Data summarization: Hive can be used to aggregate and summarize large volumes of data using various built-in functions and operators. For example, Hive can calculate the average, count, sum, min, max, etc. of a column or a group of columns in a table.
 - Data transformation: Hive can be used to transform and enrich the raw data into a more structured and meaningful format using various data manipulation functions and operators. For example, Hive can extract, filter, join, sort, group, and partition the data based on various criteria.
 - Data analysis: Hive can be used to perform complex data analysis and generate insights using various analytical functions and operators. For example, Hive can perform ranking, windowing, correlation, regression, clustering, etc. on the data.
 - Data visualization: Hive can be used to create and display various charts and graphs to visualize the data and trends using various visualization tools and libraries. For example, Hive can integrate with tools like Tableau, Power BI, QlikView, etc. to create dashboards and reports.
 - Data exploration: Hive can be used to explore and discover the data using various interactive and ad-hoc queries. For example, Hive can support SQL-like queries to query the data using various filters, joins, subqueries, etc.
- Some of the advantages of using Hive are:
 - Hive is easy to use and learn as it supports a familiar SQL-like syntax and semantics.
 - Hive is compatible with various data formats and storage systems, such as CSV, JSON, XML, ORC, Parquet, Avro, HBase, etc.
 - Hive is scalable and efficient as it leverages the parallel and distributed processing capabilities of Hadoop.
 - Hive is extensible and customizable as it supports user-defined functions (UDFs), user-defined aggregate functions (UDAFs), and user-defined table-generating functions (UDTFs) to implement custom logic and functionality.
 - Hive is open-source and widely supported by a large community of developers and users.
- Some of the disadvantages of using Hive are:

- Hive is not suitable for real-time or low-latency applications as it has a high query latency and overhead due to the MapReduce framework.
- Hive is not fully compliant with the ANSI SQL standards and has some limitations and differences in its syntax and functionality.
- Hive is not optimized for complex and nested data structures and has some performance issues when dealing with such data.
- Hive is not a full-fledged database and does not support transactions, indexes, triggers, etc.

Applications on Big Data using HBase

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS) .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases .
- HBase is well suited for real-time data processing or random read/write access to large volumes of data .
- HBase works well with Hive, a query engine for batch processing of big data, to enable fault-tolerant big data applications .
- HBase can also be integrated with Apache Spark, an open-source processing engine that provides users new ways to store and make use of big data .
- Some of the applications of HBase are:
 - Medical: HBase is used for storing genome sequences and running MapReduce on them, storing the disease history of people or an area, and many others .
 - Sports: HBase is used for storing and analyzing sports data, such as player statistics, game results, and fan engagement .
 - Social Media: HBase is used for storing and processing large-scale social media data, such as user profiles, posts, comments, likes, and shares .
 - E-commerce: HBase is used for storing and managing product catalogs, customer reviews, recommendations, and transactions .
 - Internet of Things: HBase is used for storing and processing sensor data, such as temperature, humidity, pressure, and location .
- A mnemonic to remember some of the applications of HBase is **M-S-E-I** (Medical, Sports, E-commerce, Internet of Things).

Pig

- Pig is a high-level scripting language that allows users to write complex data analysis programs for large-scale data sets.

- Pig is built on top of Hadoop, an open-source framework for distributed storage and processing of big data.
- Pig programs are compiled into a series of MapReduce jobs that are executed by Hadoop.
- Pig provides a set of built-in operators and functions that can perform common data transformations and manipulations, such as filtering, grouping, joining, sorting, aggregating, etc.
- Pig also supports user-defined functions (UDFs) that can be written in Java, Python, Ruby, or other languages and invoked from Pig scripts.
- Pig has two modes of execution: local mode and MapReduce mode. In local mode, Pig runs on a single machine without Hadoop. In MapReduce mode, Pig runs on a Hadoop cluster and utilizes its distributed file system (HDFS) and parallel processing capabilities.
- Pig has two components: Pig Latin and Pig Engine. Pig Latin is the scripting language that users write Pig programs in. Pig Engine is the component that compiles Pig Latin scripts into MapReduce jobs and executes them on Hadoop.
- Pig Latin has a simple and expressive syntax that resembles SQL. A Pig Latin script consists of a series of statements that define data flows. Each statement consists of a relation (a named data set), an operator, and one or more expressions or parameters.
- Pig Latin supports both structured and unstructured data. A relation in Pig Latin is a bag of tuples. A tuple is an ordered set of fields. A field can be of any type, such as int, float, chararray, bytearray, map, tuple, or bag.
- Pig Latin supports schema definition and inference. Users can optionally specify the schema of a relation using the AS clause. If the schema is not specified, Pig will infer it from the data at runtime.
- Pig Latin supports nested data structures, such as maps, tuples, and bags. A map is a collection of key-value pairs. A tuple is an ordered set of fields. A bag is a collection of tuples. These data structures can be nested within each other to represent complex data types.
- Pig Latin supports four types of operators: load/store operators, relational operators, diagnostic operators, and evaluation operators. Load/store operators are used to read data from or write data to external sources, such as files, databases, etc. Relational operators are used to perform data transformations and manipulations, such as filtering, grouping, joining, sorting, aggregating, etc. Diagnostic operators are used to display information about the data or the execution, such as dump, describe, explain, illustrate, etc. Evaluation operators are used to compute values from the data, such as arithmetic, string, comparison, logical, etc.
- Pig Latin supports three types of functions: built-in functions, user-defined functions, and macros. Built-in functions are predefined functions that can be used in Pig Latin scripts, such as SUM, COUNT, AVG, CONCAT, etc. User-defined functions are custom functions that can be written in Java, Python, Ruby, or other languages and invoked from Pig

Latin scripts. Macros are reusable code snippets that can be defined and called from Pig Latin scripts.

- Pig Latin supports three types of comments: single-line comments, multi-line comments, and documentation comments. Single-line comments start with `--` and end at the end of the line. Multi-line comments start with `/*` and end with `*/`. Documentation comments start with `/**` and end with `*/` and can be used to generate documentation for Pig scripts using Javadoc.

Some examples of Pig Latin scripts are:

- Load data from a file into a relation called `input` `input = LOAD 'data.txt' USING PigStorage(',') AS (name:chararray, age:int, salary:float);`
- Filter the input relation to keep only the records where age is greater than 30 `filtered = FILTER input BY age > 30;`
- Group the filtered relation by name and calculate the average salary for each name `grouped = GROUP filtered BY name; averaged = FOREACH grouped GENERATE group AS name, AVG(filtered.salary) AS avg_salary;`
- Store the averaged relation into a file `STORE averaged INTO 'output.txt' USING PigStorage(',');`
- Define a user-defined function in Python that returns the length of a string `@outputSchema("length:int") def string_length(str): return len(str)`
- Register the user-defined function and use it in a Pig Latin script `REGISTER 'string_length.py' USING jython AS myfuncs; input = LOAD 'data.txt' USING PigStorage(',') AS (name:chararray, age:int, salary:float); lengths = FOREACH input GENERATE name, myfuncs.string_length(name) AS name_length; DUMP lengths;`

Pig - Introduction to PIG

- Pig is a high-level scripting language that allows users to write complex data analysis programs for large-scale data sets.
- Pig is built on top of Hadoop, an open-source framework for distributed storage and processing of big data.
- Pig programs are compiled into a series of MapReduce jobs that are executed on the Hadoop cluster.
- Pig provides a simple and expressive syntax that is similar to SQL, but also supports user-defined functions, complex data types, and nested data structures.
- Pig can handle both structured and unstructured data, and can perform various operations such as filtering, grouping, joining, sorting, aggregating, and transforming data.
- Pig can also integrate with other languages and tools, such as Python, Java, R, and Hive, to extend its functionality and interoperability.

- Pig is designed to be scalable, fault-tolerant, and efficient, as it leverages the parallelism and reliability of the Hadoop framework.
- Pig is suitable for data analysts, data scientists, and developers who want to perform complex data analysis tasks without writing low-level code.

Some possible mnemonics and learning tricks for Pig are:

- PIG stands for Pig, I'm Great, as it is a powerful and easy-to-use language for big data analysis.
- PIG can also stand for Processing In Groups, as it can perform various operations on grouped data, such as aggregation and join.
- PIG can be pronounced as P-I-G, which sounds like pie, a common data visualization tool. This can help remember that Pig can handle different types of data and perform various transformations on them.
- PIG can also be pronounced as pig, an animal that is known for eating a lot. This can help remember that Pig can process large-scale data sets efficiently and effectively.

Execution Modes of Pig

- Apache Pig is a high-level platform for analyzing large data sets using a scripting language called Pig Latin.
- Pig can run in different execution modes depending on where the data is stored and how the Pig script is executed.
- The three main execution modes of Pig are:
 - **Local mode:** In this mode, Pig runs in a single Java Virtual Machine (JVM) on the local host and accesses the local file system. This mode is useful for development, testing and debugging of Pig scripts. To run Pig in local mode, use the `-x local` option when invoking the `pig` command.
 - **MapReduce mode:** In this mode, Pig runs on a Hadoop cluster and accesses the Hadoop Distributed File System (HDFS). This mode is suitable for processing large-scale data sets in a distributed and parallel manner. To run Pig in MapReduce mode, use the `-x mapreduce` option when invoking the `pig` command.
 - **Tez mode:** In this mode, Pig runs on a Hadoop cluster and uses Apache Tez as the execution engine. Tez is a framework for building high-performance batch and interactive data processing applications on Hadoop. Tez optimizes the execution plan of Pig scripts by minimizing data movement and reducing the number of MapReduce jobs. To run Pig in Tez mode, use the `-x tez` option when invoking the `pig` command.
- Pig also supports other execution modes such as:
 - **Interactive mode:** In this mode, Pig runs in a shell called Grunt,

where the user can enter Pig Latin commands and see the results immediately. This mode is useful for exploring and analyzing data interactively. To start the Grunt shell, simply run the `pig` command without any options.

- **Batch mode:** In this mode, Pig runs a script file that contains a sequence of Pig Latin commands. This mode is useful for automating and scheduling data processing tasks. To run a Pig script file, use the `-f` option followed by the file name when invoking the `pig` command.
 - **Embedded mode:** In this mode, Pig runs as a library within a Java program, where the user can define custom functions and operators using Java. This mode is useful for extending the functionality and performance of Pig. To run Pig in embedded mode, use the `PigServer` class in the Java program.
- A mnemonic to remember the three main execution modes of Pig is: **LMT** (Local, MapReduce, Tez).
 - An example of a Pig script that counts the number of words in a text file is:

```
-- Load the text file from the file system
A = LOAD 'input.txt' AS (line:chararray);
-- Split each line into words and flatten the result
B = FOREACH A GENERATE FLATTEN(TOKENIZE(line)) AS word;
-- Group the words by their value and count the occurrences
C = GROUP B BY word;
D = FOREACH C GENERATE group, COUNT(B);
-- Store the output in the file system
STORE D INTO 'output.txt';
```

- To run this script in local mode, use the command: `pig -x local -f wordcount.pig`
- To run this script in MapReduce mode, use the command: `pig -x mapreduce -f wordcount.pig`
- To run this script in Tez mode, use the command: `pig -x tez -f wordcount.pig`

Comparison of Pig with Databases

- Pig is a high-level data processing language that runs on top of Hadoop, a distributed file system that can store and process large amounts of data in parallel.
- Databases are structured collections of data that can be accessed and manipulated using a query language, such as SQL.
- Some of the main differences between Pig and databases are:
 - Pig is designed for batch processing, while databases are designed for

online transaction processing (OLTP) or online analytical processing (OLAP).

- Pig does not enforce a schema on the data, while databases require a predefined schema for the tables and columns.
 - Pig supports complex data types, such as maps, tuples, and bags, while databases support primitive data types, such as integers, strings, and booleans.
 - Pig can handle unstructured or semi-structured data, such as web logs, social media posts, and sensor data, while databases can handle structured or normalized data, such as customer records, sales transactions, and inventory data.
 - Pig can perform transformations and analysis on the data using a series of steps, called Pig Latin scripts, while databases can perform queries and operations on the data using a single statement, called SQL queries.
 - Pig can leverage the scalability and fault-tolerance of Hadoop, while databases can leverage the performance and consistency of relational or non-relational models.
- Some of the main similarities between Pig and databases are:
 - Both Pig and databases can store and process large amounts of data, though the scale and speed may vary depending on the use case and the system architecture.
 - Both Pig and databases can support user-defined functions (UDFs) to extend the functionality and expressiveness of the language.
 - Both Pig and databases can interact with other tools and frameworks, such as Hive, Spark, and Sqoop, to perform data integration, analysis, and visualization.
 - A possible mnemonic to remember the differences between Pig and databases is:
 - Pig is **B**ig, **S**chemaless, **C**omplex, **U**nstructured, and **S**cripted.
 - Databases are **O**nline, **S**tructured, **P**rimitive, **S**tructured, and **Q**ueried.
 - An example of a Pig Latin script that performs word count on a text file is:

```
-- Load the text file into a relation called input
input = LOAD 'text.txt' AS (line:chararray);

-- Split each line into words and store them in a relation called words
words = FOREACH input GENERATE FLATTEN(TOKENIZE(line)) AS word;

-- Group the words by their value and count the occurrences
word_groups = GROUP words BY word;
word_count = FOREACH word_groups GENERATE group, COUNT(words);
```

```

-- Store the output in a file
STORE word_count INTO 'output';

```

- An example of a SQL query that performs word count on a table is:

```

-- Create a table called input with a column called line
CREATE TABLE input (line VARCHAR(255));

-- Insert some text into the table
INSERT INTO input VALUES ('This is a test'), ('This is another test');

-- Select the words and their counts from the table
SELECT word, COUNT(*) AS count
FROM (
  -- Split each line into words and store them in a table called words
  SELECT REGEXP_SPLIT_TO_TABLE(line, '\s+') AS word
  FROM input
) AS words
-- Group the words by their value
GROUP BY word;

```

Grunt in Pig

- Grunt is an interactive shell for Apache Pig, a platform for analyzing large data sets using a high-level language called Pig Latin.
- Grunt can be used to write and execute Pig Latin scripts, as well as invoke shell commands using **sh** and **fs**.
- Grunt can be launched by typing **pig** in the command line, or by specifying a Pig Latin script file as an argument, such as **pig script.pig**.
- Grunt supports various commands and features, such as:
 - Autocompletion: Grunt can autocomplete Pig Latin keywords, operators, functions, and variables by pressing the Tab key.
 - History: Grunt can recall previous commands by using the up and down arrow keys, or by typing **history** to see the list of commands.
 - Scripting: Grunt can run Pig Latin scripts from files or URLs, or execute commands from standard input, by using the **run** and **exec** commands, respectively.
 - Debugging: Grunt can display the execution plan of a Pig Latin script, or the schema and sample data of a relation, by using the **explain** and **illustrate** commands, respectively.
 - Help: Grunt can provide help information on Pig Latin syntax, operators, functions, and commands, by using the **help** command.
- Grunt can also be embedded in Java applications, by using the **PigServer**

class, which provides methods for executing Pig Latin scripts and accessing the results.

- A possible mnemonic to remember the name and function of Grunt is: **Grunt Runs Unlimited Numbers of Tasks in Pig.**

Pig Latin

- Pig Latin is a language game or argot in which words in English are altered, usually by adding a suffix or by moving the onset or initial consonant or consonant cluster of a word to the end of the word and adding a vocalic syllable to create such a suffix.
- The objective is to conceal the meaning of the words from others not familiar with the rules.
- The reference to Latin is a deliberate misnomer, as it is simply a form of jargon, used only for its English connotations as a strange and foreign-sounding language.
- Some examples of Pig Latin are:
 - “Pig” becomes “igpay”
 - “Latin” becomes “atinlay”
 - “Banana” becomes “ananabay”
 - “Happy” becomes “appyhay”
 - “Duck” becomes “uckday”
 - “Hello” becomes “ellohay”
- The usual rules for changing standard English into Pig Latin are as follows:
 - For words that begin with consonant sounds, all letters before the initial vowel are placed at the end of the word sequence. Then, “ay” is added, as in the following examples:
 - * “Pig” becomes “igpay”
 - * “Trash” becomes “ashtray”
 - * “Glove” becomes “oveglay”
 - * “Smile” becomes “ilesmay”
 - For words that begin with vowel sounds or silent letter, one just adds “way” or “yay” to the end (or just “ay”). Examples are:
 - * “Eat” becomes “eatway” or “eatay”
 - * “Omelet” becomes “omeletway” or “omeletay”
 - * “Are” becomes “areway” or “areay”
 - * “Honest” becomes “onesthay”
 - For words that end in “y”, such as “happy”, the “y” is left alone and not counted as a vowel. Examples are:
 - * “Happy” becomes “appyhay”

- * “Silly” becomes “illysay”
- * “Candy” becomes “andycay”
- For words that contain a “qu” at the beginning, the “qu” is treated as a single consonant and moved to the end. Examples are:
 - * “Question” becomes “estionquay”
 - * “Squash” becomes “ashsquay”
 - * “Quail” becomes “ailquay”
- For words that contain a “qu” after the first consonant or consonant cluster, the “u” is treated as part of the cluster and moved to the end. Examples are:
 - * “Squeal” becomes “ealsquay”
 - * “Square” becomes “aresquay”
 - * “Squirt” becomes “irtsquay”
- Some possible mnemonics and learning tricks for Pig Latin are:
 - To remember the rule for words that begin with consonant sounds, think of the phrase “consonant to the end, add ay”.
 - To remember the rule for words that begin with vowel sounds or silent letter, think of the phrase “vowel or silent, add way or yay”.
 - To remember the rule for words that end in “y”, think of the phrase “y is not a vowel, leave it alone”.
 - To remember the rule for words that contain a “qu”, think of the phrase “qu is one sound, move it around”.

User Defined Functions in Pig

- User Defined Functions (UDFs) are functions that can be written by the user to perform custom operations on Pig data.
- UDFs can be written in Java, Python, Ruby, Groovy, or JavaScript and registered with Pig using the REGISTER statement.
- UDFs can be used in Pig scripts to transform, filter, join, group, or aggregate data.
- UDFs can be classified into four types: Eval, Filter, Load, and Store.
 - Eval functions take one or more input values and return a single output value. They can be used in expressions, projections, or assignments. For example, UPPER is an eval function that converts a string to uppercase.
 - Filter functions take a single input value and return a boolean value. They can be used in the FILTER operator to select records that satisfy a condition. For example, IsEmpty is a filter function that checks if a bag or map is empty.

- Load functions take a file name or a directory name as input and return a bag of tuples. They can be used in the LOAD operator to read data from external sources. For example, PigStorage is a load function that reads data from files in a specified format.
- Store functions take a bag of tuples as input and a file name or a directory name as output. They can be used in the STORE operator to write data to external sources. For example, PigStorage is also a store function that writes data to files in a specified format.
- UDFs can be invoked by using their fully qualified class name or by using an alias defined by the DEFINE statement. For example, `DEFINE myUpper com.example.MyUpper; A = LOAD 'data.txt' AS (name:chararray); B = FOREACH A GENERATE myUpper(name);`
- UDFs can be tested using the ILLUSTRATE operator, which shows an example of how the UDF works on a sample of data.
- UDFs can be packaged into JAR files and distributed to other users or clusters. The JAR files can be registered with Pig using the REGISTER statement or the `-Dpig.additional.jars` property.
- UDFs can be documented using the `@description`, `@param`, and `@return` annotations in the Java code. The documentation can be viewed using the DESCRIBE FUNCTION statement. For example, `DESCRIBE FUNCTION myUpper;`

Data Processing operators in Pig

- Data processing operators in Pig are the commands that allow users to manipulate and transform data in Pig Latin, a high-level scripting language for Apache Pig.
- Pig Latin operators can be classified into four categories: load/store operators, relational operators, diagnostic operators, and evaluation operators.
- Load/store operators are used to read data from and write data to the file system. The most common load/store operators are LOAD and STORE.
 - LOAD reads data from a file or a directory and creates a relation (a bag of tuples) with a specified schema.
 - STORE writes a relation to a file or a directory in a specified format.
 - Example:


```
-- Load data from a file named input.txt
A = LOAD 'input.txt' AS (name:chararray, age:int, salary:float);

-- Store the relation A to a directory named output
STORE A INTO 'output';
```
- Relational operators are used to perform common relational operations on data, such as filtering, grouping, joining, sorting, etc. The most common relational operators are FILTER, FOREACH, GROUP, JOIN, ORDER, and DISTINCT.

- FILTER selects tuples from a relation that satisfy a given condition.
- FOREACH applies a set of expressions to each tuple in a relation and generates a new relation.
- GROUP groups the tuples in a relation by one or more fields.
- JOIN joins two or more relations by a common field or a set of fields.
- ORDER sorts the tuples in a relation by one or more fields in ascending or descending order.
- DISTINCT removes duplicate tuples from a relation.
- Example:


```
-- Filter the relation A by age
B = FILTER A BY age > 25;

-- Generate a new relation C with name and salary fields
C = FOREACH B GENERATE name, salary;

-- Group the relation C by name
D = GROUP C BY name;

-- Join the relation C with another relation E by name
F = JOIN C BY name, E BY name;

-- Sort the relation F by salary in descending order
G = ORDER F BY salary DESC;

-- Remove duplicate tuples from the relation G
H = DISTINCT G;
```
- Diagnostic operators are used to display information about the data or the execution of Pig Latin scripts. The most common diagnostic operators are DUMP, DESCRIBE, EXPLAIN, and ILLUSTRATE.
 - DUMP prints the contents of a relation to the screen.
 - DESCRIBE prints the schema of a relation to the screen.
 - EXPLAIN prints the logical, physical, and map-reduce execution plans of a Pig Latin script to the screen or a file.
 - ILLUSTRATE shows an example of how a Pig Latin script operates on a small set of data.
 - Example:


```
-- Dump the relation H to the screen
DUMP H;

-- Describe the schema of the relation H
DESCRIBE H;

-- Explain the execution plan of the script to a file named plan.txt
EXPLAIN -out plan.txt;

-- Illustrate how the script operates on a small set of data
```

ILLUSTRATE;

- Evaluation operators are used to compute values from data, such as arithmetic operations, string manipulation, type conversion, etc. The most common evaluation operators are +, -, *, /, %, CONCAT, SUBSTRING, (int), (float), (chararray), etc.
 - +, -, *, /, and % perform arithmetic operations on numeric values.
 - CONCAT concatenates two or more strings.
 - SUBSTRING extracts a substring from a string.
 - (int), (float), and (chararray) convert values to different data types.
 - Example:

```
-- Add 10% bonus to the salary field in the relation C
I = FOREACH C GENERATE name, salary * 1.1 AS bonus;

-- Concatenate the name and salary fields in the relation C
J = FOREACH C GENERATE CONCAT(name, ': ', (chararray)salary);

-- Extract the first three characters of the name field in the relation C
K = FOREACH C GENERATE SUBSTRING(name, 0, 3);

-- Convert the salary field to integer in the relation C
L = FOREACH C GENERATE name, (int)salary;
```

Hive

- Hive is a data warehouse software that facilitates querying and managing large datasets residing in distributed storage.
- Hive provides a SQL-like interface called HiveQL to access structured and semi-structured data in various formats .
- Hive allows users to project structure on largely unstructured data and perform analytical queries without writing MapReduce code.
- Hive is built on top of Apache Hadoop and supports storage on HDFS, S3, ADLS, GS, etc.
- Hive can also integrate with other big data tools such as Spark, Pig, and Tez.

Some of the features and benefits of Hive are:

- Schema flexibility: Hive can handle changes in the data schema and support schema evolution.
- Data compression: Hive can compress data to reduce storage space and improve performance.
- Partitioning and bucketing: Hive can partition and bucket data based on certain columns or expressions to optimize query execution.
- User-defined functions: Hive allows users to create custom functions in Java, Python, or Scala and use them in HiveQL queries.
- External tables: Hive can create external tables that reference data stored outside the Hive warehouse, such as in S3 or HBase.

- Views: Hive can create views that are logical representations of queries and can be used to simplify complex queries or provide access control.
- Security: Hive can enforce authorization and authentication using various mechanisms, such as Kerberos, LDAP, or Ranger.

Some of the limitations and challenges of Hive are:

- Latency: Hive is not suitable for real-time or interactive queries, as it has a high latency due to the overhead of MapReduce jobs.
- Transactions: Hive does not support ACID transactions, which means it cannot guarantee data consistency and integrity in concurrent operations.
- Updates and deletes: Hive does not support updating or deleting existing records in a table, only inserting new records or overwriting the entire table.
- Joins: Hive does not support equi-joins or cross-joins, only inner and outer joins.
- Subqueries: Hive does not support subqueries in the WHERE clause, only in the FROM clause.

Some of the applications and use cases of Hive are:

- Data analysis: Hive can be used to perform complex analytical queries on large volumes of structured or semi-structured data, such as web logs, clickstream data, social media data, etc .
- Data transformation: Hive can be used to transform raw data into a more structured and standardized format, such as CSV, JSON, Parquet, etc.
- Data integration: Hive can be used to integrate data from multiple sources and formats, such as relational databases, NoSQL databases, flat files, etc.
- Data reporting: Hive can be used to generate reports and dashboards based on the aggregated and summarized data from Hive queries.
- Data mining: Hive can be used to apply data mining techniques, such as clustering, classification, association rules, etc, on the data stored in Hive tables.

Some of the mnemonics and learning tricks for Hive are:

- Remember the acronym HIVE: Hadoop, Interface, Views, External tables.
- Remember the difference between partitioning and bucketing: Partitioning splits data based on a column value, while bucketing splits data based on a hash function.
- Remember the syntax of HiveQL: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT.
- Remember the data types supported by Hive: TINYINT, SMALLINT, INT, BIGINT, FLOAT, DOUBLE, DECIMAL, STRING, VARCHAR, CHAR, BOOLEAN, BINARY, TIMESTAMP, DATE, ARRAY, MAP, STRUCT, UNION.
- Remember the file formats supported by Hive: Text, Sequence, RCFile, ORC, Parquet, Avro, JSON.

Apache Hive architecture

Apache Hive is a data warehouse system that enables analytics at a massive scale using SQL-like queries. It is built on top of Hadoop and provides a way to store, manage, and query structured and semi-structured data in a distributed environment.

The key components of the Apache Hive architecture are:

- **Hive Clients:** These are the interfaces that allow users and applications to interact with Hive. They include the Hive Shell, the Hive Web Interface, the Hive JDBC/ODBC drivers, and the Hive Thrift API.
- **Hive Services:** These are the components that process the requests from the clients and execute the queries on the Hadoop cluster. They include the HiveServer2, the Hive Metastore, and the Hive Driver.
- **HiveServer2:** This is the main server that accepts incoming requests from the clients and creates an execution plan and auto-generates a YARN job to process SQL queries. It also handles authentication, authorization, and session management for the clients.
- **Hive Metastore:** This is the central repository of metadata that stores the schema and location of the tables, partitions, columns, and other Hive objects. It can be configured to use different backends, such as MySQL, PostgreSQL, or Oracle. The Hive Metastore communicates with the HiveServer2 and the Hadoop NameNode to provide metadata information.
- **Hive Driver:** This is the component that compiles, optimizes, and executes the queries submitted by the clients. It consists of several sub-components, such as the Compiler, the Optimizer, the Executor, and the Query Plan Generator.
- **Compiler:** This is the sub-component that parses the SQL query and generates an abstract syntax tree (AST) that represents the query logic.
- **Optimizer:** This is the sub-component that applies various transformations and optimizations to the AST to produce a logical plan that is more efficient and suitable for execution on Hadoop.
- **Executor:** This is the sub-component that converts the logical plan into a physical plan that consists of a series of MapReduce or Spark jobs that can be executed on the Hadoop cluster.
- **Query Plan Generator:** This is the sub-component that generates a graphical representation of the physical plan that can be viewed by the users for debugging and performance tuning purposes.
- **Processing Framework and Resource Management:** These are the components that handle the execution of the MapReduce or Spark jobs on the Hadoop cluster. They include the Hadoop MapReduce framework, the Apache Spark framework, and the YARN resource manager.
- **Hadoop MapReduce:** This is the original framework that processes the data in a distributed and parallel manner using the map and reduce functions. It is based on the MapReduce programming model proposed

by Google.

- **Apache Spark:** This is an alternative framework that processes the data in a distributed and parallel manner using the resilient distributed datasets (RDDs) and the Spark SQL engine. It is based on the Spark programming model that supports in-memory processing and interactive analytics.
- **YARN:** This is the resource manager that allocates and manages the resources (such as CPU, memory, disk, and network) for the MapReduce or Spark jobs on the Hadoop cluster. It also monitors the status and progress of the jobs and handles the fault tolerance and recovery mechanisms.
- **Distributed Storage:** This is the component that stores the data in a distributed and fault-tolerant manner on the Hadoop cluster. It includes the Hadoop Distributed File System (HDFS) and the Hive Warehouse Directory.
- **HDFS:** This is the file system that stores the data in large blocks (typically 64 MB or 128 MB) across multiple nodes in the cluster. It provides high availability, scalability, and reliability by replicating the blocks on different nodes and handling the failures and recovery of the nodes.
- **Hive Warehouse Directory:** This is the directory on HDFS that stores the data for the Hive tables and partitions. It follows a hierarchical structure that reflects the schema and location of the tables and partitions. For example, `/user/hive/warehouse/database_name/table_name/partition_name`.

Installing Hive

Hive is a data warehouse system that runs on top of Hadoop. It provides a SQL-like interface to query and analyze large-scale structured and semi-structured data. To install Hive, you need to follow these steps:

- Download the latest version of Hive from the Apache website: <https://hive.apache.org/downloads.html>
- Extract the downloaded file to a desired location, such as `/usr/local/hive`
- Set the environment variables for Hive, such as `HIVE_HOME`, `HIVE_CONF_DIR`, and `PATH`. You can do this by editing the `.bashrc` file in your home directory and adding these lines:

```
export HIVE_HOME=/usr/local/hive
export HIVE_CONF_DIR=$HIVE_HOME/conf
export PATH=$PATH:$HIVE_HOME/bin
```

- Save and source the `.bashrc` file to apply the changes:

```
source ~/.bashrc
```

- Copy the MySQL JDBC driver to the Hive lib directory. You can download the driver from here: <https://dev.mysql.com/downloads/connector/j/>
- Create a MySQL database and user for Hive metastore. The metastore is a repository that stores the metadata of Hive tables, partitions, columns, etc. You can use these commands to create the database and user:

```
CREATE DATABASE hive;
CREATE USER 'hive'@'localhost' IDENTIFIED BY 'hive';
GRANT ALL PRIVILEGES ON hive.* TO 'hive'@'localhost';
FLUSH PRIVILEGES;
```

- Edit the hive-site.xml file in the Hive conf directory and set the following properties:

```
<property>
  <name>javax.jdo.option.ConnectionURL</name>
  <value>jdbc:mysql://localhost:3306/hive?createDatabaseIfNotExist=true</value>
  <description>JDBC connect string for a JDBC metastore</description>
</property>
```

```
<property>
  <name>javax.jdo.option.ConnectionDriverName</name>
  <value>com.mysql.jdbc.Driver</value>
  <description>Driver class name for a JDBC metastore</description>
</property>
```

```
<property>
  <name>javax.jdo.option.ConnectionUserName</name>
  <value>hive</value>
  <description>username to use against metastore database</description>
</property>
```

```
<property>
  <name>javax.jdo.option.ConnectionPassword</name>
  <value>hive</value>
  <description>password to use against metastore database</description>
</property>
```

- Initialize the Hive metastore schema by running this command:

```
schematool -dbType mysql -initSchema
```

- Start the Hive shell by running this command:

```
hive
```

- You can now use Hive to create and query tables. For example, you can create a table called employees with this command:

```
CREATE TABLE employees (
  id INT,
  name STRING,
  salary FLOAT,
  dept STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

- You can load data into the table from a local file with this command:

```
LOAD DATA LOCAL INPATH '/home/user/employees.csv' INTO TABLE employees;
```

- You can query the table with this command:

```
SELECT * FROM employees WHERE dept = 'Sales';
```

- You can exit the Hive shell with this command:

```
QUIT;
```

- A mnemonic to remember the steps of installing Hive is:

Download Hive Extract Hive Set environment variables Copy JDBC driver
Reate MySQL database and user Input hive-site.xml properties Build metastore
schema Enter Hive shell Create and query tables Quit Hive shell

DESCRIBE CQ

Hive shell

- Hive shell is a command-line interface for interacting with Hive.
- Hive shell allows users to execute Hive queries and commands, such as creating tables, loading data, querying data, and managing the Hive metastore.
- Hive shell can be launched by typing `hive` in the terminal, or by specifying a script file to run with `hive -f script.hql`.
- Hive shell supports various options and parameters, such as `-e` to execute a single query, `-v` to enable verbose mode, `-h` to display help, and `-S` to run in silent mode. To see the full list of options, type `hive --help`.
- Hive shell uses the HiveServer2 service to communicate with the Hive metastore and the execution engine. HiveServer2 can be configured to use different authentication and authorization mechanisms, such as Kerberos, LDAP, or SQL standards-based authorization.
- Hive shell supports multiple modes of operation, such as interactive mode, batch mode, and embedded mode. Interactive mode allows users to enter queries and commands one by one and see the results immediately. Batch mode allows users to run a script file containing multiple queries and commands. Embedded mode allows users to embed Hive queries and commands in a Java application using the Hive JDBC driver.
- Hive shell supports various commands and keywords, such as `SHOW`, `DESCRIBE`, `CREATE`, `DROP`, `ALTER`, `LOAD`, `INSERT`, `SELECT`, `JOIN`, `GROUP BY`, `ORDER BY`, `LIMIT`, `HAVING`, `WHERE`, `PARTITION`, `BUCKET`, `CLUSTER BY`, `DISTRIBUTE BY`, `SORT BY`, `UNION`, `INTERSECT`, `EXCEPT`, `EXPLAIN`, `ANALYZE`, `SET`, `ADD`, `DELETE`, `LIST`, `RELOAD`, `RESET`, `DFS`, `!`, and `QUIT`. To see the full list of commands and keywords, type `help` in the Hive shell.
- Hive shell supports various data types, such as `BOOLEAN`, `TINYINT`, `SMALLINT`, `INT`, `BIGINT`, `FLOAT`, `DOUBLE`, `DECIMAL`, `STRING`, `VARCHAR`, `CHAR`, `BINARY`, `DATE`, `TIMESTAMP`, `INTERVAL`, `ARRAY`, `MAP`, `STRUCT`, and

UNIONTYPE. To see the full list of data types, type `help data types` in the Hive shell.

- Hive shell supports various built-in functions, such as ABS, AVG, CONCAT, COUNT, MAX, MIN, SUM, SUBSTR, TRIM, UPPER, LOWER, CAST, COALESCE, CASE, IF, WHEN, NULLIF, NVL, ISNULL, ISNOTNULL, IN, BETWEEN, LIKE, RLIKE, REGEXP, EXPLODE, POSEXplode, INLINE, GET_JSON_OBJECT, JSON_TUPLE, FROM_UNIXTIME, UNIX_TIMESTAMP, TO_DATE, YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, WEEKOFYEAR, DATEDIFF, DATE_ADD, DATE_SUB, ADD_MONTHS, LAST_DAY, NEXT_DAY, TRUNC, ROUND, FLOOR, CEIL, SQRT, POW, LOG, LOG10, LOG2, SIN, COS, TAN, ASIN, ACOS, ATAN, ATAN2, DEGREES, RADIANS, PI, E, RAND, BIN, HEX, UNHEX, CONV, LENGTH, REVERSE, SPACE, REPEAT, LPAD, RPAD, LTRIM, RTRIM, SPLIT, REGEXP_EXTRACT, REGEXP_REPLACE, PARSE_URL, NVL2, DECODE, GREATEST, LEAST, WIDTH_BUCKET, PERCENTILE, PERCENTILE_APPROX, COLLECT_SET, COLLECT_LIST, SIZE, SORT_ARRAY, ARRAY_CONTAINS, MAP_KEYS, MAP_VALUES, STR_TO_MAP, MAP, ARRAY, STRUCT, NAMED_STRUCT, CREATE_UNION, UNION_TYPE, HASH, MD5, SHA, SHA1, SHA2, CRC32, XPATH, XPATH_BOOLEAN, 'XPATH

Hive services

Hive services are the components that enable users to interact with Hive and perform various operations on data stored in Hive tables. Hive services include:

- **HiveServer2:** This is the main service that provides a JDBC/ODBC interface for clients to execute queries and access the metadata. HiveServer2 supports multiple concurrent users and sessions, and provides security features such as authentication and authorization. HiveServer2 also supports dynamic service discovery through ZooKeeper.
- **Hive Metastore:** This is the service that stores the metadata of Hive tables, partitions, schemas, columns, etc. in a relational database. The Hive Metastore can be embedded in HiveServer2 or run as a separate service. The Hive Metastore communicates with the NameNode to get the location of data files on HDFS.
- **Hive CLI:** This is the command-line interface for users to interact with Hive. The Hive CLI can connect to HiveServer2 or directly to the Hive Metastore. The Hive CLI supports Hive QL, which is a SQL-like language for querying and manipulating data in Hive.
- **Hive Web UI:** This is the web-based interface for users to browse the Hive metadata, execute queries, and monitor the query progress and logs. The Hive Web UI can connect to HiveServer2 or directly to the Hive Metastore. The Hive Web UI also provides a graphical view of the query execution plan and statistics.
- **Beeline:** This is a JDBC client that can connect to HiveServer2 and execute queries using Hive QL. Beeline is based on SQLLine, which is a generic JDBC client. Beeline supports multiple output formats, such as

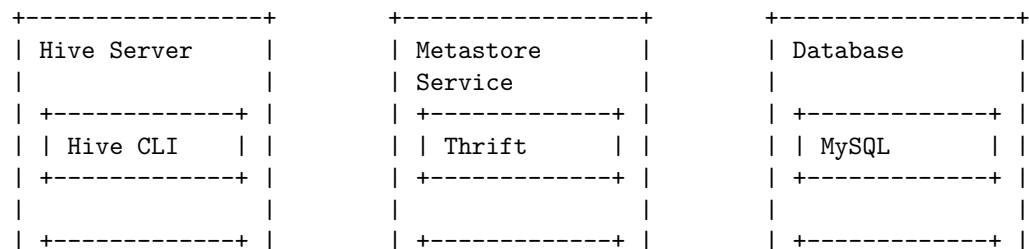
table, csv, json, etc. Beeline can also run queries from a file or a script.

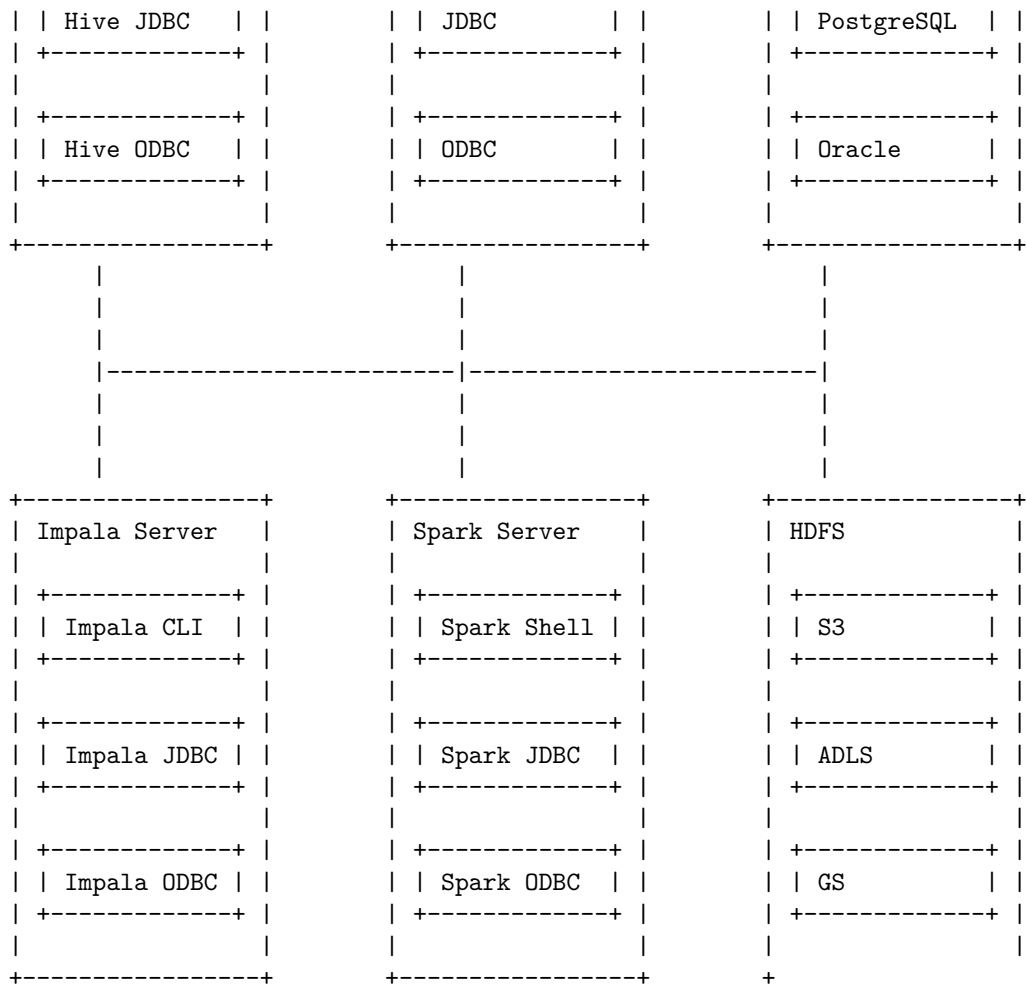
- **Hive Warehouse Connector:** This is a library that enables users to read and write data between Hive and Spark. The Hive Warehouse Connector supports ACID transactions, bucketing, and partitioning on Hive tables. The Hive Warehouse Connector also provides a DataFrame API for Spark to access Hive tables as DataFrames.

Hive metastore

- Hive metastore is a service that stores metadata related to Apache Hive and other services, such as Impala, Spark, etc. in a backend relational database, such as MySQL .
- Metadata includes information about the tables, columns, partitions, schemas, databases, views, functions, etc. that are available in the data lake.
- Hive metastore provides a central repository of metadata that can easily be analyzed to make informed, data driven decisions, and therefore it is a critical component of many data lake architectures.
- Hive metastore supports storage on various file systems, such as S3, ADLS, GS, etc. through HDFS.
- Hive metastore can be configured in three modes: embedded, local, and remote.
 - Embedded mode: The metastore service and the Hive server run in the same JVM and use the same database connection. This is the default mode and is suitable for testing and development purposes.
 - Local mode: The metastore service runs in a separate JVM from the Hive server, but they use the same database connection. This mode allows multiple Hive servers to share the same metastore service.
 - Remote mode: The metastore service runs in a separate JVM and a separate host from the Hive server, and they use different database connections. This mode allows multiple Hive servers and other services to share the same metastore service across the network.
- Hive metastore can be accessed through various interfaces, such as Thrift, JDBC, ODBC, etc.
- Hive metastore can be integrated with other components, such as Ranger, Sentry, Atlas, etc. for security, governance, and lineage purposes.

A possible ascii diagram of Hive metastore in remote mode is:





Comparison of Hive with traditional databases

- Hive is a data warehouse system that provides a SQL-like interface to query and analyze large-scale data stored in Hadoop distributed file system (HDFS).
- Traditional databases are relational database management systems (RDBMS) that store data in tables and support various operations such as insert, update, delete, join, etc.
- Some of the main differences between Hive and traditional databases are:
 - Hive is designed for batch processing and analytics, whereas traditional databases are designed for online transaction processing (OLTP) and real-time queries.

- Hive does not support transactions, concurrency control, or integrity constraints, whereas traditional databases provide these features to ensure data consistency and reliability.
 - Hive does not store data in a fixed schema, but rather in a flexible format such as text, JSON, XML, etc. Traditional databases store data in a predefined schema and enforce data types and constraints.
 - Hive queries are translated into MapReduce jobs and executed on a cluster of nodes, whereas traditional database queries are executed on a single server or a small cluster of servers.
 - Hive supports complex data types such as arrays, maps, structs, etc., whereas traditional databases support only primitive data types such as integers, strings, floats, etc.
 - Hive is scalable and fault-tolerant, as it can handle large volumes of data distributed across multiple nodes, whereas traditional databases have limitations on the size and availability of data.
- Some of the similarities between Hive and traditional databases are:
 - Both Hive and traditional databases use SQL as a query language, although Hive has some extensions and limitations compared to standard SQL.
 - Both Hive and traditional databases support various functions and operators for data manipulation and analysis, such as aggregation, sorting, filtering, etc.
 - Both Hive and traditional databases support indexing and partitioning techniques to improve query performance and data organization.
 - Both Hive and traditional databases can be integrated with various external tools and applications for data visualization, reporting, machine learning, etc.
 - A possible mnemonic to remember the differences between Hive and traditional databases is:
 - Hive is **B**ig, **B**atch, and **B**are, whereas traditional databases are **S**mall, **S**peedy, and **S**trict.
 - **B**ig: Hive can handle large-scale data, whereas traditional databases have size limitations.
 - **B**atch: Hive is suitable for batch processing and analytics, whereas traditional databases are suitable for real-time queries and transactions.
 - **B**are: Hive does not provide transactions, concurrency, or integrity features, whereas traditional databases provide these features to ensure data quality and reliability.
 - **S**mall: Traditional databases can store data in a single server or a small cluster, whereas Hive requires a large cluster of nodes.
 - **S**peedy: Traditional databases can execute queries faster and more efficiently, whereas Hive queries are translated into MapReduce jobs and take longer to run.

- **Strict:** Traditional databases enforce a fixed schema and data types, whereas Hive allows a flexible format and complex data types.

HiveQL

- HiveQL is a query language for Apache Hive, a data warehouse system for Apache Hadoop .
- HiveQL allows users to process and analyze structured data in a Metastore, which is a central repository of metadata that can easily be analyzed to make informed, data driven decisions.
- HiveQL provides the basic SQL-like operations, such as SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, HAVING, ORDER BY, etc .
- HiveQL also supports some advanced features, such as partitioning, bucketing, windowing, user-defined functions, and user-defined aggregate functions .
- HiveQL separates users from the complexity of Map Reduce programming, which is a low-level Java API for processing large-scale data in parallel .
- HiveQL reuses common concepts from relational databases, such as tables, rows, columns, and schema, to ease learning .
- HiveQL supports storage on various file systems, such as HDFS, S3, ADLS, GS, etc.
- HiveQL can be executed using various interfaces, such as Hive CLI, Hive Web UI, Hive JDBC, Hive ODBC, etc .

Some examples of HiveQL queries are:

- To create a table named employee with four columns:

```
CREATE TABLE employee (
  emp_id INT,
  name STRING,
  dept STRING,
  salary FLOAT
);
```

- To load data from a file into the employee table:

```
LOAD DATA LOCAL INPATH '/path/to/file' INTO TABLE employee;
```

- To select all the records from the employee table:

```
SELECT * FROM employee;
```

- To select the name and salary of the employees who work in the sales department:

```
SELECT name, salary FROM employee WHERE dept = 'sales';
```

- To join the employee table with another table named bonus based on the emp_id column:

```
SELECT e.name, e.salary, b.amount FROM employee e JOIN bonus b ON e.emp_id = b.emp_id;
```

- To calculate the average salary of each department and sort them in descending order:

```
SELECT dept, AVG(salary) AS avg_salary FROM employee GROUP BY dept ORDER BY avg_salary DESC;
```

Some mnemonics and learning tricks for HiveQL are:

- Remember that HiveQL is similar to SQL, so you can use your existing knowledge of SQL to learn HiveQL.
- Remember that HiveQL is case-insensitive, so you don't have to worry about the capitalization of keywords, table names, column names, etc.
- Remember that HiveQL supports comments, which are lines that start with `--` or `/*` and end with `*/`. You can use comments to explain your queries or to temporarily disable some parts of your queries.
- Remember that HiveQL supports various data types, such as INT, STRING, FLOAT, BOOLEAN, ARRAY, MAP, STRUCT, etc. You can use these data types to define the schema of your tables and to manipulate the data in your queries.
- Remember that HiveQL supports various operators, such as arithmetic operators (+, -, *, /, %), comparison operators (=, <, >, <=, >=, !=, <>, <=>), logical operators (AND, OR, NOT), etc. You can use these operators to perform calculations, comparisons, and conditions in your queries.

Tables in Hive

- Tables in Hive are similar to tables in a relational database management system. They store data in rows and columns and can be queried using HiveQL, a SQL-like language.
- Tables in Hive can be broadly classified into two types: internal and external .
- Internal tables are also known as managed tables. They are the default type of tables in Hive. They store data in the Hive data warehouse, which is located at `/hive/warehouse/` on the default storage for the cluster.
- External tables store data outside the Hive data warehouse. They can be stored on any storage accessible by the cluster, such as HDFS, Azure Blob Storage, or Amazon S3.
- The main difference between internal and external tables is how Hive handles the data when the table is dropped. When an internal table is dropped, Hive deletes both the table metadata and the data files. When an external table is dropped, Hive only deletes the table metadata, but the data files remain intact .
- Use internal tables when one of the following conditions applies:

- The data is temporary and can be deleted when the table is dropped.
 - The data is only used by Hive and not by other applications.
 - The data is loaded from local files using the `LOAD DATA` command.
- Use external tables when one of the following conditions applies:
 - The data is permanent and should not be deleted when the table is dropped.
 - The data is shared by Hive and other applications.
 - The data is loaded from external sources using the `LOCATION` clause.
- The general syntax for creating a table in Hive is:

```
CREATE [EXTERNAL] TABLE [IF NOT EXISTS] table_name
(column_name data_type [COMMENT column_comment], ...)
[COMMENT table_comment]
[PARTITIONED BY (column_name data_type [COMMENT column_comment], ...)]
[CLUSTERED BY (column_name, ...) [SORTED BY (column_name [ASC|DESC], ...)] INTO num_buckets]
[SKewed BY (column_name, ...) ON ((column_value, ...), ...) [STORED AS DIRECTORIES]]
[ROW FORMAT row_format]
[STORED AS file_format]
[LOCATION hdfs_path]
[TBLPROPERTIES (property_name=property_value, ...)];
```

- Here is an example of creating an internal table with four columns and loading data from a local file:

```
CREATE TABLE IF NOT EXISTS employees
(id INT, name STRING, dept STRING, salary FLOAT)
COMMENT 'This is a table for storing employee data';
```

```
LOAD DATA LOCAL INPATH '/home/hadoop/employees.txt' INTO TABLE employees;
```

- Here is an example of creating an external table with the same schema and loading data from an external location:

```
CREATE EXTERNAL TABLE IF NOT EXISTS employees
(id INT, name STRING, dept STRING, salary FLOAT)
COMMENT 'This is a table for storing employee data'
LOCATION 'hdfs://namenode:8020/user/hadoop/employees';
```

- Some advantages of using tables in Hive are:
 - They allow users to read and write data in various formats, such as text, JSON, Parquet, ORC, etc., using SerDes and Input/Output formats.
 - They support partitioning and bucketing to improve query performance and data organization.
 - They support data skew handling to avoid data imbalance and hotspots.

- They can be accessed by other tools and frameworks, such as Spark, Pig, MapReduce, etc.

Querying data in Hive

- Hive is a data warehouse system that allows users to query and analyze large-scale data using a SQL-like language called HiveQL.
- HiveQL is similar to standard SQL, but it also supports some features that are specific to Hive, such as partitioning, bucketing, windowing, and user-defined functions.
- HiveQL queries are translated into a series of MapReduce jobs that run on a Hadoop cluster and process the data stored in HDFS or other supported file systems.
- Hive supports various data formats, such as text, CSV, JSON, ORC, Parquet, and Avro, and allows users to define custom data formats using SerDes (serializers and deserializers).
- Hive also provides a metastore service that stores the metadata of the tables, partitions, columns, and other schema information in a relational database.
- To query data in Hive, users need to follow these steps:
 1. Connect to the Hive server using a command-line interface (CLI), a graphical user interface (GUI), or an application programming interface (API).
 2. Create and load tables using the `CREATE TABLE` and `LOAD DATA` statements, or use the `CREATE EXTERNAL TABLE` statement to reference existing data files in HDFS or other file systems.
 3. Use the `SHOW`, `DESCRIBE`, and `EXPLAIN` commands to inspect the tables, columns, partitions, and query plans.
 4. Use the `SELECT`, `JOIN`, `GROUP BY`, `ORDER BY`, `LIMIT`, and other clauses to perform various data analysis tasks on the tables.
 5. Use the `INSERT`, `UPDATE`, `DELETE`, and `MERGE` statements to modify the data in the tables, or use the `DROP TABLE` statement to delete the tables.
 6. Use the `CREATE VIEW` and `DROP VIEW` statements to create and delete views, which are logical representations of the data that can be queried as tables.
 7. Use the `CREATE FUNCTION` and `DROP FUNCTION` statements to create and delete user-defined functions, which are custom functions that can be used in HiveQL queries.
 8. Use the `SET` and `RESET` commands to configure various Hive parameters, such as the number of reducers, the compression codec, the output format, and the query timeout.

9. Use the `ADD JAR`, `ADD FILE`, and `ADD ARCHIVE` commands to add external resources, such as Java libraries, data files, and compressed archives, to the classpath of the Hive session.
 10. Use the `!` command to execute shell commands from the Hive CLI, or use the `SOURCE` command to execute HiveQL commands from a file.
- Some examples of HiveQL queries are:
 - To create a table named `customers` with four columns: `id`, `name`, `age`, and `gender`, and partition it by `gender`:


```
CREATE TABLE customers (
  id INT,
  name STRING,
  age INT
)
PARTITIONED BY (gender STRING);
```
 - To load data from a CSV file named `customers.csv` into the `customers` table:


```
LOAD DATA LOCAL INPATH 'customers.csv'
INTO TABLE customers
PARTITION (gender);
```
 - To select the name and age of the female customers who are older than 30:


```
SELECT name, age
FROM customers
WHERE gender = 'F' AND age > 30;
```
 - To join the `customers` table with another table named `orders` on the `id` column, and group the results by the `name` column, and calculate the total amount of orders for each customer:


```
SELECT c.name, SUM(o.amount) AS total
FROM customers c
JOIN orders o
ON c.id = o.customer_id
GROUP BY c.name;
```
 - To create a view named `top_customers` that shows the name and total amount of orders for the top 10 customers by order amount:


```
CREATE VIEW top_customers AS
SELECT name, total
FROM (
  SELECT c.name, SUM(o.amount) AS total
  FROM customers c
```



```

    JOIN orders o
    ON c.id = o.customer_id
    GROUP BY c.name
    ORDER BY total DESC
    LIMIT 10
) t;

```

- To create a user-defined function named **reverse** that reverses a string, and use it in a query to reverse the names of the

User defined functions in Hive

- User defined functions (UDFs) are custom functions that can be created and used in Hive queries to perform specific tasks that are not supported by the built-in functions.
- UDFs can be written in Java, Python, or any other scripting language that supports the Hive UDF interface.
- UDFs can be categorized into three types: scalar, generic, and table.
- Scalar UDFs take one or more input values and return a single output value. For example, a UDF that converts Fahrenheit to Celsius.
- Generic UDFs are similar to scalar UDFs, but they can handle complex data types such as arrays, maps, and structs. For example, a UDF that extracts the first element of an array.
- Table UDFs take one or more input values and return a table of values. For example, a UDF that splits a string into multiple rows based on a delimiter.
- To create a UDF in Java, the following steps are required:
 - Define a class that extends the UDF, GenericUDF, or GenericUDTF class, depending on the type of the UDF.
 - Implement the evaluate method that contains the logic of the UDF.
 - Compile the class into a JAR file and add it to the Hive classpath using the ADD JAR command.
 - Register the UDF using the CREATE FUNCTION command with the name and class of the UDF.
 - Use the UDF in the Hive queries with the registered name.
- To create a UDF in Python, the following steps are required:
 - Define a function that contains the logic of the UDF.
 - Save the function in a Python script file and add it to the Hive distributed cache using the ADD FILE command.
 - Register the UDF using the CREATE FUNCTION command with the name and the path of the Python script file.
 - Use the UDF in the Hive queries with the registered name.
- A mnemonic to remember the types of UDFs is: **S**calar, **G**eneric, and **T**able UDFs are **S**imple, **G**eneral, and **T**abular functions.
- An example of a scalar UDF in Java that converts Fahrenheit to Celsius is:

```

import org.apache.hadoop.hive.ql.exec.UDF;
import org.apache.hadoop.io.DoubleWritable;

public class FahrenheitToCelsius extends UDF {
    public DoubleWritable evaluate(DoubleWritable fahrenheit) {
        if (fahrenheit == null) {
            return null;
        }
        double celsius = (fahrenheit.get() - 32) * 5.0 / 9.0;
        return new DoubleWritable(celsius);
    }
}

```

- An example of a generic UDF in Java that extracts the first element of an array is:

```

import org.apache.hadoop.hive.ql.exec.UDFArgumentException;
import org.apache.hadoop.hive.ql.exec.UDFArgumentLengthException;
import org.apache.hadoop.hive.ql.metadata.HiveException;
import org.apache.hadoop.hive.ql.udf.generic.GenericUDF;
import org.apache.hadoop.hive.serde2.objectinspector.ListObjectInspector;
import org.apache.hadoop.hive.serde2.objectinspector.ObjectInspector;

public class FirstElement extends GenericUDF {
    private ListObjectInspector listOI;

    @Override
    public ObjectInspector initialize(ObjectInspector[] arguments) throws UDFArgumentException {
        if (arguments.length != 1) {
            throw new UDFArgumentLengthException("The function FIRST_ELEMENT takes exactly one arg");
        }
        if (!(arguments[0] instanceof ListObjectInspector)) {
            throw new UDFArgumentException("The function FIRST_ELEMENT takes an array as input.");
        }
        listOI = (ListObjectInspector) arguments[0];
        return listOI.getListElementObjectInspector();
    }

    @Override
    public Object evaluate(DeferredObject[] arguments) throws HiveException {
        Object list = arguments[0].get();
        if (list == null) {
            return null;
        }
        int size = listOI.getListLength(list);
        if (size == 0) {
            return null;
        }
    }
}

```

```

    }
    return listOI.getListElement(list, 0);
}

@Override
public String getDisplayString(String[] children) {
    return "first_element(" + children[0] + ")";
}
}

```

- An example of a table UDF in Java that splits a string into multiple rows based on a delimiter is:

```

import java.util.ArrayList;
import org.apache.hadoop.hive.ql.exec.UDFArgumentException;
import org.apache.hadoop.hive.ql.exec.UDFArgument

```

Sorting and aggregating in Hive

- Sorting and aggregating are common operations in data analysis that involve rearranging and summarizing data based on certain criteria.
- Hive supports two types of sorting: global sorting and bucket sorting.
- Global sorting orders the entire data set according to one or more columns, using the **ORDER BY** clause. For example, **SELECT * FROM table ORDER BY name;** will sort the table by the name column in ascending order.
- Bucket sorting partitions the data into a fixed number of buckets, each of which is sorted by one or more columns, using the **CLUSTER BY** clause. For example, **SELECT * FROM table CLUSTER BY name, age;** will divide the table into buckets based on the name and age columns, and sort each bucket by those columns.
- Bucket sorting is more efficient than global sorting, as it can be done in parallel and does not require shuffling the entire data set across the network. However, bucket sorting requires the table to be bucketed beforehand, using the **CREATE TABLE** statement with the **CLUSTERED BY** and **SORTED BY** clauses.
- Aggregating is the process of applying a function to a group of rows to produce a single value, such as the sum, average, count, minimum, or maximum. Hive supports the **GROUP BY** clause to specify the grouping columns, and the **HAVING** clause to filter the groups based on a condition. For example, **SELECT name, AVG(salary) FROM table GROUP BY name HAVING AVG(salary) > 1000;** will calculate the average salary for each name, and only return the names with an average salary greater than 1000.
- Hive also supports the **DISTINCT** keyword to eliminate duplicate values from a column or a set of columns. For example, **SELECT DISTINCT name FROM table;** will return the unique names from the table.
- A mnemonic to remember the difference between **ORDER BY** and **CLUSTER**

BY is: **O**rder by **O**ne, **C**luster by **C**hunks.

- A mnemonic to remember the syntax of the **GROUP BY** and **HAVING** clauses is: **G**roup **B**efore **H**aving.

Map Reduce scripts in Hive

- Map Reduce is a programming model for processing large-scale data sets in parallel and distributed manner.
- Hive is a data warehouse system that provides a SQL-like interface to query and analyze structured and semi-structured data stored in Hadoop.
- Hive can translate SQL queries into Map Reduce jobs and execute them on Hadoop cluster.
- Hive also supports writing custom Map Reduce scripts in various languages, such as Java, Python, Ruby, etc., and use them as user-defined functions (UDFs), user-defined aggregates (UDAFs), or user-defined table-generating functions (UDTFs).
- Map Reduce scripts in Hive can be useful for performing complex data transformations, custom aggregations, or data analysis that cannot be expressed in SQL.
- To write Map Reduce scripts in Hive, one needs to follow these steps:
 - Define the input and output formats of the script, such as TEXTFILE, SEQUENCEFILE, RCFILE, etc.
 - Write the script in the chosen language and save it as a file with the appropriate extension, such as .py, .rb, .java, etc.
 - Add the script file to the distributed cache using the ADD FILE command in Hive.
 - Use the TRANSFORM clause in the SELECT statement to invoke the script and pass the input columns as standard input to the script.
 - Specify the output columns of the script as standard output using the AS clause.
 - Optionally, use the USING clause to specify the command to run the script, such as python, ruby, java, etc.
- For example, the following Hive query uses a Python script to calculate the average rating of each movie in a table called movies:

```
ADD FILE avg_rating.py;
```

```
SELECT TRANSFORM(movie_id, rating)
  USING 'python avg_rating.py'
  AS (movie_id, avg_rating)
FROM movies
GROUP BY movie_id;
```

- The Python script `avg_rating.py` can be written as:

```
#!/usr/bin/env python
import sys

# Initialize a dictionary to store the sum and count of ratings for each movie
ratings = {}

# Read each line from the standard input
for line in sys.stdin:
    # Split the line by tab and get the movie_id and rating
    movie_id, rating = line.strip().split('\t')
    # Convert the rating to a float
    rating = float(rating)
    # If the movie_id is not in the dictionary, initialize its sum and count to zero
    if movie_id not in ratings:
        ratings[movie_id] = [0, 0]
    # Add the rating to the sum and increment the count for the movie_id
    ratings[movie_id][0] += rating
    ratings[movie_id][1] += 1

# For each movie_id in the dictionary, calculate the average rating and print it to
for movie_id in ratings:
    avg_rating = ratings[movie_id][0] / ratings[movie_id][1]
    print(movie_id + '\t' + str(avg_rating))
```

- Some advantages of using Map Reduce scripts in Hive are:
 - They can leverage the existing skills and libraries of various programming languages.
 - They can perform complex data transformations or analysis that are not possible or efficient in SQL.
 - They can be reused across different queries or tables.
- Some disadvantages of using Map Reduce scripts in Hive are:
 - They can introduce performance overhead and latency due to the serialization and deserialization of data between Hive and the script.
 - They can be difficult to debug and test, especially for distributed and parallel execution.
 - They can be less portable and maintainable than SQL queries, as they depend on the availability and compatibility of the script language and libraries on the cluster.

Joins and subqueries in Hive

- Joins are used to combine data from two or more tables based on a common column or condition.

- Subqueries are used to write a query within another query, usually to filter or aggregate data based on some criteria.
- Hive supports four types of joins: inner join, left outer join, right outer join, and full outer join.
- Hive also supports subqueries in the WHERE and HAVING clauses, but not in the SELECT or FROM clauses.
- Here are some points to remember about joins and subqueries in Hive:
 - Hive does not support join conditions that are not equality conditions, such as `t1.col1 > t2.col2`.
 - Hive does not support self-joins, which are joins of a table with itself.
 - Hive does not support cross joins, which are joins without any condition that produce a Cartesian product of the tables.
 - Hive does not support nested joins, which are joins within another join.
 - Hive does not support correlated subqueries, which are subqueries that reference columns from the outer query.
 - Hive does not support EXISTS or IN operators with subqueries, but you can use LEFT SEMI JOIN instead.
 - Hive does not support scalar subqueries, which are subqueries that return a single value, but you can use a join instead.
- Here are some examples of how to write joins and subqueries in Hive:
 - To perform an inner join of two tables `t1` and `t2` on column `id`, you can write:


```
SELECT t1.*, t2.*
FROM t1
JOIN t2
ON t1.id = t2.id;
```
 - To perform a left outer join of two tables `t1` and `t2` on column `id`, you can write:


```
SELECT t1.*, t2.*
FROM t1
LEFT OUTER JOIN t2
ON t1.id = t2.id;
```
 - To perform a right outer join of two tables `t1` and `t2` on column `id`, you can write:


```
SELECT t1.*, t2.*
FROM t1
RIGHT OUTER JOIN t2
ON t1.id = t2.id;
```

- To perform a full outer join of two tables `t1` and `t2` on column `id`, you can write:

```
SELECT t1.*, t2.*
FROM t1
FULL OUTER JOIN t2
ON t1.id = t2.id;
```

- To perform a subquery in the `WHERE` clause to filter the rows from table `t1` based on the average value of column `col1` from table `t2`, you can write:

```
SELECT *
FROM t1
WHERE t1.col1 > (SELECT AVG(col1) FROM t2);
```

- To perform a subquery in the `HAVING` clause to filter the groups from table `t1` based on the maximum value of column `col2` from table `t2`, you can write:

```
SELECT t1.col1, SUM(t1.col2) AS total
FROM t1
GROUP BY t1.col1
HAVING total < (SELECT MAX(col2) FROM t2);
```

- To perform a left semi join of two tables `t1` and `t2` on column `id`, which is equivalent to using an `IN` operator with a subquery, you can write:

```
SELECT t1.*
FROM t1
LEFT SEMI JOIN t2
ON t1.id = t2.id;
```

- To perform a join of two tables `t1` and `t2` on column `id`, which is equivalent to using a scalar subquery, you can write:

```
SELECT t1.*, (SELECT t2.col1 FROM t2 WHERE t2.id = t1.id) AS col1
FROM t1;
```

- Here are some mnemonics and learning tricks for joins and subqueries in Hive:

- To remember the four types of joins in Hive, you can use the acronym **FIRL** (Full, Inner, Right, Left).
- To remember the syntax of joins in Hive, you can use the formula **SELECT FROM JOIN ON**.
- To remember the syntax of subqueries in Hive, you can use the formula **SELECT FROM WHERE/HAVING (SELECT FROM)**.

HBase

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS) or Alluxio .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases .
- HBase is well suited for real-time data processing or random read/write access to large volumes of data .
- HBase is modeled after Google's Bigtable, a distributed storage system for structured data .
- HBase does not store relational data, and does not have a fixed database schema . This allows developers to add new data without conforming to a schema model.
- HBase is ideal for high-scale real-time applications, such as social media apps or streaming applications.
- HBase consists of tables, which are composed of rows and columns. Each table has a row key and a column family. A column family can have multiple column qualifiers, which are the actual data values. Each cell in a table can have multiple versions, which are identified by timestamps .
- HBase supports CRUD (create, read, update, delete) operations, as well as scan and filter operations on tables .
- HBase also supports secondary indexes, coprocessors, replication, snapshots, and security features .
- HBase can be accessed through various APIs, such as Java, REST, Thrift, or Avro .
- HBase can be integrated with other Hadoop components, such as MapReduce, Spark, Hive, or Pig .

A simple example of an HBase table is shown below:

Row key	Column family: info	Column family: score
Alice	name: Alice	math: 90
	age: 20	english: 85
Bob	name: Bob	math: 80
	age: 21	english: 75
Charlie	name: Charlie	math: 70
	age: 22	english: 65

Some possible mnemonics and learning tricks for HBase are:

- HBase is a **H**adoop **b**ased database that stores data in **H**orizontal columns.
- HBase is a **H**igh-scale **B**igtable-like database that supports **A**PIs, **S**cans, and **E**vents.

- HBase is a **H**uge database that can handle **B**illions of rows and **A**ny kind of data with **S**peed and **E**fficiency.

HBase concepts

- HBase is a type of NoSQL database and is classified as a key-value store. It is an open-source project and is horizontally scalable. It is a distributed column-oriented database built on top of the Hadoop file system (HDFS).
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases. It is well suited for real-time data processing or random read/write access to large volumes of data.
- HBase is a data model that is similar to Google's big table designed to provide quick random access to huge amounts of structured data. It leverages the fault tolerance provided by the Hadoop File System (HDFS).
- HBase has a master-slave architecture, where a master node manages the cluster and region servers store portions of the tables and perform the work on the data. The master node is responsible for assigning regions to region servers, balancing the load, and handling schema changes.
- HBase stores data in tables, which are composed of rows and columns. Each row has a unique identifier called a row key, and each column has a name and a value. A column name consists of a column family and a column qualifier, separated by a colon. A column family is a logical grouping of columns that share some common characteristics, such as compression, encoding, or versioning.
- HBase supports multiple versions of data for each cell, where a cell is the intersection of a row and a column. Each version of data has a timestamp associated with it, which can be either assigned by the system or specified by the user. HBase allows users to query data by specifying a range of timestamps or a maximum number of versions.
- HBase also supports some advanced features, such as filters, coprocessors, and secondary indexes. Filters are used to apply some criteria on the data returned by a scan or a get operation. Coprocessors are user-defined code that can be executed on the region server side to perform some custom logic or aggregation. Secondary indexes are used to create alternative access paths to the data based on some column values.

Some mnemonics and learning tricks for HBase concepts are:

- HBase stands for Hadoop database, which indicates its relation to HDFS and MapReduce.
- HBase is a column-oriented database, which means it stores data by columns rather than by rows. This can be remembered by the word COBRA (ColumN-Based RANdom access).
- HBase has a master-slave architecture, which can be remembered by the word MASH (MAster-Slave HBase).
- HBase stores data in tables, which can be remembered by the word TAB

(Table-Based).

- HBase supports multiple versions of data, which can be remembered by the word MUD (Multiple Data).
- HBase has a row key, a column family, and a column qualifier, which can be remembered by the word ROCK (ROw Key, COlumn family, COlumn Qualifier).

HBase clients

- HBase clients are the applications that can interact with HBase using different programming languages and APIs.
- HBase provides a native Java client that uses the HBase API to perform CRUD (create, read, update, delete) operations on HBase tables.
- HBase also supports other clients that use different protocols and interfaces, such as REST, Thrift, and Avro.
- The REST client uses HTTP requests to access HBase through a REST server that acts as a proxy between the client and the HBase cluster.
- The Thrift client uses a Thrift server that exposes a subset of the HBase API through a Thrift interface that can be accessed by various languages, such as Python, Ruby, and PHP.
- The Avro client uses an Avro server that exposes a similar interface as the Thrift server, but uses Avro as the data serialization format.
- HBase clients can be classified into two types: thick and thin.
- Thick clients are those that have direct access to the HBase cluster and can use the full features of the HBase API, such as filters, scanners, and coprocessors. The Java client is an example of a thick client.
- Thin clients are those that rely on an intermediate server to communicate with the HBase cluster and can only use a limited subset of the HBase API, such as basic CRUD operations. The REST, Thrift, and Avro clients are examples of thin clients.
- The advantages of using thick clients are that they have better performance, more functionality, and more control over the HBase cluster.
- The disadvantages of using thick clients are that they require more dependencies, more configuration, and more memory on the client side.
- The advantages of using thin clients are that they are simpler, lighter, and more portable across different languages and platforms.
- The disadvantages of using thin clients are that they have higher latency, less functionality, and less control over the HBase cluster.

Here is a table that summarizes the main features and differences of the HBase clients:

Client	Protocol	Interface	Language	Type	Pros	Cons
Java	RPC	HBase API	Java	Thick	High performance, full functionality, direct access to HBase cluster	More dependencies, more configuration, more memory
REST	HTTP	REST API	Any	Thin	Simplicity, portability, no dependencies	Higher latency, less functionality, indirect access to HBase cluster
Thrift	Thrift	Thrift API	Any	Thin	Simplicity, portability, no dependencies	Higher latency, less functionality, indirect access to HBase cluster

Client	Protocol	Interface	Language	Type	Pros	Cons
Avro	Avro	Avro API	Any	Thin	Simplicity, portability, no dependencies	Higher latency, less functionality, indirect access to HBase cluster

Here is an example of how to use the Java client to create a table and insert a row in HBase:

```
// Import the HBase classes
import org.apache.hadoop.hbase.HBaseConfiguration;
import org.apache.hadoop.hbase.TableName;
import org.apache.hadoop.hbase.client.Connection;
import org.apache.hadoop.hbase.client.ConnectionFactory;
import org.apache.hadoop.hbase.client.Table;
import org.apache.hadoop.hbase.client.Put;
import org.apache.hadoop.hbase.util.Bytes;

// Create a configuration object
Configuration config = HBaseConfiguration.create();

// Create a connection object
Connection connection = ConnectionFactory.createConnection(config);

// Get a reference to the table
TableName tableName = TableName.valueOf("test");
Table table = connection.getTable(tableName);

// Create a put object with a row key
Put put = new Put(Bytes.toBytes("row1"));

// Add a column family, qualifier, and value to the put object
put.addColumn(Bytes.toBytes("cf1"), Bytes.toBytes("q1"), Bytes.toBytes("value1"));

// Put the row to the table
table.put(put);
```

```
// Close the table and the connection  
table.close();  
connection.close();
```

HBase example

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS) .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases .
- HBase is well suited for real-time data processing or random read/write access to large volumes of data .
- HBase can be installed in two modes: standalone mode and distributed mode .
- Standalone mode is for development and testing purposes, where HBase runs on a single machine without HDFS .
- Distributed mode is for production purposes, where HBase runs on a cluster of machines with HDFS .
- An HBase table consists of rows and columns, where each row has a unique identifier called row key .
- An HBase column represents an attribute of an object; for example, if the table is storing diagnostic logs from servers, each row might be a log record, and a typical column could be the timestamp of when the log record was written, or the server name where the record originated .
- An HBase column is organized into column families, which are groups of columns that share a common prefix and are stored together on disk .
- An HBase column can have multiple versions, which are stored as timestamps .
- An HBase table can have one or more regions, which are horizontal partitions of the table based on the row key range .
- An HBase region is assigned to a region server, which is a node in the cluster that handles read and write requests for that region .
- An HBase region server can serve multiple regions, and a region can be moved from one region server to another for load balancing or fault tolerance .
- An HBase master is a node in the cluster that coordinates the region servers and handles metadata operations, such as creating and deleting tables, splitting and merging regions, and assigning regions to region servers .
- An HBase client is a node in the cluster or outside the cluster that interacts with the HBase master and region servers to perform data operations, such as inserting, updating, deleting, and querying data .
- An HBase client can use different APIs, such as Java API, REST API, Thrift API, or HBase shell, to communicate with the HBase cluster .
- An HBase shell is a command-line interface that allows the user to execute

HBase commands using Ruby syntax .

- An HBase command can be used to create, alter, drop, list, scan, put, get, delete, count, truncate, or describe tables, columns, or rows .
- An example of creating a table in HBase using the shell is:

```
hbase (main):001:0> create 'education','guru99'
0 rows (s) in 0.312 seconds
=>Hbase::Table - education
```

- The above example explains how to create a table in HBase with the name 'education' and the column family 'guru99' .
- An example of inserting data into the table using the shell is:

```
hbase (main):002:0> put 'education','row1','guru99:course','HBase'
0 rows (s) in 0.031 seconds
```

- The above example explains how to insert a row with the row key 'row1' and the column 'guru99:course' with the value 'HBase' into the table 'education' .
- An example of querying data from the table using the shell is:

```
hbase (main):003:0> get 'education','row1'
COLUMN                                CELL
guru99:course                        timestamp=1639492614, value=HBase
1 row (s) in 0.015 seconds
```

- The above example explains how to retrieve the row with the row key 'row1' from the table 'education' .
- An example of deleting data from the table using the shell is:

```
hbase (main):004:0> delete 'education','row1','guru99:course'
0 rows (s) in 0.015 seconds
```

- The above example

HBase vs RDBMS

- HBase and RDBMS are both database management systems that store and manipulate data.
- HBase is a NoSQL database that is based on the Hadoop Distributed File System (HDFS) and supports semi-structured and unstructured data. RDBMS is a relational database that is based on the Structured Query Language (SQL) and supports structured data.
- HBase is designed for horizontal scalability, high availability, and fast random access. RDBMS is designed for vertical scalability, ACID transactions, and complex queries.
- HBase stores data in tables that consist of rows and columns, where each row has a unique key and each column belongs to a column family. RDBMS stores data in tables that consist of rows and columns, where

each row has a primary key and each column has a data type and constraints.

- HBase supports sparse tables, where not every row has to have a value for every column. RDBMS supports normalized tables, where data is organized into multiple related tables to avoid redundancy and inconsistency.
- HBase does not support joins, subqueries, or aggregations natively, but relies on external tools such as Hive or Pig to perform these operations. RDBMS supports joins, subqueries, and aggregations natively, and provides various functions and operators to manipulate data.
- HBase does not enforce a schema on the data, but allows the user to define the column families and the compression and encoding options for each column family. RDBMS enforces a schema on the data, and requires the user to define the tables and the columns before inserting data.
- HBase is suitable for applications that need to handle large volumes of data, such as web analytics, social media, or sensor data. RDBMS is suitable for applications that need to handle complex transactions, such as banking, e-commerce, or inventory management.

A possible mnemonic to remember the differences between HBase and RDBMS is:

- HBase is Horizontal, Hadoop, and High-speed.
- RDBMS is Relational, SQL, and Structured.

Advanced usage of HBase

- HBase is a distributed, scalable, and column-oriented database that runs on top of the Hadoop Distributed File System (HDFS).
- HBase can store large amounts of data in a tabular format, where each table consists of rows and columns.
- HBase supports random read/write access to the data, as well as batch processing and map-reduce operations.
- HBase also provides features such as data replication, automatic sharding, load balancing, and fault tolerance.

Some of the advanced usage of HBase are:

- **Data modeling:** HBase does not have a fixed schema, which means that the columns and their types can vary for each row. This gives flexibility to the data model, but also requires careful design to optimize the performance and storage efficiency. Some of the data modeling techniques for HBase are:
 - **Denormalization:** This involves storing the data in a single table, rather than splitting it into multiple tables with foreign keys. This reduces the number of joins and improves the query speed, but also increases the data redundancy and size.
 - **Column families:** This involves grouping the columns into logical units, called column families, that are stored together on disk. This

improves the data locality and compression, but also limits the number of column families per table to a few, as each column family requires a separate file.

- **Row keys:** This involves choosing the row keys that uniquely identify each row, and also determine the sorting order of the rows. This affects the query performance and the distribution of the data across the regions. Some of the row key design patterns are:
 - * **Hash-based:** This involves using a hash function to generate the row keys, which ensures a uniform distribution of the data, but also makes the row keys meaningless and hard to query.
 - * **Time-based:** This involves using a timestamp or a sequence number as the row key, which preserves the temporal order of the data, but also creates hotspots and skewness in the data.
 - * **Composite:** This involves combining multiple attributes into the row key, which allows for more expressive queries, but also increases the row key size and complexity.
- **Data access:** HBase provides various ways to access the data, such as:
 - **HBase shell:** This is a command-line interface that allows users to perform basic operations on HBase tables, such as creating, dropping, scanning, and modifying the data. The HBase shell uses a Ruby-based syntax and supports tab completion and history.
 - **HBase API:** This is a Java-based API that allows users to perform advanced operations on HBase tables, such as creating filters, coprocessors, and custom comparators. The HBase API also supports batch operations, atomic operations, and transactions.
 - **HBase clients:** These are libraries or frameworks that provide higher-level abstractions and integrations for HBase, such as:
 - * **Thrift and REST:** These are web services that expose HBase operations as HTTP endpoints, which can be accessed by any language or platform. They also support JSON and XML formats for the data.
 - * **Phoenix:** This is a SQL engine that allows users to query HBase tables using standard SQL syntax and JDBC/ODBC drivers. It also supports secondary indexes, views, and joins.
 - * **Spark:** This is a distributed computing framework that allows users to perform analytics and machine learning on HBase tables using Scala, Python, or Java. It also supports streaming, graph, and SQL operations.
- **Data management:** HBase provides various tools and techniques to manage the data, such as:
 - **Compression:** This involves reducing the size of the data by applying algorithms such as GZIP, LZO, or Snappy. This improves the disk and network efficiency, but also adds some CPU overhead.
 - **Compaction:** This involves merging multiple small files into larger ones, which reduces the number of files and improves the read performance. HBase performs two types of compaction: minor and major.

Minor compaction merges files within a column family, while major compaction merges all the files in a column family and removes the deleted or expired data.

- **Bloom filters:** These are probabilistic data structures that allow users to check whether a row or a column exists in a file, without reading the entire file. This reduces the number of disk seeks and improves the query performance, but also consumes some memory.
- **HBase backup and restore:** This involves copying the data from one HBase cluster to another, for disaster recovery or migration purposes. HBase supports two types of backup and restore: full and incremental. Full backup and restore copies all the data from the source cluster to the destination cluster, while incremental backup and restore copies

Schema design in HBase

- HBase is a column-oriented, NoSQL database that runs on top of Hadoop. It does not support any kind of joins, but it provides the single-indexing strategy on the row key.
- HBase schema design supports denormalization with nested entities. These nested entities are nothing but a column whose name is the unique identifier for the nested entity and whose value is the entire record mashed together. Since HBase allows dynamic column definition, there's no problem.
- HBase schema design is very different compared to the relation database schema design. Below are some of general concept that should be followed while designing schema in Hbase:
 - Row key: Each table in HBase table is indexed on row key. Data is sorted lexicographically by this row key. The row key should be chosen carefully to ensure good performance and scalability. It should be unique, short, and meaningful. It should also avoid hotspots, which are regions of the table that receive more read or write requests than others. A common technique to avoid hotspots is to use salting, which is adding a random prefix to the row key to distribute the load evenly across the cluster.
 - Column family: A column family is a logical grouping of columns that share some common characteristics, such as compression, encoding, or versioning. A column family must be defined at the table creation time, and cannot be changed later. A column family can have one or more columns, which are identified by a qualifier. A column family name and a column qualifier name are separated by a colon, such as cf1:col1. A column family can have a maximum size of 2 GB, and should be kept as small as possible to reduce the I/O overhead.
 - Column: A column is a key-value pair that belongs to a column family. A column can have multiple versions, which are identified by a timestamp. A column can store any type of data, such as strings,

- numbers, or binary data. A column can be added or deleted dynamically, without affecting the schema of the table.
 - Cell: A cell is the smallest unit of data in HBase. It is the intersection of a row and a column. A cell can have a maximum size of 10 MB, and should be kept as small as possible to reduce the memory consumption and network latency.
- HBase schema design should follow some best practices, such as :
 - Use short and descriptive names for row keys, column families, and columns.
 - Avoid using composite row keys, unless they are necessary for efficient range scans or secondary indexing.
 - Use compression and encoding to reduce the disk space and network bandwidth usage.
 - Use Bloom filters to reduce the number of disk seeks for non-existent rows or columns.
 - Use filters to optimize read performance by applying predicates on the server side.
 - Use coprocessors to implement custom logic on the server side, such as aggregation, join, or trigger.
 - Use bulk loading to import large amounts of data into HBase efficiently.
 - Use compaction and major compaction to merge small files and delete obsolete data periodically.
 - Use regions and region servers to distribute and balance the data and load across the cluster.
 - Use replication and backup to ensure high availability and durability of the data.

Advanced indexing in HBase

- HBase is a distributed, column-oriented database that stores data in Hadoop Distributed File System (HDFS).
- HBase does not support secondary indexes, which are indexes on non-key columns that can speed up queries on those columns.
- To implement secondary indexes in HBase, there are two main approaches: **co-processor-based indexing** and **application-level indexing**.

Co-processor-based indexing

- Co-processor-based indexing uses HBase co-processors, which are user-defined code that runs on the same JVM as the region server and can intercept read and write requests.
- Co-processor-based indexing can be implemented using two types of co-processors: **observer co-processors** and **endpoint co-processors**.
- Observer co-processors are similar to triggers in relational databases. They can perform actions before or after a read or write operation on a table

or a region.

- Endpoint co-processors are similar to stored procedures in relational databases. They can execute custom logic on a region and return the result to the client.
- Co-processor-based indexing can maintain secondary indexes in separate tables, which are updated by observer co-processors whenever the primary table is modified.
- Co-processor-based indexing can also use endpoint co-processors to perform index lookups and join the results with the primary table.
- Co-processor-based indexing has the following advantages:
 - It is transparent to the application, as the co-processors handle the index creation and maintenance.
 - It is efficient, as the co-processors can leverage the local data access and avoid network overhead.
 - It is scalable, as the co-processors can parallelize the index operations across the region servers.
- Co-processor-based indexing has the following disadvantages:
 - It requires custom code development and deployment, which can be complex and error-prone.
 - It can introduce performance and consistency issues, as the co-processors can affect the primary table operations and the index tables can become stale or inconsistent.
 - It can increase the storage and maintenance cost, as the index tables can consume more space and resources than the primary table.

Application-level indexing

- Application-level indexing is an alternative approach that does not rely on co-processors, but rather on the application logic to create and maintain secondary indexes.
- Application-level indexing can be implemented using two techniques: **dual writes** and **index tables**.
- Dual writes is a technique that writes the data to both the primary table and the index table in the same operation, using either a **put** or a **batch** method.
- Index tables are tables that store the secondary index values and the corresponding primary key values, which can be used to retrieve the data from the primary table.
- Application-level indexing has the following advantages:
 - It is simple and flexible, as it does not require any custom code development or deployment on the HBase side.
 - It is consistent, as it ensures that the primary table and the index table are updated atomically and synchronously.
- Application-level indexing has the following disadvantages:
 - It is not transparent to the application, as the application has to handle the index creation and maintenance.

- It is not efficient, as it introduces extra write and read operations to the primary table and the index table.
- It is not scalable, as it can create hotspots and bottlenecks on the region servers that host the index table regions.

Zookeeper

- Zookeeper is a distributed coordination service that provides a consistent, reliable and scalable way to manage configuration, synchronization, naming, leader election and group membership of a cluster of nodes.
- Zookeeper is based on the concept of a **znode**, which is a data structure that represents a file or a directory in a hierarchical namespace. A znode can store data, have children znodes, and have a version number that changes whenever the znode is modified.
- Zookeeper maintains a **quorum** of servers that replicate the znode data and handle client requests. A quorum is a majority of servers that agree on the state of the system. A quorum ensures that Zookeeper can tolerate failures and network partitions without losing data consistency.
- Zookeeper provides four types of guarantees to its clients: **sequential consistency**, **atomicity**, **single system image**, and **reliability**. These guarantees mean that Zookeeper will execute client operations in the order they are received, apply them atomically, present the same view of the system to all clients, and recover from failures without losing data.
- Zookeeper exposes a simple API to its clients, consisting of four basic operations: **create**, **delete**, **read**, and **write**. Clients can also use **watchers** to get notified of changes to znodes, and **ephemeral nodes** to create znodes that are automatically deleted when the client session ends.
- Zookeeper can be used for various distributed system scenarios, such as:
 - **Configuration management**: Zookeeper can store and update configuration parameters for a cluster of nodes, and notify the nodes of any changes.
 - **Synchronization**: Zookeeper can provide barriers, locks, and queues to coordinate the actions of multiple nodes.
 - **Naming**: Zookeeper can assign unique names or identifiers to nodes or resources in a cluster, and resolve them to their locations or addresses.
 - **Leader election**: Zookeeper can elect a leader among a group of nodes, and handle leader changes in case of failures or network partitions.
 - **Group membership**: Zookeeper can keep track of the nodes that belong to a group, and their status and availability.

Zookeeper concepts

- Zookeeper is a distributed coordination service that provides a consistent, reliable and scalable way of managing configuration information, naming,

synchronization, group membership and leader election among distributed systems.

- Zookeeper consists of a set of servers, called **ZooKeeper servers**, and a set of clients, called **ZooKeeper clients**, that communicate with each other using a TCP-based protocol.
- Zookeeper maintains a hierarchical namespace of **znodes**, which are data nodes that store configuration data or other metadata. Each znode has a path, such as `/app/config`, that uniquely identifies it in the namespace. Znodes can also have children znodes, forming a tree structure.
- Zookeeper provides four types of znodes: **persistent**, **ephemeral**, **sequential** and **container**. Persistent znodes are permanent and remain in the namespace until explicitly deleted. Ephemeral znodes are temporary and are automatically deleted when the client that created them disconnects. Sequential znodes are persistent znodes that have a unique sequence number appended to their names. Container znodes are persistent znodes that are automatically deleted when they have no children.
- Zookeeper allows clients to perform various operations on znodes, such as **create**, **delete**, **read**, **write**, **setACL**, **getACL**, **getChildren**, **exists** and **sync**. These operations are atomic and linearizable, meaning that they are executed as a single unit and in the same order on all servers.
- Zookeeper also allows clients to set **watches** on znodes, which are callbacks that are triggered when a znode changes. Watches can be set for data changes, children changes or existence changes. Watches are one-time triggers and need to be re-registered after each notification.
- Zookeeper guarantees three consistency properties: **sequential consistency**, **atomicity** and **single system image**. Sequential consistency means that updates from a client will be applied in the order that they were sent. Atomicity means that updates either succeed or fail, and no partial results are visible. Single system image means that clients will see the same view of the system regardless of the server they connect to.
- Zookeeper uses a consensus protocol called **Zab** (ZooKeeper Atomic Broadcast) to ensure that all servers have the same state of the namespace. Zab works by electing a leader server that proposes updates to the namespace and broadcasts them to the follower servers. The followers acknowledge the updates and apply them to their local copies. The leader commits the updates once a majority of followers have acknowledged them.
- Zookeeper can tolerate failures of some servers as long as a majority of servers are available. This is called a **quorum**. For example, if there are five servers, then at least three servers need to be up for Zookeeper to function. If the leader server fails, then a new leader is elected from the remaining servers using a protocol called **Fast Leader Election**.
- Zookeeper can be used for various distributed system scenarios, such as **configuration management**, **service discovery**, **distributed locking**, **barrier synchronization**, **leader election** and **queue management**. For example, Zookeeper can store the configuration data of an

application in a znode and notify the clients of any changes using watches. Zookeeper can also register the available services in ephemeral znodes and allow the clients to discover them using `getChildren`. Zookeeper can also implement a distributed lock by creating an ephemeral sequential znode and checking if it is the lowest in the order.

How ZooKeeper helps in monitoring a cluster

- ZooKeeper is a distributed coordination service that helps in managing configuration information, naming, group services, and synchronization for distributed applications.
- ZooKeeper provides a hierarchical namespace of znodes (data nodes) that can store data and have children. Znodes can be ephemeral (deleted when the client session ends) or persistent (remain until explicitly deleted).
- ZooKeeper maintains a consistent view of the cluster state by using a consensus protocol called Zab (ZooKeeper Atomic Broadcast). Zab ensures that a quorum (majority) of ZooKeeper servers agree on the same order of updates to the znodes.
- ZooKeeper allows clients to register watches on znodes to get notified of any changes. This helps in monitoring the cluster state and detecting failures or configuration changes.
- ZooKeeper also provides ephemeral sequential znodes that can be used to implement leader election, locking, and queueing mechanisms. These help in coordinating distributed tasks and ensuring consistency across the cluster.
- ZooKeeper exposes various metrics and statistics through JMX (Java Management Extensions) and Prometheus endpoints. These can be used to monitor the performance, availability, and health of the ZooKeeper cluster and its clients.
- ZooKeeper can be integrated with various monitoring tools such as Sente, Prometheus, Grafana, ManageEngine, Site24x7, OpsView, InStana, SignalFx, Datadog, and AppDynamics to visualize and analyze the ZooKeeper metrics and alerts.

Some mnemonics and learning tricks for how ZooKeeper helps in monitoring a cluster are:

- ZooKeeper is like a zoo keeper who takes care of the animals (distributed applications) and their habitats (znodes).
- ZooKeeper uses Zab to keep the zoo in order and prevent chaos.
- ZooKeeper watches over the zoo and notifies the visitors (clients) of any changes or events.
- ZooKeeper uses ephemeral sequential znodes to organize the zoo activities and ensure fairness and consistency.
- ZooKeeper can be monitored by using JMX and Prometheus tools or by visiting the zoo with other tools.

How to build applications with Zookeeper

ZooKeeper is a distributed system coordinator that provides services such as configuration management, synchronization, naming, and leader election. It can be used to build reliable and scalable distributed applications that require coordination among multiple nodes or processes. Here are some steps to build applications with ZooKeeper:

- Install ZooKeeper on one or more servers. You can download a stable ZooKeeper release from the official website and follow the instructions to set up a configuration file and start the server. Alternatively, you can use a containerized solution such as Kubernetes to deploy ZooKeeper as a StatefulSet with a Headless Service and a PodDisruptionBudget.
- Connect to ZooKeeper from your application using a client library. ZooKeeper provides official Java and C client libraries, as well as bindings for other languages such as Python, Ruby, and Go. You can use the client library to create, read, update, and delete znodes, which are the basic units of data stored in ZooKeeper. Znodes can have data, children, and metadata, such as version, timestamp, and access control list.
- Use ZooKeeper to implement common distributed application patterns, such as configuration management, synchronization, naming, and leader election. For example, you can use ZooKeeper to store and update configuration parameters for your application, and watch for changes using the client library. You can also use ZooKeeper to create locks, barriers, queues, and other synchronization primitives to coordinate concurrent processes. You can use ZooKeeper to register and discover services, and assign unique names or identifiers to nodes or processes. You can also use ZooKeeper to elect a leader among a group of candidates, and handle leader changes or failures.
- Follow the best practices and recommendations for using ZooKeeper, such as choosing an appropriate session timeout, avoiding large or frequent writes, using sequential or ephemeral znodes, and handling exceptions and errors. You can also use tools and frameworks that build on top of ZooKeeper, such as Apache Curator, to simplify the development and management of your application. You can also monitor and troubleshoot your ZooKeeper cluster using tools such as JMX, ZK CLI, and ZK Web UI.

Some mnemonics and learning tricks for how to build applications with Zookeeper are:

- Remember the four-letter word CRUD (Create, Read, Update, Delete) to perform basic operations on znodes using the client library.
- Remember the acronym ZAB (ZooKeeper Atomic Broadcast) to understand how ZooKeeper achieves consensus and reliability among the servers in a cluster.
- Remember the word WATCH (Write, Access, Trigger, Change, Handle) to use the watch mechanism to get notifications of znode changes and handle

them accordingly.

IBM Big Data strategy

- IBM Big Data strategy is a corporate initiative that aims to provide solutions for storing, managing, and analyzing the huge amounts of data generated daily by various sources, such as social media, sensors, mobile devices, etc.
- IBM Big Data strategy is part of the Smarter Planet vision, which seeks to leverage smarter systems to achieve economic and sustainable growth and societal progress.
- IBM Big Data strategy consists of four main components: Big Data platform, Big Data services, Big Data solutions, and Big Data ecosystem.
- Big Data platform: IBM offers a comprehensive and integrated platform that supports various types of data, such as structured, unstructured, semi-structured, streaming, etc. The platform includes products such as IBM InfoSphere BigInsights, IBM InfoSphere Streams, IBM PureData System for Analytics, IBM PureData System for Operational Analytics, IBM DB2 with BLU Acceleration, IBM Cognos Business Intelligence, IBM SPSS, etc.
- Big Data services: IBM provides consulting, implementation, and support services to help clients design, deploy, and optimize their Big Data projects. The services include IBM Big Data and Analytics Services, IBM Big Data and Analytics Cloud Services, IBM Big Data and Analytics Managed Services, IBM Big Data and Analytics Education Services, etc.
- Big Data solutions: IBM delivers industry-specific and cross-industry solutions that address various business challenges and opportunities, such as customer analytics, fraud detection, risk management, operational efficiency, etc. The solutions include IBM Customer Insight Solutions, IBM Counter Fraud Management, IBM Watson Foundations, IBM Content Analytics, IBM Social Media Analytics, etc.
- Big Data ecosystem: IBM collaborates with a network of partners, such as technology providers, solution providers, academic institutions, research organizations, etc., to foster innovation and co-creation in the Big Data domain. The ecosystem includes IBM Big Data and Analytics Hub, IBM Big Data University, IBM Big Data Innovation Centers, IBM Big Data Partner Program, etc.
- A possible mnemonic to remember the four components of IBM Big Data strategy is **P**latform, **S**ervices, **S**olutions, and **E**cosystem, or **PSSE**.

IBM Big Data strategy

- IBM Big Data strategy is a corporate initiative that aims to provide solutions for storing, managing, and analyzing the huge amounts of data

generated daily by various sources, such as social media, sensors, mobile devices, etc.

- IBM Big Data strategy is aligned with its Smarter Planet vision, which seeks to leverage smarter systems to achieve economic and sustainable growth and societal progress.
- IBM Big Data strategy consists of four dimensions: volume, variety, velocity, and veracity.
 - Volume refers to the scale of data, which can range from terabytes to zettabytes.
 - Variety refers to the diversity of data types and sources, which can include structured, unstructured, and semi-structured data.
 - Velocity refers to the speed of data generation and processing, which can require real-time or near-real-time analysis.
 - Veracity refers to the quality and reliability of data, which can affect the accuracy and trustworthiness of insights.
- IBM Big Data strategy offers a comprehensive portfolio of products and services that cover the entire data lifecycle, from data ingestion, integration, and governance, to data storage, processing, and analysis, to data visualization, exploration, and discovery.
- IBM Big Data strategy leverages various technologies and platforms, such as:
 - IBM Cloud Pak for Data, which is a cloud-native data and AI platform that enables users to collect, organize, and analyze data across hybrid cloud environments.
 - IBM Watson, which is a suite of AI services and applications that can understand, reason, and learn from data, and provide natural language and conversational interfaces.
 - IBM Db2, which is a family of relational database management systems that can handle structured and unstructured data, and support various data models and languages.
 - IBM Netezza, which is a data warehouse appliance that can perform high-performance analytics on large-scale data sets.
 - IBM InfoSphere, which is a platform for data integration, quality, and governance, that can handle various data sources, formats, and standards.
 - IBM Cognos, which is a business intelligence and analytics software that can provide interactive dashboards, reports, and visualizations.
 - IBM SPSS, which is a statistical analysis and predictive modeling software that can perform various data mining and machine learning techniques.
 - IBM Streams, which is a platform for real-time data processing and analytics, that can handle streaming data from various sources and applications.
- IBM Big Data strategy also involves various partnerships and collaborations with other companies and organizations, such as Cloudera, Hortonworks, Apache, and Open Data Platform Initiative, to support open source

and industry standards for Big Data technologies and solutions.

- IBM Big Data strategy aims to help its clients and partners to gain insights and value from their data, and to drive innovation and transformation in various domains and industries, such as healthcare, retail, banking, education, government, etc.

Introduction to Infosphere

- Infosphere is a term that refers to the global network of information and communication technologies (ICTs) that enable the creation, storage, access, and exchange of data and knowledge.
- Infosphere is also a concept that describes the social, cultural, and ethical implications of living in a world where information is abundant, pervasive, and influential.
- Infosphere is composed of three layers: the physical layer, the syntactic layer, and the semantic layer.
 - The physical layer consists of the hardware and infrastructure that support the transmission and storage of data, such as computers, cables, satellites, servers, etc.
 - The syntactic layer consists of the software and protocols that enable the encoding, decoding, and processing of data, such as languages, formats, algorithms, etc.
 - The semantic layer consists of the meaning and interpretation of data, such as concepts, categories, values, norms, etc.
- Infosphere is characterized by some features, such as:
 - Interconnectivity: the ability of different devices, systems, and agents to communicate and interact with each other across time and space.
 - Interoperability: the ability of different devices, systems, and agents to exchange and use data and information in a compatible and consistent way.
 - Interactivity: the ability of different devices, systems, and agents to influence and respond to each other in a dynamic and reciprocal way.
 - Intelligence: the ability of different devices, systems, and agents to perform tasks that require reasoning, learning, adaptation, and problem-solving.
- Infosphere poses some challenges, such as:
 - Information overload: the difficulty of managing and processing the large amount of data and information that is available and accessible.
 - Information quality: the reliability, accuracy, relevance, and timeliness of data and information that is produced and consumed.
 - Information security: the protection of data and information from unauthorized access, use, modification, and disclosure.
 - Information ethics: the moral and legal principles that govern the production, distribution, and use of data and information in the infosphere.
- Infosphere offers some opportunities, such as:

- Information literacy: the skills and competencies that enable individuals and groups to access, evaluate, and use data and information effectively and responsibly in the infosphere.
- Information empowerment: the benefits and advantages that individuals and groups can gain from accessing, creating, and sharing data and information in the infosphere.
- Information innovation: the creation and development of new and improved data and information products, services, and solutions in the infosphere.
- Information democracy: the participation and representation of individuals and groups in the governance and regulation of data and information in the infosphere.

Introduction to BigInsights

- BigInsights is an IBM product that helps firms analyze the increasing volume, velocity and veracity of data of their interest .
- BigInsights does not replace a relational database management system (DBMS) or a traditional data warehouse, but rather complements them by providing a scalable and flexible platform for processing and analyzing diverse types of data, including data that is loosely structured or largely unstructured .
- BigInsights is based on the open source Apache Hadoop framework, which is a distributed system that allows parallel processing of large amounts of data across multiple nodes in a cluster.
- BigInsights enhances the Hadoop framework with additional features and tools, such as:
 - Big SQL: a SQL engine that allows querying data stored in Hadoop using standard SQL syntax and JDBC/ODBC drivers.
 - BigSheets: a spreadsheet-like interface that allows users to explore, visualize and analyze data in Hadoop without writing code.
 - Big R: an integration of R, a popular statistical programming language, with Hadoop, that allows users to apply advanced analytics and machine learning algorithms to data in Hadoop.
 - BigInsights Text Analytics: a tool that allows users to extract insights from unstructured text data, such as social media posts, emails, documents, etc., using natural language processing and text mining techniques.
 - BigInsights Enterprise Management: a set of tools that help users to manage, monitor and secure their BigInsights clusters, such as Ambari, Knox, Ranger, etc..
- BigInsights can be deployed on-premise, on cloud or in a hybrid mode, depending on the needs and preferences of the users.
- BigInsights can be used for various applications and use cases, such as:
 - Customer analytics: to understand customer behavior, preferences, sentiment, churn, etc., from various sources of data, such as transac-

- tions, web logs, social media, etc..
- Fraud detection: to identify and prevent fraudulent activities, such as credit card fraud, insurance fraud, etc., by analyzing data from multiple sources and applying anomaly detection and pattern recognition techniques.
- Risk management: to assess and mitigate various types of risks, such as credit risk, market risk, operational risk, etc., by analyzing data from internal and external sources and applying predictive modeling and simulation techniques.
- Sentiment analysis: to measure and monitor the opinions, emotions and attitudes of customers, employees, competitors, etc., from various sources of text data, such as reviews, surveys, blogs, tweets, etc..
- Recommendation systems: to provide personalized and relevant suggestions to customers, such as products, services, content, etc., based on their past behavior, preferences, context, etc., by analyzing data from multiple sources and applying collaborative filtering and content-based filtering techniques.

Introduction to Big Sheets

- Big Sheets is a web-based spreadsheet application that can process large-scale data sets using Apache Hadoop and IBM BigInsights.
- Big Sheets allows users to explore, visualize, and analyze data using familiar spreadsheet functions and formulas, without requiring any programming skills.
- Big Sheets can handle data sets that are too large or complex for traditional spreadsheet applications, such as Excel or Google Sheets.
- Big Sheets can import data from various sources, such as HDFS, Hive, Pig, Flume, or Sqoop, and export data to CSV, PDF, or HTML formats.
- Big Sheets can also integrate with other BigInsights components, such as Big SQL, Big R, or BigSheetsML, to perform advanced analytics and machine learning on the data.
- Big Sheets provides a web-based interface that consists of the following elements:
 - A workbook, which is a collection of sheets that contain data and formulas.
 - A sheet, which is a grid of cells that can store data, formulas, or charts.
 - A cell, which is the smallest unit of data or formula in a sheet.
 - A formula bar, which is a text box that displays and edits the formula or value of the selected cell.
 - A toolbar, which provides various options and commands for working with the workbook, sheet, or cell.
 - A status bar, which shows information about the workbook, sheet, or cell, such as the number of rows and columns, the cell format, or the calculation status.

- A sidebar, which provides access to various features and functions, such as data sources, filters, charts, macros, or functions.
- A possible mnemonic to remember the elements of the Big Sheets interface is: **Wish Cell Formula Tool Status Side** (Workbook, Sheet, Cell, Formula bar, Toolbar, Status bar, Sidebar).
- A possible learning trick to understand the concept of Big Sheets is to compare it with a traditional spreadsheet application, such as Excel or Google Sheets, and note the similarities and differences. For example:
 - Similarities: Both Big Sheets and traditional spreadsheet applications use the same basic concepts of workbooks, sheets, cells, formulas, and charts. Both Big Sheets and traditional spreadsheet applications provide a range of functions and formulas to manipulate and analyze data. Both Big Sheets and traditional spreadsheet applications allow users to export data to various formats, such as CSV, PDF, or HTML.
 - Differences: Big Sheets can handle much larger and more complex data sets than traditional spreadsheet applications, using Apache Hadoop and IBM BigInsights. Big Sheets can import data from various sources, such as HDFS, Hive, Pig, Flume, or Sqoop, while traditional spreadsheet applications usually import data from local files or databases. Big Sheets can integrate with other BigInsights components, such as Big SQL, Big R, or BigSheetsML, to perform advanced analytics and machine learning on the data, while traditional spreadsheet applications usually require external tools or add-ons for such tasks. Big Sheets provides a web-based interface, while traditional spreadsheet applications usually provide a desktop-based interface.

Introduction to Big SQL

- Big SQL is a SQL engine for Apache Hadoop that allows you to query and analyze data stored in Hadoop using the standard SQL syntax.
- Big SQL is compatible with the ANSI SQL:2011 standard and supports most of the SQL features, such as joins, subqueries, aggregations, window functions, common table expressions, etc.
- Big SQL can access data from various sources, such as Hive tables, HBase tables, Spark tables, ORC files, Parquet files, CSV files, JSON files, etc.
- Big SQL can also integrate with other Hadoop components, such as YARN, MapReduce, Tez, Spark, HDFS, HBase, Hive, Ranger, Knox, etc.
- Big SQL uses a cost-based optimizer to generate efficient query plans and a parallel execution framework to execute queries in a distributed manner across the cluster nodes.
- Big SQL provides a JDBC/ODBC driver and a command-line interface (CLI) for connecting to the SQL engine and executing queries.
- Big SQL also provides a web console and a REST API for managing and monitoring the SQL engine and the queries.
- Big SQL can be deployed on-premise or on cloud platforms, such as IBM

Cloud Pak for Data, Amazon Web Services, Microsoft Azure, Google Cloud Platform, etc.

Some of the advantages of Big SQL are:

- It allows you to leverage your existing SQL skills and tools to query and analyze big data in Hadoop.
- It provides high performance, scalability, and reliability for processing large and complex queries on big data.
- It supports various data formats and sources, enabling you to access and integrate data from different systems and applications.
- It supports various Hadoop components and platforms, enabling you to leverage the benefits and features of the Hadoop ecosystem.
- It supports various security and governance mechanisms, such as authentication, authorization, encryption, auditing, etc.

Some of the disadvantages of Big SQL are:

- It may not support some of the advanced SQL features or functions that are specific to other SQL engines or databases.
- It may not be compatible with some of the SQL dialects or extensions that are used by other SQL engines or databases.
- It may require additional configuration and tuning to optimize the performance and resource utilization of the SQL engine and the queries.

Some of the applications of Big SQL are:

- You can use Big SQL to perform ad-hoc queries and exploratory analysis on big data in Hadoop.
- You can use Big SQL to create and populate data warehouses or data marts on Hadoop for reporting and analytics purposes.
- You can use Big SQL to perform data transformation and data integration tasks on Hadoop using SQL scripts or ETL tools.
- You can use Big SQL to expose data in Hadoop to other applications or users using SQL interfaces or APIs.